

BEDROCK ARCHITECTURE

Programmer's Reference Manual

Bounded Word-Oriented CISC Architecture

CONTENTS

1	Overview	1
1.1	Design Goals	1
1.2	Architectural Profile	1
1.3	Instruction Stream	2
1.4	Word 0 Format	2
1.5	Prefix Model	2
2	Terminology	4
2.1	Address Values	4
2.2	Segmentation And Paging	4
2.3	Control Flow And Linkage	5
2.4	Architectural State	5
2.5	Compatibility	6
3	Reserved and Compatibility Rules	6
3.1	Terminology	6
3.2	Reserved Fields	7
3.3	Reserved Encodings	7
3.4	CPUID Compatibility	8
4	Programming Model	9
4.1	Register Model	9
4.2	Data Register Banks	10
4.3	FLAGS and STATUS Registers	11
4.4	Floating-Point Register Model	11
4.5	Segment Registers	13
4.6	Segment Register Operand Class	14
4.7	Special Registers	14
4.8	Control Registers	15
5	CPUID Feature Discovery	18
5.1	Overview	18
5.2	Query Model	18
5.3	Discovery Classes	18

5.4	Leaf Directory	19
5.5	Bit Fields	22
6	SAVE/RESTORE Processor-State Save Area	28
7	Data Formats	31
7.1	Integer Data Sizes	31
7.2	Little-Endian Memory Organization	31
7.3	Instruction Payload Ordering	32
7.4	Floating-Point Data Formats	33
8	Condition Codes	34
9	Prefixes	35
9.1	Address-Update Operand Syntax	35
9.2	User Access Operand Syntax	35
9.3	Prefix Semantics	36
9.4	Conditional Repeat Prefix REPcc	36
9.5	Grouped Repeat Prefix REPG	39
10	Effective Addressing Modes	42
10.1	Compact EA Addressing Modes	43
10.2	Extended EA Descriptor	59
10.3	Extended EA Addressing Modes	60
11	Memory Address Translation	86
11.1	Translation Pipeline	86
11.2	Combined Segment and Page Translation	86
11.3	Segmentation Stage	88
11.4	Paging Stage	88
11.5	LA48 Four-Level Paging	89
11.6	LA57 Five-Level Paging	90
11.7	Page-Table Entry Format	91
12	Memory Model	93
12.1	Scope	93
12.2	Memory Access Domains	93
12.3	Access Size, Alignment, and Tearing	93

12.4 Atomic Access Requirements	94
12.5 Visibility and Ordering	94
12.6 Fault and Partial-Commit Rules	95
12.7 Instruction Fetch and Self-Modifying Code	95
12.8 Address-Translation Coherency	95
13 Supervisor / Privileged Programming Model	97
13.1 Privileged Model Rules	97
13.2 Privilege State	97
13.3 SYSCALL and SYSRET	98
13.4 Interrupt and Control-State Rules	98
14 Exception Processing Reference	99
14.1 Interrupt Vector Table	99
14.2 Vector Assignment	100
14.3 Entry and Return Rules	101
14.4 Supervisor Entry Stack Frame	102
14.5 Interrupt Reset State	104
15 Instruction Word Formats	105
15.1 Representative Encodings	105
16 Instruction Execution Model	106
16.1 Instruction Boundary and Default Execution Rules	106
16.2 Instruction Boundary, Operand Evaluation, and EA Defaults	106
16.3 Mnemonic Suffix Rules	106
16.4 Operand and Status Notation	107
16.5 Shared Side-Effect Rules	107
17 Streaming Execution Model	109
17.1 Architectural Scalar Semantics	109
17.2 Implementation Freedom	109
17.3 Streaming Candidates	109
17.4 Compiler Model	111
17.5 Relationship to Other Acceleration	111
18 Instruction Set Summary	112

18.1 Instruction Attribute Matrix	113
19 Condition Code Computation	118
19.1 Flag Meanings	118
19.2 Common Integer Rules	118
19.3 Shift, Rotate, and Bit Operations	118
19.4 Conditional Consumers	119
19.5 Floating-Point Interactions	119
20 General Instructions Summary	120
21 General Instructions Descriptions	123
21.1 Reading an Instruction Description	123
22 Virtualization Acceleration Instructions Summary	376
23 Virtualization Acceleration Instructions Descriptions	377
24 Floating-Point Instructions Summary	379
25 Floating-Point Instructions Descriptions	380
26 Floating-Point Transcendental Instructions Summary	457
27 Floating-Point Transcendental Instructions Descriptions	458
28 C Library Instruction Examples	477
28.1 REP Counter Convention	477
28.2 memcpy	477
28.3 memmove	477
28.4 memset	478
28.5 memcmp	478
28.6 Bulk Character Extension	478
29 Runtime Instruction Examples	479
29.1 CPUID-Defined Program Context Save	479

LIST OF TABLES

1-1	Word 0 Fields	2
1-2	Instruction Length Encoding	2
1-3	Prefix Word Rules	3
3-1	Compatibility Terms	7
3-2	Compatibility Policy Values	8
4-1	Register Classes	10
4-2	Data Register Bank Selector	10
4-3	Data Register Bank Rules	10
4-4	Data Register Bank Availability Tiers	10
4-5	FLAGS Bits	11
4-6	STATUS Bits	11
4-7	Floating-Point Register File	12
4-8	FFLAGS Bits	12
4-9	FSTATUS Fields	12
4-10	FSTATUS Rounding Modes	12
4-11	Segment Registers	13
4-12	Segment Register Fields	13
4-13	Segment Register Operand Class	14
4-14	RDSEG/WRSEG Segment Register Selector Encoding	14
4-15	Special Registers	15
4-16	Control Registers	15
4-17	Control Register Selector Encoding	16
4-18	PTCR Fields	16
4-19	PTCR PABITS Selector Encoding	16
4-20	ASCR Fields	17
4-21	ICR Fields	17
5-1	CPUID Calling Convention	18
5-2	CPUID Query Selector	18
5-3	CPUID Discovery Policy	19
5-4	CPUID Classes	19
5-5	CPUID Leaves	20
7-1	Integer Data Sizes	31
7-2	Immediate and Displacement Payload Order	32
7-3	Immediate Operand Interpretation	33

8-1	Condition Codes	34
9-1	Prefix Encodings	35
9-2	Address-Update Encodings	35
9-3	User-Access Operand Encodings	35
9-4	REPcc Prefix Rules	37
9-5	REPcc Observed Values	37
9-6	REPG Prefix Rules	40
9-7	REPG Optimization Classes	41
11-1	Table 3-1. Address Translation Stages	86
11-2	Segment Pre-Translation Modes	87
11-3	Segment and Paging Composition	87
11-4	Page-Table Entry Low Attribute Bits	91
11-5	Page-Walk Level Rules	91
11-6	PTE Attribute Semantics	92
11-7	PTE User-Permission Rules	92
12-1	Memory Access Domains	93
12-2	Scalar Access Sizes and Alignment	93
12-3	Atomic Memory-Order Selectors	94
12-4	Fence Instructions	95
13-1	Privileged Model Rules	97
13-2	Privilege State and Access Domains	97
13-3	SYSCALL/SYSRET Rules	98
13-4	Privileged Execution Rules	98
14-1	Interrupt Vector Table Entry Fields	99
14-2	Interrupt Vector Ranges	100
14-3	CPU-Owned Interrupt Vectors	101
14-4	Exception Entry and Return Rules	101
14-5	Supervisor Entry Stack Frame	102
14-6	Supervisor Stack Frame Types	102
14-7	Supervisor Payload Block Slots	103
14-8	FRAME_CONTROL Fields	103
14-9	Repeat Fault Auxiliary Fields	104
14-10	Interrupt and Translation Reset State	104
16-1	Execution Defaults	106
16-2	Conditional Mnemonic Suffix Rules	106

16-3 Semantic Notation	107
18-1 Instruction Classes	113
18-2 Instruction Attribute Matrix Legend	113
18-3 Instruction Attribute Matrix	117
19-1 Integer Condition-Code Bits	118
19-2 Common Integer Flag Computation	118
19-3 Shift and Rotate Count Rules	119
19-4 Shift and Rotate Flag Computation	119
20-1 General Instructions Summary	122
22-1 Virtualization Acceleration Instructions Summary	376
24-1 Floating-Point Instructions Summary	379
26-1 Floating-Point Transcendental Instructions Summary	457

LIST OF FIGURES

1-1 Word 0 Format	2
1-2 Prefix Word Format	3
4-1 User Programming Model	9
4-2 FLAGS Register Format	11
4-3 STATUS Register Format	11
4-4 Segment Register Format	13
4-5 PTCR Format	16
4-6 ASCR Format	17
4-7 ICR Format	17
7-1 Integer Register Operand Subfields	31
7-2 Little-Endian Byte Order for a 64-Bit Value	32
7-3 Instruction Word Storage in Memory	32
10-1 Compact Effective-Address Field	42
10-2 Compact EA Form Map	42
10-3 Compact EA Payload Sequence	43
10-4 Extended EA Word Sequence	59
10-5 Extended Effective-Address Descriptor	59
10-6 Representative Indexed Extended-EA Extra Field	60
11-1 Address Translation Pipeline	86
11-2 LA48 Four-Level Page Walk	89
11-3 LA57 Five-Level Page Walk	90
11-4 PTE Low Attribute Bits	91
14-1 Interrupt Vector Table Entry	99
14-2 IVT Entry Control Byte	99
14-3 Supervisor Entry Stack Frame	102
14-4 FRAME_CONTROL Format	103
15-1 Instruction Word Format Families	105
15-2 Encoding layout for MOV.X(z:L/Q) <ea(e)>, Dn(d)	105
15-3 Encoding layout for MOV.X(z:L/Q) Dn(d), <ea(e)>	105
15-4 Encoding layout for Jcc.X(z:W/L) imm	105

1 OVERVIEW

Bedrock defines a bounded, 16-bit-word-oriented CISC instruction set architecture. It includes selected register-memory forms, compact register-register forms, explicit instruction lengths, and a fixed maximum instruction size.

The base architecture combines simplified effective addressing with page-granular segmented address pre-translation, optional page-table translation, and ordinary memory-mapped device access.

1.1 Design Goals

Bedrock is intended for high-performance general-purpose systems rather than as a minimal embedded instruction set. The architecture keeps the visible programming model scalar and bounded, while giving implementations explicit repeated-operation structures that may be executed by microarchitectural streaming or internal SIMD machinery.

- preserve selected CISC code-density advantages without unbounded instruction forms
- make instruction boundaries and maximum instruction size explicit
- support register-memory operations and compact hot-path register-register operations
- expose scalar architectural state while allowing wider internal execution resources
- accelerate fast, regular loops through REP, REPcc, REPG, and update-eligible effective addresses
- let common loop kernels scale with implementation width without adding programmer-visible vector state or width-specific opcode families
- avoid making vector width or internal streaming resources part of the architectural ABI
- leave specialized cryptographic, image, matrix, tensor, and accelerator workloads to extensions or devices

1.2 Architectural Profile

- 16-bit instruction words
- explicit instruction length encoded in word 0
- maximum instruction length of eight words
- at most one prefix word containing two prefix bytes
- register-register and register-memory instruction forms
- bounded compact and extended effective-address forms
- page-granular segment pre-translation before optional paging
- memory-mapped device access through the normal memory map
- no separate I/O port address space
- no direct physical-memory load/store instructions

1.3 Instruction Stream

The instruction stream is composed of 16-bit words. Instructions are aligned to 16-bit boundaries. The program counter addresses bytes, but instruction length is defined in units of 16-bit words.

$$next_pc = pc + 2 \times instruction_length_in_words$$

The maximum instruction length is eight words, or 16 bytes. No instruction may exceed this bound.

1.4 Word 0 Format

Every instruction begins with word 0. The instruction boundary is determined from word 0 alone.

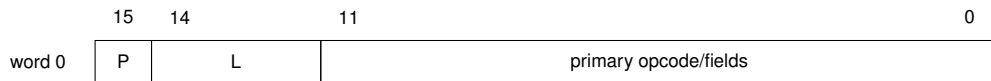


Figure 1-1. Word 0 Format

Field	Bits	Meaning
P	15	Prefix-present bit. If set, word 1 is interpreted as the prefix word.
L	14:12	Total instruction length encoded as length minus one.
primary opcode/fields	11:0	Primary payload for compact operand fields or an extended opcode root.

Table 1-1. Word 0 Fields

L Field	Total Words	Total Bytes
000	1	2
001	2	4
010	3	6
011	4	8
100	5	10
101	6	12
110	7	14
111	8	16

Table 1-2. Instruction Length Encoding

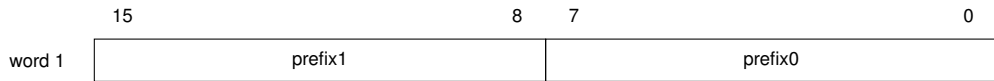
$$length = word0.L + 1 \quad next_pc = pc + 2 \times length$$

1.5 Prefix Model

If P is set, word 1 is a prefix word containing two independent 8-bit prefix slots. The low byte, bits 7..0, is `prefix0`; the high byte, bits 15..8, is `prefix1`. Prefixes do not determine instruction length.

Property	Spec Value
Prefix bytes	2
Fill order	low, high

Property	Spec Value
Decode order	low, high
Unused slot encoding	NPX
Conflict resolution	last prefix wins
Conflict note	Prefix effects are applied in byte order. If two prefixes in the same semantic group or otherwise conflicting prefixes appear in the prefix word, the later decoded prefix overrides the earlier one.

Table 1-3. Prefix Word Rules**Figure 1-2. Prefix Word Format**

2 TERMINOLOGY

2.1 Address Values

effective address (EA)

The address value or operand designator produced by an addressing mode before segment pre-translation. Memory EA forms produce an address value; register and immediate EA forms name non-memory operands.

pre-segment virtual address

The ABI-visible address value before segment pre-translation. Ordinary pointers, ELF symbol values, near CALL targets, and near JMP targets use this address space unless a rule explicitly says otherwise.

linear address

The address produced by segment pre-translation. If paging is enabled, the page-table walker consumes the linear address. If paging is disabled, the linear address is used directly by the memory system.

physical address

A byte-addressed memory address derived by page-table translation for ordinary byte-addressed memory. When paging is disabled, the linear address is not called a physical address by this specification; it is used directly as the memory-system address.

memory-system address

The address value presented to the memory subsystem after all active architectural translation stages. With paging enabled this value is produced from the PTE frame number and page offset. With paging disabled, the linear address is used directly as the memory-system address. The memory-system address may be interpreted as an ordinary byte address or as an externally acknowledged bus-sized access according to the final access attributes.

address generation

The instruction-local calculation that combines registers, displacement, index, scale, and immediate fields to produce an effective-address value. Address generation does not by itself read memory.

2.2 Segmentation And Paging

segment pre-translation

The stage between effective-address calculation and paging. Disabled segments perform no segment-window check and no base addition. Translated-window segments check an offset within the segment span and add the segment base. Bounds-only segments check an already-linear address without adding the segment base.

segment register image

The 64-bit architectural value stored in a segment register. It contains the segment base page, size exponent, size mantissa, and bounds-only enable bit.

disabled segment	A segment whose size mantissa is zero. A disabled segment performs no segment-window check and passes the effective-address value through as the linear address.
translated segment	An enabled segment with bounds-only mode clear. The effective-address value is checked as an offset within the segment span, then the segment base is added.
bounds-only segment	An enabled segment with bounds-only mode set. The effective-address value is already treated as a linear address; the segment only checks that it lies inside the segment window.
page-table translation	The optional translation stage controlled by PTCR.PE. It maps a canonical linear address to a physical frame and applies PTE permissions, cache policy, and addressing type.
PFN	Physical frame number. A leaf PTE supplies the PFN used with the page offset to form the physical address.

2.3 Control Flow And Linkage

near control transfer	A CALL, RET, JMP, or conditional branch that changes only PC within the current code-segment context.
segment-changing control transfer	A control transfer that updates CS and PC as one architectural commit. Long calls and long returns belong to this category.
linkage domain	An ABI-visible region in which ordinary code addresses name near-call entry points and function pointers do not carry an explicit CS value.
veneer	A small runtime-generated or linker-generated near-call entry point that hides a more complex transfer sequence from ordinary callers.

2.4 Architectural State

data register	A D-class general-purpose integer register.
address register	An A-class general-purpose register used preferentially for address values and pointer arguments.
stack pointer register	The SP register. SP is its own register class and is not an A-class register.
segment register operand	A 64-bit segment register exposed through the S operand class. The S operand class names the segment registers defined by the SREG selector table; it does not name FLAGS, STATUS, PC, or SP.

state register	A named architectural state register such as FLAGS or STATUS. State registers are accessed through their dedicated load/write instructions rather than through the S operand class.
control register	A privileged architectural register exposed through the CR operand class, such as PTCR, ASCR, ICR, SPC, SCS, and SDS.
prefix	An 8-bit modifier stored in the optional prefix word. Prefix bytes are interpreted from low byte to high byte; when two prefixes conflict, the later prefix overrides the earlier one.

2.5 Compatibility

reserved	Architecturally held for a future definition. Software must not depend on the current value or behavior of a reserved field, encoding, selector, vector, or result bit.
must be zero	Software must write zero. A nonzero value is invalid unless a later architectural revision assigns the field or value.
read-as-zero	Hardware returns zero for the field on this implementation. Software must still treat the field as reserved unless it is explicitly software-defined.
ignored	Hardware accepts the field but does not use it for the current operation. Ignored fields are not automatically available for software-defined state.
preserved	Software must retain the previous value when rewriting the containing architectural object.
software-defined	Hardware does not interpret the field and reserves it for software use. Future architectural extensions do not consume the field without redefining the containing structure.
implementation-defined	The implementation chooses the value or behavior and exposes the relevant contract through CPUID, fixed platform documentation, or firmware.
illegal	Use of a disallowed instruction opcode encoding, effective-address form, selector, or operation raises the exception named by the defining rule. Prefix bytes use the prefix model's own unassigned value rule.

3 RESERVED AND COMPATIBILITY RULES

This section defines the default compatibility contract for reserved fields, optional features, and future architectural growth. A later section may narrow a rule for a specific structure, but it must say so explicitly.

3.1 Terminology

Term	Meaning
<code>reserved</code>	Architecturally held for a future definition. Software must not depend on its current value or behavior.
<code>must be zero</code>	Software must write zero. A nonzero value is invalid unless a later architectural revision assigns the field.
<code>read-as-zero</code>	Hardware returns zero for the field on this implementation. Software must still treat the field as reserved unless it is explicitly software-defined.
<code>ignored</code>	Hardware accepts the field but does not use it for the current operation. Ignored fields are not automatically available for software-defined state.
<code>preserved</code>	Software must retain the previous value when rewriting the containing object.
<code>software-defined</code>	Hardware does not interpret the field and reserves it for software use. Future architectural extensions do not consume this field without redefining the containing structure.
<code>implementation-defined</code>	The implementation chooses the value or behavior and exposes the relevant contract through CPUID, fixed documentation, or platform firmware.
<code>illegal</code>	Use of a disallowed instruction opcode encoding, effective-address form, selector, or operation raises the exception named by the defining rule. Prefix bytes use the prefix model's unassigned-value rule.

Table 3-1. Compatibility Terms

3.2 Reserved Fields

Architected registers and architectural memory structures use the following defaults:

- Reserved bits in architected registers read as zero and must be written as zero.
- Reserved bits in control registers or selector fields cause `INVALID_CONTROL_STATE` when a nonzero or reserved value is written.
- Reserved bits in page-table entries cause `PAGE_FAULT` when they are consumed by page-table translation.
- Reserved bits in interrupt-vector-table entries and supervisor-entry frames must be zero. A reserved CPU vector has no predefined frame type; attempted delivery follows the double-fault rule described by the exception-processing model.
- Software-defined fields are not reserved fields. Software may store values in them, and hardware ignores them except where the containing structure explicitly says otherwise.

3.3 Reserved Encodings

Reserved instruction opcodes, reserved extension opcode encodings, reserved effective-address forms, and unsupported optional instruction-group encodings raise the exception named in the compatibility policy table. Bedrock keeps the decode contract simple: if the implementation does not support an optional instruction group, encodings from that group are not distinguished from other illegal instruction encodings by a separate extension-availability vector.

Prefix bytes are not instruction opcodes. Unassigned prefix values follow the prefix policy values in the compatibility policy table and remain reserved for future prefix definitions. Prefix bytes are consumed only as prefix metadata; instruction decode then proceeds from the following opcode, operands, and addressing forms.

Noncanonical encodings are invalid unless the instruction description explicitly defines an alias or priority rule. Assemblers must emit canonical encodings by default. Disassemblers must prefer canonical spellings when more than one spelling names the same operation.

3.4 CPUID Compatibility

CPUID is the compatibility boundary for optional architectural behavior. Software must not use an optional instruction group, optional register state, or optional state image unless the corresponding CPUID bit or leaf reports support.

Unknown CPUID selectors, reserved CPUID result bits, privilege, serialization, and runtime mutability are governed by the compatibility policy table unless a CPUID leaf explicitly gives a narrower rule.

Rule	Spec Value
Reserved opcode exception	ILLEGAL_INSTRUCTION
Reserved extension opcode exception	ILLEGAL_INSTRUCTION
Reserved EA form exception	ILLEGAL_INSTRUCTION
Unsupported optional group exception	ILLEGAL_INSTRUCTION
Extension unavailable exception defined	False
Unassigned prefix exception	none
Unassigned prefix behavior	no_architectural_effect
Unknown CPUID class result	zero
Unknown CPUID leaf result	zero
Unknown CPUID index result	zero
Reserved CPUID result bit software action	ignore
CPUID privilege	unprivileged
CPUID serialization	none
CPUID stable after reset	True

Table 3-2. Compatibility Policy Values

4 PROGRAMMING MODEL

4.1 Register Model

The programming model exposes separate data, address, stack-pointer, program-counter, floating-point, segment-register, and control-register classes. SP is an independent stack register, not an alias of the A-register class. The S class is used by RDSEG and WRSEG, while the CR class is used by RDCR and WRCR.

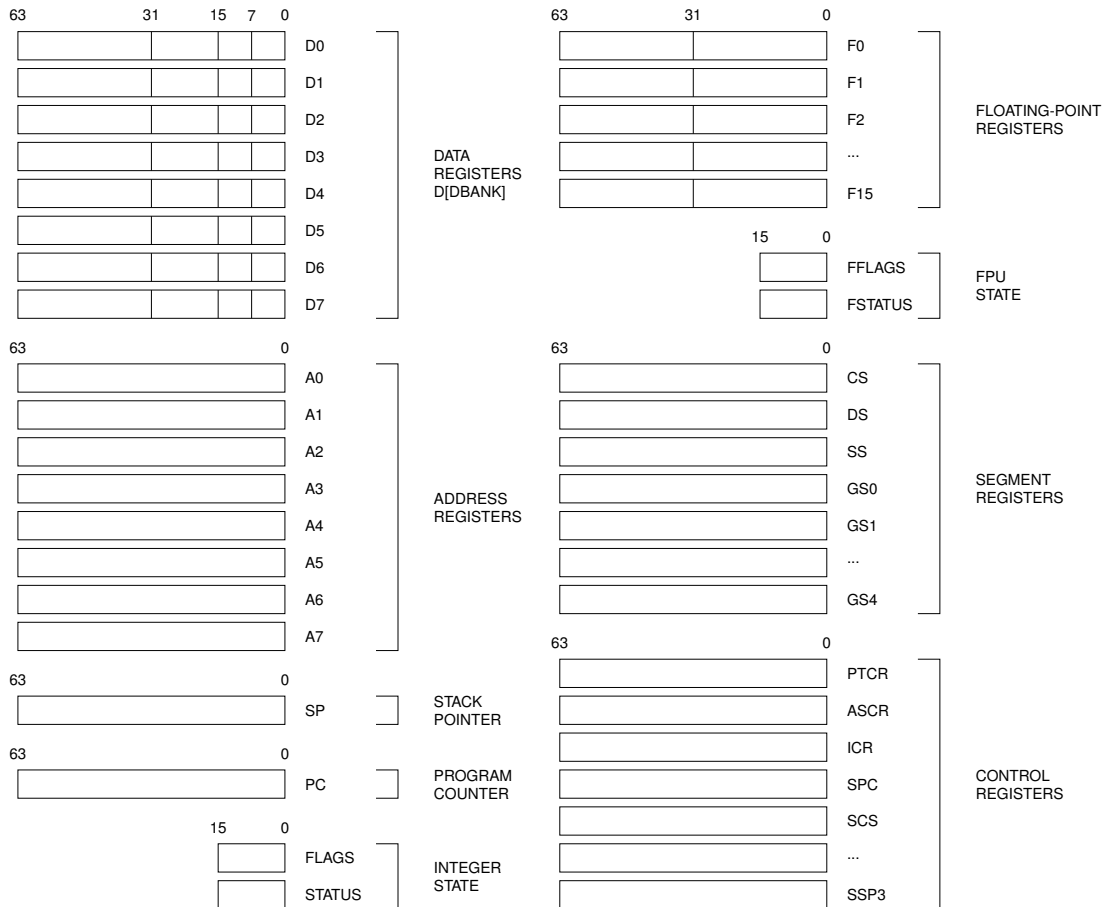


Figure 4-1. User Programming Model

Class	Count	Width	Role	Allocatable
D	8	64	data	True
A	8	64	address	True
SP	1	64	stack pointer	False
PC	1	64	program counter	False
F	16	64	floating-point	True
S	8	64	segment	False
CR	18	64	control	False

Table 4-1. Register Classes

4.2 Data Register Banks

D-register banking extends the data-register file without changing the 3-bit DREG fields used by ordinary instructions. The current DBANK selector chooses which physical D0 through D7 bank is visible. Source-language ABIs do not expose DBANK; compilers, runtimes, hand-written assembly, and repeat/block optimizers may use nonzero banks when the target profile provides them.

Property	Value
Selector	DBANK
Selector width	4 bits
Architectural namespace	16 banks
Base required count	1
Discovery	CPUID implementation properties.DATA_REGISTER_BANKS
Object attribute	BEDROCK_ATTR_REQUIRED_DBANK_COUNT

Table 4-2. Data Register Bank Selector

Rule	Meaning
Visible D register	$D_n \text{ names } D[\text{DBANK}][n]$
Ordinary D operands	ordinary DREG operands use the current DBANK selector
Bank 0	ABI-visible data-register bank
Public ABI boundary	DBANK must be 0 at public ABI boundaries
Handler entry	exception, interrupt, trap, and syscall handlers save the interrupted DBANK selector and execute with DBANK set to 0
Saved state	supervisor-entry and syscall frames save the interrupted DBANK selector; IRET and SYSRET restore it from the saved frame image

Table 4-3. Data Register Bank Rules

Tier	Banks	Meaning
DBANK1	1	only bank 0 is implemented; bank-switching instructions exist but nonzero bank selectors fault
DBANK4	4	banks 0 through 3 are available for backend and runtime scratch allocation
DBANK16	16	the full 4-bit architectural bank namespace is available

Table 4-4. Data Register Bank Availability Tiers

4.3 FLAGS and STATUS Registers

FLAGS and STATUS are accessed through dedicated 16-bit W-sized read/write instructions, not through the S segment-register operand class. Reserved bits read as zero and must remain zero; FLAGS has four meaningful bits and STATUS has 6 meaningful bits.

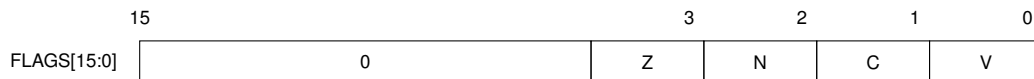


Figure 4-2. FLAGS Register Format

Bit	Position	Meaning
Z	3	zero
N	2	negative
C	1	carry or borrow
V	0	overflow

Table 4-5. FLAGS Bits

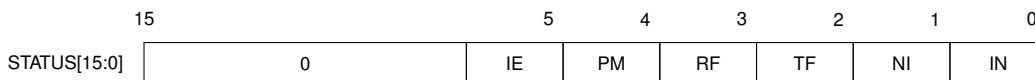


Figure 4-3. STATUS Register Format

Bit	Position	Meaning
IE	5	interrupt enable
PM	4	privileged mode; 0 = user, 1 = supervisor
RF	3	resume flag
TF	2	trap flag
NI	1	NMI in-progress
IN	0	interrupt nesting enable

Table 4-6. STATUS Bits

4.4 Floating-Point Register Model

The floating-point extension defines 16 Q-sized floating-point registers, F0-F15. Single-precision and double-precision operations use the same register file. FFLAGS holds accrued floating-point exception flags. FSTATUS holds floating-point control and status-mode state, including trap enables, rounding mode, and non-default arithmetic modes.

Property	Value
Registers	F0-F15
Count	16
Architectural width	64 bits
Scalar formats	S, D
FFLAGS width	16 bits
FSTATUS width	16 bits
FFLAGS reset	0
FSTATUS reset	0
Unsupported extension instruction	ILLEGAL_INSTRUCTION

Table 4-7. Floating-Point Register File

Bit	Position	Meaning
NV	4	accrued invalid operation flag
DZ	3	accrued divide-by-zero flag
OF	2	accrued overflow flag
UF	1	accrued underflow flag
NX	0	accrued inexact flag

Table 4-8. FFLAGS Bits

Field	Position	Meaning
NV_EN	4	invalid operation trap enable
DZ_EN	3	divide-by-zero trap enable
OF_EN	2	overflow trap enable
UF_EN	1	underflow trap enable
NX_EN	0	inexact trap enable
RM	6:5	rounding mode
FTZ	7	flush tiny results to signed zero
DAZ	8	treat subnormal inputs as signed zero
DN	9	default NaN result mode
reserved	15:10	reserved, must be zero

Table 4-9. FSTATUS Fields

Value	Rounding Mode
0	roundTiesToEven
1	roundTowardZero
2	roundTowardNegative
3	roundTowardPositive

Table 4-10. FSTATUS Rounding Modes

- Set the corresponding accrued FFLAGS bit and continue.
- Raise `FLOATING_POINT_FAULT` when the corresponding exception occurs.

- Reserved FSTATUS bits must be zero; nonzero reserved writes raise `INVALID_CONTROL_STATE`.
- `FSTATUS=0` selects `roundTiesToEven`, disables exception traps, and leaves `FTZ`, `DAZ`, and `DN` disabled

4.5 Segment Registers

Segment registers define a currently addressable memory window plus a simple address translation step. They are not the memory-protection mechanism; paging supplies memory access permissions and final address translation. When the segment mantissa field is zero, the segment entry is disabled. When the mantissa is nonzero, an address passes through segmentation first and then through paging.

The bounds-only bit enables a check-only mode for segment translation: the segment bounds are tested, but the segment base address is not added to the translated address.

Register	Selector	Width
CS	code	64
DS	data	64
SS	stack	64
GS0	general	64
GS1	general	64
GS2	general	64
GS3	general	64
GS4	general	64

Table 4-11. Segment Registers

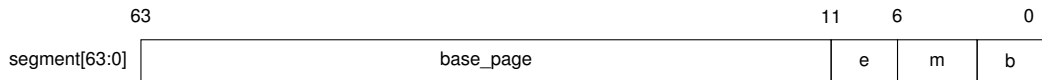


Figure 4-4. Segment Register Format

Field	Bits	Meaning
<code>base_page</code>	63:12	page-unit segment base index
<code>e</code>	11:7	segment size exponent
<code>m</code>	6:1	segment size mantissa
<code>b</code>	0	bounds-only mode enable; check segment bounds without adding the segment base address

Table 4-12. Segment Register Fields

$$\text{base byte address} = \text{base_page} \times 4096$$

$$\text{segment size} = m \times 2^e \times 4096$$

$$\text{limit byte address} = (\text{base_page} + m \times 2^e) \times 4096$$

$$b = 0 : \text{segmented address} = \text{base byte address} + \text{offset}, \quad 0 \leq \text{offset} < \text{segment size}$$

$$b = 1 : \text{segmented address} = \text{offset}, \quad \text{base byte address} \leq \text{offset} < \text{limit byte address}$$

The base, segment size, and limit are computed in an unsigned domain wide enough to represent the full 64-bit linear-address space plus one. If the limit exceeds 2^{64} , the segment register image is invalid and any access through it reports `PAGE_FAULT`.

If `m` is zero, segmentation is disabled for that segment selector and the effective address is already the linear address. If `m` is nonzero, the segment bounds are checked before paging. In normal mode (`b = 0`), the segment base is added to the offset after the bounds check. In bounds-only mode (`b = 1`), the base is not added; the segment only constrains which linear-address window may be accessed.

4.6 Segment Register Operand Class

The S class is the Q-sized segment-register operand class used by `RDSEG` and `WRSEG`. Its selector values are defined by the `SREG` selector table. `SS` is distinct from the `SP` stack-pointer register, and `FLAGS` and `STATUS` are accessed through dedicated `RD`/`WR` instructions.

Class	Width	Bits	Role	Registers
S	64	3	segment	CS DS SS GS0 GS1 GS2 GS3 GS4

Table 4-13. Segment Register Operand Class

Bits	Register	Reserved Access
000	CS	-
001	DS	-
010	SS	-
011	GS0	-
100	GS1	-
101	GS2	-
110	GS3	-
111	GS4	-

Table 4-14. RDSEG/WRSEG Segment Register Selector Encoding

4.7 Special Registers

Name	Width	Class	Role	Privilege	Implicit
PC	64	PC	-	any	True
SP	64	SP	-	any	True
FLAGS	16	FLAGS	condition flags	any	True
STATUS	16	STATUS	state	supervisor	False
FSTATUS	16	FSTATUS	floating-point control and status mode	unprivileged	False

Name	Width	Class	Role	Privilege	Implicit
FFLAGS	16	FFLAGS	floating-point accrued exception flags	unprivileged	False

Table 4-15. Special Registers

4.8 Control Registers

Control registers are Q-sized privileged registers used for translation, interrupt, boot, counter, and privileged-entry state. PTCR and ASCR configure the memory-address translation pipeline described in the Memory Address Translation section; ICR configures interrupt-vector state; SPC, SCS, and SDS hold supervisor-entry control state.

Group	Register(s)	Width	Privilege	Role
translation	PTCR	64	supervisor	control
	ASCR			control
	ICR			control
supervisor entry	SPC	64	supervisor	supervisor entry program counter
	SCS			supervisor entry code segment selector
	SDS			supervisor entry data segment selector
interrupt stack banks	SSS0	64	supervisor	interrupt stack segment selector bank 0
	SSP0			interrupt stack pointer bank 0
	SSS1			interrupt stack segment selector bank 1
	SSP1			interrupt stack pointer bank 1
	SSS2			interrupt stack segment selector bank 2
	SSP2			interrupt stack pointer bank 2
	SSS3			interrupt stack segment selector bank 3
	SSP3			interrupt stack pointer bank 3
boot	B00TPC	64	supervisor	control
	B00TCFG			control
counters	PTC	64	supervisor	control
	PMC			control

Table 4-16. Control Registers

RDCR and WRCR use 16-bit control-register selectors. Selectors are grouped by function, leaving space inside each group for related control state.

Group	Selector / Register
translation	0x0000 PTCR
	0x0001 ASCR
	0x0002 ICR
supervisor entry	0x0100 SPC
	0x0101 SCS
	0x0102 SDS
interrupt stack banks	0x0200 SSS0
	0x0201 SSP0
	0x0210 SSS1
	0x0211 SSP1
	0x0220 SSS2
	0x0221 SSP2
	0x0230 SSS3
	0x0231 SSP3
boot	0x1000 B00TPC
	0x1001 B00TCFG
counters	0x1100 PTC
	0x1101 PMC
unassigned	all other selectors
	INVALID_CONTROL_STATE

Table 4-17. Control Register Selector Encoding

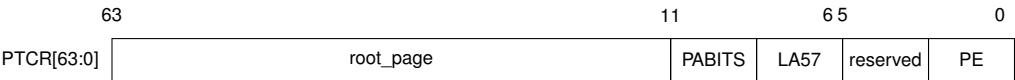


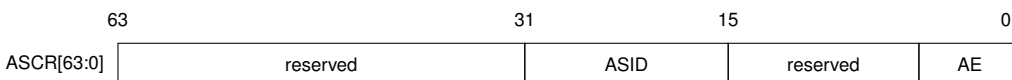
Figure 4-5. PTCR Format

Field	Bits	Meaning
root_page	63:12	page-table root physical frame number
PABITS_SEL	11:8	physical address width selector
LA57	7	5-level paging enable
reserved	6:1	reserved, must be zero
PE	0	page-table translation enable

Table 4-18. PTCR Fields

Selector	PABITS	Reserved Access
0	48	-
1	56	-
2..15	reserved	INVALID_CONTROL_STATE

Table 4-19. PTCR PABITS Selector Encoding

**Figure 4-6. ASCR Format**

ASCR holds the current address-space identifier. SWPT Dn performs a page-table swap that clears ASCR. SWPTA Dn, Dn performs a context-aware page-table swap: PTCR is updated, ASCR[31:16] is set from the low 16 bits of the second data-register operand, and ASCR.AE is set.

Field	Bits	Meaning
reserved_high	63:32	reserved, must be zero
ASID	31:16	current address-space identifier
reserved_low	15:1	reserved, must be zero
AE	0	ASID enable

Table 4-20. ASCR Fields**Figure 4-7. ICR Format**

Field	Bits	Meaning
ivt_page	63:12	interrupt vector table frame number / page number
MAX_IDEPT	11:8	maximum maskable interrupt nesting depth
NMI_PENDING	7	hardware-managed NMI pending bit
NMI_LATCH	6	latch NMI while STATUS.NI is set
DF_ENABLE	5	double-fault handling enable
reserved	4:1	reserved, must be zero
IVT_VALID	0	interrupt vector table valid

Table 4-21. ICR Fields

The detailed interrupt-vector and supervisor-entry frame layouts are described in the Exception Processing Reference section.

5 CPUID FEATURE DISCOVERY

5.1 Overview

CPUID is the architectural discovery instruction. It exposes base-profile identity, optional architectural extensions, and program-visible tuning properties. It is not a dump of internal microarchitectural structures.

A leaf describes one meaningful information group. The result index selects one Q-sized result from that group.

5.2 Query Model

The instruction form is `CPUID Dn`. The selected D register contains a 64-bit query selector before execution and receives the selected 64-bit result after execution. The selector is divided into a 32-bit class, a 16-bit leaf within that class, and a 16-bit result index.

Property	Spec Value
input	Dn contains the CPUID query selector
output	Dn receives one selected 64-bit result
unsupported class	result = 0
unsupported leaf	result = 0
unsupported index	result = 0
privilege	unprivileged
serialization	not serializing
reserved result bits	software must ignore
runtime mutability	static after reset unless a leaf explicitly says otherwise

Table 5-1. CPUID Calling Convention

Bits	Field	Meaning
0..15	index	result index within the selected leaf
16..31	leaf	information leaf within the selected class
32..63	class	CPUID information class

Table 5-2. CPUID Query Selector

5.3 Discovery Classes

Base profile. The base Bedrock ISA profile contains ordinary integer, data movement, control-flow, atomic, data-register banking, cache/TLB, segmentation, paging, control-register, and repeat-prefix instruction groups. Instruction groups with their own optional-extension leaves, such as virtualization acceleration, are not implied by the base profile.

Floating-point. Floating-point discovery is intentionally coarse. An implementation reports whether the base floating-point extension is available, and separately whether the transcendental floating-point group is available. Double precision, fused multiply-add/subtract, floating-point conditional move, conversion, classification, and ordinary floating-point memory/register forms are base features of the floating-point extension.

Data-register banking. Data-register banking instructions are part of the base instruction set. DBANK implementation-property discovery reports the implemented bank count and selector width; object attributes record the minimum bank count required by a linked image. Source-language ABIs still normalize DBANK to zero at public boundaries.

Virtualization acceleration. ENCINST availability is discovered through the optional-extension class. The instruction reference groups ENCINST under Virtualization Acceleration because it supports canonical binary emission for JITs, hypervisors, and binary translators rather than ordinary application execution.

Unavailable optional groups. If an optional instruction group is not reported by CPUID, executing an instruction from that group raises `ILLEGAL_INSTRUCTION`. Extension availability uses the ordinary instruction-encoding fault policy.

Implementation properties. Implementation-property leaves expose only information a program can reasonably use to select code paths or tune algorithms. Structures such as register renaming, reorder buffers, issue queues, and physical register files are not architectural discovery bits. The initial execution-property leaf exposes only `OUT_OF_ORDER`. Processor-topology discovery reports the total exposed hardware-thread count and the current hardware-thread identity; it does not report the set of CPUs currently online under an operating system.

Policy	Class	Meaning
Bedrock base ISA	0x00000000	The base Bedrock ISA profile contains the ordinary integer, data movement, data-register banking, control-flow, atomic, cache/TLB, segmentation, paging, control-register, and repeat-prefix instruction groups. Optional instruction groups that have their own CPUID leaves are not implied by the base profile.
optional extensions	0x00000001	Optional extension class 1 describes architectural features that may be absent while the base ISA remains usable. The initial optional extensions are floating-point execution, floating-point transcendental operations, and virtualization acceleration.
implementation properties	0x00000002	Implementation-property class 2 exposes only information that software can reasonably use when choosing code paths or tuning algorithms. Internal structures such as register renaming, reorder buffers, issue queues, or physical-register files are not exposed as architectural discovery bits.

Table 5-3. CPUID Discovery Policy

Class	Name	Meaning
0x00000000	standard	basic identification, vendor strings, and base-profile constants
0x00000001	isa extensions	optional architectural extensions outside the base ISA
0x00000002	implementation properties	program-visible implementation properties and tuning hints
0x00000003	memory topology	cache, TLB, and memory-topology discovery
0x40000000	implementation private	implementation-defined leaves
0x80000000	extended identification	extended brand/model strings and long-form identification

Table 5-4. CPUID Classes

5.4 Leaf Directory

Class	Leaf	Name	Indexes	Summary
0x00000000	0x0000	STANDARD_ROOT	0, 1, 2	Standard CPUID root.
0x00000000	0x0001	VENDOR_STRING	0..n	Vendor or implementation identifier string.
0x00000001	0x0000	EXTENSION_ROOT	0	Optional ISA extension root.
0x00000001	0x0001	OPTIONAL_EXTENSION_FEATURES	0	Optional ISA extension availability.
0x00000002	0x0000	IMPLEMENTATION_ROOT	0	Implementation-property root.
0x00000002	0x0001	EXECUTION_PROPERTIES	0	Program-visible execution properties.
0x00000002	0x0002	DATA_REGISTER_BANKS	0	Implemented data-register bank resources.
0x00000002	0x0003	PROCESSOR_TOPOLOGY	0, 1, 2..n	Processor topology and current hardware-thread identity.
0x00000002	0x0004	SAVE_AREA_LAYOUT	0, 1, 2..n	Processor-state save area layout.

Table 5-5. CPUID Leaves

CPUID Leaf Details

STANDARD_ROOT (class 0x00000000, leaf 0x0000)

Standard CPUID root.

Result Indexes:

- index 0: maximum supported standard leaf in class 0
- index 1: architecture revision and base profile identifier
- index 2: instruction stream and prefix geometry limits

VENDOR_STRING (class 0x00000000, leaf 0x0001)

Vendor or implementation identifier string.

Result Indexes:

- index 0..n: Vendor or implementation identifier string chunk. Each result returns up to eight bytes in little-endian byte order. The string is zero-terminated; indexes beyond the end return zero.

EXTENSION_ROOT (class 0x00000001, leaf 0x0000)

Optional ISA extension root.

Result Indexes:

- index 0: maximum supported optional-extension leaf in class 1

OPTIONAL_EXTENSION_FEATURES (class 0x00000001, leaf 0x0001)

Optional ISA extension availability.

Result Indexes:

- index 0: optional extension availability feature bits

IMPLEMENTATION_ROOT (class 0x00000002, leaf 0x0000)

Implementation-property root.

Result Indexes:

- index 0: maximum supported implementation-property leaf in class 2

EXECUTION_PROPERTIES (class 0x00000002, leaf 0x0001)

Program-visible execution properties.

Result Indexes:

- index 0: program-visible execution property feature bits

DATA_REGISTER_BANKS (class 0x00000002, leaf 0x0002)

Implemented data-register bank resources.

Result Indexes:

- index 0: implemented data-register bank count, selector width, and availability tier

PROCESSOR_TOPOLOGY (class 0x00000002, leaf 0x0003)

Processor topology and current hardware-thread identity.

Processor topology is reported as a stable implementation-property leaf. Index 0 reports topology metadata and the total number of exposed hardware threads in this CPUID namespace. Index 1 identifies the current hardware thread. Indexes 2..n describe the contiguous bit fields inside the current topology id. Software may use this information for scheduling, locality decisions, and per-thread data placement. The processor id and topology id are not memory addresses and are not required to match interrupt-controller numbering or an operating system CPU number. The hardware-thread count is not an operating system online-CPU count.

Result Indexes:

- index 0: topology metadata and total exposed hardware-thread count
- index 1: current hardware-thread identity
- index 2..n: topology level descriptor
- index 2..n extraction: $\text{level_id} = (\text{CURRENT_TOPOLOGY_ID} \gg \text{LEVEL_SHIFT}) \& ((1 \ll \text{LEVEL_WIDTH}) - 1)$. Descriptors are ordered from the finest level to the coarsest level. Unknown or unused descriptor indexes return zero.

Topology Level Types:

- 0: invalid
- 1: hardware thread

- 2: core
- 3: cluster
- 4: die
- 5: package

SAVE_AREA_LAYOUT (class 0x00000002, leaf 0x0004)

Processor-state save area layout.

SAVE and RESTORE use a fixed 0x0c0-byte base save area followed by CPUID-described 64-byte-aligned extension component slots. The base area contains base header fields, a state-block bitmap, bank-0 D registers, A registers, and GS segment registers. FLAGS, STATUS, and the current DBANK selector are packed into the base header fields. PC, SP, CS, DS, and SS are carried by architectural control transfers or supervisor-entry stack frames and are not duplicated in the SAVE area. The save-area base address must be 4 KiB aligned. Extension component slots are described by numeric byte offsets and may include additional data-register banks and extension-owned architectural state. The first extension component begins no earlier than SAVE_HEADER_SIZE, 0x0c0 for the base profile. CPUID component descriptor index order is the architectural extension-component order; component offsets must be strictly increasing in that order and must be 64-byte aligned. The fixed state-block bitmap at offset 0x008 uses bit *i* for descriptor index 2+*i*, so SAVE_COMPONENT_COUNT must be no larger than 64. Each extension component begins with component-defined header fields for the state inside that component. The memory image does not contain a per-component size table. SAVE may skip architecturally clean components, using an XSAVEOPT-like policy for extension state blocks. There is no architectural upper bound on SAVE_AREA_SIZE; software must allocate the maximum save area size reported by this leaf for the selected implementation/profile.

Result Indexes:

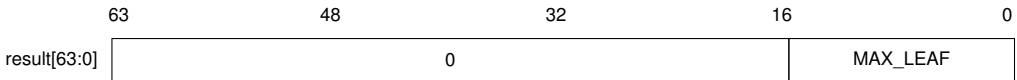
- index 0: save area size requirement
- index 1: save area fixed-header and state-component metadata
- index 2..n: save component descriptor
- index 2..n extraction: Component descriptors are indexed from 2 upward. Descriptor index order is the architectural extension-component order, and COMPONENT_OFFSET values must be strictly increasing in that order. Offsets are 64-byte-aligned numeric byte offsets from the save-area base; the save image itself does not contain a variable component table. Each extension component begins with component-defined header fields. Known base-defined extension component identifiers include additional data-register banks. Optional extensions define their own state component identifiers.

5.5 Bit Fields

CPUID Bit Field Layouts

STANDARD_ROOT (class 0x00000000, leaf 0x0000)

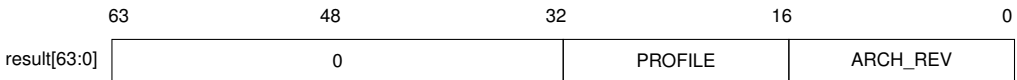
Result index 0



CPUID Payload: STANDARD_ROOT index 0

Bits	Field	Meaning
0..15	MAX_STANDARD_LEAF	maximum supported standard leaf in class 0
16..63	reserved	reserved, reads as zero

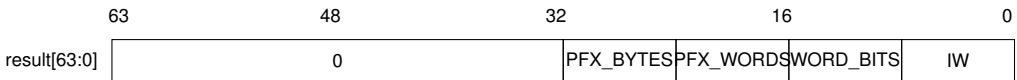
Result index 1



CPUID Payload: STANDARD_ROOT index 1

Bits	Field	Meaning
0..15	ARCH_REVISION	Bedrock architecture revision identifier
16..31	BASE_PROFILE_ID	base ISA profile identifier
32..63	reserved	reserved, reads as zero

Result index 2



CPUID Payload: STANDARD_ROOT index 2

Bits	Field	Meaning
0..7	MAX_INSTRUCTION_WORDS	maximum architectural instruction length in 16-bit words
8..15	WORD_BITS	instruction word size in bits
16..23	MAX_PREFIX_WORDS	maximum prefix words per instruction
24..31	MAX_PREFIX_BYTES	maximum prefix bytes per instruction
32..63	reserved	reserved, reads as zero

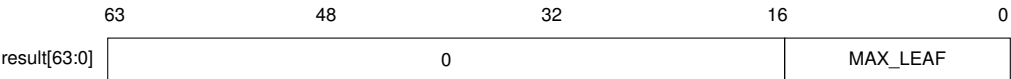
VENDOR_STRING (class 0x00000000, leaf 0x0001)

Result index 0..n

Bits	Field	Meaning
0..63	VENDOR_BYTES	eight-byte little-endian vendor string chunk

EXTENSION_ROOT (class 0x00000001, leaf 0x0000)

Result index 0

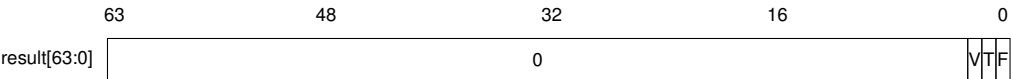


CPUID Payload: EXTENSION_ROOT index 0

Bits	Field	Meaning
0..15	MAX_EXTENSION_LEAF	maximum supported optional-extension leaf in class 1
16..63	reserved	reserved, reads as zero

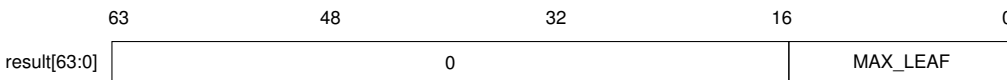
OPTIONAL_EXTENSION_FEATURES (class 0x00000001, leaf 0x0001)

Result index 0



CPUID Payload: OPTIONAL_EXTENSION_FEATURES index 0

Bits	Field	Meaning
0	FP	The base floating-point extension is available. When set, the implementation supports the Bedrock floating-point register model, FSTATUS, FFLAGS, S and D scalar formats, floating-point moves, compares, arithmetic, fused multiply-add/subtract, conversion, classification, bounds, conditional move, and ordinary floating-point memory/register forms.
1	FPTRANS	The floating-point transcendental operation group is available. This covers FSIN, FCOS, FSINCOS, FTAN, inverse trigonometric, hyperbolic, logarithmic, and exponential floating-point operations.
2	VIRTACCEL	The virtualization acceleration instruction group is available. This group includes ENCINST for canonical instruction emission by hypervisors, JITs, and binary translators.
3..63	reserved	reserved, reads as zero

IMPLEMENTATION_ROOT (class 0x00000002, leaf 0x0000)**Result index 0****CPUID Payload: IMPLEMENTATION_ROOT index 0**

Bits	Field	Meaning
0..15	MAX_IMPLEMENTATION_LEAF	maximum supported implementation-property leaf in class 2
16..63	reserved	reserved, reads as zero

EXECUTION_PROPERTIES (class 0x00000002, leaf 0x0001)**Result index 0****CPUID Payload: EXECUTION_PROPERTIES index 0**

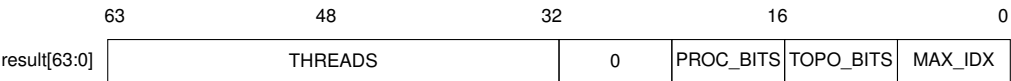
Bits	Field	Meaning
0	OUT_OF_ORDER	The implementation may execute independent architectural instruction instances out of program order while preserving the architectural exception and retirement model. If clear, no out-of-order execution property is advertised.
1..63	reserved	reserved, reads as zero

DATA_REGISTER_BANKS (class 0x00000002, leaf 0x0002)**Result index 0****CPUID Payload: DATA_REGISTER_BANKS index 0**

Bits	Field	Meaning
0..7	DBANK_COUNT	number of implemented D-register banks including bank 0
8..11	DBANK_SELECTOR_BITS	implemented selector width; at most the 4-bit architectural DBANK namespace
12..15	DBANK_TIER	0=DBANK1, 1=DBANK4, 2=DBANK16, other values reserved
16..63	reserved	reserved, reads as zero

PROCESSOR_TOPOLOGY (class 0x00000002, leaf 0x0003)

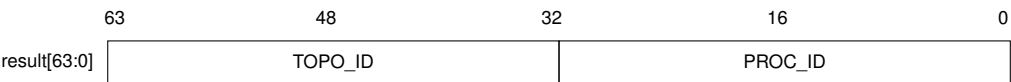
Result index 0



CPUID Payload: PROCESSOR_TOPOLOGY index 0

Bits	Field	Meaning
0..7	MAX_TOPOLOGY_INDEX	highest supported result index for this leaf; 1 means no detailed topology descriptors
8..15	TOPOLOGY_ID_BITS	number of valid low bits in CURRENT_TOPOLOGY_ID
16..23	PROCESSOR_ID_BITS	number of valid low bits in CURRENT_PROCESSOR_ID
24..31	reserved	reserved, reads as zero
32..63	HARDWARE_THREAD_COUNT	total exposed hardware threads in this CPUID namespace; not an OS online-CPU count

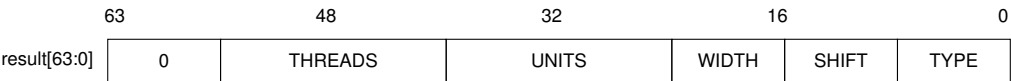
Result index 1



CPUID Payload: PROCESSOR_TOPOLOGY index 1

Bits	Field	Meaning
0..31	CURRENT_PROCESSOR_ID	stable implementation-assigned id of the current hardware thread
32..63	CURRENT_TOPOLOGY_ID	implementation-assigned topology id of the current hardware thread

Result index 2..n



CPUID Payload: PROCESSOR_TOPOLOGY index 2..n

Bits	Field	Meaning
0..7	LEVEL_TYPE	topology level type; 1=hardware thread, 2=core, 3=cluster, 4=die, 5=package
8..15	LEVEL_SHIFT	low bit position of this level inside CURRENT_TOPOLOGY_ID
16..23	LEVEL_WIDTH	number of bits occupied by this level; zero marks an unused descriptor
24..39	UNITS_IN_PARENT	number of units of this level in the immediately containing parent for the current topology
40..55	THREADS_IN_UNIT	number of hardware threads contained in one unit of this level, including lower levels
56..63	reserved	reserved, reads as zero

SAVE_AREA_LAYOUT (class 0x00000002, leaf 0x0004)**Result index 0**

Bits	Field	Meaning
0..63	SAVE_AREA_SIZE	maximum bytes required for the CPUID-defined save area layout; Bedrock defines no smaller architectural upper bound

Result index 1**CPUID Payload: SAVE_AREA_LAYOUT index 1**

Bits	Field	Meaning
0..15	SAVE_HEADER_SIZE	bytes occupied by the fixed base save area; base Bedrock reports 0x0c0
16..31	SAVE_COMPONENT_COUNT	number of CPUID-described extension components after the fixed base area, in canonical descriptor order; must be no larger than 64
32..47	SAVE_BITMAP_WORDS	number of 64-bit state-block bitmap words in the fixed base area; base Bedrock uses one word at offset 0x008
48..63	reserved	reserved, reads as zero

Result index 2..n**CPUID Payload: SAVE_AREA_LAYOUT index 2..n**

Bits	Field	Meaning
0..15	COMPONENT_ID	save component identifier
16..31	COMPONENT_FLAGS	component property flags
32..63	COMPONENT_OFFSET	64-byte-aligned byte offset of the component slot in the CPUID-defined save area

6 SAVE/RESTORE PROCESSOR-STATE SAVE AREA

Save-area bytes								Offset		
63..56	55..48		47..40	39..32	31..24	23..16	15..8	7..0		
reserved		DBANK	STATUS		FLAGS		reserved		GS_VALID	0x000
state block bitmap										0x008
D0, bank 0										0x010
D1, bank 0										0x018
D2, bank 0										0x020
D3, bank 0										0x028
D4, bank 0										0x030
D5, bank 0										0x038
D6, bank 0										0x040
D7, bank 0										0x048
A0										0x050
A1										0x058
A2										0x060
A3										0x068
A4										0x070
A5										0x078
A6										0x080
A7										0x088
GS0										0x090
GS1										0x098
GS2										0x0a0
GS3										0x0a8
GS4										0x0b0
reserved padding to end of fixed base block										0x0b8
extension state block area may begin										0x0c0

Base Header Fields:

Offset	Bits	Field	Meaning
0x000	0..4	GS_VALID	bit n validates GS _n at 0x090 + 8*n; RESTORE updates only GS slots whose bits are set
0x000	5..15	reserved	SAVE writes zero and RESTORE ignores
0x000	16..31	FLAGS	always-saved integer condition-code FLAGS image
0x000	32..47	STATUS	always-saved STATUS image; user-mode RESTORE ignores this field, supervisor RESTORE applies STATUS write rules
0x000	48..51	DBANK	current data-register bank selector; RESTORE applies the SELDB validity rules and updates the selector after bank payloads
0x000	52..63	reserved	SAVE writes zero and RESTORE ignores
0x008	i	component descriptor 2+i present	i=0..SAVE_COMPONENT_COUNT-1 and SAVE_COMPONENT_COUNT must be no larger than 64
0x008	all other bits	reserved	SAVE writes zero and RESTORE ignores

Extension Component Formats:**Extension Component: Additional Data-Register Banks**

Component ID 0x0001
Component Name DATA_REGISTER_BANKS
Component Size 0x40 * DBANK_COUNT

Component-relative bytes								Offset
63..56	55..48	47..40	39..32	31..24	23..16	15..8	7..0	
reserved						D_BANK_VALID		+0x000
reserved padding to first bank record								+0x008
D bank records								+0x040

Component Header Fields:

Offset	Bits	Field	Meaning
+0x000	0	reserved	bank 0 lives in the fixed base block
+0x000	1..15	D_BANK_VALID	bit b validates D bank b at +0x040*b for b=1..DBANK_COUNT-1
+0x000	16..63	reserved	bits for unimplemented banks and future state are reserved

Extension Component: Floating-Point State

Component ID 0x0002
Component Name FLOATING_POINT
Extension Requirement floating_point
Component Size 0x00c0

Component-relative bytes								Offset
63..56	55..48	47..40	39..32	31..24	23..16	15..8	7..0	
FSTATUS		FFLAGS		reserved		F_VALID		+0x000
F0								+0x008
F1								+0x010
F2								+0x018
F3								+0x020
F4								+0x028
F5								+0x030
F6								+0x038
F7								+0x040
F8								+0x048
F9								+0x050
F10								+0x058
F11								+0x060
F12								+0x068
F13								+0x070
F14								+0x078
F15								+0x080
reserved padding to 0x0c0								+0x088

Component Header Fields:

Offset	Bits	Field	Meaning
+0x000	0..15	F_VALID	bit n validates Fn at +0x008 + 8*n
+0x000	16..31	reserved	SAVE writes zero and RESTORE ignores
+0x000	32..47	FFLAGS	floating-point accrued exception flags, always present when this state block is present
+0x000	48..63	FSTATUS	floating-point control and status-mode register, always present when this state block is present

7 DATA FORMATS

7.1 Integer Data Sizes

Integer operands use four architectural sizes. The size suffix selects the number of low-order bits used by the operation and the number of bytes transferred for memory operands.

Suffix	Bits	Bytes	Name
B	8	1	byte
W	16	2	word
L	32	4	long word
Q	64	8	quad word

Table 7-1. Integer Data Sizes

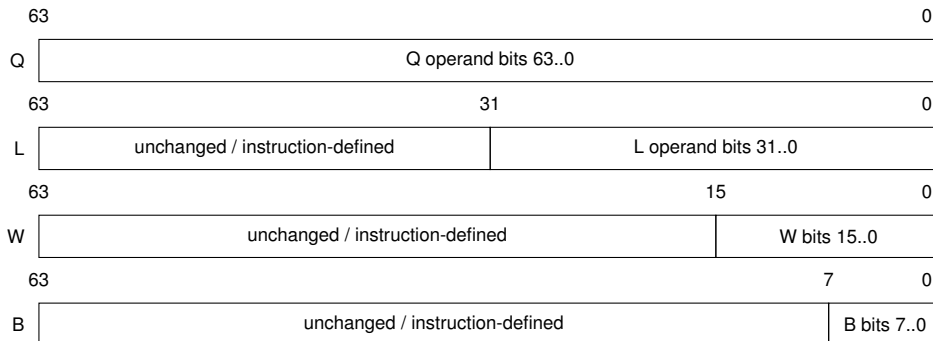


Figure 7-1. Integer Register Operand Subfields

Byte, word, and long operations use the low-order subfield of the selected register. Q-sized operations use the full 64-bit register. Instructions that explicitly extend a value define whether the upper bits are sign-extended or zero-extended; otherwise, the instruction semantics define whether upper bits are preserved, cleared, or replaced.

7.2 Little-Endian Memory Organization

Bedrock data memory is little-endian. For an N-byte integer value stored at byte address A, byte A contains bits 7..0, byte A+1 contains bits 15..8, and so on. The least significant byte is stored at the lowest address.

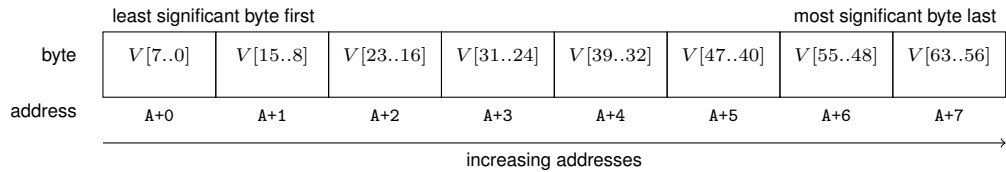


Figure 7-2. Little-Endian Byte Order for a 64-Bit Value

$$mem[A + i] = (value \gg (8i)) \& 255, \quad 0 \leq i < N$$

The same byte order is used for instruction words, immediate payloads, displacement payloads, absolute-address payloads, and data transferred by load/store instructions.

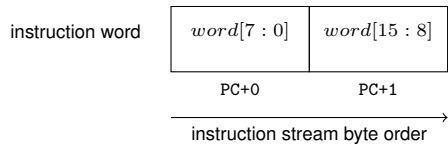


Figure 7-3. Instruction Word Storage in Memory

7.3 Instruction Payload Ordering

Instruction length is measured in 16-bit words, but each instruction word is itself stored in little-endian byte order. Word 0 occupies the first two bytes of the instruction. If a prefix word, descriptor word, immediate, displacement, or bitmap follows, those words occupy increasing addresses in decode order.

Payload	Words	Memory Order
imm16	1 word	bits 7..0 at the lower byte, bits 15..8 at the next byte
imm32	2 words	least significant word first, then most significant word
imm64	4 words	least significant word first through most significant word
disp16	1 word	signed two's-complement value
disp32	2 words	signed two's-complement value, least significant word first
disp64	4 words	signed two's-complement value, least significant word first

Table 7-2. Immediate and Displacement Payload Order

Multiword immediates and displacements are stored least significant word first. Signed displacement payloads use two's-complement representation at their declared width before any sign extension performed by the consuming instruction or effective-address form.

When an instruction operand has an explicit operation size and encodes an immediate operand narrower than that operation size, the operand kind defines how the encoded value is extended before use. Immediate operands on forms without an operation size are interpreted by that instruction form rather than by this operation-size rule.

Operand	Bits	Value	Range	Operation-Size Rule	Applies When
imm6	6	unsigned	0..63	zero-extend to the operation size	integer forms with an explicit B/W/L/Q operation size

Table 7-3. Immediate Operand Interpretation

7.4 Floating-Point Data Formats

Floating-point scalar formats are little-endian in memory as well. The S format is 32 bits and the D format is 64 bits. The bit-level floating-point encoding, NaN policy, exception flags, and rounding behavior are described by the floating-point programming model and instruction metadata; their memory byte order follows the same least-significant-byte-first rule as integer data.

8 CONDITION CODES

Conditional instructions encode a four-bit condition code. The zero-valued condition is T, which is also used for canonical aliases such as JMP for Jcc.T and TRAP for TRAPcc.T. The condition-code bits are the low four meaningful bits of FLAGS, as shown in the Register Model section.

Code	Value	Aliases	Expression
T	0x0	-	true
F	0x1	-	false
EQ	0x2	Z	Z == 1
NE	0x3	NZ	Z == 0
ULT	0x4	C	C == 1
UGE	0x5	NC	C == 0
MI	0x6	N	N == 1
PL	0x7	NN	N == 0
VS	0x8	V	V == 1
VC	0x9	NV	V == 0
ULE	0xa	-	C == 1 Z == 1
UGT	0xb	-	C == 0 && Z == 0
LT	0xc	-	N != V
GE	0xd	-	N == V
LE	0xe	-	Z == 1 N != V
GT	0xf	-	Z == 0 && N == V

Table 8-1. Condition Codes

9 PREFIXES

Prefix words modify the following instruction. One prefix word contains two 8-bit slots; the low byte is filled and decoded first, and the high byte is filled and decoded second. The table below lists standalone prefix spellings and assembler-generated repeat-boundary bytes. Address-update and user-access forms are shown separately because they are selected by operand syntax.

Syntax	Byte/Pattern	Meaning
NPX	0x00	no effect
NOSPEC	0x01	speculation boundary
SAT	0x02	saturating arithmetic
NT	0x03	non-temporal memory hint
REP{cc} Dn, instr	1ccc cddd	repeat while counter and condition remain active
REPG Dn, {...}	0111 0ddd	repeat grouped instructions using the selected counter
ENDG	0x78	end grouped repeat

Table 9-1. Prefix Encodings

9.1 Address-Update Operand Syntax

Address update is selected by the memory operand spelling, not by a standalone prefix mnemonic. The assembler emits the corresponding prefix byte only for update-eligible indirect EA forms. Because legality depends on the selected EA form, address update is not listed as a separate per-mnemonic column in the instruction attribute matrix.

Operand Syntax	Byte	Update
[An++]	0x04	Address-update prefix for update-eligible indirect EA forms; the address register is incremented after the access.
[++An]	0x05	Address-update prefix for update-eligible indirect EA forms; the address register is incremented before the access.
[An--]	0x06	Address-update prefix for update-eligible indirect EA forms; the address register is decremented after the access.
[--An]	0x07	Address-update prefix for update-eligible indirect EA forms; the address register is decremented before the access.

Table 9-2. Address-Update Encodings

9.2 User Access Operand Syntax

Memory operands use the current-domain C by default. Assembly source may annotate memory operands with U: to request a user-domain access. The assembler maps the annotated source and destination domains to one encoded access-domain prefix. Explicit C: is allowed for readability but is usually omitted. Operand positions that are not memory accesses do not contribute to prefix selection. If no user-domain memory access is present, no access-domain prefix is needed.

Operand Domains	Byte	Encoded Meaning
U:[src], C:[dst]	0x08	user-domain source to current-domain destination
C:[src], U:[dst]	0x09	current-domain source to user-domain destination
U:[src], U:[dst]	0x0a	user-domain memory operands

Table 9-3. User-Access Operand Encodings

9.3 Prefix Semantics

Prefix bytes are interpreted as modifiers of the immediately following instruction only. Assemblers should avoid redundant or contradictory prefixes, but architectural prefix interpretation remains deterministic because later prefix slots override earlier conflicting effects.

NPX:	Neutral prefix byte and canonical filler for an unused prefix slot.
NOSPEC:	Marks the following instruction as a speculation boundary or speculation-limited operation.
SATURATE:	Requests the saturating form of arithmetic instructions that define one. Applies to: arithmetic
NONTEMPORAL:	Provides a cache-allocation hint for memory operations without changing the architectural loaded or stored value. Applies to: memory
U2C:	Access-domain prefix selected by the U:[src], C:[dst] operand annotation. Applies to: memory Examples: MOV.Q U: [A0] , DO, MOV.B U: [A0++] , C: [A1++]
C2U:	Access-domain prefix selected by the C:[src], U:[dst] operand annotation. Applies to: memory Examples: MOV.Q DO, U: [A1], MOV.B C: [A0++] , U: [A1++]
U2U:	Access-domain prefix selected by the U:[src], U:[dst] operand annotation. Applies to: memory Examples: MOV.B U: [A0++] , U: [A1++] , INC.L U: [A0]

9.4 Conditional Repeat Prefix REPcc

REPcc is the conditional repeat prefix family. It combines a D-register repeat counter with a full condition-code selector, then applies that repeat contract to the following instruction. REP is the canonical alias for REPT.

The selected D register remains an ordinary signed two's-complement value. REP/REPcc tests that value for zero before each iteration. After the instruction body completes, REPFLAGS are computed from the observed value; the counter moves one step toward zero only when the selected REP condition remains true. A completed terminating iteration whose REP condition is false commits the instruction body's ordinary side effects but leaves the counter unchanged. If the same D register appears in an indexed effective address, EA evaluation uses the same architectural value before the iteration update.

Item	Value
Syntax	REPcc Dn, <instruction>
Alias	REP = REPT

Item	Value
FPU conditional subset	FCMP FTEST FBNDII FBNDIX FBNDXI FBNDXX
FPU status predicates	not supported
Counter	DREG, signed counter moves toward zero
Counter encoding	signed two's-complement; bits 63..0
Zero test	zero counter performs no iteration
REP index value	architectural counter value before iteration update
Condition-true update	positive counters decrement and negative counters increment toward zero
Condition-false update	counter is not updated
Commit rule	a condition-false terminating iteration commits the body and leaves the counter unchanged
Condition source	REPFLAGS from the iteration observed value
Architectural flags	body instruction rules apply to the last completed iteration
FFLAGS	bitwise inclusive-or of completed iterations

Table 9-4. REPcc Prefix Rules

Repeats the following instruction using the selected D-register counter and condition-code selector.

Mnemonic	Observed Value	REPFLAGS Rule
CMP	right operand minus left operand	compute subtract flags from right operand minus left operand
EXTSL	result value	set Z/N from observed value and clear C/V
EXTSQ	result value	set Z/N from observed value and clear C/V
EXTSW	result value	set Z/N from observed value and clear C/V
EXTZL	result value	set Z/N from observed value and clear C/V
EXTZQ	result value	set Z/N from observed value and clear C/V
EXTZW	result value	set Z/N from observed value and clear C/V
FBNDII	-	set V from interval check
FBNDIX	-	set V from interval check
FBNDXI	-	set V from interval check
FBNDXX	-	set V from interval check
FCMP	-	floating-point compare flag rules
FTEST	-	floating-point compare flag rules
MOV	source value	set Z/N from observed value and clear C/V
TEST	left operand bitwise-and right operand	set Z/N from observed value and clear C/V

Table 9-5. REPcc Observed Values

REPcc Operation Examples. The following examples show how the observed value, REPFLAGS, counter update, and terminating iteration interact for common operation classes. In each case, the counter is tested before the body executes. The body then computes an observed value, derives REPFLAGS from that value, and updates the counter only when the selected REP condition remains true.

Counted Move. Plain REP is the REPT alias, so the condition is always true. This form copies exactly the requested count, and the counter reaches zero when the transfer finishes.

```
; D0 = -n, A0 = src + n, A1 = dst + n
REP D0, MOV.B [A0 + D0 * 1], [A1 + D0 * 1]
```

For MOV, the observed value is the source value. REPT does not use it to stop; the body commits and the counter moves toward zero after every completed iteration.

Scan Until Zero. MOV can also be used with REPNE to scan a stream while the loaded value remains nonzero.

```
; D0 = n, A0 = base
MOV.L D0, D1
REPNE D1, MOV.L [A0++], D2
SUB.L D1, D0
```

The MOV body loads the candidate element into D2 and uses that loaded value as the observed value. If the value is nonzero, REPNE is true and D1 is decremented. If the value is zero, the load and address update still commit, but D1 is not decremented. The final subtraction therefore produces the zero element index without a separate off-by-one correction.

Compare While Equal. CMP uses the subtraction result as the observed value. REPEQ continues while the comparison is equal and stops on the first non-equal comparison or when the counter reaches zero.

```
; D0 = -n, A0 = a + n, A1 = b + n
REPEQ D0, CMP.B [A0 + D0 * 1], [A1 + D0 * 1]
```

When a mismatch is found, the terminating CMP commits its FLAGS result and the counter is not updated for that terminating iteration. Software can use the final FLAGS state to derive the ordered result.

Test While Nonzero. TEST derives REPFLAGS from the logical test result. A memory scan can therefore continue while a masked value remains nonzero, or stop when a selected bit range clears.

```
; D0 = n, A0 = base, D3 = mask
REPNE D0, TEST.L [A0++], D3
```

The memory operand and mask are tested each iteration. A nonzero tested result updates the counter and continues; a zero tested result commits the TEST FLAGS and leaves the counter at the terminating element.

Bulk Extension. The EXTS and EXTZ memory forms use the widened result as the observed value. An unconditional REP form expresses a compact bulk conversion, while a conditional form can stop when a converted value matches the selected condition.

```
; D0 = n, A0 = byte source, A1 = long destination
REP D0, EXTZL.B [A0++], [A1++]
```

The source byte is read, zero-extended to a long value, and written to the destination element. Because REP is REPT, every completed conversion updates the counter.

9.5 Grouped Repeat Prefix REPG

REPG repeats an ENDG-terminated block of instructions with a D-register counter. The first grouped instruction carries the REPG prefix byte; the final grouped instruction carries the ENDG prefix byte. Instructions between those endpoints are ordinary instruction records with their own word 0 length fields. The grouped instructions execute in program order and have no condition-code selector.

Architecturally, REPG is a compact spelling for a counted scalar loop over a short straight-line instruction trace. One group iteration executes every instruction from the REPG-marked instruction through the ENDG-marked instruction exactly once, in instruction-stream order. After a successful group iteration, the selected D register is moved one step toward zero. A zero counter performs no group iteration. Positive counters count down; negative counters count up toward zero.

Group Discovery. A REPG group is discovered entirely from the architectural instruction stream. When a REPG prefix is seen, instruction interpretation follows the explicit word 0 length fields of subsequent instruction records in the same naturally aligned 64-byte grouping window until it reaches an instruction carrying ENDG. ENDG may appear on the same instruction as REPG, which forms a one-instruction group. A well-formed REPG group has exactly one active REPG start marker, exactly one ENDG terminator, and no nested repeat prefix inside the group.

Fast-Contract Source Form. REPGF is a source-level assertion that uses the same architectural encoding as REPG. It does not allocate a second prefix byte and does not force a particular implementation strategy. It marks the group as a REPG-fast candidate: the selected counter register is outside the group writes, the effective addresses are regular, and the group avoids control, state, and nested-repeat operations. Runtime properties such as ordinary mapped memory remain architectural preconditions of the fast class rather than extra encoded bits.

Grouping Window. The complete byte range occupied by the grouped instructions must fit within the naturally aligned 64-byte grouping window that contains the first grouped instruction. The first instruction does not need to be aligned to the beginning of the window; only containment matters. This keeps architectural group recognition bounded while avoiding any dependency on a particular implementation's instruction-fetch granularity.

Execution and Faults. Each grouped instruction retains its ordinary operand decoding, effective-address calculation, privilege checks, FLAGS or FFLAGS behavior, and memory side

effects. Address-update operands update as they would in scalar execution for each completed instruction.

If a fault occurs, completed iterations and completed instructions in the current iteration remain committed, and the fault is precise at the faulting grouped instruction. The counter update for an iteration occurs only after the whole group iteration completes successfully. A REPG fault frame saves the PC of the faulting grouped instruction and a compact repeat continuation in `FAULT_AUX`. The group start is not stored as a full address; it is reconstructed from a five-bit unsigned negative word displacement:

$$\text{group_start_pc} = \text{fault_pc} - 2 \times \text{group_start_delta_words}$$

The group index is not saved. If the handler returns without editing the frame, `IRET` resumes at the saved PC under the active REPG continuation and then proceeds through the rest of the group.

Group Model. A REPG group is intended to be straight-line code. Branches, calls, returns, traps, system-call returns, interrupt returns, atomics, stack operations, nested repeat prefixes, and state-mutating system, TLB, cache, or fence operations are outside the REPG group model. REPG optimization is divided into REPG-fast and REPG-general. REPG-fast identifies groups that are suitable for internal SIMD or streaming recognition. REPG-general preserves exact scalar semantics for less regular groups, including groups with selected-counter dependencies.

Item	Value
Syntax	source uses <code>REPG Dn</code> , followed by a braced instruction list
Fast-contract syntax	source may use <code>REPGF Dn</code> , followed by a braced instruction list; the emitted encoding is the same as <code>REPG</code>
Condition selector	none
Counter	D register
Group termination	<code>ENDG</code> prefix on the final grouped instruction
Fast-contract marker	REPG-fast candidate when the group satisfies the fast rules
Grouping window	grouped instruction bytes must fit in one naturally aligned 64-byte window
Commit rule	completed iterations and completed prior group instructions commit
Fault continuation	saved PC is the faulting grouped instruction; group-start delta is a 5-bit negative word displacement; no group index is saved
FFLAGS	OR of completed iterations

Table 9-6. REPG Prefix Rules

Class	Availability	Implementation
REPG-fast	counter <code>Dn</code> is not written regular effective addresses ordinary memory no control or state changes	internal SIMD or streaming optimization is possible

REPG-general	counter Dn may be read or written state-query instructions may appear wait-like instructions may appear examples: WAIT, CPUID	exact scalar semantics, with no fast streaming guarantee
--------------	--	---

Table 9-7. REPG Optimization Classes**Example.**

```
REPG D2, {  
    MOV.L [A1++], D4  
    MADD.L [A0++], D4, D1  
}
```

The example repeats a two-instruction dot-product step. Each iteration loads one source value, performs one multiply-add using the other source stream, advances the update-eligible addresses, and then updates D2 toward zero after the whole group completes.

10 EFFECTIVE ADDRESSING MODES

Effective-address operands are encoded through a compact six-bit EA field. Most instructions that accept EA therefore share the same operand sublayout, which keeps register and memory forms visually regular. The high three bits select the addressing family and the low three bits normally carry the register number or a small form selector. Forms that need displacement, absolute address, immediate data, or an extended descriptor append payload words after the instruction word.

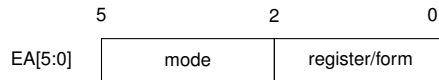


Figure 10-1. Compact Effective-Address Field

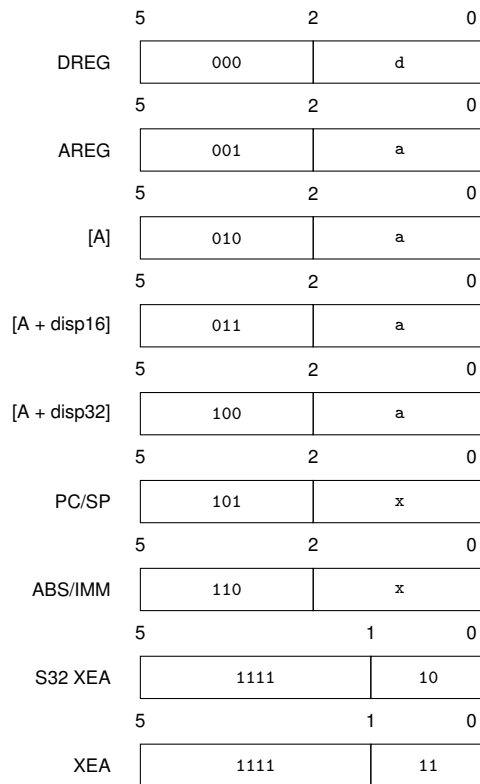


Figure 10-2. Compact EA Form Map

Compact EA forms keep the register selector in EA[2:0] whenever the form is register-based. The PC/SP, absolute, and immediate groups reuse those low bits as form selectors. The S32 XEA and XEA compact forms are escapes to the descriptor shown below.

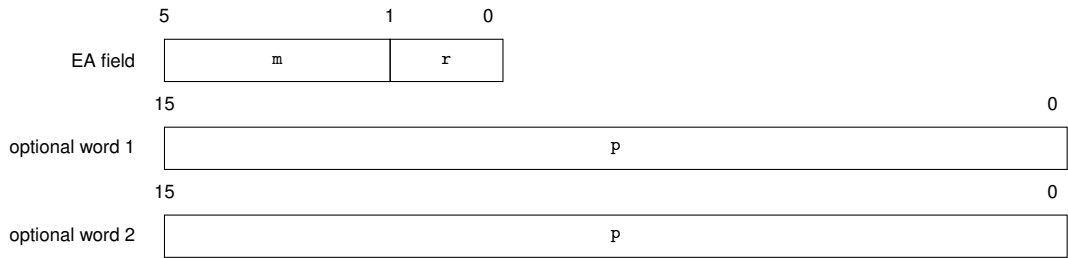


Figure 10-3. Compact EA Payload Sequence

10.1 Compact EA Addressing Modes

Data Register Direct

Direct data-register operand. The EA field selects D_n and does not imply a memory access.

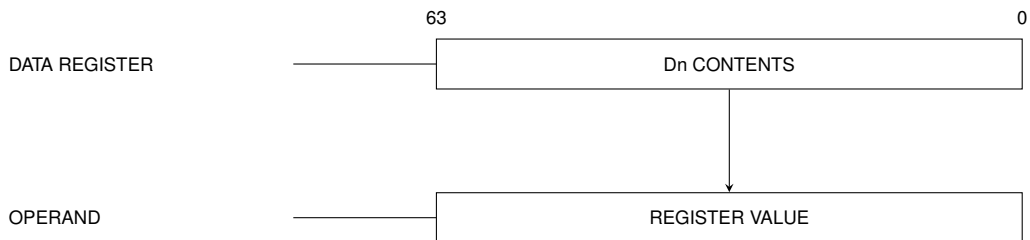
Generation: operand = contents(D_n)

Assembler Syntax: D_n

Encoding: 000 ddd

Payload: no payload words

Address Update: not eligible



Direct operand selection for Data Register Direct

Address Register Direct

Direct address-register operand. The EA field selects A_n and does not imply a memory access.

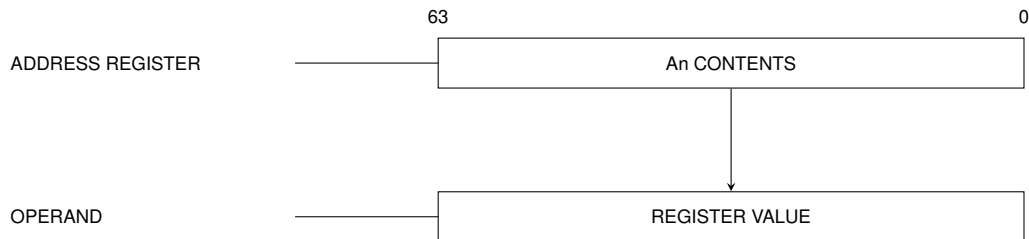
Generation: operand = contents(A_n)

Assembler Syntax: A_n

Encoding: 001 aaa

Payload: no payload words

Address Update: not eligible



Direct operand selection for Address Register Direct

Address Register Indirect

address-register indirect memory operand. address-update prefixes may use this form.

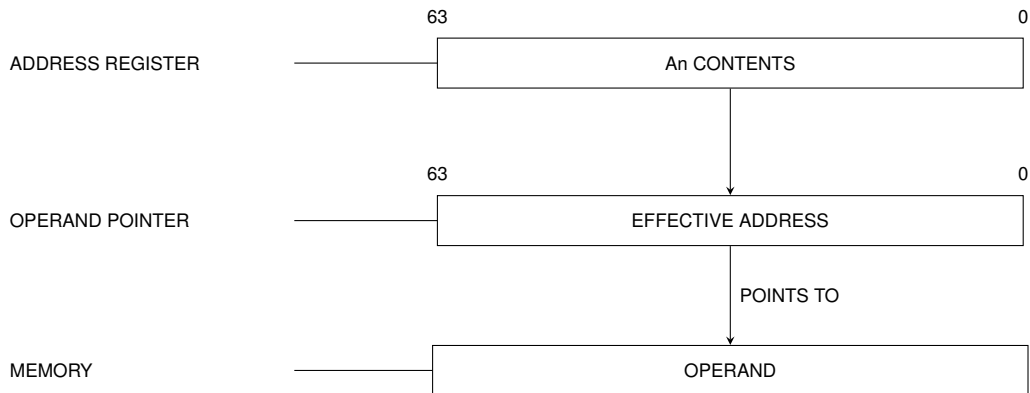
Generation: EA = contents(A_n)

Assembler Syntax: [A]

Encoding: 010 aaa

Payload: no payload words

Address Update: eligible



Address calculation for Address Register Indirect

Address Register Indirect with disp16

address-register relative memory operand. with a signed 16-bit displacement payload.

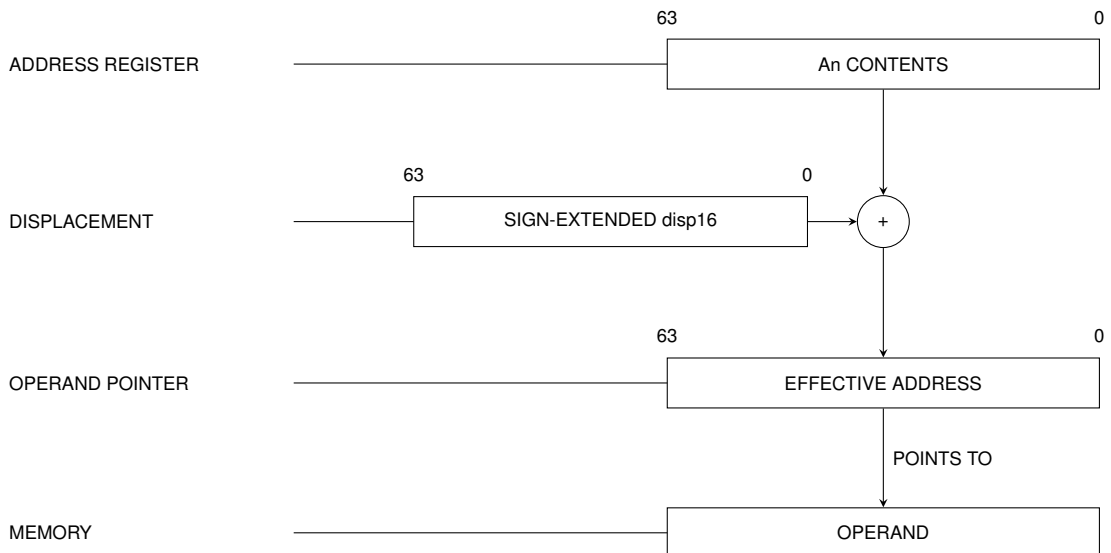
Generation: $EA = \text{contents}(A_n) + \text{disp16}$

Assembler Syntax: $[A + \text{disp16}]$

Encoding: 011 aaa

Payload: one signed 16-bit displacement word

Address Update: not eligible



Address calculation for Address Register Indirect with disp16

Address Register Indirect with disp32

address-register relative memory operand. with a signed 32-bit displacement payload.

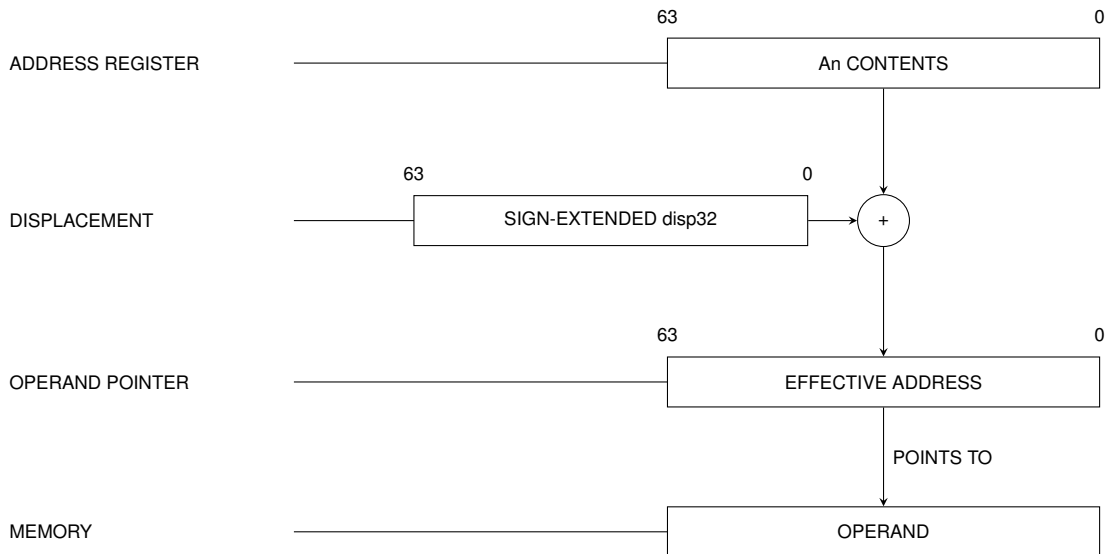
Generation: $EA = \text{contents}(A_n) + \text{disp32}$

Assembler Syntax: `[A + disp32]`

Encoding: 100 aaa

Payload: two signed 32-bit displacement words

Address Update: not eligible



Address calculation for Address Register Indirect with disp32

Program Counter Relative with disp16

program-counter relative memory operand. with a signed 16-bit displacement payload. using fixed CS segment selection.

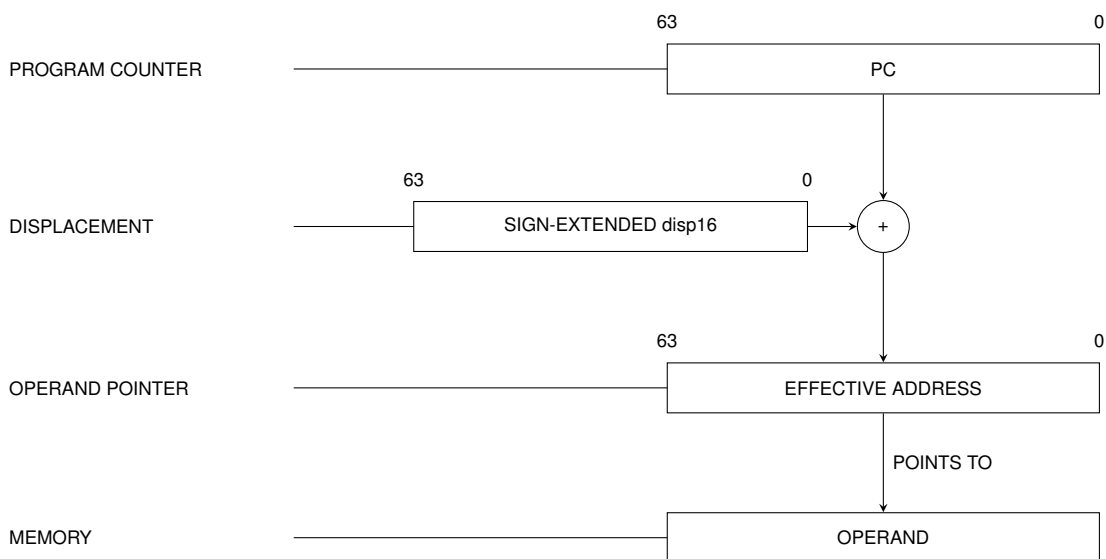
Generation: $EA = PC + \text{disp16}$

Assembler Syntax: $[PC + \text{disp16}]$

Encoding: 101000

Payload: one signed 16-bit displacement word

Address Update: not eligible



Address calculation for Program Counter Relative with disp16

Program Counter Relative with disp32

program-counter relative memory operand. with a signed 32-bit displacement payload. using fixed CS segment selection.

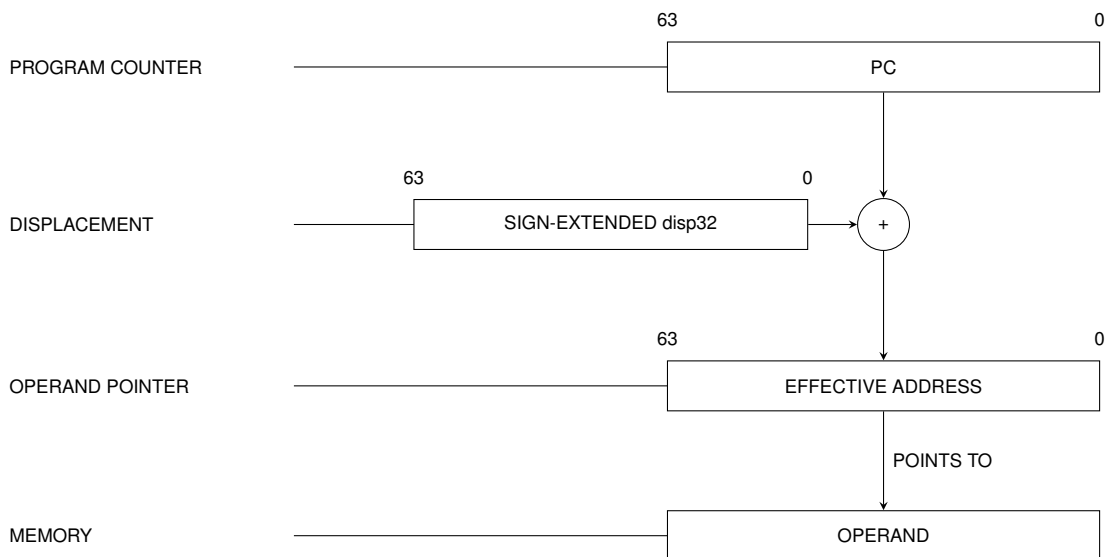
Generation: $EA = PC + \text{disp32}$

Assembler Syntax: $[PC + \text{disp32}]$

Encoding: 101001

Payload: two signed 32-bit displacement words

Address Update: not eligible



Address calculation for Program Counter Relative with disp32

Program Counter Relative with disp64

program-counter relative memory operand. with a signed 64-bit displacement payload. using fixed CS segment selection.

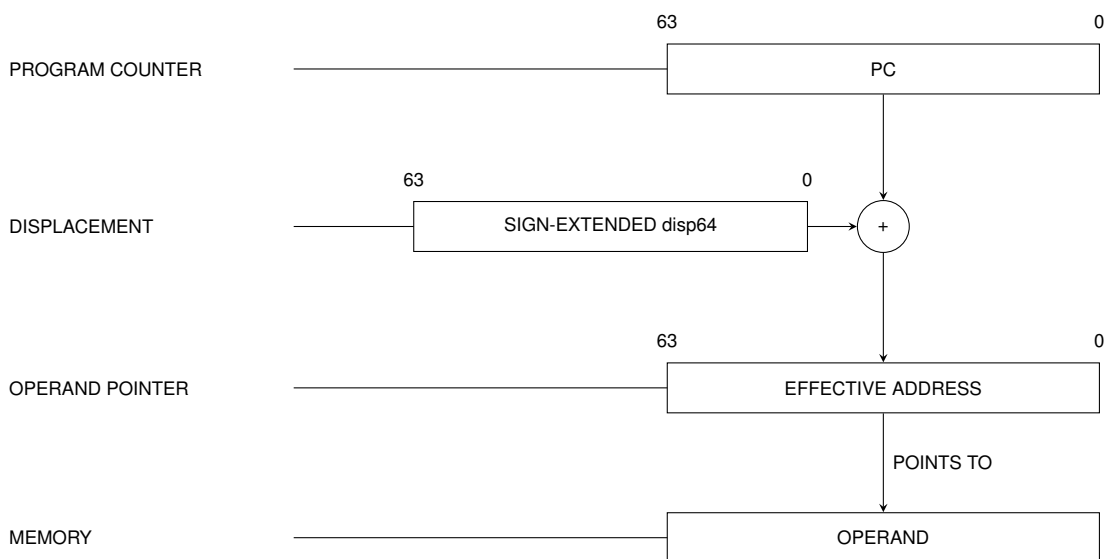
Generation: $EA = PC + \text{disp64}$

Assembler Syntax: $[PC + \text{disp64}]$

Encoding: 101010

Payload: four signed 64-bit displacement words

Address Update: not eligible



Address calculation for Program Counter Relative with disp64

Stack Pointer Relative with disp16

stack-pointer relative memory operand. with a signed 16-bit displacement payload. using fixed SS segment selection.

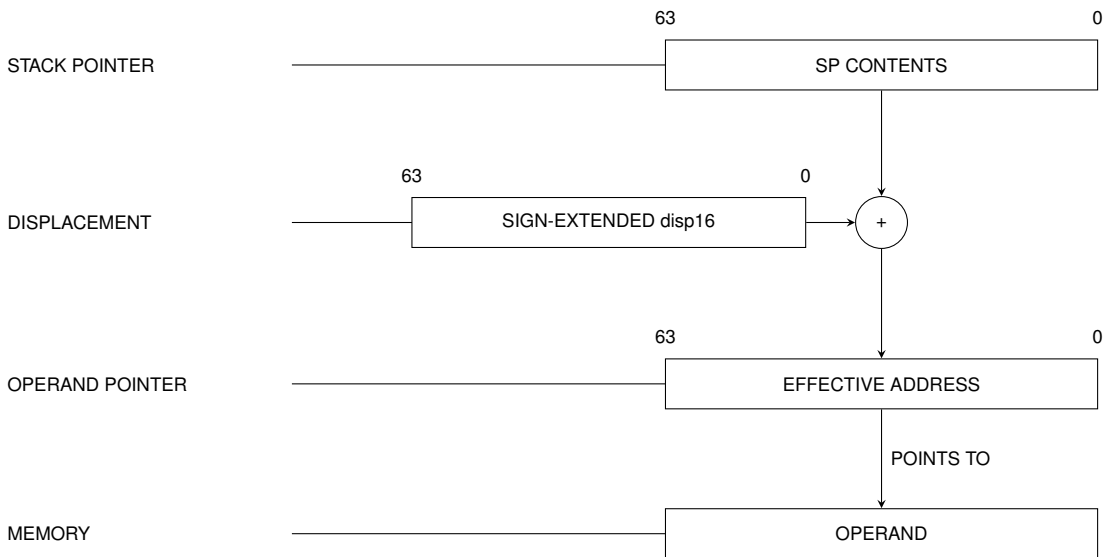
Generation: $EA = \text{contents}(SP) + \text{disp16}$

Assembler Syntax: `[SP + disp16]`

Encoding: 101100

Payload: one signed 16-bit displacement word

Address Update: not eligible



Address calculation for Stack Pointer Relative with disp16

Stack Pointer Relative with disp32

stack-pointer relative memory operand. with a signed 32-bit displacement payload. using fixed SS segment selection.

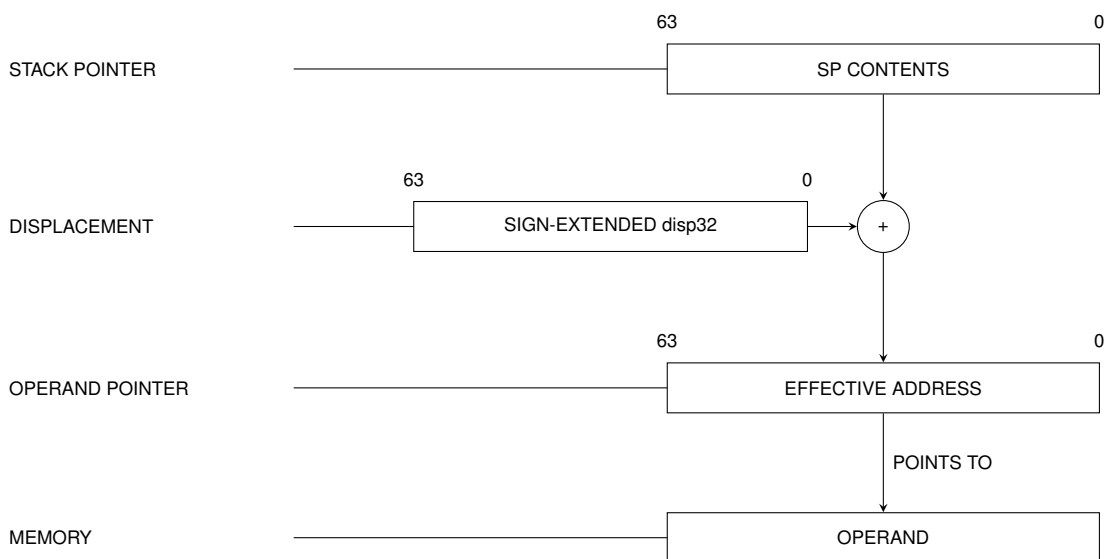
Generation: $EA = \text{contents}(SP) + \text{disp32}$

Assembler Syntax: $[SP + \text{disp32}]$

Encoding: 101101

Payload: two signed 32-bit displacement words

Address Update: not eligible



Address calculation for Stack Pointer Relative with disp32

Stack Pointer Relative with disp64

stack-pointer relative memory operand. with a signed 64-bit displacement payload. using fixed SS segment selection.

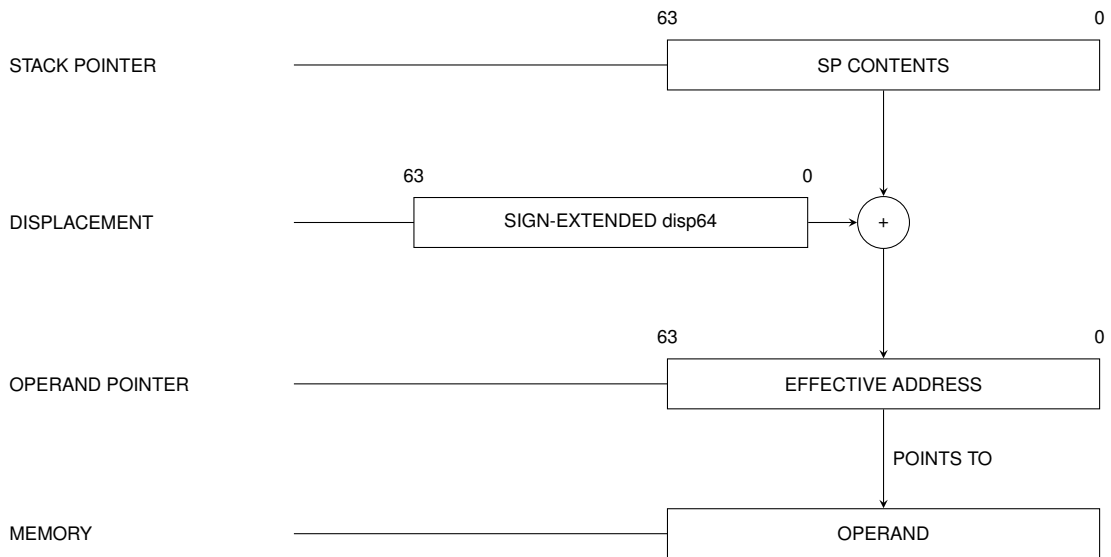
Generation: $EA = \text{contents}(SP) + \text{disp64}$

Assembler Syntax: `[SP + disp64]`

Encoding: 101110

Payload: four signed 64-bit displacement words

Address Update: not eligible



Address calculation for Stack Pointer Relative with disp64

Stack Pointer Direct

Direct stack-pointer operand. The EA field selects SP and does not imply a memory access.

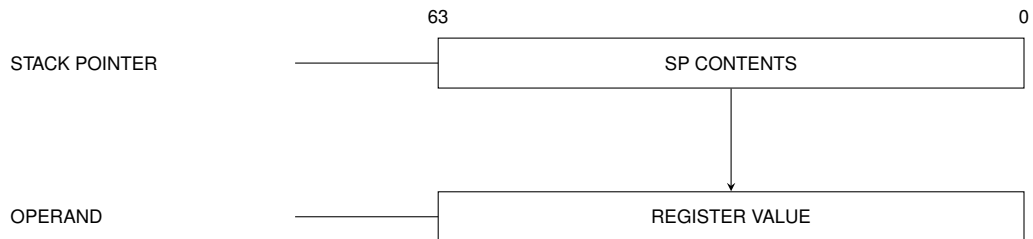
Generation: operand = contents(SP)

Assembler Syntax: SP

Encoding: 101111

Payload: no payload words

Address Update: not eligible



Direct operand selection for Stack Pointer Direct

Absolute Memory 32-bit

absolute memory operand. using a signed 32-bit address payload.

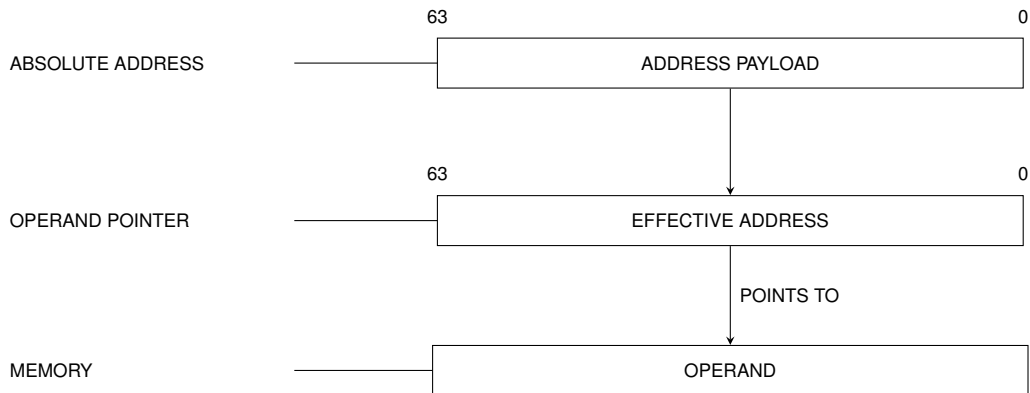
Generation: EA = absolute payload

Assembler Syntax: [imm32]

Encoding: 110000

Payload: a two-word signed 32-bit absolute address

Address Update: not eligible



Address calculation for Absolute Memory 32-bit

Absolute Memory 64-bit

absolute memory operand. using a unsigned 64-bit address payload.

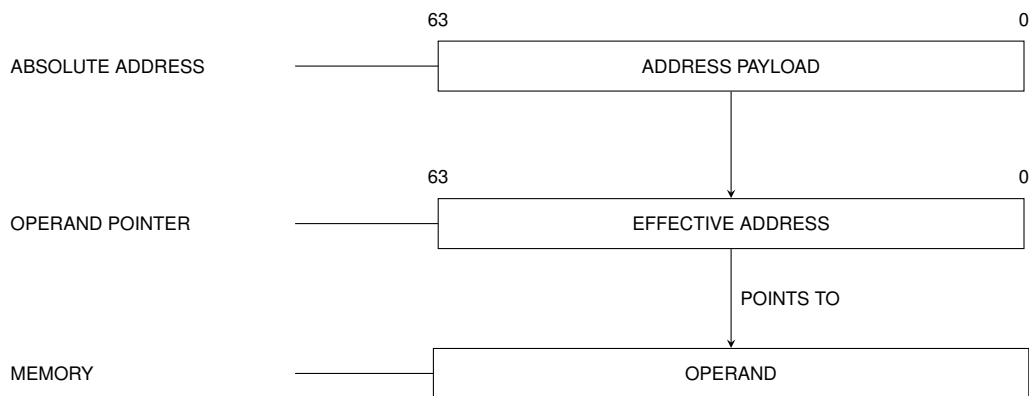
Generation: EA = absolute payload

Assembler Syntax: [imm64]

Encoding: 110001

Payload: a four-word unsigned 64-bit absolute address

Address Update: not eligible



Address calculation for Absolute Memory 64-bit

Immediate Operand 16

Immediate operand supplied by following payload words. The value is sign-extended to the instruction operand size.

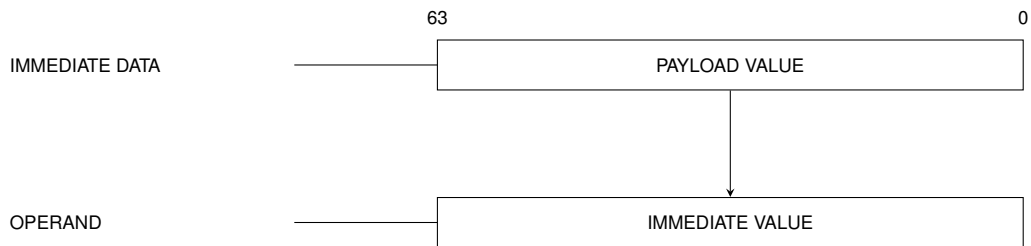
Generation: operand = immediate payload

Assembler Syntax: imm16

Encoding: 110010

Payload: one immediate payload word

Address Update: not eligible



Direct operand selection for Immediate Operand 16

Immediate Operand 32

Immediate operand supplied by following payload words. The value is sign-extended to the instruction operand size.

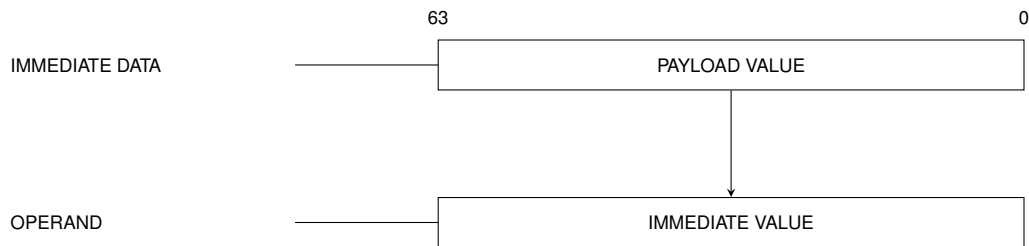
Generation: operand = immediate payload

Assembler Syntax: imm32

Encoding: 110011

Payload: two immediate payload words

Address Update: not eligible



Direct operand selection for Immediate Operand 32

Immediate Operand 64

Immediate operand supplied by following payload words. The value is not sign-extended.

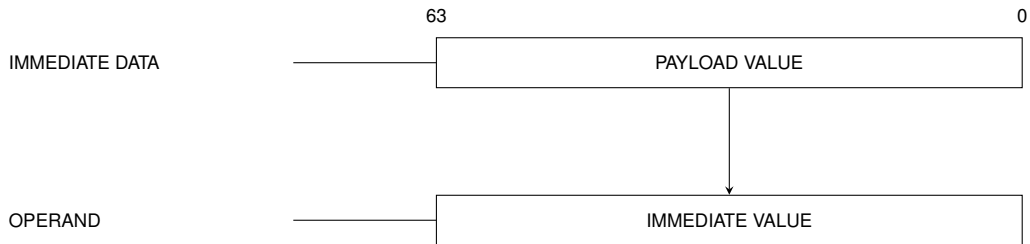
Generation: operand = immediate payload

Assembler Syntax: imm64

Encoding: 110100

Payload: four immediate payload words

Address Update: not eligible



Direct operand selection for Immediate Operand 64

10.2 Extended EA Descriptor

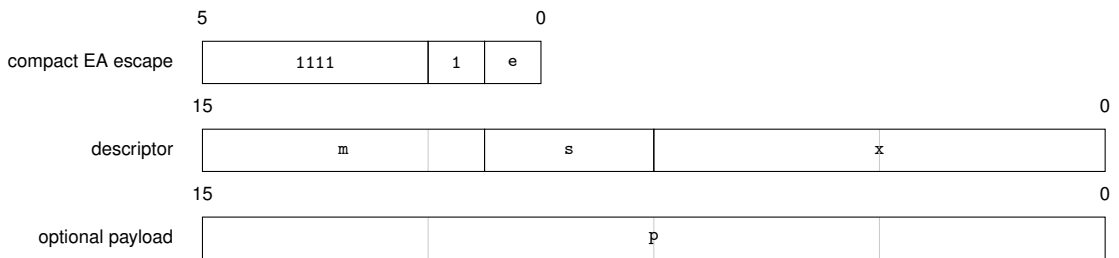


Figure 10-4. Extended EA Word Sequence

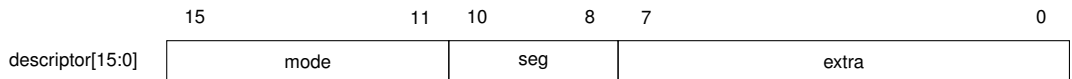


Figure 10-5. Extended Effective-Address Descriptor

The compact XEA form uses the six-bit value 111111; the S32 XEA form uses 111110 and selects the same descriptor layout with signed 32-bit index extension. Each escape is followed by a 16-bit descriptor. The descriptor mode field selects the extended EA form. Segment-selectable forms use the segment field to select a concrete segment register. When assembly syntax omits the segment on a segment-selectable A-indexed form, the assembler encodes DS; the processor does not decode a separate default-segment value. Fixed-segment forms reserve that field

and use their architectural segment. The three-bit segment field uses the SREG selector table. The extra field carries mode-specific register, index, scale, or selector bits. Displacements and absolute addresses remain in following payload words instead of being split across opcode fields. Segment-qualified memory addresses pass through segmentation first when the selected segment is enabled, and then through paging.

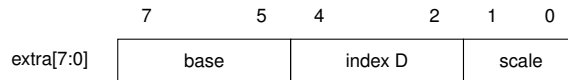


Figure 10-6. Representative Indexed Extended-EA Extra Field

For indexed extended EA forms, the extra field is interpreted as a compact base/index/scale descriptor. The S32 XEA escape uses the same descriptor fields, but sign-extends the low 32 bits of the D-index register before scaling. Assembly syntax for indexed EA forms always writes the scale term explicitly, including the scale-one case; $[A0 + D1 * 1]$ is valid, while $[A0 + D1]$ is not an indexed-EA spelling. Other extended modes use the same descriptor word but assign the low bits to the fields needed by that mode.

10.3 Extended EA Addressing Modes

Indexed

Segment-selectable indexed memory operand. using An, Dn, and scale. omitted segment syntax assembles to the default data segment.

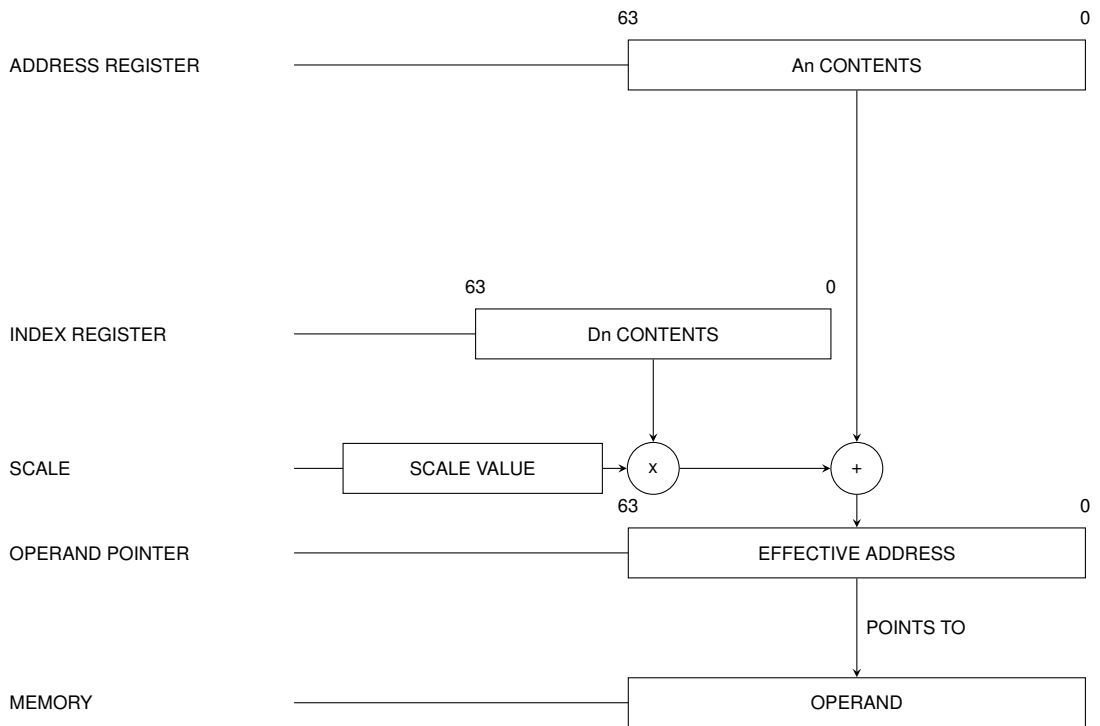
Generation: $EA = \text{contents}(An) + Dn * \text{scale}$

Assembler Syntax: $[A + D * \text{scale}] / [\text{seg}:A + D * \text{scale}]$

Encoding: EXTENDED descriptor mode 0x0

Payload: descriptor word only

Address Update: not eligible



Address calculation for Indexed

Indexed with disp16

Segment-selectable indexed memory operand. using An, Dn, and scale. with a signed 16-bit displacement payload. omitted segment syntax assembles to the default data segment.

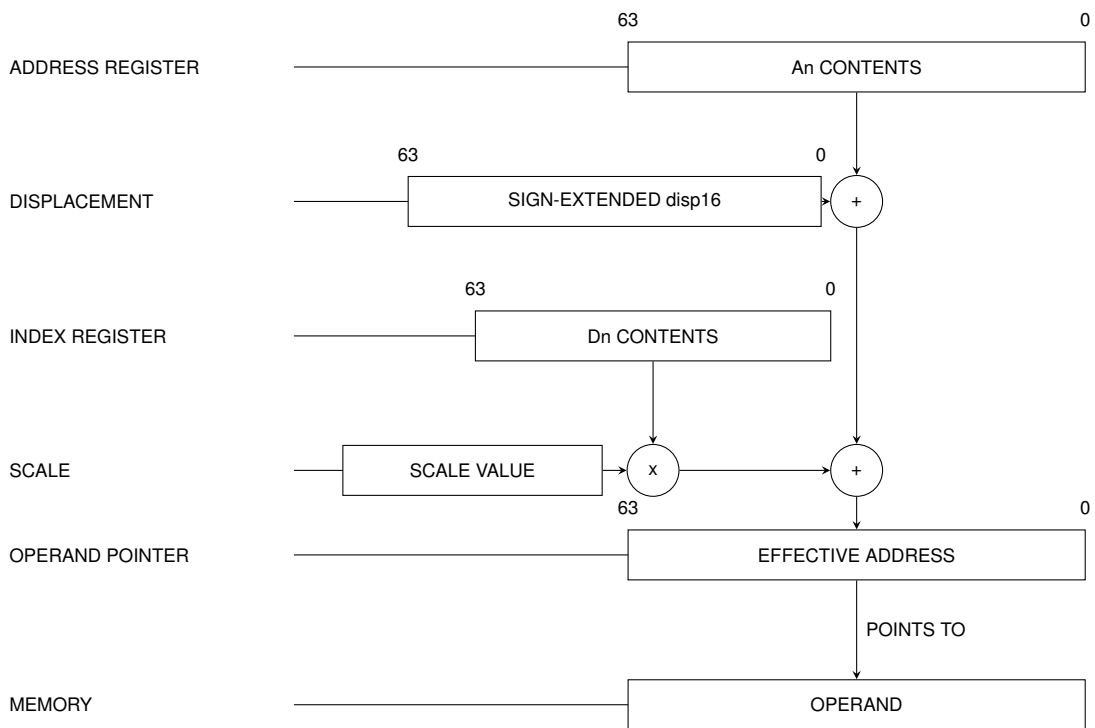
Generation: $EA = \text{contents}(An) + Dn * \text{scale} + \text{disp16}$

Assembler Syntax: $[A + D * \text{scale} + \text{disp16}] / [\text{seg:A} + D * \text{scale} + \text{disp16}]$

Encoding: EXTENDED descriptor mode 0x1

Payload: descriptor word plus one signed 16-bit displacement word

Address Update: not eligible



Address calculation for Indexed with disp16

Indexed with disp32

Segment-selectable indexed memory operand. using An, Dn, and scale. with a signed 32-bit displacement payload. omitted segment syntax assembles to the default data segment.

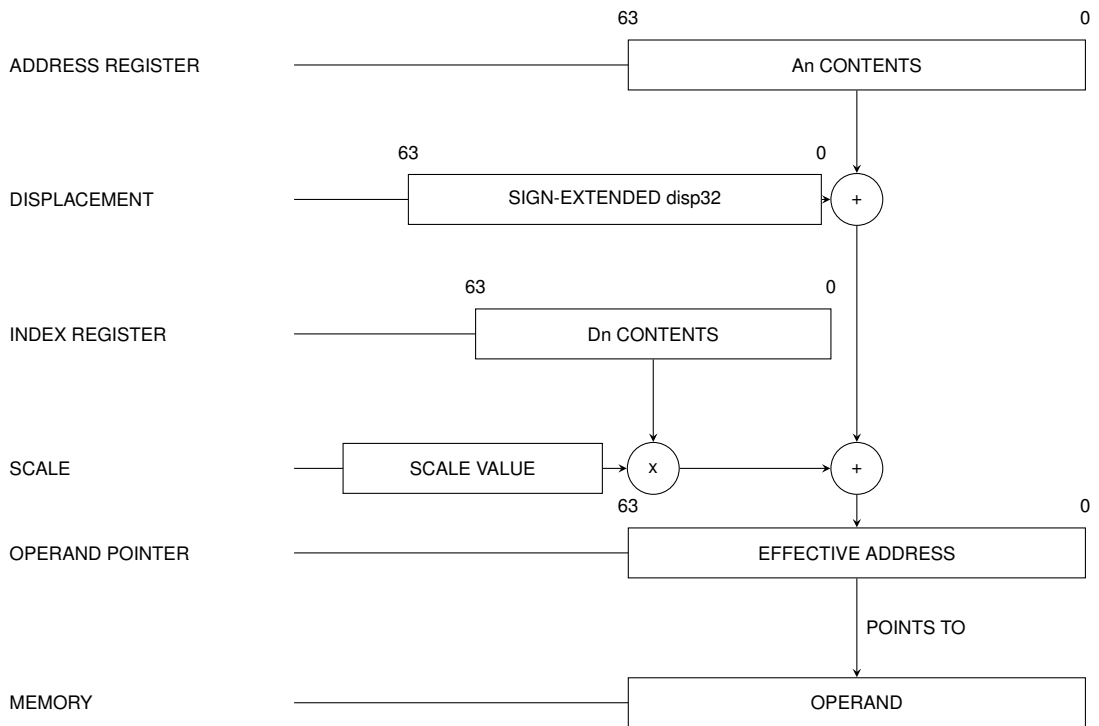
Generation: $EA = \text{contents}(An) + Dn * \text{scale} + \text{disp32}$

Assembler Syntax: `[A + D * scale + disp32] / [seg:A + D * scale + disp32]`

Encoding: EXTENDED descriptor mode 0x2

Payload: descriptor word plus two signed 32-bit displacement words

Address Update: not eligible



Address calculation for Indexed with disp32

Indexed with disp64

Segment-selectable indexed memory operand. using An, Dn, and scale. with a signed 64-bit displacement payload. omitted segment syntax assembles to the default data segment.

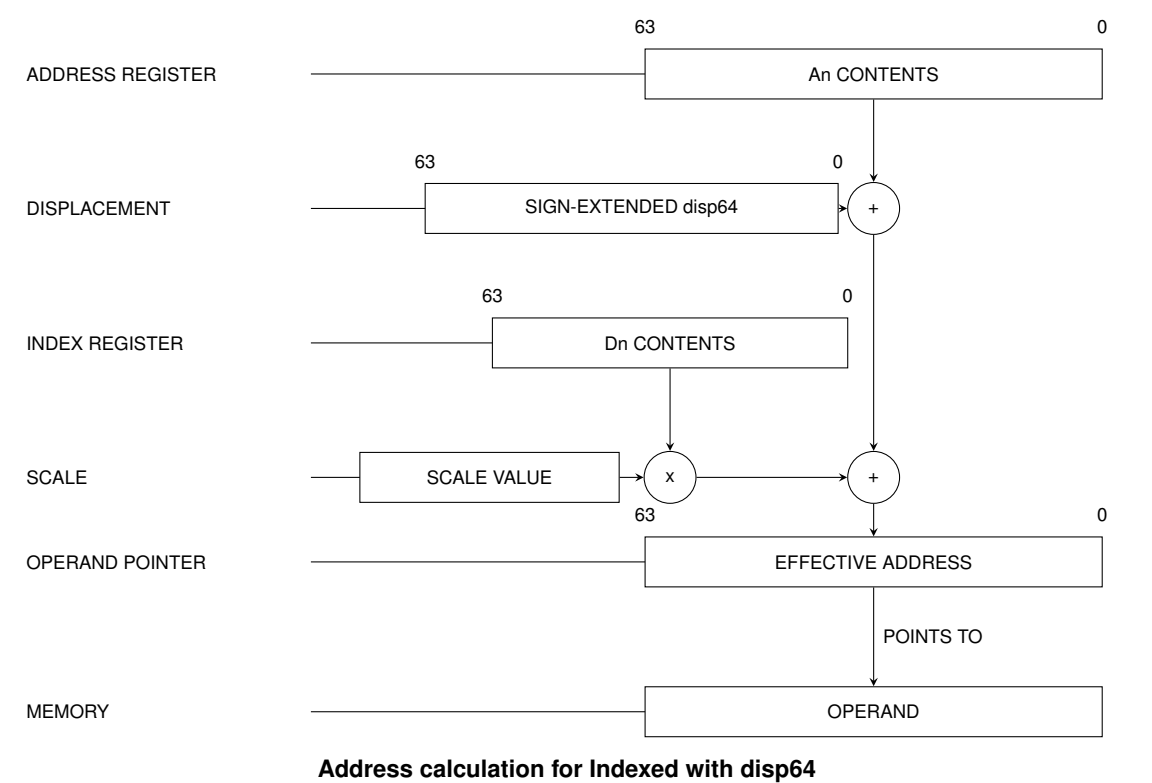
Generation: EA = contents(An) + Dn * scale + disp64

Assembler Syntax: [A + D * scale + disp64] / [seg:A + D * scale + disp64]

Encoding: EXTENDED descriptor mode 0x3

Payload: descriptor word plus four signed 64-bit displacement words

Address Update: not eligible



Segment-Qualified Address Register Indirect

Segment-selectable address-register indirect memory operand. address-update prefixes may use this form.

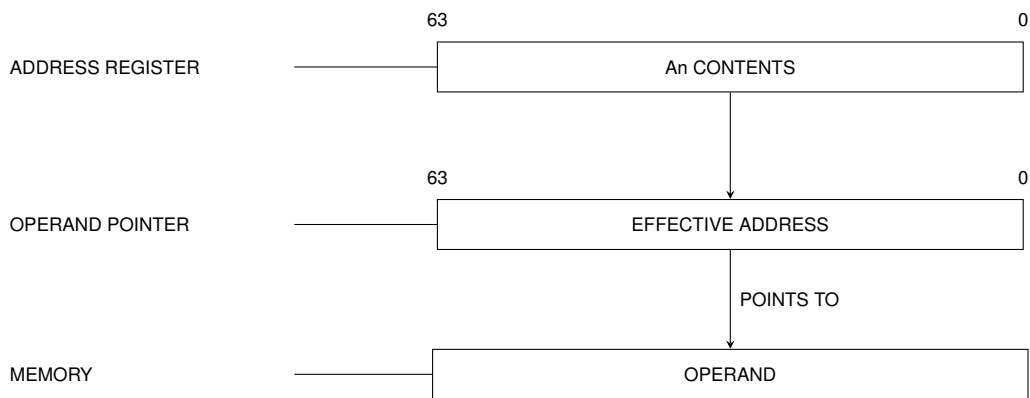
Generation: EA = contents(A_n)

Assembler Syntax: [$seg:A$]

Encoding: EXTENDED descriptor mode 0x4

Payload: descriptor word only

Address Update: eligible



Address calculation for Segment-Qualified Address Register Indirect

Segment-Qualified Address Register Indirect with disp16

Segment-selectable address-register relative memory operand. with a signed 16-bit displacement payload.

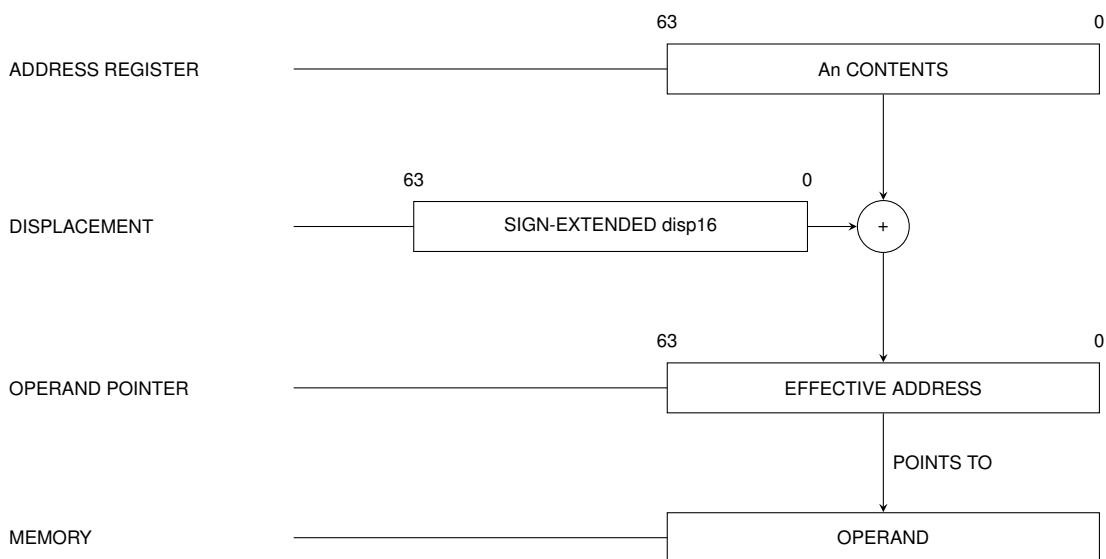
Generation: $EA = \text{contents}(An) + \text{disp16}$

Assembler Syntax: `[seg:A + disp16]`

Encoding: EXTENDED descriptor mode 0x5

Payload: descriptor word plus one signed 16-bit displacement word

Address Update: not eligible



Address calculation for Segment-Qualified Address Register Indirect with disp16

Segment-Qualified Address Register Indirect with disp32

Segment-selectable address-register relative memory operand. with a signed 32-bit displacement payload.

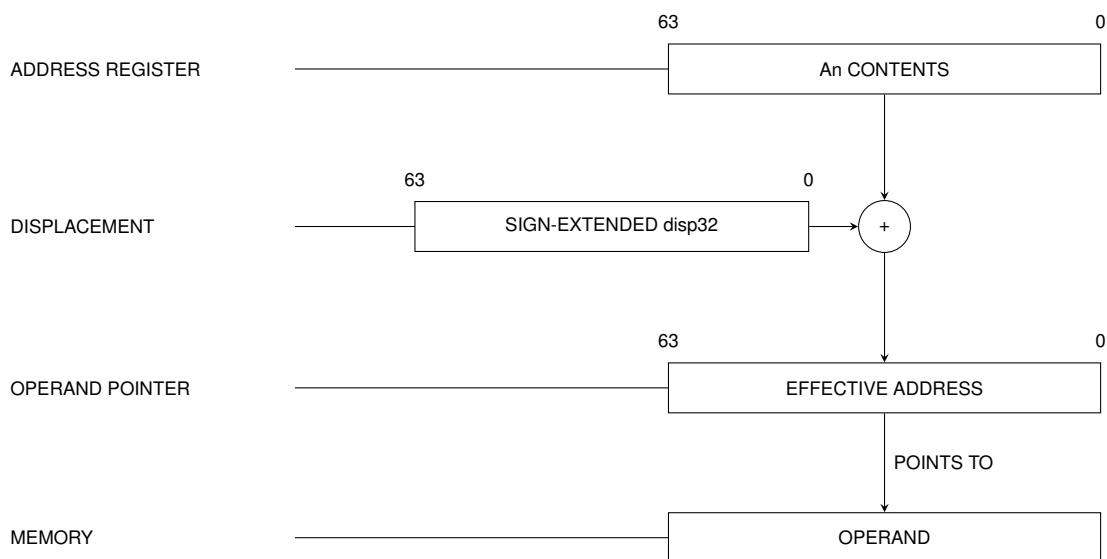
Generation: $EA = \text{contents}(An) + \text{disp32}$

Assembler Syntax: `[seg:A + disp32]`

Encoding: EXTENDED descriptor mode 0x6

Payload: descriptor word plus two signed 32-bit displacement words

Address Update: not eligible



Address calculation for Segment-Qualified Address Register Indirect with disp32

Segment-Qualified Absolute Memory 32-bit

Segment-selectable absolute memory operand. using a signed 32-bit address payload.

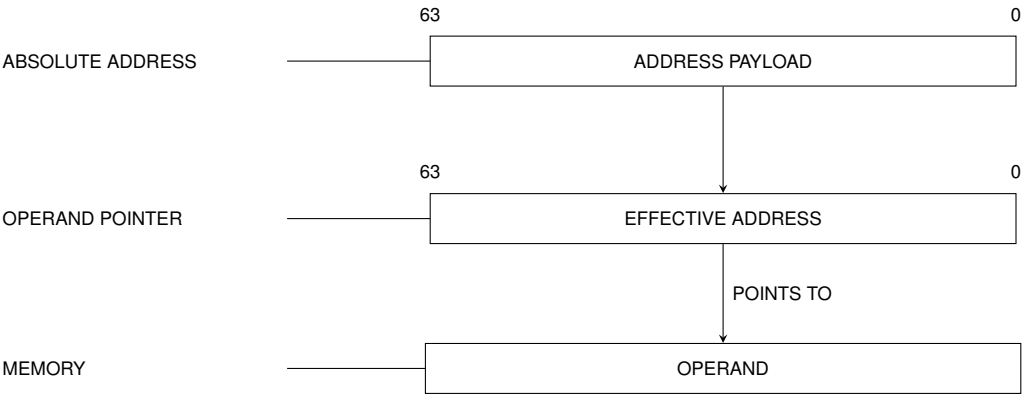
Generation: EA = absolute payload

Assembler Syntax: [seg:imm32]

Encoding: EXTENDED descriptor mode 0x7

Payload: descriptor word plus a two-word signed 32-bit absolute address

Address Update: not eligible



Address calculation for Segment-Qualified Absolute Memory 32-bit

Segment-Qualified Absolute Memory 64-bit

Segment-selectable absolute memory operand. using a unsigned 64-bit address payload.

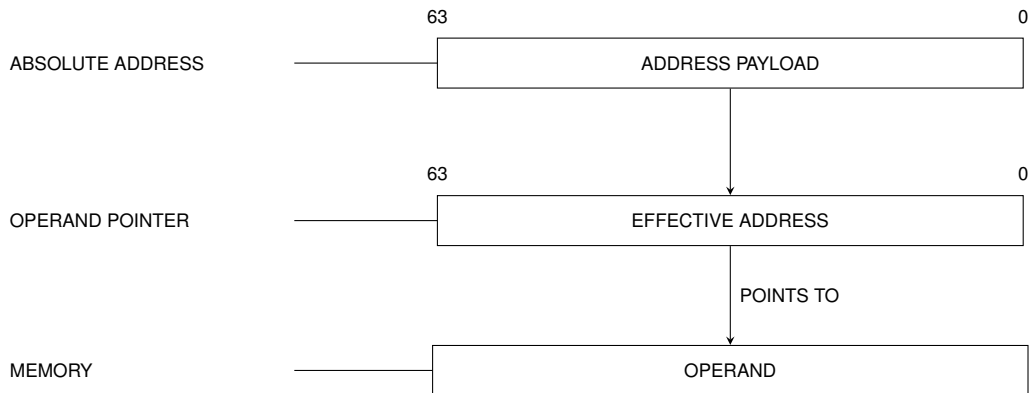
Generation: EA = absolute payload

Assembler Syntax: [seg:imm64]

Encoding: EXTENDED descriptor mode 0x8

Payload: descriptor word plus a four-word unsigned 64-bit absolute address

Address Update: not eligible



Address calculation for Segment-Qualified Absolute Memory 64-bit

Indexed with disp16

indexed memory operand. using SP, Dn, and scale. with a signed 16-bit displacement payload. using fixed SS segment selection. with the descriptor segment field reserved.

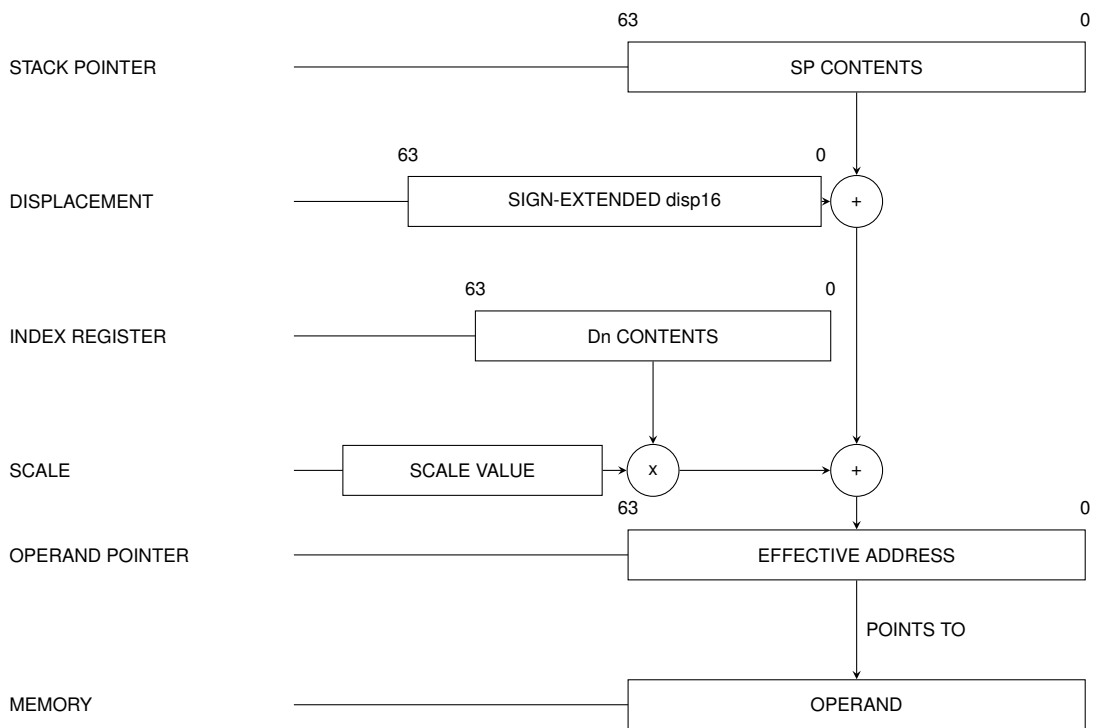
Generation: $EA = \text{contents}(SP) + Dn * \text{scale} + \text{disp16}$

Assembler Syntax: `[SP + D * scale + disp16]`

Encoding: EXTENDED descriptor mode 0x9

Payload: descriptor word plus one signed 16-bit displacement word

Address Update: not eligible



Address calculation for Indexed with disp16

Indexed with disp32

indexed memory operand. using SP, Dn, and scale. with a signed 32-bit displacement payload. using fixed SS segment selection. with the descriptor segment field reserved.

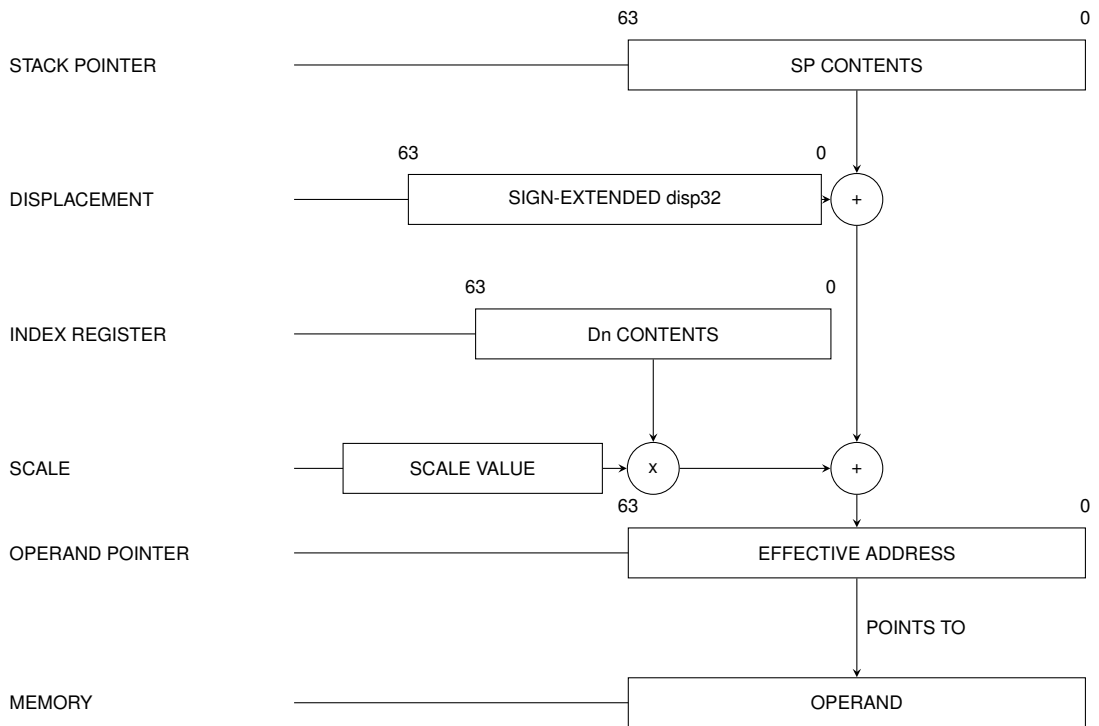
Generation: $EA = \text{contents}(SP) + D_n * \text{scale} + \text{disp32}$

Assembler Syntax: `[SP + D * scale + disp32]`

Encoding: EXTENDED descriptor mode 0xa

Payload: descriptor word plus two signed 32-bit displacement words

Address Update: not eligible



Address calculation for Indexed with disp32

Indexed with disp64

indexed memory operand. using SP, Dn, and scale. with a signed 64-bit displacement payload. using fixed SS segment selection. with the descriptor segment field reserved.

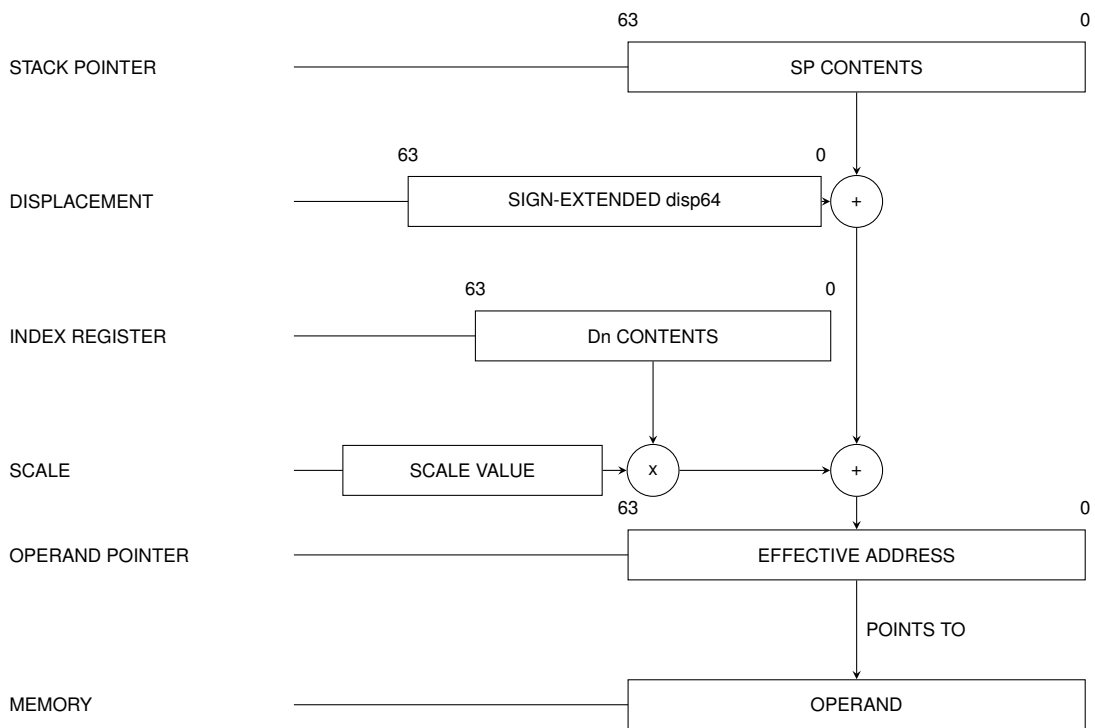
Generation: $EA = \text{contents}(SP) + Dn * \text{scale} + \text{disp64}$

Assembler Syntax: `[SP + D * scale + disp64]`

Encoding: EXTENDED descriptor mode 0xb

Payload: descriptor word plus four signed 64-bit displacement words

Address Update: not eligible



Address calculation for Indexed with disp64

Indexed with disp16

indexed memory operand. using PC, Dn, and scale. with a signed 16-bit displacement payload. using fixed CS segment selection. with the descriptor segment field reserved.

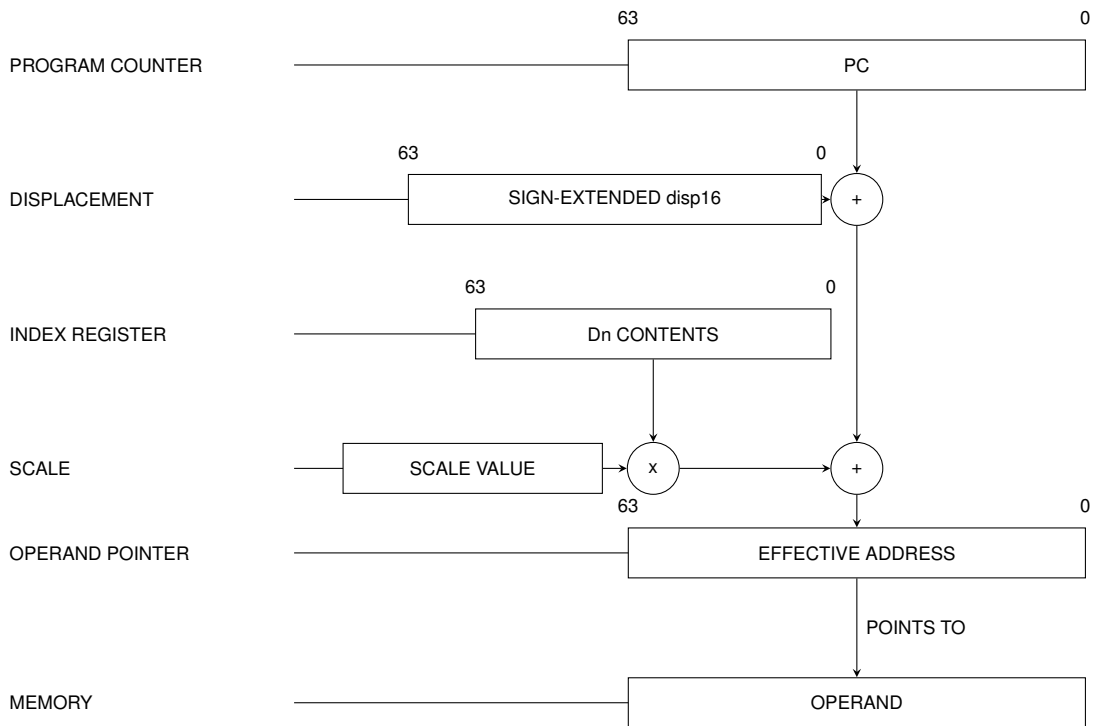
Generation: $EA = PC + D_n * scale + disp16$

Assembler Syntax: `[PC + D * scale + disp16]`

Encoding: EXTENDED descriptor mode 0xc

Payload: descriptor word plus one signed 16-bit displacement word

Address Update: not eligible



Address calculation for Indexed with disp16

Indexed with disp32

indexed memory operand. using PC, Dn, and scale. with a signed 32-bit displacement payload.
using fixed CS segment selection. with the descriptor segment field reserved.

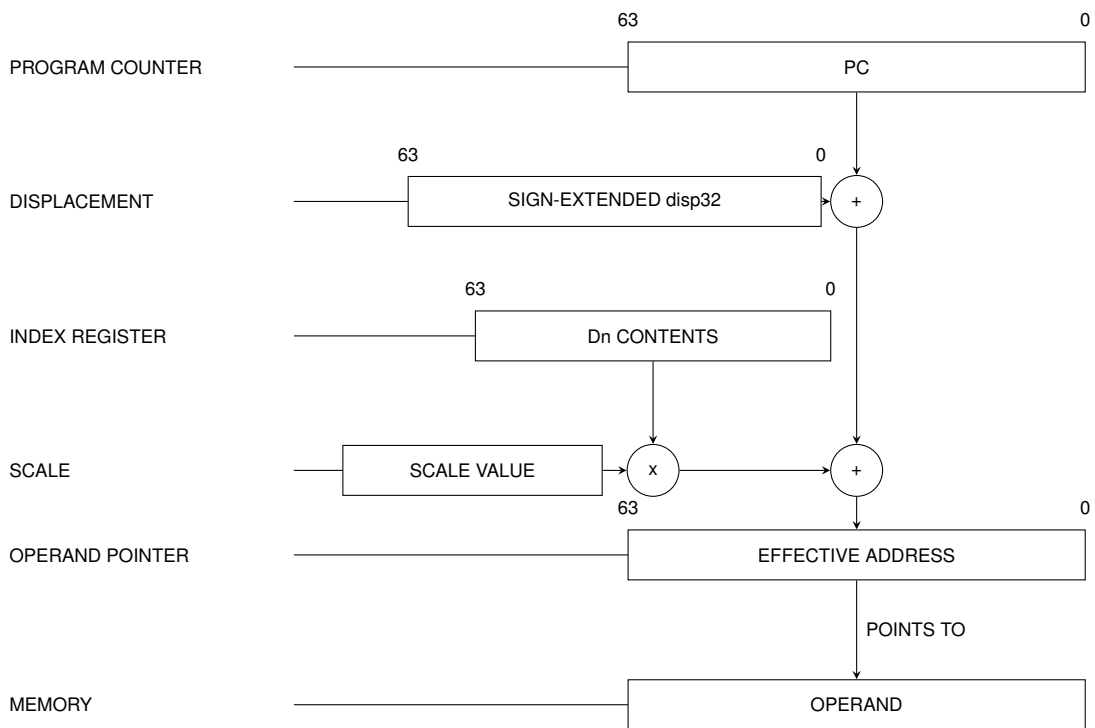
Generation: $EA = PC + D_n * scale + disp32$

Assembler Syntax: `[PC + D * scale + disp32]`

Encoding: EXTENDED descriptor mode 0xd

Payload: descriptor word plus two signed 32-bit displacement words

Address Update: not eligible



Address calculation for Indexed with disp32

Indexed with disp64

indexed memory operand. using PC, Dn, and scale. with a signed 64-bit displacement payload. using fixed CS segment selection. with the descriptor segment field reserved.

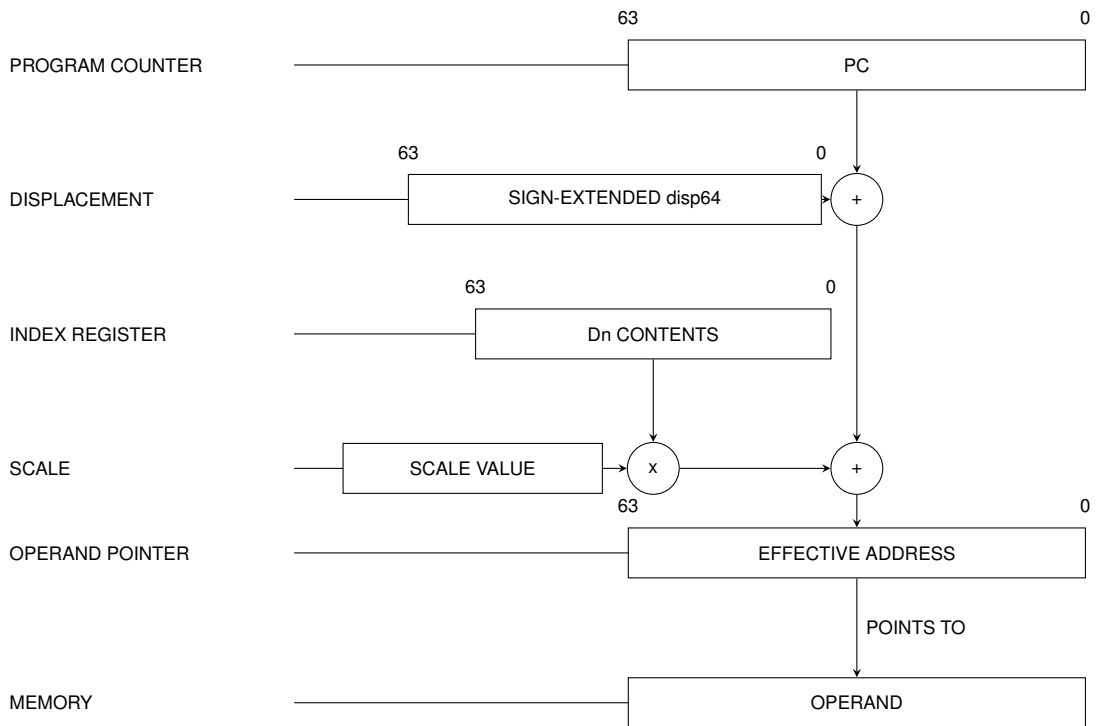
Generation: $EA = PC + D_n * scale + disp64$

Assembler Syntax: `[PC + D * scale + disp64]`

Encoding: EXTENDED descriptor mode 0xe

Payload: descriptor word plus four signed 64-bit displacement words

Address Update: not eligible



Address calculation for Indexed with disp64

Signed 32-bit Indexed

Segment-selectable signed-32 indexed memory operand. using An, Dn[31:0], and scale. omitted segment syntax assembles to the default data segment.

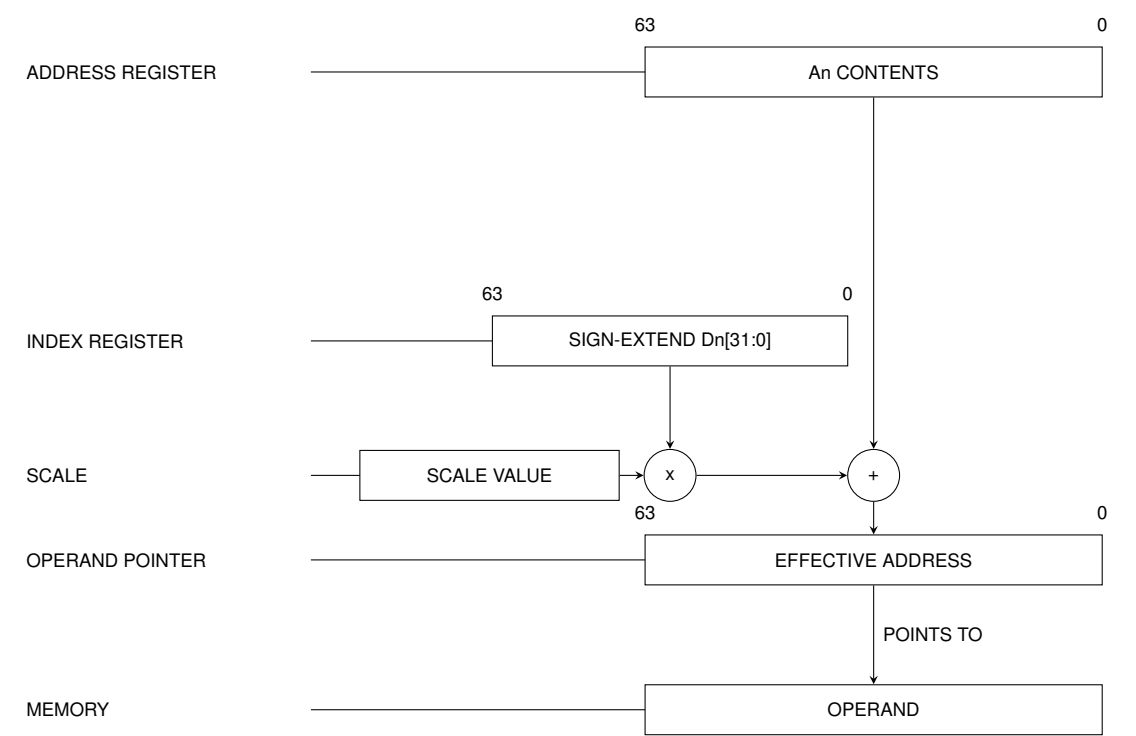
Generation: EA = contents(An) + sign_extend(Dn[31:0]) * scale

Assembler Syntax: [A + D.L * scale] / [seg:A + D.L * scale]

Encoding: S32_INDEXED_EXTENDED descriptor mode 0x0

Payload: descriptor word only

Address Update: not eligible



Address calculation for Signed 32-bit Indexed

Signed 32-bit Indexed with disp16

Segment-selectable signed-32 indexed memory operand. using An, Dn[31:0], and scale. with a signed 16-bit displacement payload. omitted segment syntax assembles to the default data segment.

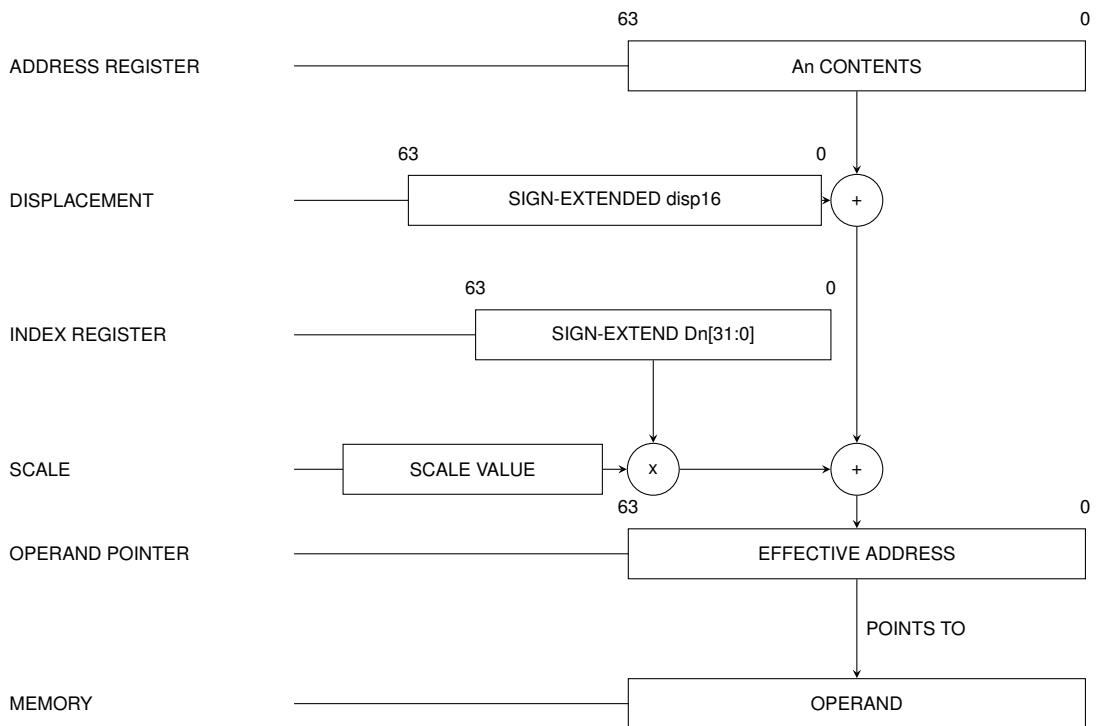
Generation: $EA = \text{contents}(An) + \text{sign_extend}(Dn[31:0]) * \text{scale} + \text{disp16}$

Assembler Syntax: `[A + D.L * scale + disp16] / [seg:A + D.L * scale + disp16]`

Encoding: S32_INDEXED_EXTENDED descriptor mode 0x1

Payload: descriptor word plus one signed 16-bit displacement word

Address Update: not eligible



Address calculation for Signed 32-bit Indexed with disp16

Signed 32-bit Indexed with disp32

Segment-selectable signed-32 indexed memory operand. using An, Dn[31:0], and scale. with a signed 32-bit displacement payload. omitted segment syntax assembles to the default data segment.

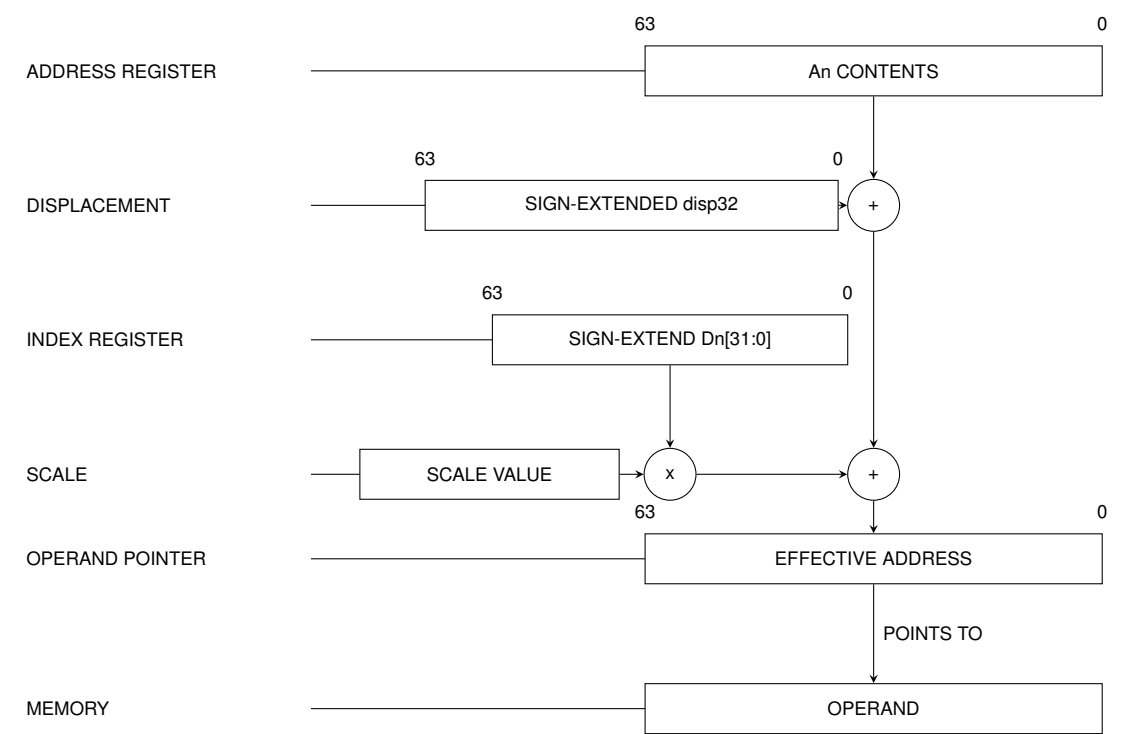
Generation: EA = contents(An) + sign_extend(Dn[31:0]) * scale + disp32

Assembler Syntax: [A + D.L * scale + disp32] / [seg:A + D.L * scale + disp32]

Encoding: S32_INDEXED_EXTENDED descriptor mode 0x2

Payload: descriptor word plus two signed 32-bit displacement words

Address Update: not eligible



Address calculation for Signed 32-bit Indexed with disp32

Signed 32-bit Indexed with disp64

Segment-selectable signed-32 indexed memory operand. using An, Dn[31:0], and scale. with a signed 64-bit displacement payload. omitted segment syntax assembles to the default data segment.

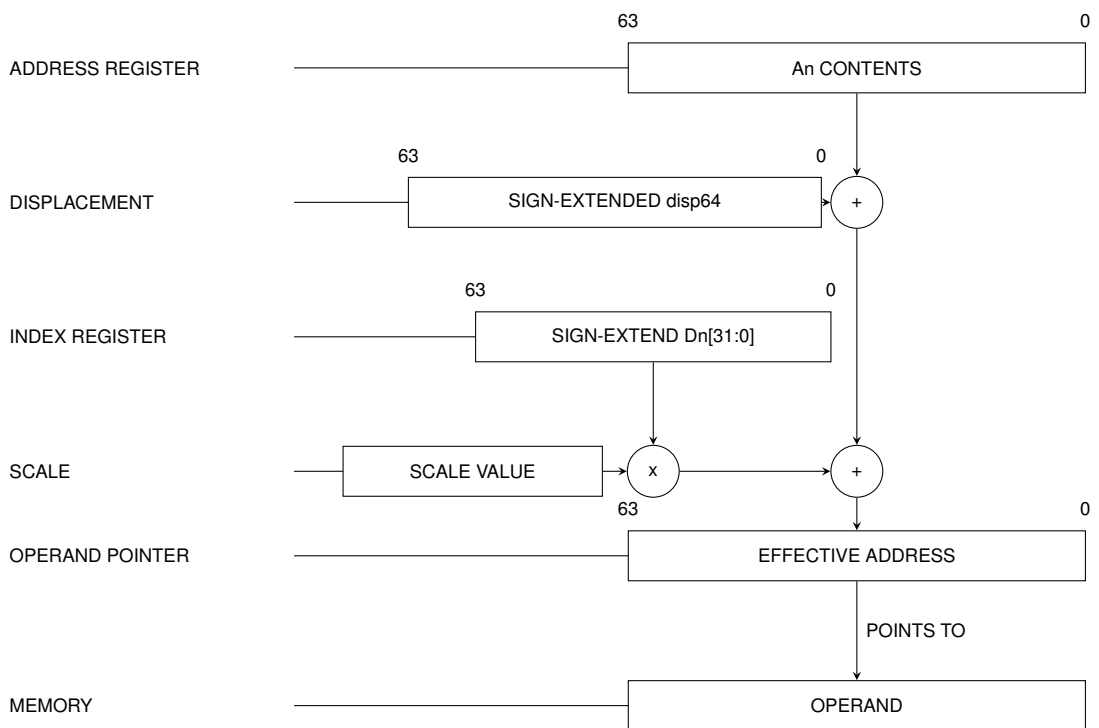
Generation: $EA = \text{contents}(An) + \text{sign_extend}(Dn[31:0]) * \text{scale} + \text{disp64}$

Assembler Syntax: `[A + D.L * scale + disp64] / [seg:A + D.L * scale + disp64]`

Encoding: S32_INDEXED_EXTENDED descriptor mode 0x3

Payload: descriptor word plus four signed 64-bit displacement words

Address Update: not eligible



Address calculation for Signed 32-bit Indexed with disp64

Signed 32-bit Indexed with disp16

signed-32 indexed memory operand. using SP, Dn[31:0], and scale. with a signed 16-bit displacement payload. using fixed SS segment selection. with the descriptor segment field reserved.

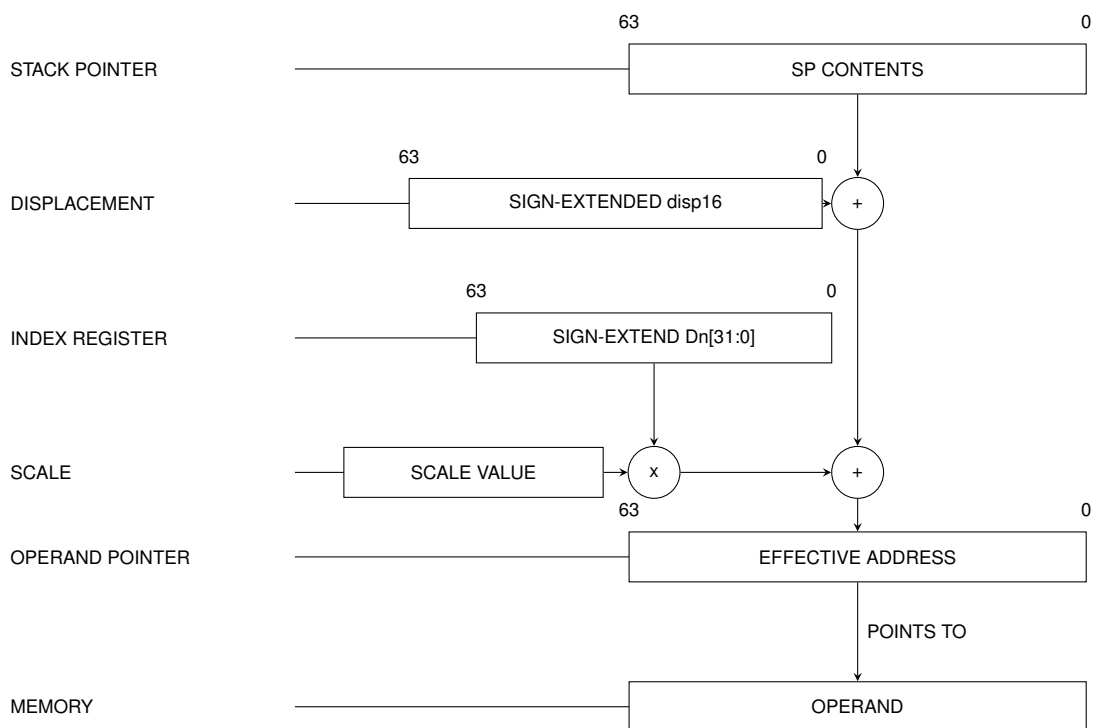
Generation: $EA = \text{contents}(SP) + \text{sign_extend}(Dn[31:0]) * \text{scale} + \text{disp16}$

Assembler Syntax: `[SP + D.L * scale + disp16]`

Encoding: S32_INDEXED_EXTENDED descriptor mode 0x9

Payload: descriptor word plus one signed 16-bit displacement word

Address Update: not eligible



Address calculation for Signed 32-bit Indexed with disp16

Signed 32-bit Indexed with disp32

signed-32 indexed memory operand. using SP, Dn[31:0], and scale. with a signed 32-bit displacement payload. using fixed SS segment selection. with the descriptor segment field reserved.

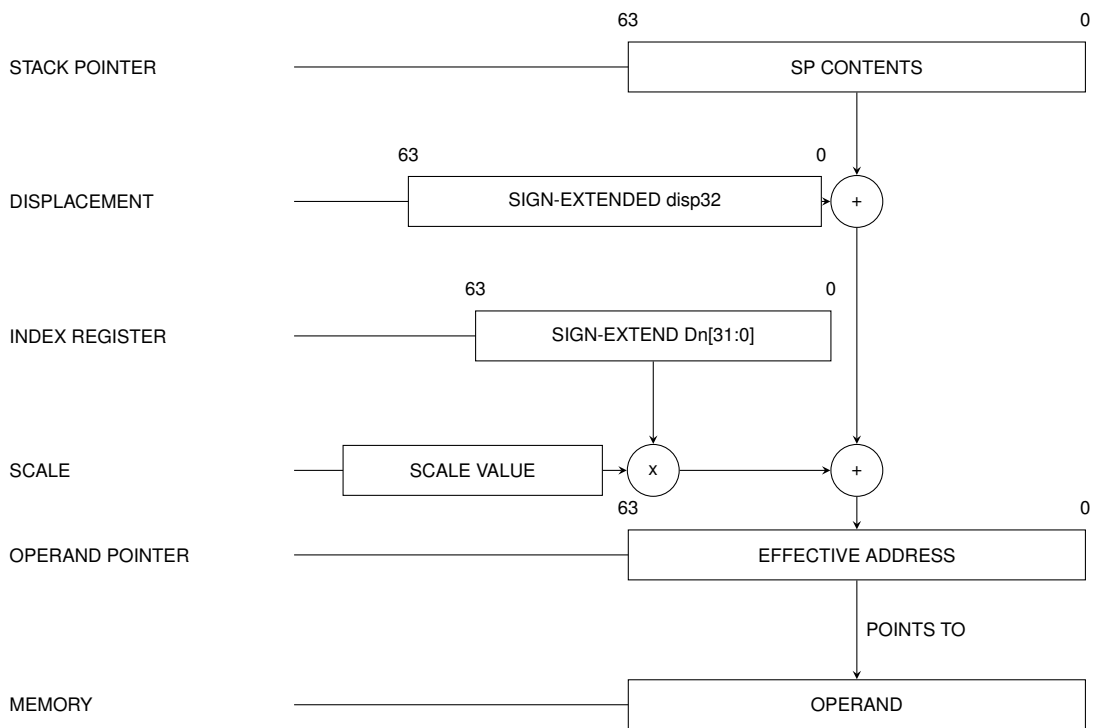
Generation: $EA = \text{contents}(SP) + \text{sign_extend}(Dn[31:0]) * \text{scale} + \text{disp32}$

Assembler Syntax: `[SP + D.L * scale + disp32]`

Encoding: S32_INDEXED_EXTENDED descriptor mode 0xa

Payload: descriptor word plus two signed 32-bit displacement words

Address Update: not eligible



Address calculation for Signed 32-bit Indexed with disp32

Signed 32-bit Indexed with disp64

signed-32 indexed memory operand. using SP, Dn[31:0], and scale. with a signed 64-bit displacement payload. using fixed SS segment selection. with the descriptor segment field reserved.

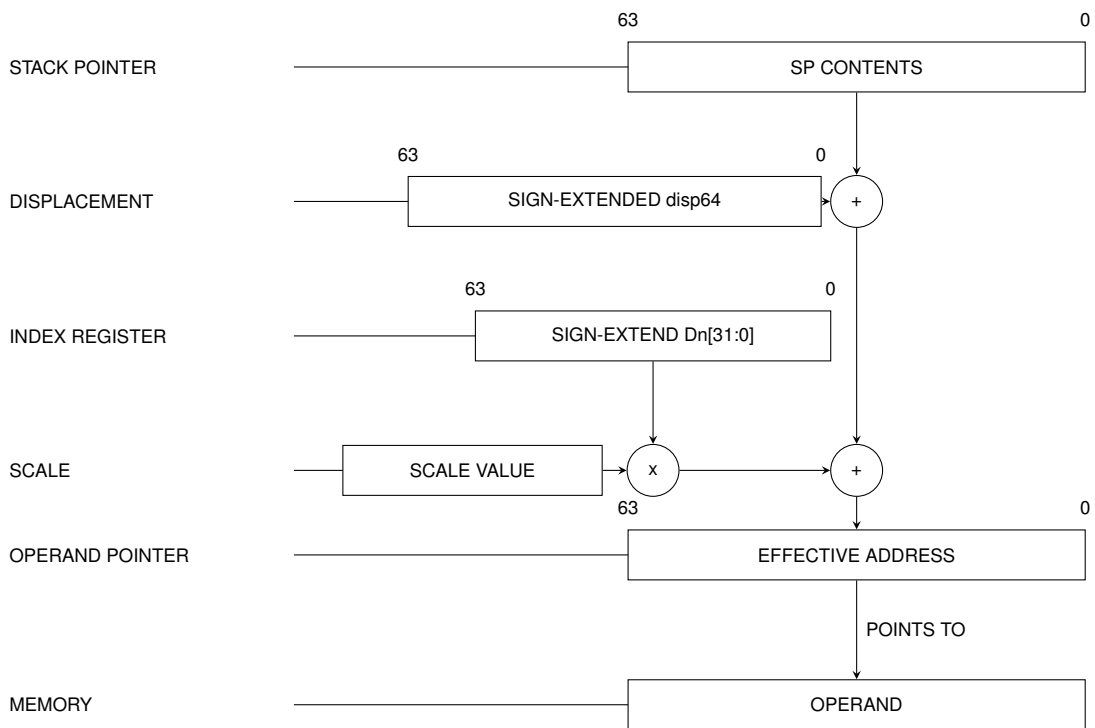
Generation: $EA = \text{contents}(SP) + \text{sign_extend}(Dn[31:0]) * \text{scale} + \text{disp64}$

Assembler Syntax: $[SP + D.L * \text{scale} + \text{disp64}]$

Encoding: S32_INDEXED_EXTENDED descriptor mode 0xb

Payload: descriptor word plus four signed 64-bit displacement words

Address Update: not eligible



Address calculation for Signed 32-bit Indexed with disp64

Signed 32-bit Indexed with disp16

signed-32 indexed memory operand. using PC, Dn[31:0], and scale. with a signed 16-bit displacement payload. using fixed CS segment selection. with the descriptor segment field reserved.

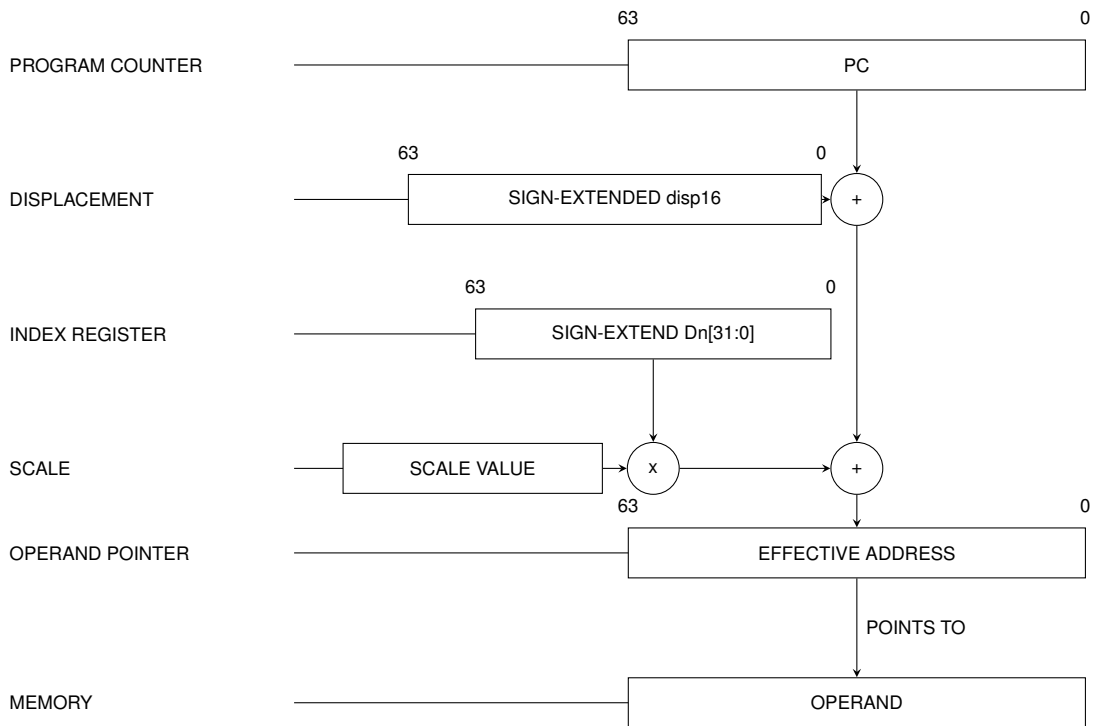
Generation: $EA = PC + \text{sign_extend}(Dn[31:0]) * \text{scale} + \text{disp16}$

Assembler Syntax: $[PC + D.L * \text{scale} + \text{disp16}]$

Encoding: S32_INDEXED_EXTENDED descriptor mode 0xc

Payload: descriptor word plus one signed 16-bit displacement word

Address Update: not eligible



Address calculation for Signed 32-bit Indexed with disp16

Signed 32-bit Indexed with disp32

signed-32 indexed memory operand. using PC, Dn[31:0], and scale. with a signed 32-bit displacement payload. using fixed CS segment selection. with the descriptor segment field reserved.

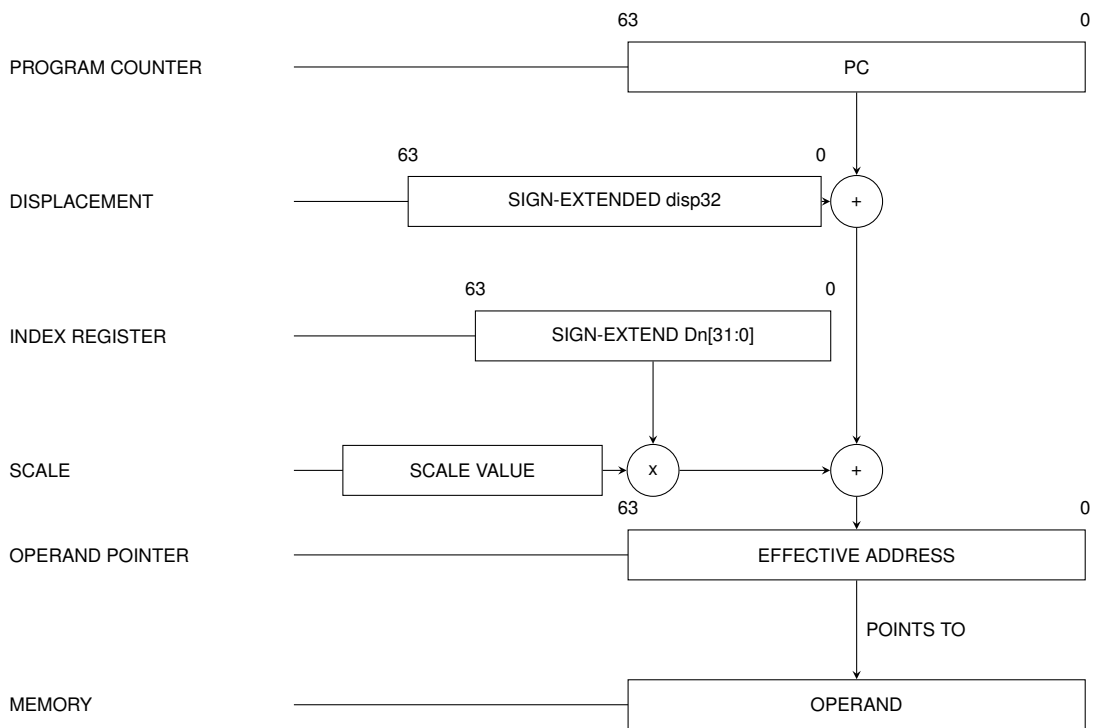
Generation: $EA = PC + \text{sign_extend}(Dn[31:0]) * \text{scale} + \text{disp32}$

Assembler Syntax: $[PC + D.L * \text{scale} + \text{disp32}]$

Encoding: S32_INDEXED_EXTENDED descriptor mode 0xd

Payload: descriptor word plus two signed 32-bit displacement words

Address Update: not eligible



Address calculation for Signed 32-bit Indexed with disp32

Signed 32-bit Indexed with disp64

signed-32 indexed memory operand. using PC, Dn[31:0], and scale. with a signed 64-bit displacement payload. using fixed CS segment selection. with the descriptor segment field reserved.

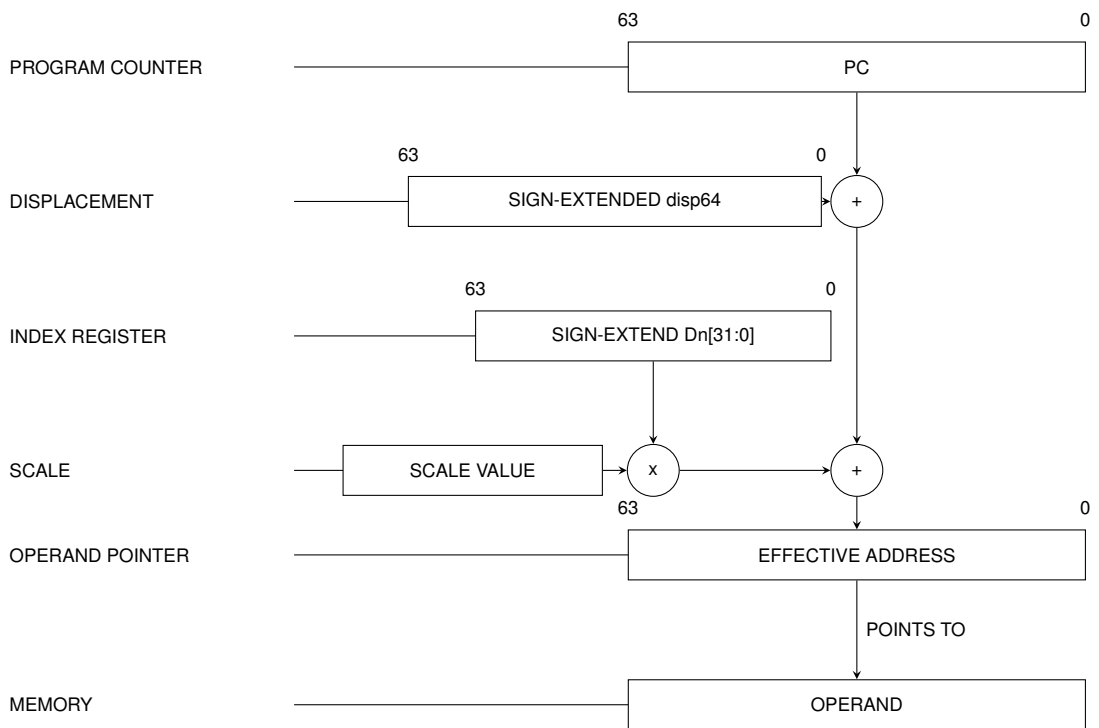
Generation: $EA = PC + \text{sign_extend}(Dn[31:0]) * \text{scale} + \text{disp64}$

Assembler Syntax: $[PC + D.L * \text{scale} + \text{disp64}]$

Encoding: S32_INDEXED_EXTENDED descriptor mode 0xe

Payload: descriptor word plus four signed 64-bit displacement words

Address Update: not eligible



Address calculation for Signed 32-bit Indexed with disp64

11 MEMORY ADDRESS TRANSLATION

Memory address translation is a separate pipeline after effective-address calculation. Effective-address evaluation produces an EA value or address expression; segmentation optionally turns that value into a linear address, and paging optionally turns the linear address into a memory-system address.

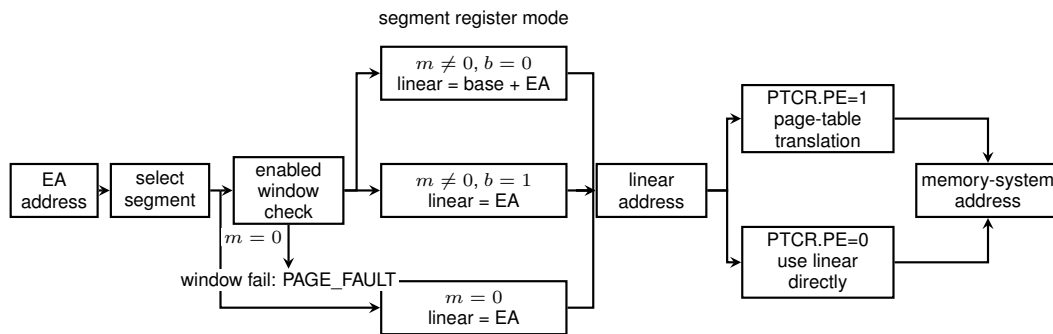


Figure 11-1. Address Translation Pipeline

11.1 Translation Pipeline

Stage	Result
Effective address	address generated by the selected EA form
Segment pre-translation	disabled segment passes the address through; enabled segment checks a byte-addressed window with page-granular base and span segment output, or the EA directly when segmentation is disabled
Linear address	walk page tables from PTCR.root_page and apply PTE attributes
Paging, PTCR.PE = 1	use the linear address directly
Paging, PTCR.PE = 0	

Table 11-1. Table 3-1. Address Translation Stages

11.2 Combined Segment and Page Translation

Segment pre-translation and page-table translation are composed in a fixed order. The effective-address unit first computes an address value from the selected addressing mode. A segment register is then selected by the access class or by an explicit segment-qualified effective-address form. The selected segment produces the linear address that is either used directly or passed to the paging stage. Architectural pointer values used by ordinary code are the address values before this segment stage. If software needs the linear address produced by segment pre-translation, it uses `SEGLEA`; the resulting value is not the normal representation of a portable pointer under the ABI.

EA → segment pre-translation → linear address

PTCR.PE = 1 : linear address → page walk → memory-system address

PTCR.PE = 0 : linear address → memory-system address

Instruction fetches use the code-segment context, stack accesses use the stack-segment context, and ordinary data accesses use the data-segment context unless the effective address explicitly selects another segment. The segment stage does not replace paging as the memory-protection mechanism. Disabled segments perform no window check and no base addition. Translated-window segments check an offset and add the segment base. Bounds-only segments check an already-linear address without adding the base. In both enabled modes, the checked byte-addressed window has a page-granular base and span.

Let x be the address produced by effective-address calculation, and let the selected segment register define:

$$base = base_page \times 4096, \quad span = m \times 2^e \times 4096, \quad limit = base + span.$$

The base, span, and limit are computed in an unsigned domain wide enough to represent the full 64-bit linear-address space plus one. If $limit > 2^{64}$, the segment register image is invalid and any access through it reports `PAGE_FAULT`.

Mode	Segment State	Check	Linear Address
disabled	$m=0$	no segment-window check	$linear = x$
translated window	$m!=0, b=0$	$0 \leq x < span$	$linear = base + x$
bounds-only window	$m!=0, b=1$	$base \leq x < limit$	$linear = x$

Table 11-2. Segment Pre-Translation Modes

The translated-window mode is useful when an address is interpreted as an offset inside the selected segment. The bounds-only mode is useful when the effective address is already a linear address but should still be constrained to the segment window. In both enabled segment modes, a failed segment-window check is reported through the page-fault exception class.

Segment Result	Paging State	Final Translation Behavior
$linear = x$ from disabled segment	<code>PTCR.PE=0</code>	The effective address is used directly as the memory-system address. PTE presence, permission, cache, and addressing-type attributes are not applied.
$linear = x$ from disabled segment	<code>PTCR.PE=1</code>	The effective address is interpreted as a linear address. Canonical-address checking, page walk, PTE permissions, PTE cache policy, and PTE addressing type are applied.
$linear = base + x$ from translated segment	<code>PTCR.PE=0</code>	The segment performs the window check and base addition. The resulting linear address is used directly by the memory system.
$linear = base + x$ from translated segment	<code>PTCR.PE=1</code>	The segment first checks and translates the offset. The resulting linear address is then canonical-checked and translated through the page tables.
$linear = x$ from bounds-only segment	either state	The segment constrains the already-linear address to the selected window. If paging is enabled, the checked linear address is then translated by the page tables; otherwise it is used directly.

Table 11-3. Segment and Paging Composition

When paging is enabled, PTE permissions are evaluated after segment pre-translation. For example, an access may pass the segment-window check and still fault because the leaf PTE is not

present, is not writable for a store, is not executable for an instruction fetch, or is not accessible at the current privilege level. Conversely, paging cannot rescue an address that fails the enabled segment-window check, because the page walk is not attempted for that access.

Privilege checks that do not depend on address translation are performed before page-fault reporting. Address failures from the active translation stages, including enabled segment-window checks, canonical-address checks when paging is enabled, page-table walks, PTE permission checks, and stack-memory translation, are reported through the `PAGE_FAULT` vector.

11.3 Segmentation Stage

Segmentation is a page-granular address-window and pre-translation mechanism, not the primary memory-protection mechanism. A segment with $m = 0$ is disabled and passes the effective-address result through unchanged. A translated segment checks an offset against the segment span and then adds the segment base. A bounds-only segment checks an already-linear address against the segment window without adding the base.

$$\begin{aligned}
 base &= base_page \times 4096 \\
 span &= m \times 2^e \times 4096 \\
 limit &= base + span \\
 m = 0 : \quad linear &= x \\
 m \neq 0, b = 0 : \quad 0 \leq x < span, \quad linear &= base + x \\
 m \neq 0, b = 1 : \quad base \leq x < limit, \quad linear &= x
 \end{aligned}$$

The base, span, and limit are computed in an unsigned domain wide enough to represent the full 64-bit linear-address space plus one. If $limit > 2^{64}$, the segment register image is invalid and any access through it reports `PAGE_FAULT`.

11.4 Paging Stage

Paging performs final address translation and supplies the protection attributes. LA57 selects either four-level paging with 48-bit canonical linear addresses or five-level paging with 57-bit canonical linear addresses. `PTE[0..11]` are architecturally assigned low attribute bits, `PTE[12..PABITS-1]` contains the physical frame number or next-table frame number, and `PTE[PABITS..63]` is software-defined. Bit 10 is reserved for software use.

11.5 LA48 Four-Level Paging

When PTCR.LA57 is zero, paging uses a 48-bit canonical linear address. Bits 63..48 must be the canonical sign extension of bit 47. The remaining address bits select four nine-bit page-table indexes followed by a twelve-bit page offset.

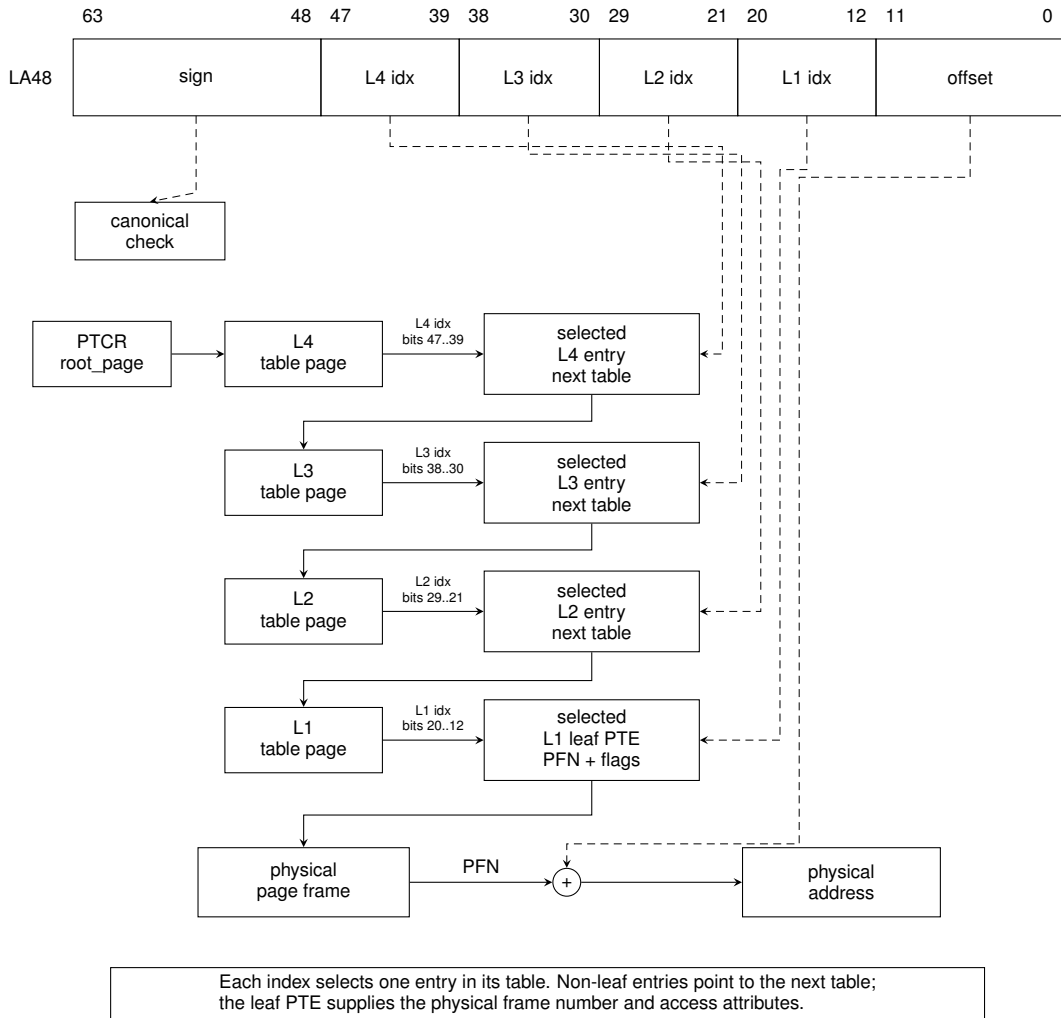
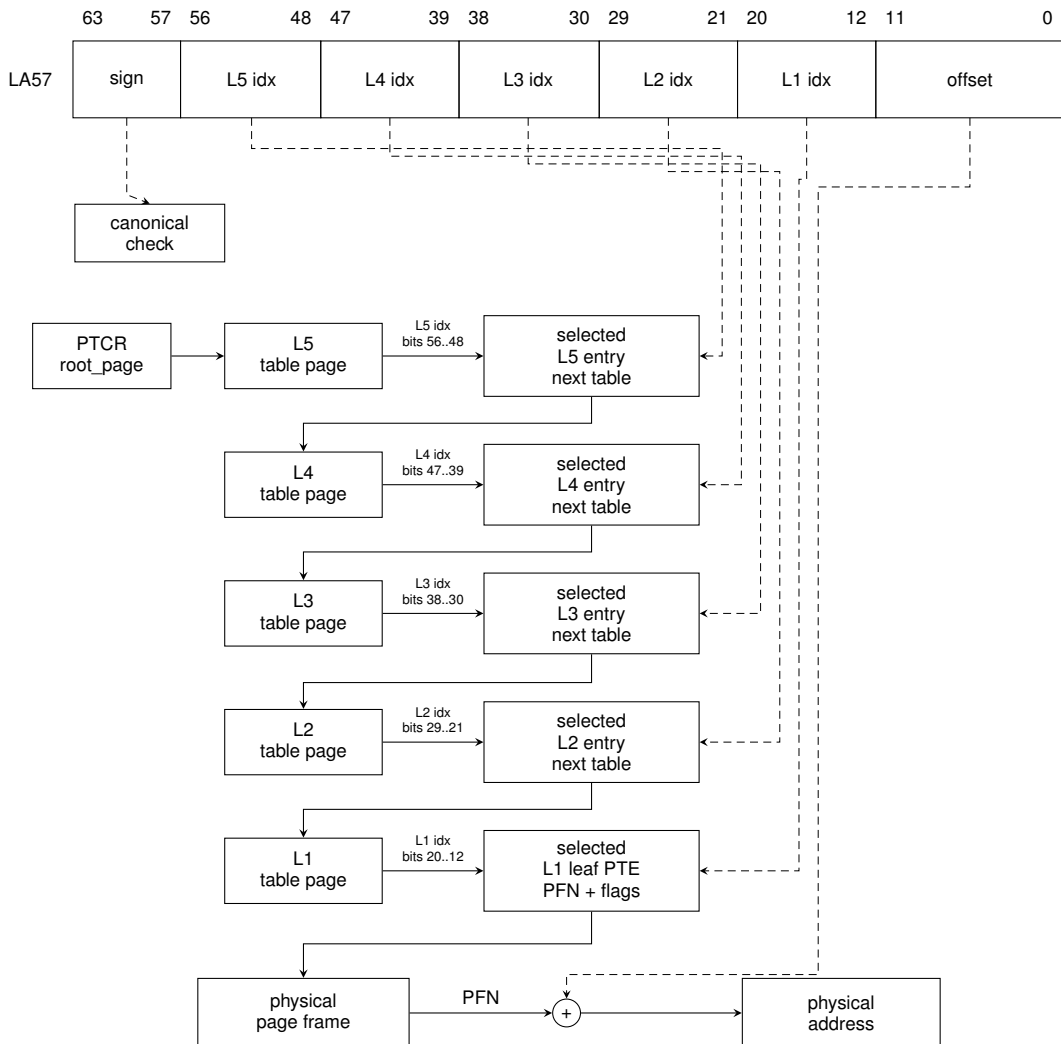


Figure 11-2. LA48 Four-Level Page Walk

11.6 LA57 Five-Level Paging

When PTCR.LA57 is one, paging uses a 57-bit canonical linear address. Bits 63..57 must be the canonical sign extension of bit 56. The additional L5 index selects the top-level table before the same L4 through L1 walk used by LA48.



Each index selects one entry in its table. Non-leaf entries point to the next table; the leaf PTE supplies the physical frame number and access attributes.

Figure 11-3. LA57 Five-Level Page Walk

Field	Rule
leaf W_X_U	access permissions for stores, instruction fetches, and user-mode access
leaf A	set when the leaf mapping is successfully used
leaf D	set on a successful store through the leaf mapping
leaf G	marks the resulting translation as global
leaf CP	cache policy for the mapped access
leaf AT	addressing type for the mapped access; does not change PFN plus offset address formation
leaf SW0	software-defined and ignored by hardware

Table 11-6. PTE Attribute Semantics

Mode	Condition	Result
user	effective U permission is 0	PRIVILEGE_FAULT
user	effective U permission is 1	allowed subject to present, W/X, and access-type checks
supervisor	effective U permission is 0	allowed for C-domain accesses subject to present, W/X, and access-type checks
supervisor	effective U permission is 1 and access domain is C	PAGE_FAULT
supervisor	effective U permission is 1 and access domain is U	allowed subject to present, W/X, and access-type checks
supervisor	effective U permission is 0 and access domain is U	PAGE_FAULT

Table 11-7. PTE User-Permission Rules

12 MEMORY MODEL

12.1 Scope

The memory model defines the architectural behavior of instruction fetches, loads, stores, atomic read-modify-write operations, cache maintenance, and translation-cache maintenance after effective-address calculation and address translation. Language rules such as C `volatile`, C atomics, and process startup policy are ABI concerns; this section describes the guarantees provided directly by the ISA.

12.2 Memory Access Domains

All instruction fetch, data, stack, and device accesses use the same effective-address and address-translation pipeline. The final page-table attributes classify the access domain.

Attribute	Domain	Architectural Meaning
PTE.AT=0	byte-addressed memory	Normal memory is addressed in bytes and may be cached according to PTE.CP.
PTE.AT=1	externally acknowledged memory	The access is acknowledged by an external memory or device agent and may have bus-sized side effects.
PTE.CP=00	cacheable	Normal coherent cacheable memory.
PTE.CP=01	uncacheable	Accesses are not allocated into normal data caches.
PTE.CP=10	write-through	Stores become visible through a write-through cache policy.
PTE.CP=11	write-combining	Stores may be gathered and combined before visibility; explicit ordering is required when software depends on completion order.

Table 12-1. Memory Access Domains

Externally acknowledged memory is not treated as ordinary cacheable memory. An implementation must not silently replace an externally acknowledged access with an unrelated cached access. The translated address is still formed as PFN plus page offset; PTE.AT=1 changes the memory transaction domain and acknowledgement behavior, not the address arithmetic. Ordering requirements for device protocols are expressed with mapped memory attributes and explicit fence or cache-synchronization instructions.

12.3 Access Size, Alignment, and Tearing

The architectural scalar memory sizes are byte, word, long word, and quad word. A memory access of size N bytes is naturally aligned when its byte address is a multiple of N .

Suffix	Bytes	Natural Alignment	Tear-Free Guarantee
B	1	any byte address	aligned normal-memory load/store is tear-free
W	2	address divisible by 2	aligned normal-memory load/store is tear-free
L	4	address divisible by 4	aligned normal-memory load/store is tear-free
Q	8	address divisible by 8	aligned normal-memory load/store is tear-free

Table 12-2. Scalar Access Sizes and Alignment

Unaligned ordinary memory accesses are architecturally permitted. An unaligned load or store does not fault merely because of its alignment, although it may still fault because one of the bytes it touches fails address translation, permission checking, or device acknowledgement. Unaligned accesses may be implemented as multiple aligned memory beats and have no tear-free or atomicity guarantee.

Architectural accesses wider than eight bytes, including block transfers and repeated string-like operations, are not single atomic memory accesses. They are described as a sequence of scalar accesses unless a specific instruction form states otherwise.

12.4 Atomic Access Requirements

Atomic instructions require a naturally aligned memory operand for the selected operand size. A misaligned atomic memory operand is invalid for the atomic operation and must not complete as a non-atomic read-modify-write. If a fault is reported for such an instruction, the fault is precise and the atomic memory location is not updated.

For an aligned atomic read-modify-write instruction, the read, modification, and write of the selected memory location are observed as one indivisible operation by other agents that can access the same physical location. Atomic read-modify-write instructions carry a memory-order selector as part of the atomic instruction format. The selector constrains visibility ordering around the indivisible update; it is not a general-purpose prefix.

Selector	Code	Architectural Ordering Effect
RELAXED	0	Atomicity only. No ordering is imposed on unrelated memory operations.
ACQUIRE	1	Later memory operations by the same hardware thread are ordered after the atomic operation.
RELEASE	2	Earlier memory operations by the same hardware thread are ordered before the atomic operation.
ACQREL	3	Applies both acquire and release ordering.
SEQCST	4	Applies acquire-release ordering and participates in the global sequentially consistent order for atomic operations.
RESERVED5	5	Reserved selector encoding; using it is invalid for the atomic form.
RESERVED6	6	Reserved selector encoding; using it is invalid for the atomic form.
RESERVED7	7	Reserved selector encoding; using it is invalid for the atomic form.

Table 12-3. Atomic Memory-Order Selectors

The `consume` memory order is not a separate architectural selector. Software that needs consume-style source semantics uses the acquire selector. A full ordering point for ordinary unrelated accesses can also be expressed with `AFENCE` when the program needs one independently of an atomic operation.

12.5 Visibility and Ordering

Normal cacheable memory is coherent at the architectural level: once a store becomes visible to another agent, later coherent accesses to the same physical location observe a value consistent with the coherence order for that location. The ISA does not require independent ordinary loads

and stores to become visible to other agents in program order unless an instruction dependency, an atomic operation, or a fence instruction imposes that order.

Instruction	Ordering Effect
RFENCE	Orders prior reads before later reads issued by the same hardware thread.
WFENCE	Orders prior writes before later writes issued by the same hardware thread.
AFENCE	Orders prior memory operations before later memory operations issued by the same hardware thread.

Table 12-4. Fence Instructions

Fence instructions constrain visibility ordering. They do not perform cache-line writeback, cache invalidation, TLB invalidation, or instruction-cache synchronization by themselves unless the instruction description explicitly says so.

12.6 Fault and Partial-Commit Rules

Memory faults are precise with respect to the faulting instruction. A faulting load does not update its destination register. A faulting scalar store does not commit bytes of that store. Atomic read-modify-write instructions either complete the whole aligned atomic operation or leave the memory location unchanged.

Repeated operations commit completed instruction bodies only. For `REPcc`, a completed terminating iteration whose repeat condition is false still commits the instruction body's ordinary side effects, but it does not update the repeat counter. If a `REP` or `REPcc` operation faults after some iterations have completed, the completed iterations remain committed and the saved restart state describes the remaining work. `REPG` uses the same precise-fault rule, extended to the current grouped iteration: completed prior group instructions in the faulting iteration remain committed, later group instructions do not execute, and the counter is not updated until the whole group iteration succeeds. The repeat-fault auxiliary state allows return from the fault to continue at the faulting grouped instruction under the active `REPG` context.

12.7 Instruction Fetch and Self-Modifying Code

Stores to memory that is later executed as instructions are ordinary data stores until an explicit synchronization sequence makes the modified bytes visible to instruction fetch. Code patching, dynamic relocation into executable mappings, and JIT-style code generation therefore require instruction/data cache synchronization before execution may rely on the new instruction bytes.

`SYNCCACHE` synchronizes instruction/data cache visibility for modified executable memory. `INVICACHE`, `INVDCACHE`, `FLSHDCACHE`, and `WRBKDCACHE` provide explicit cache-maintenance operations for the cache block or range selected by their operands.

12.8 Address-Translation Coherency

Page-table memory is ordinary memory until a page-table update is made visible to the translation machinery. Software that modifies page tables must order the page-table stores before invalidating or replacing translation-cache entries. `INVTLB`, `INVPAGE`, `INVASID`, `SWPT`, and `SWPTA` define the architectural translation-cache maintenance operations.

When ASID matching is enabled, address-space identifiers participate in translation-cache lookup. Global entries are matched according to the page-table attributes described in the Memory Address Translation section. When ASID matching is disabled, translation-cache entries are matched without an address-space identifier.

13 SUPERVISOR / PRIVILEGED PROGRAMMING MODEL

The privileged programming model defines how execution moves between user mode and supervisor mode, which saved state is used for return, and which control state is directly accessible.

13.1 Privileged Model Rules

The following rules define the supervisor and exception-control behavior used by the rest of this manual.

No.	Subject	Rule
1	privilege mode encoding	STATUS.PM=0 is user mode; STATUS.PM=1 is supervisor mode
2	user-memory access control	Supervisor-mode memory accesses use the current-domain C by default; U2C, C2U, and U2U prefixes select user-domain access for the annotated memory operands of that instruction only
3	saved STATUS return	STATUS is restored verbatim from the saved frame image; SYSCALL, trap, interrupt, and exception entry set STATUS.PM=1 only and do not mask interrupts
4	supervisor entry DBANK normalization	SYSCALL, trap, interrupt, and exception entry save the interrupted DBANK selector and then set DBANK=0 before fetching the handler
5	SYSCALL frame and entry	SYSCALL saves FRAME_CONTROL, SS:SP, CS:PC, and DS, then sets DBANK=0 and uses a 32-byte supervisor-entry table with 16-byte entry-address alignment
6	return instruction split	SYSRET is only for SYSCALL return; IRET is for interrupt, trap, and exception return
7	IRET restore policy	IRET restores all saved interrupt-frame state and does not validate FRAME_CONTROL.frame_type
8	malformed return frames	malformed frames are software errors and do not raise a separate architectural exception
9	NMI and double-fault stacks	NMI and DOUBLE_FAULT use the SN field of their IVT entries
10	interrupt nesting limit	when hidden_current_iddepth equals ICR.MAX_IDDEPTH, maskable interrupts are implicitly masked
11	control-register access	RDCR and WRCR are always privileged; user-visible control state uses dedicated gated instructions
12	reserved CPU vectors	reserved CPU vectors have no frame type; attempted delivery immediately becomes DOUBLE_FAULT

Table 13-1. Privileged Model Rules

13.2 Privilege State

STATUS.PM selects the current privilege level. Memory operands use the current-domain C by default. In supervisor mode, operand annotations such as U: [A0] select user-domain access for that instruction only through the encoded U2C, C2U, or U2U prefix. In user mode C and U are identical, so these prefixes have no additional architectural effect.

Field	Value	Meaning
STATUS.PM	0	user mode
STATUS.PM	1	supervisor mode
access domain	C	current privilege-mode memory access domain
access domain	U	user memory access domain selected per instruction by U2C, C2U, or U2U

Table 13-2. Privilege State and Access Domains**13.3 SYSCALL and SYSRET**

SYSCALL is an explicit supervisor-entry path rather than an IVT-vector event. It behaves like an extended long call: it saves the user control state, uses a 32-byte supervisor-entry table with 16-byte entry-address alignment, loads the supervisor entry state from SPC/SCS/SDS, and sets STATUS.PM to supervisor mode. SYSRET is the matching return instruction and is only for this syscall frame shape.

Property	Rule
Entry vector	none
Entry target	SPC, SCS, SDS
Frame style	expanded long-call frame
Saved state	FRAME CONTROL, SS:SP, CS:PC, DS
Entry table size	32 bytes
Entry address alignment	16 bytes
Entry STATUS change	set STATUS.PM to 1 only
Entry DBANK change	save interrupted DBANK selector and set DBANK to 0
Return instruction	SYSRET
Return policy	read the syscall frame as written, restore saved DBANK, and return

Table 13-3. SYSCALL/SYSRET Rules**13.4 Interrupt and Control-State Rules**

Trap, interrupt, and exception entry also set STATUS.PM to supervisor mode, but do not automatically mask interrupts. The saved STATUS image is restored verbatim by the matching return path. Malformed return frames are software errors; the architecture does not raise a second exception merely because the return frame is nonsensical.

Rule	Meaning
Entry STATUS update	set STATUS.PM to 1
Entry DBANK update	save interrupted DBANK selector and set DBANK to 0
Saved STATUS return	restored verbatim from the saved frame image
Entry interrupt masking	no automatic masking
Interrupt stack selection	IVT entry SN selects SSSn:SSPn
NMI stack selection	use the IVT entry SN field
Double-fault stack selection	use the IVT entry SN field
Interrupt frame save	atomic
SYSRET	syscall frame only; restores saved DBANK from the syscall frame
IRET	restore all saved interrupt frame state including DBANK; frame_type is not validated by hardware
Malformed return frame	software error; no architectural exception is raised
ICR.MAX_IDEPTH	when hidden_current_idepth equals ICR.MAX_IDEPTH, maskable interrupts are implicitly masked
RDCR/WRCR	RDCR = supervisor; WRCR = supervisor
User-visible control state	user-visible control state must use dedicated instructions gated by control-register policy bits

Table 13-4. Privileged Execution Rules

14 EXCEPTION PROCESSING REFERENCE

Exception processing covers synchronous restartable CPU exceptions, traps, NMIs, external interrupts, and the supervisor-entry state needed to return from them. The processor records a restartable context for CPU exceptions; the operating system decides whether that context is re-tried, emulated, reported, or terminated. ICR selects the interrupt vector table page and controls interrupt nesting; each IVT entry selects a handler and one of the supervisor stack banks.

14.1 Interrupt Vector Table

The interrupt vector table occupies one 4096-byte page selected by `ICR.ivt_page`. It contains 256 fixed-size entries, each 16 bytes. The vector number selects the entry at `ICR.ivt_page * 4096 + vector * 16`.

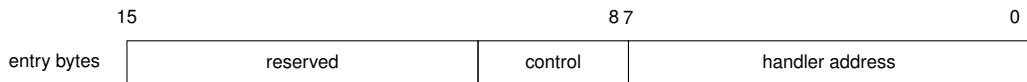


Figure 14-1. Interrupt Vector Table Entry

Bytes 0 through 7 contain the 64-bit interrupt handler address. Byte 8 is the entry control byte. HP is the handler-present bit; if it is clear, delivery immediately becomes `DOUBLE_FAULT`. SN selects one of four interrupt stack banks, `SSS0:SSP0` through `SSS3:SSP3`. Byte 8 bit 1 and bits 7 through 4 are reserved, and bytes 9 through 15 are reserved.

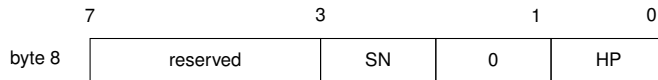


Figure 14-2. IVT Entry Control Byte

Field	Location	Meaning
handler_address	bytes 0..7	interrupt handler address
HP	byte 8 bit 0	handler present; if clear, delivery becomes <code>DOUBLE_FAULT</code>
reserved_low	byte 8 bit 1	reserved, must be zero
SN	byte 8 bit 3:2	selects <code>SSSn:SSPn</code> interrupt stack set 0 = <code>SSS0:SSP0</code> 1 = <code>SSS1:SSP1</code> 2 = <code>SSS2:SSP2</code> 3 = <code>SSS3:SSP3</code>
reserved_high	byte 8 bit 7:4	reserved, must be zero
reserved	bytes 9..15	reserved, must be zero

Table 14-1. Interrupt Vector Table Entry Fields

14.2 Vector Assignment

Vectors 0x00..0x1F are owned by the CPU architecture. Assigned CPU vectors are grouped by supervisor stack frame type. Reserved CPU vectors do not predefine a frame type. TRAP and TRAPcc use vector 0x05 in the BASIC-frame group. PRIVILEGE_FAULT is ordered before PAGE_FAULT. Length, segment, and canonical-address faults do not have separate CPU vectors; invalid length is an invalid instruction encoding, and segment or canonical-address failures are reported through PAGE_FAULT. Vectors 0x20..0x3F are reserved for future architectural use. Operating systems, platforms, and devices may assign vectors 0x40..0xFF.

SYSCALL does not allocate or consume an IVT vector. It enters supervisor mode through the SPC/SCS/SDS supervisor-entry path.

Range	Owner	Meaning
0x00..0x03	CPU	assigned BASIC-frame CPU exceptions
0x04	CPU	reserved CPU vector; frame type not predefined
0x05	CPU	assigned BASIC-frame CPU trap
0x06..0x07	CPU	reserved CPU vectors; frame type not predefined
0x08..0x0E	CPU	assigned ERROR/PAGE_FAULT-frame CPU exceptions
0x0F	CPU	reserved CPU vector; frame type not predefined
0x10..0x17	CPU	reserved CPU vectors; frame type not predefined
0x18..0x1A	CPU	assigned AUX_FAULT-frame CPU exceptions
0x1B..0x1F	CPU	reserved CPU vectors; frame type not predefined
0x20..0x3F	CPU	reserved for future architectural vectors
0x40..0xFF	OS/platform/device	assignable interrupt and event vectors

Table 14-2. Interrupt Vector Ranges

Vector	Name	Source	Frame
0x00	DEBUG TRACE	debug, trace, watchpoint, or single-step entry	BASIC
0x01	NMI	non-maskable interrupt	BASIC
0x02	BREAKPOINT	BKPT instruction	BASIC
0x03	ILLEGAL INSTRUCTION	illegal, undefined, reserved, noncanonical, or disabled extension instruction encoding	BASIC
0x05	TRAP	TRAP instruction or true TRAPcc condition	BASIC
0x06	RESERVED CPU 06	reserved CPU vector	-
0x07	RESERVED CPU 07	reserved CPU vector	-
0x08	PRIVILEGE FAULT	privileged instruction, CR access, or supervisor-only state violation	ERROR
0x09	PAGE FAULT	segment pre-translation, canonical address when paging is enabled, page-table translation, permission, presence, or stack memory translation failure	PAGE FAULT
0x0A	DIVIDE ERROR	integer divide, modulo, or quotient overflow	ERROR
0x0B	ARITHMETIC TRAP	reserved arithmetic trap class	ERROR
0x0C	BOUNDS FAULT	bounds-check exception class	ERROR
0x0D	INVALID CONTROL STATE	invalid control-register, status, or return state	ERROR
0x0E	FLOATING POINT FAULT	floating-point exception trap	ERROR
0x0F	RESERVED CPU 0F	reserved CPU vector	-
0x10	RESERVED CPU 10	reserved CPU vector	-
0x11	RESERVED CPU 11	reserved CPU vector	-
0x12	RESERVED CPU 12	reserved CPU vector	-

Vector	Name	Source	Frame
0x13	RESERVED CPU 13	reserved CPU vector	-
0x14	RESERVED CPU 14	reserved CPU vector	-
0x15	RESERVED CPU 15	reserved CPU vector	-
0x16	RESERVED CPU 16	reserved CPU vector	-
0x17	RESERVED CPU 17	reserved CPU vector	-
0x18	DOUBLE FAULT	fault during exception delivery, reserved CPU vector delivery, or IVT HP=0	AUX FAULT
0x19	MACHINE CHECK	machine-check or hardware integrity failure	AUX FAULT
0x1A	BUS ERROR	externally acknowledged or bus-sized access failure	AUX FAULT
0x1B	RESERVED CPU 1B	reserved CPU vector	-
0x1C	RESERVED CPU 1C	reserved CPU vector	-
0x1D	RESERVED CPU 1D	reserved CPU vector	-
0x1E	RESERVED CPU 1E	reserved CPU vector	-
0x1F	RESERVED CPU 1F	reserved CPU vector	-

Table 14-3. CPU-Owned Interrupt Vectors

14.3 Entry and Return Rules

The processor changes only the privilege bit on entry: STATUS.PM is set to supervisor mode. It also saves the interrupted DBANK selector and sets DBANK to zero before fetching the handler. Interrupts are not automatically masked merely because an exception or interrupt was taken. The interrupt stack frame is saved atomically. When returning, the saved STATUS image is restored exactly as recorded in the frame, and the saved DBANK selector is restored from the frame.

Rule	Meaning
CPU exception model	synchronous, restartable
Restart policy	CPU exceptions identify the faulting instruction or operation at a restartable boundary. The operating system decides whether the saved context should be retried, emulated, reported to user space, or terminated.
Interrupt frame save	atomic
Entry STATUS update	set STATUS.PM to 1 only; do not automatically mask interrupts
Return STATUS update	restore saved STATUS image verbatim
Entry DBANK update	save interrupted DBANK selector and set DBANK to 0 before handler fetch
Return DBANK update	restore saved DBANK selector from the interrupt frame
SYSRET	syscall frame only; restores saved DBANK from the syscall frame
IRET	restore all saved interrupt frame state including DBANK; frame_type is not validated by hardware
IRET frame-type check	none; IRET does not validate FRAME_CONTROL.frame_type
Malformed return frame	software error; no separate architectural exception
Absent IVT handler	IVT HP=0 immediately becomes DOUBLE_FAULT
Reserved CPU vector	reserved CPU vectors have no frame type; attempted delivery immediately becomes DOUBLE_FAULT
Fault priority	privilege fault, page fault
Collapsed fault classes	length_fault: invalid instruction encoding; no separate vector; segment_fault: reported as PAGE_FAULT; canonical_address_fault: reported as PAGE_FAULT
Address fault vector	PAGE_FAULT

Table 14-4. Exception Entry and Return Rules

14.4 Supervisor Entry Stack Frame

The fixed supervisor-entry frame is 64 bytes: 8 64-bit slots. `FRAME_CONTROL.frame_size_units` records the total frame size in 8-byte units, so the base frame records 8 units before any optional payload. Offsets are measured from the saved stack pointer.

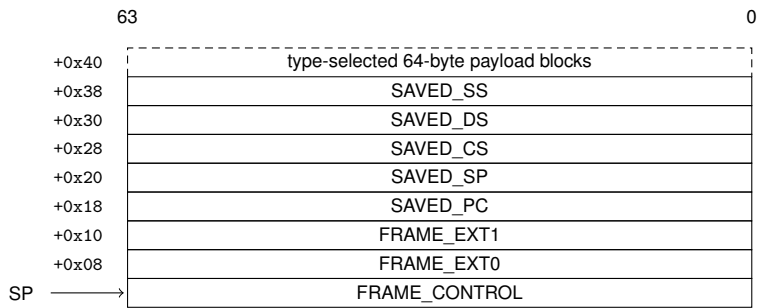


Figure 14-3. Supervisor Entry Stack Frame

Offset	Slot	Meaning
+0x00	FRAME_CONTROL	frame metadata, entry flags, saved DBANK, saved FLAGS, and saved STATUS
+0x08	FRAME_EXT0	reserved for future architected base-frame state, must be zero
+0x10	FRAME_EXT1	reserved for future architected base-frame state, must be zero
+0x18	SAVED_PC	saved program counter
+0x20	SAVED_SP	saved stack pointer
+0x28	SAVED_CS	saved code segment selector
+0x30	SAVED_DS	saved data segment selector
+0x38	SAVED_SS	saved stack segment selector

Table 14-5. Supervisor Entry Stack Frame

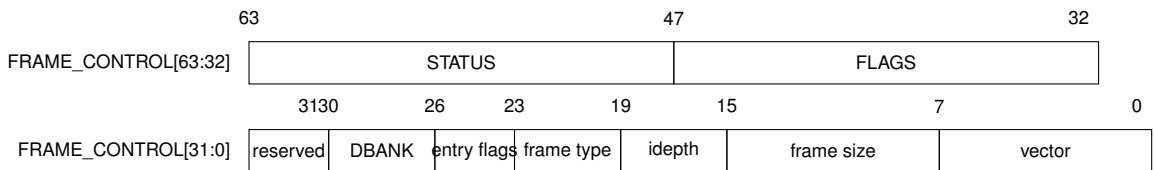
`FRAME_CONTROL.frame_type` documents the stack frame format, M68000-style. Payload, if any, is appended after the fixed frame in whole 64-byte blocks. `FRAME_CONTROL.frame_size_units` gives the total number of 8-byte slots, and the frame type code determines how software interprets the allocated payload blocks. `IRET` does not validate the frame type.

Code	Type	Payload Blocks	Payload	Meaning
0x0	BASIC	0	none	fixed frame only
0x1	ERROR	1	ERROR CODE	fixed frame plus one 64-byte payload block containing the exception error code
0x2	PAGE_FAULT	1	ERROR CODE FAULT EA FAULT LINEAR	fixed frame plus one 64-byte payload block containing page-fault code, effective-address context, and linear address
0x3	AUX_FAULT	1	ERROR CODE FAULT EA FAULT LINEAR FAULT AUX	fixed frame plus one 64-byte payload block containing full auxiliary fault context

Table 14-6. Supervisor Stack Frame Types

FRAME_CONTROL packs the saved DBANK selector, FLAGS image, and STATUS image with the frame metadata. The fixed frame also reserves two 64-bit extension slots for future architected base-frame state.

Offset	Slot	Meaning
+0x00	ERROR_CODE	exception-specific error code
+0x08	FAULT_EA	faulting effective-address operand information
+0x10	FAULT_LINEAR	faulting linear address
+0x18	FAULT_AUX	auxiliary fault information

Table 14-7. Supervisor Payload Block Slots**Figure 14-4. FRAME_CONTROL Format**

Field	Bits	Meaning
vector	0..7	interrupt, exception, or trap vector number
frame_size_units	8..15	total frame size in 8-byte units
saved_idepth	16..19	saved interrupt nesting depth
frame_type	20..23	supervisor stack frame type code
from_user	24	entry was taken from user mode
nmi_frame	25	frame was created by NMI entry
rep_fault	26	fault occurred during REP, REPcc, or REPG repeated execution
saved_dbank	27..30	saved interrupted DBANK selector
reserved	31	reserved, must be zero
flags	32..47	saved FLAGS image
status	48..63	saved STATUS image

Table 14-8. FRAME_CONTROL Fields

Repeat Fault Continuation. When FRAME_CONTROL.rep_fault is set, the FAULT_AUX payload records the active repeat context. FAULT_AUX.repeat_kind distinguishes REP/REPcc, REPG-general, and REPG-fast continuations. For REPG, the saved PC is the faulting grouped instruction, and the group start is reconstructed as $\text{group_start_pc} = \text{fault_pc} - 2 * \text{group_start_delta_words}$. The delta is an unsigned negative displacement measured in 16-bit words, so five bits cover the full 64-byte grouping window. Returning with IRET preserves the continuation state: execution resumes at the saved faulting instruction under the active repeat context. Software that wants to abandon or emulate the repeated operation may edit the frame before returning.

Field	Bits	Meaning
counter_register	0..2	D-register counter number, D0-D7
group_start_delta_words	3..7	REPG-general/REPG-fast unsigned negative displacement in 16-bit words from the saved fault_pc to the first grouped instruction; 0..31 words covers one 64-byte grouping window
repeat_kind	8..9	repeat context kind 0: REP or REPcc 1: REPG-general 2: REPG-fast 3: reserved
reserved	10..63	reserved, must be zero

Table 14-9. Repeat Fault Auxiliary Fields

14.5 Interrupt Reset State

State	Reset Value
ICR	0
ICR.IVT_VALID	0
STATUS.IE	0
STATUS.IN	0
STATUS.NI	0
FSTATUS	0
FFLAGS	0
hidden_current_iddepth	0
hidden_nmi_pending	0
PTCR.PE	0
ASCR	0

Table 14-10. Interrupt and Translation Reset State

15 INSTRUCTION WORD FORMATS

The first instruction word contains a four-bit prefix/length area and a twelve-bit primary payload. Compact instructions place operand fields directly in this primary payload when the field set fits. Extended instructions use a primary root and a following descriptor/opcode word, with more payload words added only when needed.

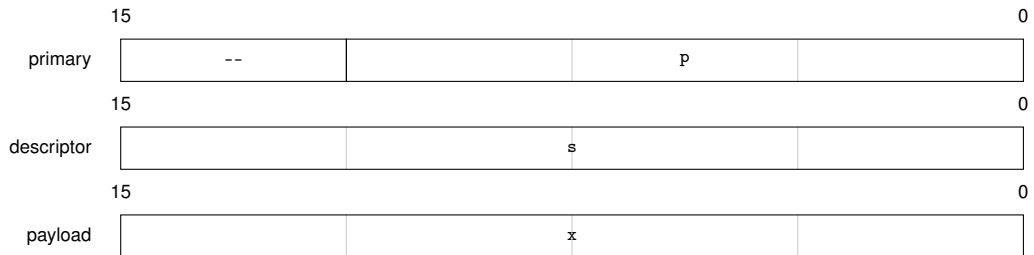


Figure 15-1. Instruction Word Format Families

15.1 Representative Encodings

<ea(e)>, Dn(d): MOV.X(z:L/Q) <ea(e)>, Dn(d)

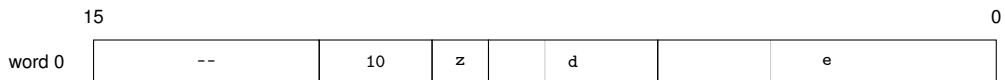


Figure 15-2. Encoding layout for MOV.X(z:L/Q) <ea(e)>, Dn(d)

Dn(d), <ea(e)>: MOV.X(z:L/Q) Dn(d), <ea(e)>

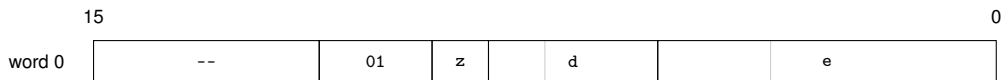


Figure 15-3. Encoding layout for MOV.X(z:L/Q) Dn(d), <ea(e)>

imm: Jcc.X(z:W/L) imm



Figure 15-4. Encoding layout for Jcc.X(z:W/L) imm

16 INSTRUCTION EXECUTION MODEL

This section defines the common execution contract used by the instruction descriptions. An individual instruction may add stricter rules, but it does not silently replace the rules below. The purpose of this section is to make operand evaluation, side effects, repetition, and status updates readable before the per-instruction pages.

16.1 Instruction Boundary and Default Execution Rules

The instruction boundary is fixed by word 0 before operand evaluation begins. An implementation may reject an instruction before any architectural state is changed if the encoded length cannot contain the selected form.

Topic	Spec Value
Overlong Encoding	allowed
Undersized Encoding	length/decode fault
Memory-Memory Operands	operation opt in
Unmentioned FLAGS/FFLAGS	unchanged
Operand Evaluation Order	decode order

Table 16-1. Execution Defaults

16.2 Instruction Boundary, Operand Evaluation, and EA Defaults

- Word 0 length selects the instruction boundary; prefixes and extension words never extend it implicitly.
- Operands are decoded in instruction order; source reads complete before the final destination write unless an atomic form says otherwise.
- EA calculation may produce an address without reading memory; memory is read only when the operation needs the operand value.

16.3 Mnemonic Suffix Rules

Conditional mnemonic forms use the condition names listed in the Condition Codes section.

Rule	Value
Placement Applies To	mnemonic suffix
	DJcc
	IJcc
	Jcc
	MOVcc
	SETcc
	TRAPcc
	FMOVcc
	REPcc

Table 16-2. Conditional Mnemonic Suffix Rules

16.4 Operand and Status Notation

The generated operation fields use compact notation. The following terms define when an operand is only addressed, when it is read, and which status register is affected.

Term	Meaning
Operand Read	read operand value after applying its addressing mode and size
Operand Write	write result back to destination using operand size
EA Address	compute effective address without reading memory unless operation says read
Selector	selector operands may name a D register or an encoded immediate selector
Overlong	extra trailing instruction words are ignored as padding payload
FLAGS ZNCV	integer flags Z, N, C, V
FP Status	floating-point accrued exception/status bits in FFLAGS
FFLAGS	NV: invalid operation DZ: division by zero OF: overflow UF: underflow NX: inexact

Table 16-3. Semantic Notation

16.5 Shared Side-Effect Rules

These rows summarize execution rules that apply to whole instruction families. The instruction descriptions list exact forms and operands; this block records the common side-effect model.

Integer ALU

- Memory operands: at most one memory operand

Compare/Test

- Memory operands: memory memory explicit form

Extension

- Memory operands: bulk extend memory memory
- Source sizes for W destination: B
- Source sizes for L destination: B, W
- Source sizes for Q destination: B, W, L
- FLAGS: unchanged

Data Movement

- FLAGS: unchanged

Data Register Banking

- FLAGS: unchanged

Control Transfer

- FLAGS: unchanged

Atomics

- Atomic: True
- Memory operands: addressable memory operand required

TLB and Context

- Privilege: supervisor

Cache

- FLAGS: unchanged

Floating-Point Transcendental

- CPUID feature: OPTIONAL EXTENSION FEATURES.FPTRANS
- Implementation: hardware implementation or internal assist

17 STREAMING EXECUTION MODEL

This section defines the architectural role of repeated scalar execution in Bedrock. The architecture exposes scalar register state and bounded repeated instruction forms. A high-performance implementation may recognize regular repeated operations and execute them with wider internal datapaths, but that internal width is not architectural state.

17.1 Architectural Scalar Semantics

REP, REPcc, and REPG are architecturally scalar. Their visible behavior is the same as executing the repeated instruction or instruction group in program order under the counter and termination rules of the selected repeat prefix. For REPcc, the instruction body of the terminating condition-false iteration commits, while the repeat counter is left unchanged. These operations have no architected vector register file, and software must not depend on any implementation's internal vector width.

Repeated execution preserves the ordinary instruction semantics:

- register and memory operands are evaluated according to the repeated instruction's normal rules;
- update-eligible effective-address forms update as if each iteration had executed the scalar instruction;
- faults and exceptions are precise at the repeated-operation boundary defined by the REP or REPG rules;
- FLAGS and FFLAGS are updated according to the repeated instruction's metadata and repeat-prefix accumulation rules.

17.2 Implementation Freedom

An implementation may execute a repeated scalar operation by using an internal micro-loop, a loop buffer, a streaming execution engine, or a wider internal SIMD datapath. These mechanisms are invisible to software. They must retire the same architectural state that the scalar repetition would have produced.

The implementation may choose scalar execution for any repeated operation. This is permitted even when the operation is marked as a streaming candidate. The streaming marker indicates that the operation shape is suitable for microarchitectural recognition; it is not a promise of a particular throughput, latency, or internal vector width.

17.3 Streaming Candidates

Streaming candidates are repeated operations with simple, bounded dataflow and regular memory access. Typical candidates include contiguous copies and fills, integer reductions, bulk extension or conversion, floating-point arithmetic, and fused multiply-add loops.

REPG optimization is divided into two execution classes:

- | | |
|------------------|---|
| REPG-fast | The selected D register used as the repeat counter is not written by the group body, effective-address forms are regular, memory references are |
|------------------|---|

ordinary memory, and the group does not perform control or architectural state changes. PC-relative addressing is not part of the fast-contract source form, because it ties a memory reference to the location of a grouped instruction rather than to the repeated data stream. An implementation may recognize this form as a candidate for internal SIMD or streaming execution. Source may spell the group as `REPGF` when the programmer wants the assembler to enforce these static fast-contract rules; `REPGF` emits the same instruction encoding as `REPG`.

REPG-general

The selected counter register may be read or written inside the group body, and effective-address or data dependencies may be less regular than the fast class. The base `REPG` group model still applies: control flow, atomics, stack operations, nested repeat, and state-changing system, TLB, cache, or fence operations are outside the grouped execution model. The implementation must preserve exact scalar `REPG` semantics, but no fast streaming execution is guaranteed.

The `REPG` prefix is encoded on the first instruction in the group, and the `ENDG` prefix is encoded on the final instruction. Instructions between those endpoints keep their own word 0 length fields and are decoded as ordinary instruction records under the active `REPG` group. The group may start at any address, but the complete grouped instruction sequence must fit within one naturally aligned 64-byte grouping window.

`REPG` group discovery is deliberately bounded by an architectural grouping window. Because every instruction record carries an explicit length, instruction interpretation can walk from the `REPG`-marked instruction to the `ENDG`-marked instruction without interpreting the full semantics of each grouped instruction. A well-formed `REPG` group contains one `REPG` start marker and one `ENDG` boundary marker inside the grouping window.

The repeat counter is ordinary architectural D-register state. A zero counter executes no group iterations. A positive counter moves toward zero by decrementing after each successful group iteration; a negative counter moves toward zero by incrementing after each successful group iteration. The counter update occurs after the entire group iteration succeeds, not after each instruction inside the group.

Within one iteration, grouped instructions execute in program order. Each instruction keeps its normal operand rules, effective-address calculation, prefix effects, privilege checks, `FLAGS` or `FFLAGS` behavior, and memory side effects. This means a `REPG` group can be reasoned about as a scalar loop first; any internal SIMD or streaming treatment is an implementation choice layered under the same architectural retirement rules.

```
REP D0, ADD.L [A0++], D1
REPGF D0, {
    FMOV.S [A0++], F0
    FMADD.S [A1++], F0, F1
}
```

The examples above are scalar programs. A compact implementation may execute them directly as repeated scalar operations. A wider implementation may map the same instruction stream onto

internal lanes, multiple accumulators, or a streaming load/operate/store pipeline while preserving scalar architectural results.

Faults remain precise at the repeated-operation boundary. Completed iterations are committed. If a fault occurs partway through an iteration, instructions in that iteration that completed before the fault remain visible according to their ordinary scalar semantics, and the fault is reported at the faulting grouped instruction. The architectural counter update for the current iteration has not completed unless the whole group iteration completed. For REPG, the saved PC is the faulting grouped instruction. The repeat-fault auxiliary state records the selected counter register and a five-bit unsigned negative displacement from the saved PC back to the first grouped instruction:

$$\text{group_start_pc} = \text{fault_pc} - 2 \times \text{group_start_delta_words}.$$

The group index is not saved. Returning with the frame unchanged resumes at the saved PC under the active REPG continuation; software may instead edit the frame to abandon or emulate the repeated operation.

17.4 Compiler Model

A compiler targeting Bedrock should treat streaming optimization as a lowering problem rather than as ordinary vector-instruction selection. The compiler forms REP or REPG when it can preserve scalar ordering, fault behavior, and flag behavior. The implementation may then choose whether the repeated form is profitable to execute with internal SIMD resources.

The preferred lowering order is:

- use REP for a single counted operation;
- use REPG for a short, bounded group of counted operations;
- prefer REPG-fast when the counter register can stay outside the group dataflow;
- use REPG-general when correctness needs counter-register dependencies or less regular scalar dataflow;
- prefer update-eligible effective-address forms for linear streams;
- keep the whole group inside one naturally aligned 64-byte grouping window;
- keep the group small and regular enough for implementation recognition;
- fall back to an ordinary scalar loop when ordering, aliasing, or exception semantics cannot be preserved.

17.5 Relationship to Other Acceleration

The streaming execution model is intended for fast, relatively simple loops in general-purpose code. It is not a substitute for every form of data-parallel acceleration. Workloads that require wide shuffles, irregular gather/scatter, large matrix kernels, image pipelines, or specialized cryptographic primitives may use separate architectural extensions or external accelerators. The base streaming model keeps architectural state scalar while allowing high-performance implementations to optimize the common repeated-loop cases aggressively.

18 INSTRUCTION SET SUMMARY

This section groups the instruction set by architectural purpose. The generated instruction summary tables later in this manual list every mnemonic alphabetically; this section is intended as the shorter map that explains which families exist and what they are for.

Class	Mnemonics	Notes
Core Control	AFENCE, BKPT, HALT, ILLEGAL, IRET, NOP, RESET, RFENCE, SYSCALL, SYSRET, WAIT, WFENCE, YIELD	Flags: unchanged
Integer ALU	ADD, AND, OR, SUB, SUM, XOR	Memory: at most one memory operand
Integer Compare	CMP, TEST	Memory: memory memory explicit form
Integer Carry ALU	ADC, SBB	Memory: at most one memory operand; Flags: update Z/N/C/V
Integer Minmax	MAXS, MAXU, MINS, MINU	Memory: at most one memory operand; Flags: may update Z/N
Data Movement	MOV, XCHG	Flags: unchanged
Stack	POP, POPM, PUSH, PUSHM	Flags: unchanged
Register Set Transfer	MOVSETAD, MOVSETDA, XCHGSETAD, XCHGSETDA	Flags: unchanged
Data Register Banking	GETDB, MOVSETDD, SELDB, XCHGSETDD	Flags: unchanged
Integer Unary	ABS, CLR, DEC, DECN, INC, INCN, NEG, NOT	
Integer Extend	EXTSL, EXTSQ, EXTSW, EXTZL, EXTZQ, EXTZW	Memory: bulk extend memory memory; Flags: unchanged
Bit Reverse	REVBIT, REVBYTE	Flags: unchanged
Shifts Rotates	RCL, RCR, ROL, ROR, SAR, SHL, SHR	Flags: mode: per shift
Bit Ops	BCHG, BCLR, BSET, BTEST	Flags: set Z when the old bit was zero; preserve N/C/V
Bit Counts	CLS, CLZ, CTS, CTZ, PARITY, POPCNT	
Multiply Divide	CLMUL, CLMULH, DIVMODS, DIVMODU, DIVS, DIVU, MADD, MODS, MODU, MSUB, MULHS, MULHSU, MULHU, MULS, MULU	Traps: divide by zero, signed overflow trap when defined
Bounds	BNDSII, BNDSIX, BNDSXI, BNDSXX, BNDUII, BNDUIX, BNDUXI, BNDUXX	Flags: V: set when the value is out of bounds
Control Flow	CALL, DJcc, IJcc, JMP, Jcc, LCALL, LJMP, LRET, MOVcc, RET, SETcc, TRACE, TRAP, TRAPcc	Flags: unchanged
EA Utility	LEA, SEGLEA, TESTCANON	
Atomics	CMPXCHG, FETCHADD, FETCHAND, FETCHOR, FETCHSUB, FETCHXOR	Memory: addressable memory operand required; Atomic: True
System Registers	CPUID, RDCR, RDFLAGS, RDFSTATUS, RDPMC, RDPTC, RDSEG, RDSTATUS, WRCR, WRFLAGS, WRFSTATUS, WRSEG, WRSTATUS	
TLB Context	INVASID, INVPAGE, INVTLB, PTATTR, PTQUERY, RESTORE, SAVE, SWPT, SWPTA, VTOP	Privilege: supervisor
Cache	FLSHDCACHE, INVDCACHE, INVICACHE, PREFETCH, SYNCCACHE, WRBKDCACHE	Flags: unchanged
FPU Move Compare	FCLASS, FCLR, FCMP, FCVT, FCVTU, FMOV, FMOVCR, FMOVcc, FTEST, FXCHG	
FPU Arithmetic	FABS, FADD, FBNDII, FBNDIX, FBNDXX, FCEIL, FCOPYSIGN, FDIV, FFLOOR, FGETEXP, FGETMAN, FINT, FINTRZ, FMADD, FMAX, FMIN, FMOD, FMSUB, FMUL, FNEG, FNMADD, FNMSUB, FREM, FROUND, FSCALE, FSQRT, FSUB, FTRUNC	

Class	Mnemonics	Notes
FPU Transcendental	FACOS, FASIN, FATAN, FATANH, FCOS, FCOSH, FETOX, FETOXM1, FLOG10, FLOG2, FLOGN, FLOGNP1, FSIN, FSINCOS, FSINH, FTAN, FTANH, FTENTOX, FTWOTOX	Implementation: hardware implementation or internal assist
Virtualization Acceleration	ENCINST	
FPU Stack	FPOPM, FPUSHM	

Table 18-1. Instruction Classes

18.1 Instruction Attribute Matrix

The following table is a compact availability index. NOSPEC is a common prefix and is omitted from the per-instruction prefix list. Address-update forms are operand spellings described in the Effective Addressing and Prefix Model sections, not a separate per-mnemonic prefix column.

Column	Cell	Meaning
Priv	U	all listed forms are unprivileged
Priv	U*	all listed forms are user-accessible under a listed architectural condition
Priv	S	all listed forms require supervisor privilege
Priv	P	at least one listed form is policy-controlled or configurable
Priv	mixed	the mnemonic has both unprivileged and privileged forms
Flag	Y	update permission for this FLAGS or FFLAGS bit
Flag	0	the instruction writes this flag as cleared
Flag	U	the flag is unchanged
Prefix	Y	at least one form of the mnemonic supports the prefix
Prefix	-	the prefix is not applicable to this mnemonic
SAT	Y/-	SATURATE prefix is applicable or not applicable
NT	Y/-	NONTEMPORAL hint is applicable or not applicable to memory forms

Table 18-2. Instruction Attribute Matrix Legend

Instr.	Priv	Z	N	C	V	NV	DZ	OF	UF	NX	SAT	NT
ABS	U	Y	U	U	Y	U	U	U	U	U	Y	Y
ADC	U	Y	Y	Y	Y	U	U	U	U	U	Y	Y
ADD	U	Y	U	U	Y	U	U	U	U	U	Y	Y
AFENCE	U	U	U	U	U	U	U	U	U	U	-	-
AND	U	Y	U	U	Y	U	U	U	U	U	-	Y
BCHG	U	Y	U	U	U	U	U	U	U	U	-	Y
BCLR	U	Y	U	U	U	U	U	U	U	U	-	Y
BKPT	U	U	U	U	U	U	U	U	U	U	-	-
BNDSII	U	U	U	U	Y	U	U	U	U	U	-	Y
BNDSIX	U	U	U	U	Y	U	U	U	U	U	-	Y
BNDSXI	U	U	U	U	Y	U	U	U	U	U	-	Y
BNDSXX	U	U	U	U	Y	U	U	U	U	U	-	Y
BNDUII	U	U	U	U	Y	U	U	U	U	U	-	Y
BNDUIX	U	U	U	U	Y	U	U	U	U	U	-	Y
BNDUXI	U	U	U	U	Y	U	U	U	U	U	-	Y
BNDUXX	U	U	U	U	Y	U	U	U	U	U	-	Y
BSET	U	Y	U	U	U	U	U	U	U	U	-	Y
BTEST	U	Y	U	U	U	U	U	U	U	U	-	Y
CALL	U	U	U	U	U	U	U	U	U	U	-	-
CLMUL	U	U	U	U	U	U	U	U	U	U	-	Y
CLMULH	U	U	U	U	U	U	U	U	U	U	-	Y

Instr.	Priv	Z	N	C	V	NV	DZ	OF	UF	NX	SAT	NT
CLR	U	U	U	U	U	U	U	U	U	U	-	Y
CLS	U	U	U	U	U	U	U	U	U	U	-	Y
CLZ	U	U	U	U	U	U	U	U	U	U	-	Y
CMP	U	Y	U	U	Y	U	U	U	U	U	-	Y
CMPXCHG	U	Y	U	U	Y	U	U	U	U	U	-	Y
CPUID	U	U	U	U	U	U	U	U	U	U	-	-
CTS	U	U	U	U	U	U	U	U	U	U	-	Y
CTZ	U	U	U	U	U	U	U	U	U	U	-	Y
DEC	U	Y	U	U	Y	U	U	U	U	U	Y	Y
DECN	U	U	U	U	U	U	U	U	U	U	Y	Y
DIVMODS	U	U	U	U	U	U	U	U	U	U	-	Y
DIVMODU	U	U	U	U	U	U	U	U	U	U	-	Y
DIVS	U	U	U	U	U	U	U	U	U	U	-	Y
DIVU	U	U	U	U	U	U	U	U	U	U	-	Y
DJcc	U	U	U	U	U	U	U	U	U	U	-	-
ENCINST	U*	U	U	U	U	U	U	U	U	U	-	Y
EXTSL	U	U	U	U	U	U	U	U	U	U	-	Y
EXTSQ	U	U	U	U	U	U	U	U	U	U	-	Y
EXTSW	U	U	U	U	U	U	U	U	U	U	-	Y
EXTZL	U	U	U	U	U	U	U	U	U	U	-	Y
EXTZQ	U	U	U	U	U	U	U	U	U	U	-	Y
EXTZW	U	U	U	U	U	U	U	U	U	U	-	Y
FABS	U	U	U	U	U	U	U	U	U	U	-	Y
FACOS	U	U	U	U	U	Y	U	U	Y	Y	-	-
FADD	U	U	U	U	U	Y	U	Y	Y	Y	-	Y
FASIN	U	U	U	U	U	Y	U	U	Y	Y	-	-
FATAN	U	U	U	U	U	Y	U	U	Y	Y	-	-
FATANH	U	U	U	U	U	Y	U	U	Y	Y	-	-
FBNDII	U	U	U	U	Y	U	U	U	U	U	-	Y
FBNDIX	U	U	U	U	Y	U	U	U	U	U	-	Y
FBNDXI	U	U	U	U	Y	U	U	U	U	U	-	Y
FBNDXX	U	U	U	U	Y	U	U	U	U	U	-	Y
FCEIL	U	U	U	U	U	Y	U	U	U	Y	-	Y
FCLASS	U	U	U	U	U	U	U	U	U	U	-	-
FCLR	U	U	U	U	U	U	U	U	U	U	-	Y
FCMP	U	U	U	U	U	Y	U	U	U	U	-	Y
FCOPYSIGN	U	U	U	U	U	U	U	U	U	U	-	-
FCOS	U	U	U	U	U	Y	U	Y	Y	Y	-	-
FCOSH	U	U	U	U	U	Y	U	Y	Y	Y	-	-
FCVT	U	U	U	U	U	Y	U	Y	Y	Y	-	-
FCVTU	U	U	U	U	U	Y	U	Y	Y	Y	-	-
FDIV	U	U	U	U	U	Y	U	Y	Y	Y	-	Y
FETCHADD	U	U	U	U	U	U	U	U	U	U	-	Y
FETCHAND	U	U	U	U	U	U	U	U	U	U	-	Y
FETCHOR	U	U	U	U	U	U	U	U	U	U	-	Y
FETCHSUB	U	U	U	U	U	U	U	U	U	U	-	Y
FETCHXOR	U	U	U	U	U	U	U	U	U	U	-	Y
FETOX	U	U	U	U	U	Y	U	Y	Y	Y	-	-
FETOXM1	U	U	U	U	U	Y	U	Y	Y	Y	-	-
FFLOOR	U	U	U	U	U	Y	U	U	U	Y	-	Y
FGETEXP	U	U	U	U	U	Y	U	U	U	U	-	Y
FGETMAN	U	U	U	U	U	Y	U	U	U	Y	-	Y
FINT	U	U	U	U	U	Y	U	U	U	Y	-	Y
FINTRZ	U	U	U	U	U	Y	U	U	U	Y	-	Y
FLOG10	U	U	U	U	U	Y	Y	U	Y	Y	-	-
FLOG2	U	U	U	U	U	Y	Y	U	Y	Y	-	-
FLOGN	U	U	U	U	U	Y	Y	U	Y	Y	-	-
FLOGNP1	U	U	U	U	U	Y	Y	U	Y	Y	-	-
FLSHDCACHE	P	U	U	U	U	U	U	U	U	U	-	Y
FMADD	U	U	U	U	U	Y	U	Y	Y	Y	-	Y
FMAX	U	U	U	U	U	Y	U	U	U	U	-	Y

Instr.	Priv	Z	N	C	V	NV	DZ	OF	UF	NX	SAT	NT
FMIN	U	U	U	U	U	Y	U	U	U	U	-	Y
FMOD	U	U	U	U	U	Y	U	U	U	U	-	Y
FMOV	U	U	U	U	U	U	U	U	U	U	-	Y
FMOVCR	U	U	U	U	U	U	U	U	U	U	-	-
FMOV _{cc}	U	U	U	U	U	U	U	U	U	U	-	Y
FMSUB	U	U	U	U	U	Y	U	Y	Y	Y	-	Y
FMUL	U	U	U	U	U	Y	U	Y	Y	Y	-	Y
FNEG	U	U	U	U	U	U	U	U	U	U	-	Y
FNMADD	U	U	U	U	U	Y	U	Y	Y	Y	-	Y
FNMSUB	U	U	U	U	U	Y	U	Y	Y	Y	-	Y
FPOPM	U	U	U	U	U	U	U	U	U	U	-	-
FPUSHM	U	U	U	U	U	U	U	U	U	U	-	-
FREM	U	U	U	U	U	Y	U	U	U	U	-	Y
FROUND	U	U	U	U	U	Y	U	U	U	Y	-	Y
FSCALE	U	U	U	U	U	Y	U	Y	Y	Y	-	Y
FSIN	U	U	U	U	U	Y	U	Y	Y	Y	-	-
FSINCOS	U	U	U	U	U	Y	U	Y	Y	Y	-	-
FSINH	U	U	U	U	U	Y	U	Y	Y	Y	-	-
FSQRT	U	U	U	U	U	Y	U	U	U	Y	-	Y
FSUB	U	U	U	U	U	Y	U	Y	Y	Y	-	Y
FTAN	U	U	U	U	U	Y	U	Y	Y	Y	-	-
FTANH	U	U	U	U	U	Y	U	Y	Y	Y	-	-
FTENTOX	U	U	U	U	U	Y	U	Y	Y	Y	-	-
FTEST	U	U	U	U	U	Y	U	U	U	U	-	Y
FTRUNC	U	U	U	U	U	Y	U	U	U	Y	-	Y
FTWOTOX	U	U	U	U	U	Y	U	Y	Y	Y	-	-
FXCHG	U	U	U	U	U	U	U	U	U	U	-	-
GETDB	U	U	U	U	U	U	U	U	U	U	-	-
HALT	S	U	U	U	U	U	U	U	U	U	-	-
IJ _{cc}	U	U	U	U	U	U	U	U	U	U	-	Y
ILLEGAL	U	U	U	U	U	U	U	U	U	U	-	-
INC	U	Y	U	U	Y	U	U	U	U	U	Y	Y
INCN	U	U	U	U	U	U	U	U	U	U	Y	Y
INVASID	S	U	U	U	U	U	U	U	U	U	-	-
INVDCACHE	P	U	U	U	U	U	U	U	U	U	-	Y
INVICACHE	P	U	U	U	U	U	U	U	U	U	-	Y
INVPAGE	S	U	U	U	U	U	U	U	U	U	-	Y
INVTLB	S	U	U	U	U	U	U	U	U	U	-	-
IRET	S	U	U	U	U	U	U	U	U	U	-	-
JMP	U	U	U	U	U	U	U	U	U	U	-	-
J _{cc}	U	U	U	U	U	U	U	U	U	U	-	-
LCALL	U	U	U	U	U	U	U	U	U	U	-	-
LEA	U	U	U	U	U	U	U	U	U	U	-	-
LJMP	U	U	U	U	U	U	U	U	U	U	-	-
LRET	U	U	U	U	U	U	U	U	U	U	-	-
MADD	U	U	U	U	U	U	U	U	U	U	-	Y
MAXS	U	Y	Y	U	U	U	U	U	U	U	Y	Y
MAXU	U	Y	Y	U	U	U	U	U	U	U	Y	Y
MINS	U	Y	Y	U	U	U	U	U	U	U	Y	Y
MINU	U	Y	Y	U	U	U	U	U	U	U	Y	Y
MODS	U	U	U	U	U	U	U	U	U	U	-	Y
MODU	U	U	U	U	U	U	U	U	U	U	-	Y
MOV	U	U	U	U	U	U	U	U	U	U	-	Y
MOVSETAD	U	U	U	U	U	U	U	U	U	U	-	-
MOVSETDA	U	U	U	U	U	U	U	U	U	U	-	-
MOVSETDD	U	U	U	U	U	U	U	U	U	U	-	-
MOV _{cc}	U	U	U	U	U	U	U	U	U	U	-	Y
MSUB	U	U	U	U	U	U	U	U	U	U	-	Y
MULHS	U	U	U	U	U	U	U	U	U	U	-	Y
MULHSU	U	U	U	U	U	U	U	U	U	U	-	Y
MULHU	U	U	U	U	U	U	U	U	U	U	-	Y

Instr.	Priv	Z	N	C	V	NV	DZ	OF	UF	NX	SAT	NT
MULS	U	U	U	U	U	U	U	U	U	U	-	Y
MULU	U	U	U	U	U	U	U	U	U	U	-	Y
NEG	U	Y	U	U	Y	U	U	U	U	U	Y	Y
NOP	U	U	U	U	U	U	U	U	U	U	-	-
NOT	U	Y	U	U	Y	U	U	U	U	U	-	Y
OR	U	Y	U	U	Y	U	U	U	U	U	-	Y
PARITY	U	U	U	U	U	U	U	U	U	U	-	Y
POP	U	U	U	U	U	U	U	U	U	U	-	-
POPCNT	U	U	U	U	U	U	U	U	U	U	-	Y
POPM	U	U	U	U	U	U	U	U	U	U	-	-
PREFETCH	U	U	U	U	U	U	U	U	U	U	-	Y
PTATTR	S	U	U	U	U	U	U	U	U	U	-	Y
PTQUERY	S	U	U	U	U	U	U	U	U	U	-	Y
PUSH	U	U	U	U	U	U	U	U	U	U	-	-
PUSHM	U	U	U	U	U	U	U	U	U	U	-	-
RCL	U	U	U	U	U	U	U	U	U	U	-	Y
RCR	U	U	U	U	U	U	U	U	U	U	-	Y
RDCR	S	U	U	U	U	U	U	U	U	U	-	-
RDFLAGS	U	U	U	U	U	U	U	U	U	U	-	-
RDFSTATUS	U	U	U	U	U	U	U	U	U	U	-	-
RDPMC	P	U	U	U	U	U	U	U	U	U	-	-
RDPTC	P	U	U	U	U	U	U	U	U	U	-	-
RDSEG	P	U	U	U	U	U	U	U	U	U	-	-
RDSTATUS	U	U	U	U	U	U	U	U	U	U	-	-
RESET	S	U	U	U	U	U	U	U	U	U	-	-
RESTORE	U	U	U	U	U	U	U	U	U	U	-	Y
RET	U	U	U	U	U	U	U	U	U	U	-	-
REVBIT	U	U	U	U	U	U	U	U	U	U	-	Y
REVBYTE	U	U	U	U	U	U	U	U	U	U	-	Y
RFENCE	U	U	U	U	U	U	U	U	U	U	-	-
ROL	U	U	U	U	U	U	U	U	U	U	-	Y
ROR	U	U	U	U	U	U	U	U	U	U	-	Y
SAR	U	U	U	U	U	U	U	U	U	U	-	Y
SAVE	U	U	U	U	U	U	U	U	U	U	-	Y
SBB	U	Y	Y	Y	Y	U	U	U	U	U	Y	Y
SEGLEA	U	U	U	U	Y	U	U	U	U	U	-	-
SELDB	U	U	U	U	U	U	U	U	U	U	-	-
SETcc	U	U	U	U	U	U	U	U	U	U	-	Y
SHL	U	U	U	U	U	U	U	U	U	U	-	Y
SHR	U	U	U	U	U	U	U	U	U	U	-	Y
SUB	U	Y	U	U	Y	U	U	U	U	U	Y	Y
SUM	U	U	U	U	U	U	U	U	U	U	-	-
SWPT	S	U	U	U	U	U	U	U	U	U	-	-
SWPTA	S	U	U	U	U	U	U	U	U	U	-	-
SYNCCACHE	P	U	U	U	U	U	U	U	U	U	-	Y
SYSCALL	U	U	U	U	U	U	U	U	U	U	-	-
SYSRET	S	U	U	U	U	U	U	U	U	U	-	-
TEST	U	Y	U	U	Y	U	U	U	U	U	-	Y
TESTCANON	U	U	U	U	Y	U	U	U	U	U	-	-
TRACE	U	U	U	U	U	U	U	U	U	U	-	-
TRAP	U	U	U	U	U	U	U	U	U	U	-	-
TRAPcc	U	U	U	U	U	U	U	U	U	U	-	-
VTOP	S	U	U	U	U	U	U	U	U	U	-	Y
WAIT	P	U	U	U	U	U	U	U	U	U	-	-
WFENCE	U	U	U	U	U	U	U	U	U	U	-	-
WRBKDCACHE	P	U	U	U	U	U	U	U	U	U	-	Y
WRCR	S	U	U	U	U	U	U	U	U	U	-	-
WRFLAGS	U	U	U	U	U	U	U	U	U	U	-	-
WRFSTATUS	U	U	U	U	U	U	U	U	U	U	-	-
WRSEG	P	U	U	U	U	U	U	U	U	U	-	-
WRSTATUS	S	U	U	U	U	U	U	U	U	U	-	-

Instr.	Priv	Z	N	C	V	NV	DZ	OF	UF	NX	SAT	NT
XCHG	U	U	U	U	U	U	U	U	U	U	-	Y
XCHGSETAD	U	U	U	U	U	U	U	U	U	U	-	-
XCHGSETDA	U	U	U	U	U	U	U	U	U	U	-	-
XCHGSETDD	U	U	U	U	U	U	U	U	U	U	-	-
XOR	U	Y	U	U	Y	U	U	U	U	U	-	Y
YIELD	U	U	U	U	U	U	U	U	U	U	-	-

Table 18-3. Instruction Attribute Matrix

The grouping is semantic rather than an opcode-space promise. Related instructions are preferably kept near each other by the allocator, but the authoritative encodings are still the instruction forms shown in the generated instruction description pages.

19 CONDITION CODE COMPUTATION

Integer condition codes are held in the FLAGS register. Most integer operations either update the Z, N, C, and V bits from a temporary result, update only a subset, or leave FLAGS unchanged. Unless an instruction description says otherwise, unmentioned FLAGS bits are preserved.

19.1 Flag Meanings

Bit	Name	Meaning
Z	zero	Set when the result value is zero.
N	negative	Set from the most significant bit of the result at the operand size.
C	carry/borrow	Set on unsigned carry out for addition, or unsigned borrow for subtraction.
V	overflow	Set on signed overflow, or by instructions that explicitly report exceptional conditions through V.

Table 19-1. Integer Condition-Code Bits

19.2 Common Integer Rules

For an operand width of n bits, arithmetic results are evaluated modulo 2^n unless the instruction explicitly defines a wider intermediate. The sign bit is bit $n - 1$. The formulas below describe the ordinary integer ALU rules; individual instruction pages remain authoritative for special cases.

Operation Class	FLAGS Result
ADD, ADC	Z and N come from the stored result. C is the unsigned carry out. V is set when the source and destination signs match but the result sign differs.
SUB, SBB, CMP	Z and N come from the subtraction result. C is the unsigned borrow. V is set when the minuend and subtrahend signs differ and the result sign differs from the minuend. CMP does not store the temporary result. Address-destination ADD/SUB forms do not update FLAGS.
AND, OR, XOR, TEST	Z and N come from the logical result. C and V are cleared. TEST does not store the temporary result.
INC, DEC	Use the ADD or SUB rule with an implicit operand of one when the instruction form updates FLAGS. INCN and DECN leave FLAGS unchanged.
NEG	Computes $0 - dst$; FLAGS follow the subtraction rule.
ABS	Updates FLAGS from the absolute-value result and reports overflow if the most negative signed value cannot be represented.
CLR	Writes zero and leaves FLAGS unchanged in the current Bedrock definition.
Bounds checks	Set V when the value is outside the encoded interval; Z, N, and C are unchanged.

Table 19-2. Common Integer Flag Computation

19.3 Shift, Rotate, and Bit Operations

Shift and rotate counts are read as unsigned values. Let W be the selected operand width in bits. A zero shift count leaves the destination and FLAGS unchanged. For rotates, the architectural count is first reduced to an effective count; an effective count of zero also leaves the destination and FLAGS unchanged.

Operation	Count Rule
SHL, SHR, SAR	The count is not reduced modulo W . Counts greater than W are well-defined: SHL and SHR produce zero; SAR produces sign fill.
ROL, ROR	The effective count is $count \bmod W$. Effective count zero leaves the destination and FLAGS unchanged.
RCL, RCR	The carry bit is part of the rotate ring. The effective count is $count \bmod (W + 1)$. Effective count zero leaves the destination and FLAGS unchanged.

Table 19-3. Shift and Rotate Count Rules

Operation	FLAGS Result for Nonzero Effective Count
SHL	Z and N come from the result. C is the last bit shifted out; if the count is greater than W , C is zero. V is set when the signed source value multiplied by 2^{count} is not representable in the selected width.
SHR	Z and N come from the result. C is the last bit shifted out; if the count is greater than W , C is zero. V is cleared.
SAR	Z and N come from the result. C is the last bit shifted out; if the count is greater than W , C is the original sign bit. V is cleared.
ROL	Z and N come from the result. C is the least significant result bit. V is cleared.
ROR	Z and N come from the result. C is the most significant result bit. V is cleared.
RCL, RCR	Z and N come from the result. C is the carry slot after the through-carry rotate. V is cleared.

Table 19-4. Shift and Rotate Flag Computation

Bit test/set/clear/change instructions observe the selected bit before modifying it. They set Z when that old bit was zero and leave N, C, and V unchanged. BTEST does not modify its destination; BSET, BCLR, and BCHG apply the requested write after recording the old-bit result in Z.

19.4 Conditional Consumers

Conditional control and conditional move instructions read FLAGS but do not modify them. The condition-code table defines how each condition mnemonic interprets Z, N, C, and V. Unconditional aliases such as JMP and TRAP correspond to the true condition and use the unconditional spelling as the canonical disassembly.

19.5 Floating-Point Interactions

Floating-point comparison and test instructions update the integer FLAGS so that ordinary conditional branches can consume the result. An unordered floating-point comparison sets V. Floating-point exception/status bits are tracked separately in FFLAGS and are described on the floating-point instruction pages.

20 GENERAL INSTRUCTIONS SUMMARY

Mnemonic	Summary	Forms
ABS	Absolute Value	2
ADC	Add With Carry	3
ADD	Add	6
AFENCE	Address Fence	1
AND	Bitwise And	5
BCHG	Bit Change	3
BCLR	Bit Clear	3
BKPT	Breakpoint	1
BNDSCII	Signed Bounds Check Inclusive-Inclusive	1
BNDSEX	Signed Bounds Check Inclusive-Exclusive	1
BNDSEI	Signed Bounds Check Exclusive-Inclusive	1
BNDSEX	Signed Bounds Check Exclusive-Exclusive	1
BNDUII	Unsigned Bounds Check Inclusive-Inclusive	1
BNDUIX	Unsigned Bounds Check Inclusive-Exclusive	1
BNDUXI	Unsigned Bounds Check Exclusive-Inclusive	1
BNDUXX	Unsigned Bounds Check Exclusive-Exclusive	1
BSET	Bit Set	3
BTEST	Bit Test	3
CALL	Call	4
CLMUL	Carryless Multiply	2
CLMULH	Carryless Multiply High	2
CLR	Clear	3
CLS	Count Leading Set Bits	1
CLZ	Count Leading Zeros	1
CMP	Compare	7
CMPXCHG	Compare And Exchange	1
CPUID	CPU Identification	1
CTS	Count Trailing Set Bits	1
CTZ	Count Trailing Zeros	1
DEC	Decrement	2
DECN	Decrement Without Flags	2
DIVMODS	Signed Divide With Remainder	1
DIVMODU	Unsigned Divide With Remainder	1
DIVS	Signed Divide	2
DIVU	Unsigned Divide	2
DJcc	Decrement And Jump Conditional	1
EXTSL	Sign Extend To Long	4
EXTSQ	Sign Extend To Quad	4
EXTSW	Sign Extend To Word	4
EXTZL	Zero Extend To Long	4
EXTZQ	Zero Extend To Quad	4
EXTZW	Zero Extend To Word	1
FETCHADD	Atomic Fetch Add	1
FETCHAND	Atomic Fetch And	1
FETCHOR	Atomic Fetch Or	1
FETCHSUB	Atomic Fetch Subtract	1
FETCHXOR	Atomic Fetch Xor	1
FLSHDCACHE	Flush Data Cache	1
GETDB	Get Data Register Bank	1

Mnemonic	Summary	Forms
HALT	Halt	1
IJcc	Increment And Jump Conditional	1
ILLEGAL	Illegal Instruction	1
INC	Increment	2
INCN	Increment Without Flags	2
INVASID	Invalidate TLB By ASID	1
INVDCACHE	Invalidate Data Cache	1
INVICACHE	Invalidate Instruction Cache	1
INVPAGE	Invalidate TLB Page	1
INVTLB	Invalidate TLB	1
IRET	Interrupt Return	1
JMP	Jump	2
Jcc	Jump Conditional	2
LCALL	Long Call	1
LEA	Load Effective Address	1
LJMP	Long Jump	1
LRET	Long Return	1
MADD	Multiply Add	1
MAXS	Signed Maximum	2
MAXU	Unsigned Maximum	2
MINS	Signed Minimum	2
MINU	Unsigned Minimum	2
MODS	Signed Modulo	2
MODU	Unsigned Modulo	2
MOV	Move	6
MOVSETAD	Move D Registers To A Registers	2
MOVSETDA	Move A Registers To D Registers	2
MOVSETDD	Move D Register Set Between Banks	1
MOVcc	Move Conditional	4
MSUB	Multiply Subtract	1
MULHS	Signed Multiply High	2
MULHSU	Signed By Unsigned Multiply High	2
MULHU	Unsigned Multiply High	2
MULS	Signed Multiply	2
MULU	Unsigned Multiply	2
NEG	Negate	2
NOP	No Operation	1
NOT	Bitwise Not	2
OR	Bitwise Or	5
PARITY	Parity Test	1
POP	Pop	2
POPCNT	Population Count	1
POPM	Pop Multiple	1
PREFETCH	Prefetch	1
PTATTR	Page Table Attributes	1
PTQUERY	Page Table Query	1
PUSH	Push	2
PUSHM	Push Multiple	1
RCL	Rotate Through Carry Left	3
RCR	Rotate Through Carry Right	3
RDCR	Read Control Register	1

Mnemonic	Summary	Forms
RDFLAGS	Read Flags	1
RDFSTATUS	Read Floating-Point Status	1
RDPMC	Read Performance Counter	1
RDPTC	Read Page Table Counter	1
RDSEG	Read Segment Register	1
RDSTATUS	Read Status	1
RESET	Reset	1
RESTORE	Restore Processor State	1
RET	Return	1
REVBIT	Reverse Bits	3
REVBYTE	Reverse Bytes	3
RFENCE	Read Fence	1
ROL	Rotate Left	3
ROR	Rotate Right	3
SAR	Shift Arithmetic Right	3
SAVE	Save Processor State	1
SBB	Subtract With Borrow	3
SEGLEA	Segment Load Effective Address	1
SELDB	Select Data Register Bank	2
SETcc	Set Conditional	1
SHL	Shift Left	3
SHR	Shift Right	3
SUB	Subtract	6
SUM	Register Set Sum	2
SWPT	Switch Page Table	1
SWPTA	Switch Page Table With Context	1
SYNCCACHE	Synchronize Caches	1
SYSCALL	System Call	1
SYSRET	System Return	1
TEST	Test	5
TESTCANON	Test Canonical Address	1
TRACE	Trace Marker	1
TRAP	Trap	1
TRAPcc	Trap Conditional	1
VTOP	Virtual To Physical Query	1
WAIT	Wait	1
WFENCE	Write Fence	1
WRBKDCACHE	Write Back Data Cache	1
WRCR	Write Control Register	1
WRFLAGS	Write Flags	1
WRFSTATUS	Write Floating-Point Status	1
WRSEG	Write Segment Register	1
WRSTATUS	Write Status	1
XCHG	Exchange	2
XCHGSETAD	Exchange D And A Register Set	2
XCHGSETDA	Exchange A And D Register Set	2
XCHGSETDD	Exchange D Register Sets Between Banks	1
XOR	Bitwise Xor	5
YIELD	Yield	1

Table 20-1. General Instructions Summary

21.1 Reading an Instruction Description

The instruction pages below use a common layout. The shortened example shows how to read the fields before the alphabetical instruction descriptions begin. It is a reading guide; the generated form tables and diagrams on each instruction page are the normative encoding reference.

ADD**Add****ADD**

Summary:	Adds the source operand to the destination operand.
Operation:	Read the source and destination values, add them, write the result, and update FLAGS according to the form.
Assembler Syntax:	ADD.<L/Q> Dn, Dn ADD.<B/W/L/Q> <ea>, Dn
Attributes:	Size, word length, privilege, prefix availability, and flag behavior for the listed forms.
Description:	Longer prose records form-specific behavior and alias notes that do not fit in the summary line.
Condition Codes:	Z N C V A star marks a flag that may be updated; a dash marks an unchanged flag. * * * *

Instruction Forms:

ADD.X(z:L/Q) Dn(d), Dn(D)

Form	Dn(d), Dn(D)
Encoding	compact primary or extended opcode form, depending on the generated allocation
Words	minimum and maximum instruction length in 16-bit words
Privilege	user/supervisor policy for this form
Operands	source operand is d; destination operand is D
Notes	payload words, EA words, padding, or descriptor requirements

d	symbolic source-register field used by this form.
D	symbolic destination-register field used by this form.
z	symbolic size selector used by this form.

- The page header repeats the mnemonic on both sides and places the instruction title in the center.
- The assembler syntax block shows programmer-facing spellings. Size suffixes appear after the dot.
- The instruction forms are grouped by allocated encoding form. Each form begins with the full assembler syntax, then lists the operand shape, length range, privilege rule, and notes.
- Field names in parentheses, such as d, D, e, s, or z, are symbolic encoding fields. The diagram and field list under each form map those symbols to exact bit positions.
- Opcode roots and ranges are allocation details; the visible contract is mnemonic, operands, sizes, privilege, flags, and EA legality.

ABS

Absolute Value

ABS

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the absolute value operation.

if (((dst_v >> 63) & 1) == 0) goto <abs_done>;
dst_v = 0 - dst_v;
<abs_done>
ZF = dst_v == 0;
NF = dst_v s< 0;
CF = 0;
VF = 0;

ABS.<B/W/L/Q> Dn
ABS.<B/W/L/Q> <ea>

Size = (Byte, Word, Long, Quad)
Words = (1-8, 2-8)
Privilege = unprivileged
Flags = Z: result == 0; N: most significant bit(result); C: source operand is negative; V: source operand is the most negative signed value

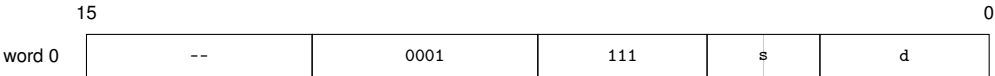
Z N C V
* - - *

Z: result == 0; N: most significant bit(result); C: source operand is negative; V: source operand is the most negative signed value

Instruction Forms:

ABS.X(s:B/W/L/Q) Dn(d)

Form	Dn(d)
Encoding	primary opcode range 0x1e0 to 0x1ff
Words	1-8
Privilege	unprivileged
Operands	destination operand is data register number

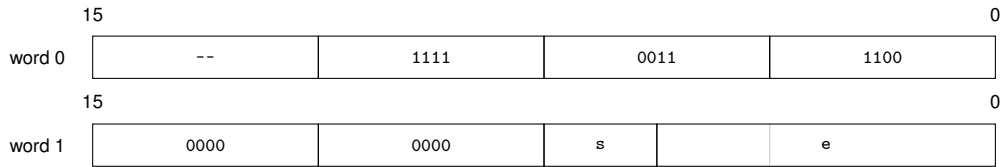


Instruction format for ABS.X(s:B/W/L/Q) Dn(d)

- d
- data register number for the destination operand (primary word bits 2:0).
- s
- size selector for the operand size (primary word bits 4:3).

ABS.X(s:B/W/L/Q) <ea(e)>

Form	<ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	destination operand is compact effective-address field



Instruction format for ABS.X(s:B/W/L/Q) <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 7:6).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

ADC

Add With Carry

ADC

Summary: Performs the add with carry operation.

Operation: `carry_v = CF;`
`tmp_v = dst_v + src_v;`
`result_v = tmp_v + carry_v;`
`dst_v = result_v;`
`ZF = result_v == 0;`
`NF = result_v s< 0;`
`CF = carry(dst_old_v, src_v) || carry(tmp_v, carry_v);`
`VF = scarry(dst_old_v, src_v) || scarry(tmp_v, carry_v);`

Assembler Syntax: `ADC.<B/W/L/Q> Dn, <ea>`
`ADC.<B/W/L/Q> <ea>, Dn`
`ADC.<B/W/L/Q> <imm6>, <ea>`

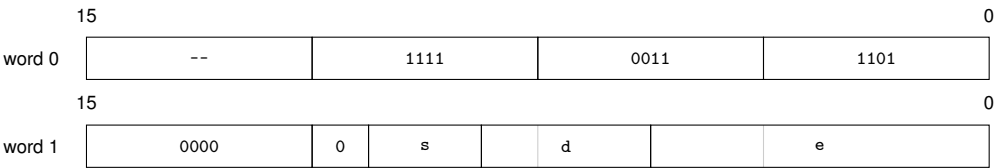
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = update Z/N/C/V

Condition Codes: **Z N C V**
* * * *
update Z/N/C/V

Instruction Forms:

ADC.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form	Dn(d), <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is data register number; destination operand is compact effective-address field

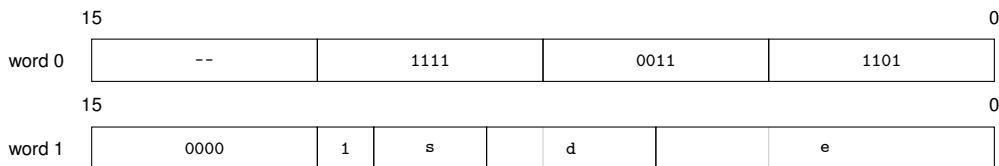


Instruction format for ADC.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the source operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

ADC.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form <ea(e)>, Dn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is data register number

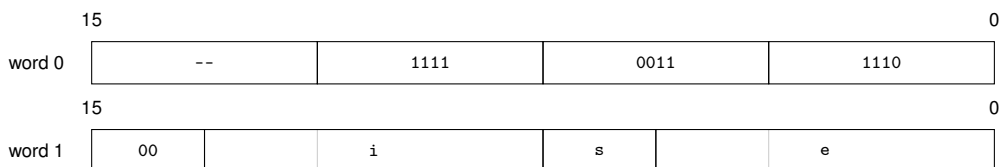


Instruction format for ADC.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

ADC.X(s:B/W/L/Q) <imm(i)>, <ea(e)>

Form <imm(i)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands imm is unsigned 6-bit immediate literal; destination operand is compact effective-address field



Instruction format for ADC.X(s:B/W/L/Q) <imm(i)>, <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 7:6).
- i unsigned 6-bit immediate literal for the imm (extended opcode word bits 13:8). The encoded value is zero-extended before use for integer forms with an explicit B/W/L/Q operation size.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

ADD**Add****ADD**

Summary:	Adds the source operand to the destination operand.
Operation:	<pre> result_v = dst_v + src_v; dst_v = result_v; ZF = result_v == 0; NF = result_v < 0; CF = carry(dst_old_v, src_v); VF = scarry(dst_old_v, src_v); </pre>
Assembler Syntax:	<pre> ADD.<L/Q> Dn, Dn ADD.<W/L> <imm>, Dn ADD <ea>, An ADD.<B/W/L/Q> Dn, <ea> ADD.<B/W/L/Q> <ea>, Dn ADD.<B/W/L/Q> <imm6>, <ea> </pre>
Attributes:	Size = (Byte, Word, Long, Quad, Long, Quad, Word, Long) Words = (1-8, 2-8) Privilege = unprivileged Flags = Z: result == 0; N: most significant bit(result); C: unsigned carry out; V: signed overflow
Condition Codes:	Z N C V * _ _ * Z: result == 0; N: most significant bit(result); C: unsigned carry out; V: signed overflow
Instruction Forms:	

ADD.X(z:L/Q) Dn(d), Dn(D)

Form Dn(d), Dn(D)
Encoding primary opcode range 0x080 to 0x0ff
Words 1-8
Privilege unprivileged
Operands source operand is data register number; destination operand is data register number



Instruction format for ADD.X(z:L/Q) Dn(d), Dn(D)

d data register number for the source operand (primary word bits 2:0).
D data register number for the destination operand (primary word bits 5:3).
z size selector for the operand size (primary word bits 6).

ADD.X(z:W/L) imm, Dn(d)

Form imm, Dn(d)
Encoding primary opcode range 0x400 to 0x7c0
Words 2-8
Privilege unprivileged
Operands imm; destination operand is data register number

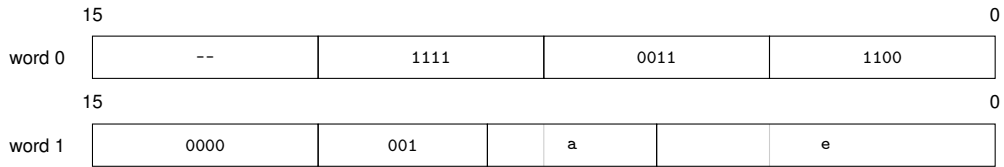


Instruction format for ADD.X(z:W/L) imm, Dn(d)

d data register number for the destination operand (primary word bits 8:6).
z size selector for the operand size (primary word bits 9).

ADD <ea(e)>, An(a)

Form <ea(e)>, An(a)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is address register number

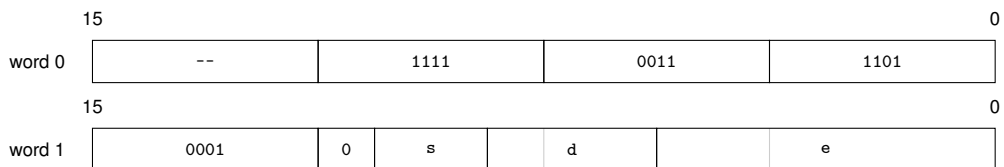


Instruction format for ADD <ea(e)>, An(a)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- a address register number for the destination operand (extended opcode word bits 8:6).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

ADD.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

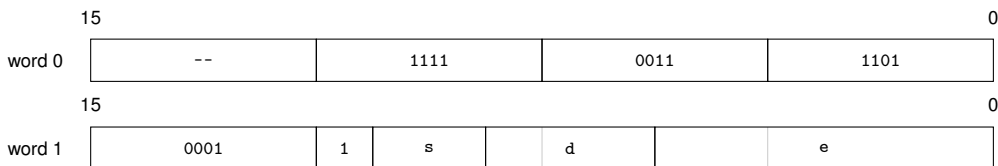


Instruction format for ADD.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the source operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

ADD.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form <ea(e)>, Dn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is data register number

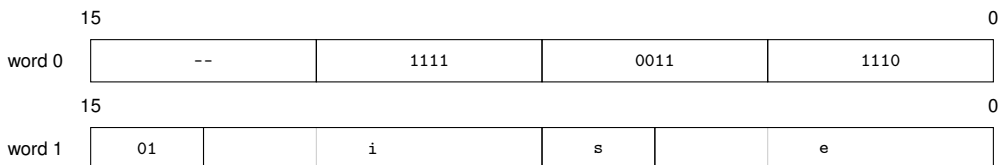


Instruction format for ADD.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

ADD.X(s:B/W/L/Q) <imm(i)>, <ea(e)>

Form <imm(i)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands imm is unsigned 6-bit immediate literal; destination operand is compact effective-address field



Instruction format for ADD.X(s:B/W/L/Q) <imm(i)>, <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 7:6).
- i unsigned 6-bit immediate literal for the imm (extended opcode word bits 13:8). The encoded value is zero-extended before use for integer forms with an explicit B/W/L/Q operation size.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

AFENCE

Address Fence

AFENCE

Summary:

Order prior memory operations before later memory operations.

Operation:

arch_fence_address();

Assembler Syntax:

AFENCE

Attributes:

Words = (1-8)
Privilege = unprivileged
Flags = unchanged

Condition Codes:

Z N C V
- - - -
unchanged

Instruction Forms:

AFENCE

Form

AFENCE

Encoding

primary opcode 0x005

Words

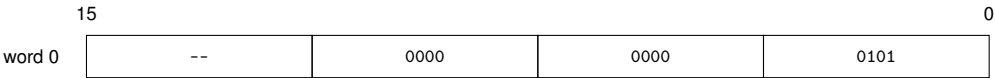
1-8

Privilege

unprivileged

Operands

no explicit operands



Instruction format for AFENCE

No explicit operand fields are encoded in this form.

AND**Bitwise And****AND**

Summary: Computes the bitwise AND of the source and destination operands.

Operation: `result_v = dst_v & src_v;`
`dst_v = result_v;`
`ZF = result_v == 0;`
`NF = result_v < 0;`
`CF = 0;`
`VF = 0;`

Assembler Syntax: `AND.<W/L> <imm>, Dn`
`AND.<L/Q> Dn, Dn`
`AND.<B/W/L/Q> <ea>, Dn`
`AND.<B/W/L/Q> Dn, <ea>`
`AND.<B/W/L/Q> <imm6>, <ea>`

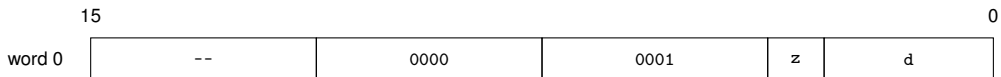
Attributes: Size = (Byte, Word, Long, Quad, Long, Quad, Word, Long)
Words = (1-8, 2-8)
Privilege = unprivileged
Flags = Z: result == 0; N: most significant bit(result); C: 0; V: 0

Condition Codes: **Z N C V**
* - - *
Z: result == 0; N: most significant bit(result); C: 0; V: 0

Instruction Forms:

`AND.X(z:W/L) imm, Dn(d)`

Form imm, Dn(d)
Encoding primary opcode range 0x010 to 0x01f
Words 2-8
Privilege unprivileged
Operands imm; destination operand is data register number



Instruction format for AND.X(z:W/L) imm, Dn(d)

d data register number for the destination operand (primary word bits 2:0).
z size selector for the operand size (primary word bits 3).

AND.X(z:L/Q) Dn(d), Dn(D)

Form	Dn(d), Dn(D)
Encoding	primary opcode range 0x300 to 0x37f
Words	1-8
Privilege	unprivileged
Operands	source operand is data register number; destination operand is data register number

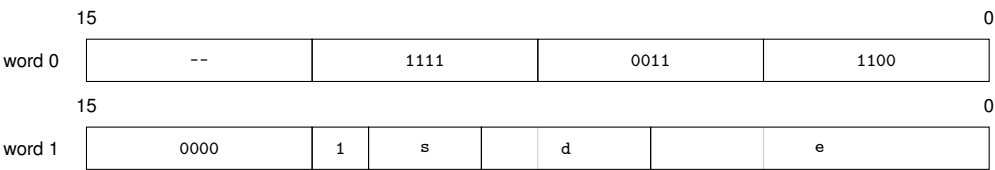


Instruction format for AND.X(z:L/Q) Dn(d), Dn(D)

- d data register number for the source operand (primary word bits 2:0).
- D data register number for the destination operand (primary word bits 5:3).
- z size selector for the operand size (primary word bits 6).

AND.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number

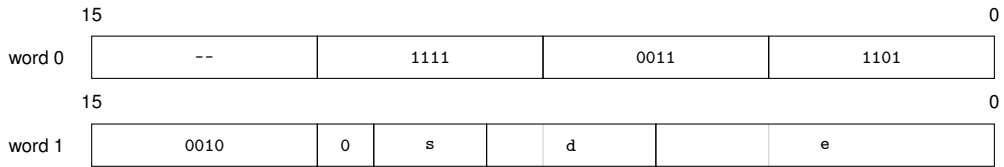


Instruction format for AND.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the destination operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

AND.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

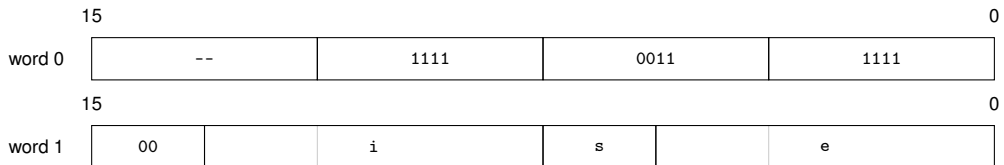


Instruction format for AND.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
d data register number for the source operand (extended opcode word bits 8:6).
s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

AND.X(s:B/W/L/Q) <imm(i)>, <ea(e)>

Form <imm(i)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands imm is unsigned 6-bit immediate literal; destination operand is compact effective-address field



Instruction format for AND.X(s:B/W/L/Q) <imm(i)>, <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
s size selector for the operand size (extended opcode word bits 7:6).
i unsigned 6-bit immediate literal for the imm (extended opcode word bits 13:8). The encoded value is zero-extended before use for integer forms with an explicit B/W/L/Q operation size.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

BCHG

Bit Change

BCHG

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the bit change operation.

tmp_v = 1 << (bit_index_v & 63);

dst_v = dst_v ^ tmp_v;

ZF = dst_v == 0;

NF = dst_v s< 0;

CF = 0;

VF = 0;

BCHG.<B/W/L/Q> Dn, Dn

BCHG.<B/W/L/Q> Dn, <ea>

BCHG.<B/W/L/Q> <bit_index:imm6>, <ea>

Size = (Byte, Word, Long, Quad)

Words = (2-8)

Privilege = unprivileged

Flags = set Z when the old bit was zero; preserve N/C/V

Z N C V

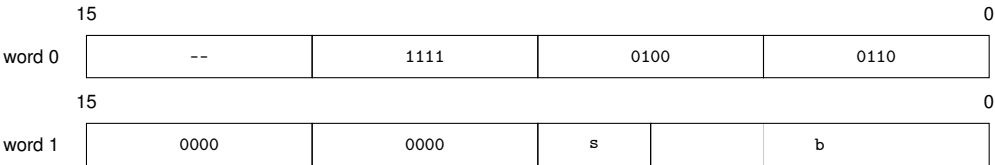
* - - -

set Z when the old bit was zero; preserve N/C/V

Instruction Forms:

BCHG.X(s:B/W/L/Q) Dn(b), Dn(b)

Form	Dn(b), Dn(b)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	bit index is data register number; destination operand is data register number

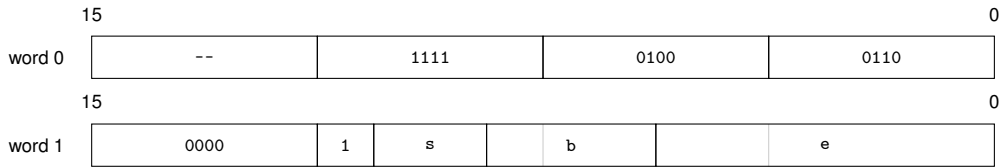


Instruction format for BCHG.X(s:B/W/L/Q) Dn(b), Dn(b)

- b data register number for the bit index (extended opcode word bits 2:0).
- b data register number for the bit index (extended opcode word bits 5:3).
- s size selector for the operand size (extended opcode word bits 7:6).

BCHG.X(s:B/W/L/Q) Dn(b), <ea(e)>

Form Dn(b), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands bit index is data register number; destination operand is compact effective-address field

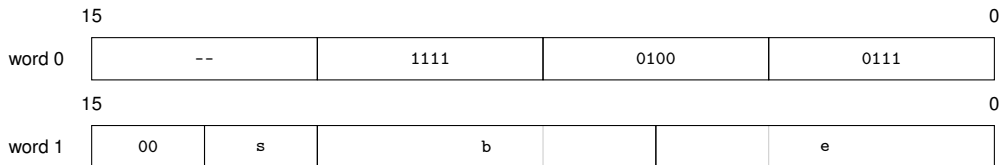


Instruction format for BCHG.X(s:B/W/L/Q) Dn(b), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - b data register number for the bit index (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

BCHG.X(s:B/W/L/Q) <bit_index(b)>, <ea(e)>

Form <bit_index(b)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands bit index is selector imm6; destination operand is compact effective-address field



Instruction format for BCHG.X(s:B/W/L/Q) <bit_index(b)>, <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - b selector6 for the bit index (extended opcode word bits 11:6).
 - s size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

BCLR

Bit Clear

BCLR

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the bit clear operation.

tmp_v = 1 << (bit_index_v & 63);

dst_v = dst_v & ~tmp_v;

ZF = dst_v == 0;

NF = dst_v s< 0;

CF = 0;

VF = 0;

BCLR.<B/W/L/Q> Dn, Dn

BCLR.<B/W/L/Q> Dn, <ea>

BCLR.<B/W/L/Q> <bit_index:imm6>, <ea>

Size = (Byte, Word, Long, Quad)

Words = (2-8)

Privilege = unprivileged

Flags = set Z when the old bit was zero; preserve N/C/V

Z N C V

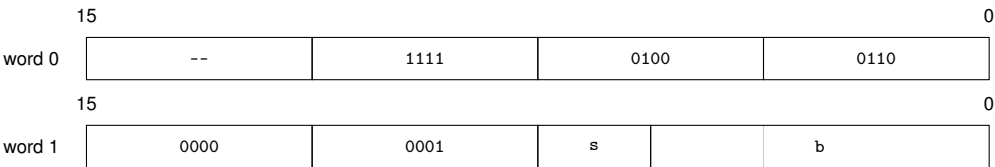
* - - -

set Z when the old bit was zero; preserve N/C/V

Instruction Forms:

BCLR.X(s:B/W/L/Q) Dn(b), Dn(b)

Form	Dn(b), Dn(b)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	bit index is data register number; destination operand is data register number

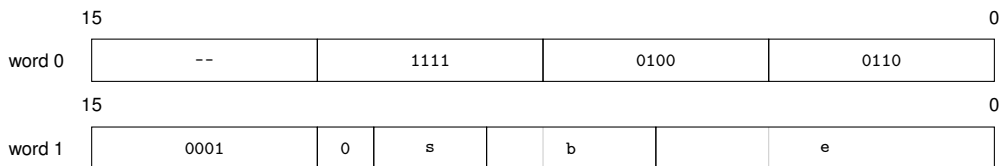


Instruction format for BCLR.X(s:B/W/L/Q) Dn(b), Dn(b)

- b data register number for the bit index (extended opcode word bits 2:0).
- b data register number for the bit index (extended opcode word bits 5:3).
- s size selector for the operand size (extended opcode word bits 7:6).

BCLR.X(s:B/W/L/Q) Dn(b), <ea(e)>

Form Dn(b), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands bit index is data register number; destination operand is compact effective-address field

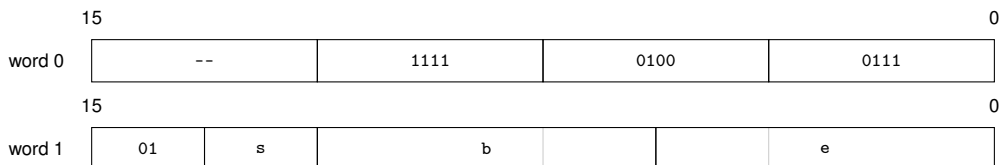


Instruction format for BCLR.X(s:B/W/L/Q) Dn(b), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - b data register number for the bit index (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

BCLR.X(s:B/W/L/Q) <bit_index(b)>, <ea(e)>

Form <bit_index(b)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands bit index is selector imm6; destination operand is compact effective-address field



Instruction format for BCLR.X(s:B/W/L/Q) <bit_index(b)>, <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - b selector6 for the bit index (extended opcode word bits 11:6).
 - s size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

BKPT

Breakpoint

BKPT

Summary:

Raise breakpoint/debug exception.

Operation:

arch_raise_breakpoint();

Assembler Syntax:

BKPT

Attributes:

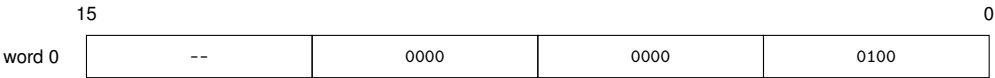
Words = (1-8)
Privilege = unprivileged
Flags = unchanged

Condition Codes:

Z N C V
- - - -
unchanged

Instruction Forms:

BKPT	
Form	BKPT
Encoding	primary opcode 0x004
Words	1-8
Privilege	unprivileged
Operands	no explicit operands



Instruction format for BKPT

No explicit operand fields are encoded in this form.

BNDSCII**Signed Bounds Check Inclusive-Inclusive****BNDSCII**

Summary: Checks signed value membership in the inclusive-inclusive interval [lo, hi].

Operation: ZF = 0;
 NF = 0;
 CF = 0;
 VF = 0;
 if (value_v < lo_v) goto <bnduii_bad>;
 if (value_v > hi_v) goto <bnduii_bad>;
 goto <bnduii_done>;
 <bnduii_bad>
 VF = 1;
 <bnduii_done>

Assembler Syntax: BNDSCII.<B/W/L/Q> Dn, <ea>, Dn

Attributes: Size = (Byte, Word, Long, Quad)
 Words = (2-8)
 Privilege = unprivileged
 Flags = V: set when the value is out of bounds

Description: BNDSCII treats both bounds as inclusive. It sets V when signed(value) is outside [signed(lo), signed(hi)]; Z, N, and C are unchanged.

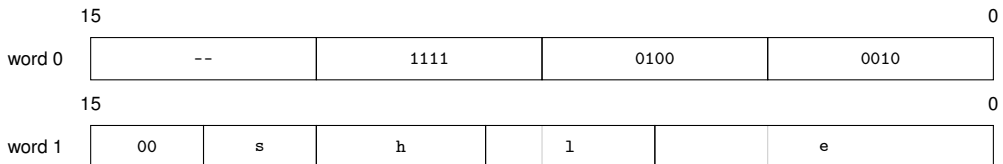
Condition Codes: **Z N C V**
 - - - *

V: set when the value is out of bounds

Instruction Forms:

BNDSCII.X(s:B/W/L/Q) Dn(l), <ea(e)>, Dn(h)

Form Dn(l), <ea(e)>, Dn(h)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands lower bound operand is data register number; value is compact effective-address field; upper bound operand is data register number



Instruction format for BNDSCII.X(s:B/W/L/Q) Dn(l), <ea(e)>, Dn(h)

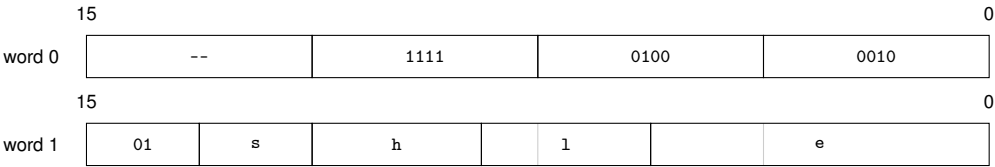
- e compact effective-address field for the value (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - l data register number for the lower bound operand (extended opcode word bits 8:6).
 - h data register number for the upper bound operand (extended opcode word bits 11:9).
 - s size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

BNDSEX**Signed Bounds Check Inclusive-Exclusive****BNDSEX**

Summary:	Checks signed value membership in the inclusive-exclusive interval [lo, hi).
Operation:	<pre> ZF = 0; NF = 0; CF = 0; VF = 0; if (value_v < lo_v) goto <bnduix_bad>; if (value_v >= hi_v) goto <bnduix_bad>; goto <bnduix_done>; <bnduix_bad> VF = 1; <bnduix_done> </pre>
Assembler Syntax:	<code>BNDSEX.<B/W/L/Q> Dn, <ea>, Dn</code>
Attributes:	Size = (Byte, Word, Long, Quad) Words = (2-8) Privilege = unprivileged Flags = V: set when the value is out of bounds
Description:	BNDSEX treats the low bound as inclusive and the high bound as exclusive. It sets V when signed(value) is outside [signed(lo), signed(hi)); Z, N, and C are unchanged.
Condition Codes:	Z N C V - - - * V: set when the value is out of bounds
Instruction Forms:	

BNDSIX.X(s:B/W/L/Q) Dn(l), <ea(e)>, Dn(h)

Form	Dn(l), <ea(e)>, Dn(h)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	lower bound operand is data register number; value is compact effective-address field; upper bound operand is data register number



Instruction format for BNDSIX.X(s:B/W/L/Q) Dn(l), <ea(e)>, Dn(h)

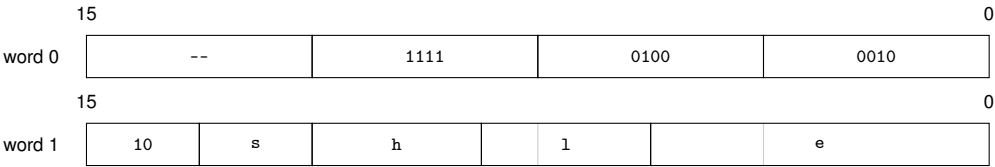
- e compact effective-address field for the value (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - l data register number for the lower bound operand (extended opcode word bits 8:6).
 - h data register number for the upper bound operand (extended opcode word bits 11:9).
 - s size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

BNDXSI**Signed Bounds Check Exclusive-Inclusive****BNDXSI**

Summary:	Checks signed value membership in the exclusive-inclusive interval [lo, hi].
Operation:	<pre> ZF = 0; NF = 0; CF = 0; VF = 0; if (value_v <= lo_v) goto <bnduxi_bad>; if (value_v > hi_v) goto <bnduxi_bad>; goto <bnduxi_done>; <bnduxi_bad> VF = 1; <bnduxi_done> </pre>
Assembler Syntax:	<code>BNDXSI.<B/W/L/Q> Dn, <ea>, Dn</code>
Attributes:	Size = (Byte, Word, Long, Quad) Words = (2-8) Privilege = unprivileged Flags = V: set when the value is out of bounds
Description:	BNDXSI treats the low bound as exclusive and the high bound as inclusive. It sets V when signed(value) is outside (signed(lo), signed(hi)]; Z, N, and C are unchanged.
Condition Codes:	Z N C V - - - * V: set when the value is out of bounds
Instruction Forms:	

BNDSXI.X(s:B/W/L/Q) Dn(l), <ea(e)>, Dn(h)

Form	Dn(l), <ea(e)>, Dn(h)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	lower bound operand is data register number; value is compact effective-address field; upper bound operand is data register number



Instruction format for BNDSXI.X(s:B/W/L/Q) Dn(l), <ea(e)>, Dn(h)

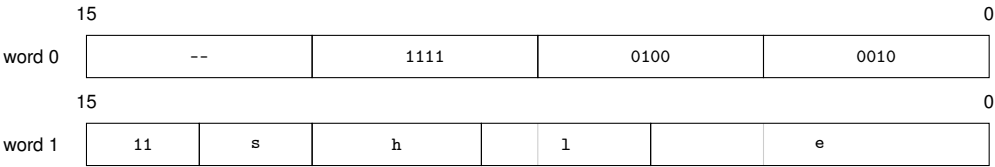
- e compact effective-address field for the value (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - l data register number for the lower bound operand (extended opcode word bits 8:6).
 - h data register number for the upper bound operand (extended opcode word bits 11:9).
 - s size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

BNDSEX**Signed Bounds Check Exclusive-Exclusive****BNDSEX**

Summary:	Checks signed value membership in the exclusive-exclusive interval (lo, hi).
Operation:	<pre> ZF = 0; NF = 0; CF = 0; VF = 0; if (value_v <= lo_v) goto <bndsex_bad>; if (value_v >= hi_v) goto <bndsex_bad>; goto <bndsex_done>; <bndsex_bad> VF = 1; <bndsex_done> </pre>
Assembler Syntax:	<code>BNDSEX.<B/W/L/Q> Dn, <ea>, Dn</code>
Attributes:	Size = (Byte, Word, Long, Quad) Words = (2-8) Privilege = unprivileged Flags = V: set when the value is out of bounds
Description:	BNDSEX treats both bounds as exclusive. It sets V when signed(value) is outside (signed(lo), signed(hi)); Z, N, and C are unchanged.
Condition Codes:	Z N C V - - - * V: set when the value is out of bounds
Instruction Forms:	

BNDXXX.X(s:B/W/L/Q) Dn(l), <ea(e)>, Dn(h)

Form	Dn(l), <ea(e)>, Dn(h)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	lower bound operand is data register number; value is compact effective-address field; upper bound operand is data register number



Instruction format for BNDXXX.X(s:B/W/L/Q) Dn(l), <ea(e)>, Dn(h)

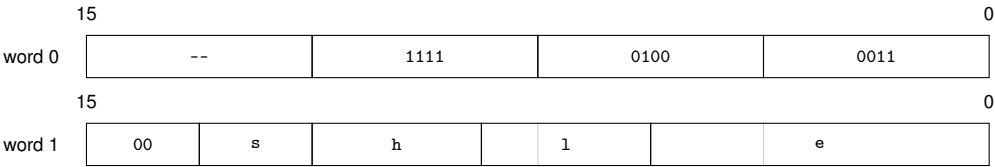
- e compact effective-address field for the value (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - l data register number for the lower bound operand (extended opcode word bits 8:6).
 - h data register number for the upper bound operand (extended opcode word bits 11:9).
 - s size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

BNDUII**Unsigned Bounds Check Inclusive-Inclusive****BNDUII**

Summary:	Checks unsigned value membership in the inclusive-inclusive interval [lo, hi].
Operation:	<pre> ZF = 0; NF = 0; CF = 0; VF = 0; if (value_v < lo_v) goto <bnduii_bad>; if (value_v > hi_v) goto <bnduii_bad>; goto <bnduii_done>; <bnduii_bad> VF = 1; <bnduii_done> </pre>
Assembler Syntax:	<code>BNDUII.<B/W/L/Q> Dn, <ea>, Dn</code>
Attributes:	Size = (Byte, Word, Long, Quad) Words = (2-8) Privilege = unprivileged Flags = V: set when the value is out of bounds
Description:	BNDUII treats both bounds as inclusive. It sets V when unsigned(value) is outside [unsigned(lo), unsigned(hi)]; Z, N, and C are unchanged.
Condition Codes:	Z N C V - - - * V: set when the value is out of bounds
Instruction Forms:	

BNDUII.X(s:B/W/L/Q) Dn(l), <ea(e)>, Dn(h)

Form	Dn(l), <ea(e)>, Dn(h)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	lower bound operand is data register number; value is compact effective-address field; upper bound operand is data register number



Instruction format for BNDUII.X(s:B/W/L/Q) Dn(l), <ea(e)>, Dn(h)

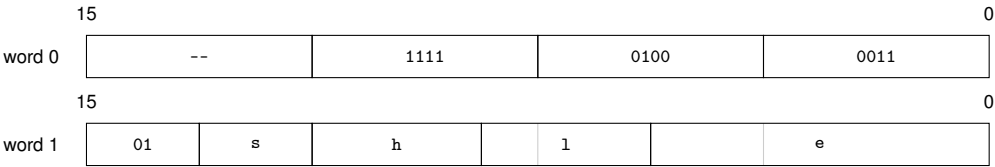
- e compact effective-address field for the value (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - l data register number for the lower bound operand (extended opcode word bits 8:6).
 - h data register number for the upper bound operand (extended opcode word bits 11:9).
 - s size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

BNDUIX**Unsigned Bounds Check Inclusive-Exclusive****BNDUIX**

Summary:	Checks unsigned value membership in the inclusive-exclusive interval [lo, hi).
Operation:	<pre> ZF = 0; NF = 0; CF = 0; VF = 0; if (value_v < lo_v) goto <bnduix_bad>; if (value_v >= hi_v) goto <bnduix_bad>; goto <bnduix_done>; <bnduix_bad> VF = 1; <bnduix_done> </pre>
Assembler Syntax:	<code>BNDUIX.<B/W/L/Q> Dn, <ea>, Dn</code>
Attributes:	Size = (Byte, Word, Long, Quad) Words = (2-8) Privilege = unprivileged Flags = V: set when the value is out of bounds
Description:	BNDUIX treats the low bound as inclusive and the high bound as exclusive. It sets V when unsigned(value) is outside [unsigned(lo), unsigned(hi)); Z, N, and C are unchanged.
Condition Codes:	Z N C V - - - * V: set when the value is out of bounds
Instruction Forms:	

BNDUIX.X(s:B/W/L/Q) Dn(l), <ea(e)>, Dn(h)

Form	Dn(l), <ea(e)>, Dn(h)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	lower bound operand is data register number; value is compact effective-address field; upper bound operand is data register number



Instruction format for BNDUIX.X(s:B/W/L/Q) Dn(l), <ea(e)>, Dn(h)

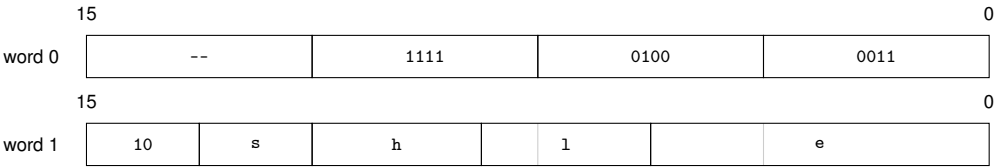
- e compact effective-address field for the value (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - l data register number for the lower bound operand (extended opcode word bits 8:6).
 - h data register number for the upper bound operand (extended opcode word bits 11:9).
 - s size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

BNDUXI**Unsigned Bounds Check Exclusive-Inclusive****BNDUXI**

Summary:	Checks unsigned value membership in the exclusive-inclusive interval (lo, hi].
Operation:	<pre> ZF = 0; NF = 0; CF = 0; VF = 0; if (value_v <= lo_v) goto <bnduxi_bad>; if (value_v > hi_v) goto <bnduxi_bad>; goto <bnduxi_done>; <bnduxi_bad> VF = 1; <bnduxi_done> </pre>
Assembler Syntax:	<code>BNDUXI.<B/W/L/Q> Dn, <ea>, Dn</code>
Attributes:	Size = (Byte, Word, Long, Quad) Words = (2-8) Privilege = unprivileged Flags = V: set when the value is out of bounds
Description:	BNDUXI treats the low bound as exclusive and the high bound as inclusive. It sets V when unsigned(value) is outside (unsigned(lo), unsigned(hi)]; Z, N, and C are unchanged.
Condition Codes:	Z N C V - - - * V: set when the value is out of bounds
Instruction Forms:	

BNDUXI.X(s:B/W/L/Q) Dn(l), <ea(e)>, Dn(h)

Form	Dn(l), <ea(e)>, Dn(h)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	lower bound operand is data register number; value is compact effective-address field; upper bound operand is data register number



Instruction format for BNDUXI.X(s:B/W/L/Q) Dn(l), <ea(e)>, Dn(h)

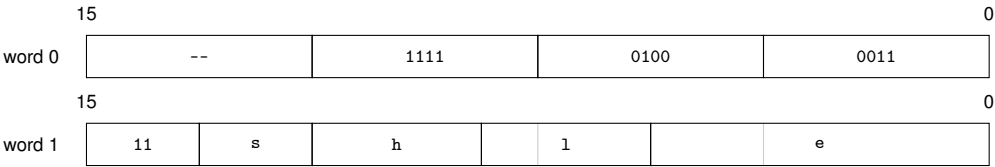
- e compact effective-address field for the value (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - l data register number for the lower bound operand (extended opcode word bits 8:6).
 - h data register number for the upper bound operand (extended opcode word bits 11:9).
 - s size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

BNDUXX**Unsigned Bounds Check Exclusive-Exclusive****BNDUXX**

Summary:	Checks unsigned value membership in the exclusive-exclusive interval (lo, hi).
Operation:	<pre> ZF = 0; NF = 0; CF = 0; VF = 0; if (value_v <= lo_v) goto <bnduxx_bad>; if (value_v >= hi_v) goto <bnduxx_bad>; goto <bnduxx_done>; <bnduxx_bad> VF = 1; <bnduxx_done> </pre>
Assembler Syntax:	<code>BNDUXX.<B/W/L/Q> Dn, <ea>, Dn</code>
Attributes:	Size = (Byte, Word, Long, Quad) Words = (2-8) Privilege = unprivileged Flags = V: set when the value is out of bounds
Description:	BNDUXX treats both bounds as exclusive. It sets V when unsigned(value) is outside (unsigned(lo), unsigned(hi)); Z, N, and C are unchanged.
Condition Codes:	Z N C V - - - * V: set when the value is out of bounds
Instruction Forms:	

BNDUXX.X(s:B/W/L/Q) Dn(l), <ea(e)>, Dn(h)

Form	Dn(l), <ea(e)>, Dn(h)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	lower bound operand is data register number; value is compact effective-address field; upper bound operand is data register number



Instruction format for BNDUXX.X(s:B/W/L/Q) Dn(l), <ea(e)>, Dn(h)

- e compact effective-address field for the value (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - l data register number for the lower bound operand (extended opcode word bits 8:6).
 - h data register number for the upper bound operand (extended opcode word bits 11:9).
 - s size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

BSET**Bit Set****BSET**

Summary: Performs the bit set operation.

Operation: `tmp_v = 1 << (bit_index_v & 63);`
`dst_v = dst_v | tmp_v;`
`ZF = dst_v == 0;`
`NF = dst_v < 0;`
`CF = 0;`
`VF = 0;`

Assembler Syntax: `BSET.<B/W/L/Q> Dn, Dn`
`BSET.<B/W/L/Q> Dn, <ea>`
`BSET.<B/W/L/Q> <bit_index:imm6>, <ea>`

Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = set Z when the old bit was zero; preserve N/C/V

Condition Codes: **Z N C V**

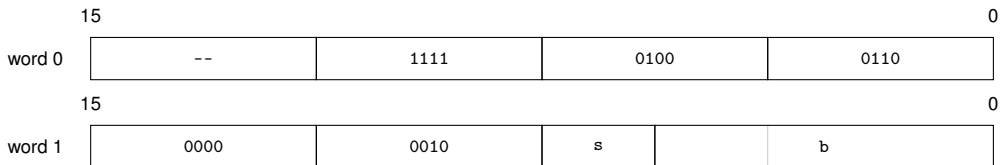
* - - -

set Z when the old bit was zero; preserve N/C/V

Instruction Forms:

`BSET.X(s:B/W/L/Q) Dn(b), Dn(b)`

Form Dn(b), Dn(b)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands bit index is data register number; destination operand is data register number

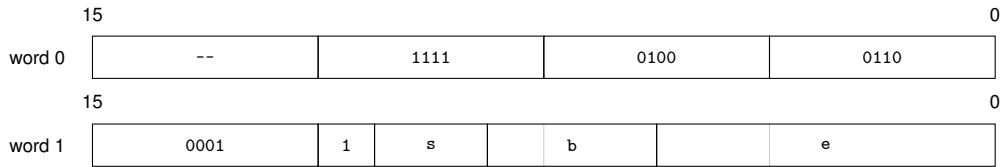


Instruction format for BSET.X(s:B/W/L/Q) Dn(b), Dn(b)

- b data register number for the bit index (extended opcode word bits 2:0).
- b data register number for the bit index (extended opcode word bits 5:3).
- s size selector for the operand size (extended opcode word bits 7:6).

BSET.X(s:B/W/L/Q) Dn(b), <ea(e)>

Form Dn(b), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands bit index is data register number; destination operand is compact effective-address field

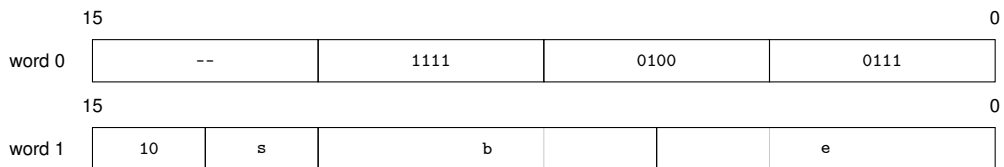


Instruction format for BSET.X(s:B/W/L/Q) Dn(b), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - b data register number for the bit index (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

BSET.X(s:B/W/L/Q) <bit_index(b)>, <ea(e)>

Form <bit_index(b)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands bit index is selector imm6; destination operand is compact effective-address field



Instruction format for BSET.X(s:B/W/L/Q) <bit_index(b)>, <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - b selector6 for the bit index (extended opcode word bits 11:6).
 - s size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

BTEST**Bit Test****BTEST**

Summary: Performs the bit test operation.

Operation: $\text{tmp_v} = (\text{dst_v} \gg (\text{bit_index_v} \& 63)) \& 1;$
 $\text{ZF} = \text{tmp_v} == 0;$

Assembler Syntax: `BTEST.<B/W/L/Q> Dn, Dn`
`BTEST.<B/W/L/Q> Dn, <ea>`
`BTEST.<B/W/L/Q> <bit_index:imm6>, <ea>`

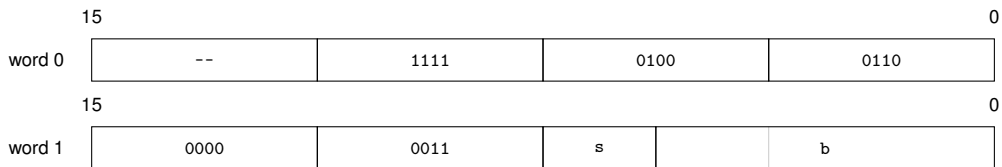
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = set Z when the old bit was zero; preserve N/C/V

Condition Codes: **Z N C V**
* - - -
set Z when the old bit was zero; preserve N/C/V

Instruction Forms:

`BTEST.X(s:B/W/L/Q) Dn(b), Dn(b)`

Form Dn(b), Dn(b)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands bit index is data register number; destination operand is data register number

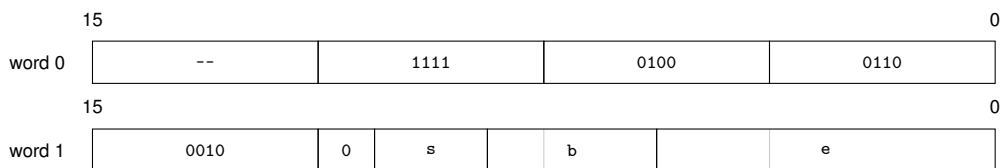


Instruction format for BTEST.X(s:B/W/L/Q) Dn(b), Dn(b)

- b data register number for the bit index (extended opcode word bits 2:0).
- b data register number for the bit index (extended opcode word bits 5:3).
- s size selector for the operand size (extended opcode word bits 7:6).

BTEST.X(s:B/W/L/Q) Dn(b), <ea(e)>

Form Dn(b), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands bit index is data register number; destination operand is compact effective-address field

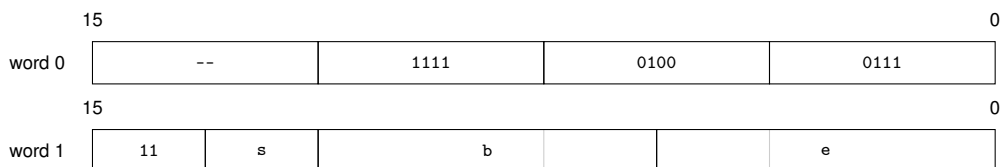


Instruction format for BTEST.X(s:B/W/L/Q) Dn(b), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - b data register number for the bit index (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

BTEST.X(s:B/W/L/Q) <bit_index(b)>, <ea(e)>

Form <bit_index(b)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands bit index is selector imm6; destination operand is compact effective-address field



Instruction format for BTEST.X(s:B/W/L/Q) <bit_index(b)>, <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - b selector6 for the bit index (extended opcode word bits 11:6).
 - s size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

CALL**Call****CALL**

Summary: Performs the call operation.

Operation: local return_pc_v:8 = inst_next;
 SP = SP - 8;
 *:8 SP = return_pc_v;
 call [target_v];

Assembler Syntax: CALL <imm32>
 CALL <imm64>
 CALL <imm16>
 CALL <ea>

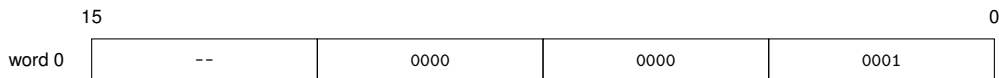
Attributes: Words = (2-8, 3-8, 5-8)
 Privilege = unprivileged
 Flags = unchanged

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

CALL imm32

Form imm32
Encoding primary opcode 0x001
Words 3-8
Privilege unprivileged
Operands branch target is imm32

**Instruction format for CALL imm32**

No explicit operand fields are encoded in this form.

CALL imm64

Form	imm64
Encoding	primary opcode 0x002
Words	5-8
Privilege	unprivileged
Operands	branch target is imm64



Instruction format for CALL imm64

No explicit operand fields are encoded in this form.

CALL imm16

Form	imm16
Encoding	primary opcode 0x003
Words	2-8
Privilege	unprivileged
Operands	branch target is 16-bit immediate selector

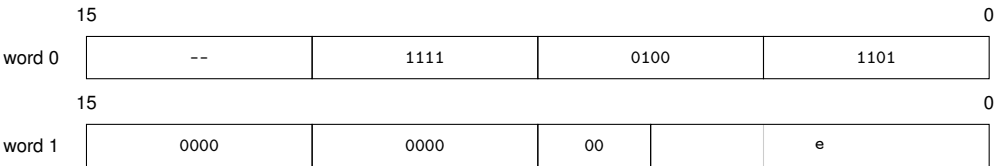


Instruction format for CALL imm16

No explicit operand fields are encoded in this form.

CALL <ea(e)>

Form	<ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	branch target is compact effective-address field



Instruction format for CALL <ea(e)>

- e compact effective-address field for the branch target (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms. EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

CLMUL**Carryless Multiply****CLMUL**

Summary: Performs the carryless multiply operation.

Operation: `dst_v = arch_carryless_multiply_low(dst_v, src_v, size_v);`

Assembler Syntax: `CLMUL.<B/W/L/Q> Dn, <ea>`
`CLMUL.<B/W/L/Q> <ea>, Dn`

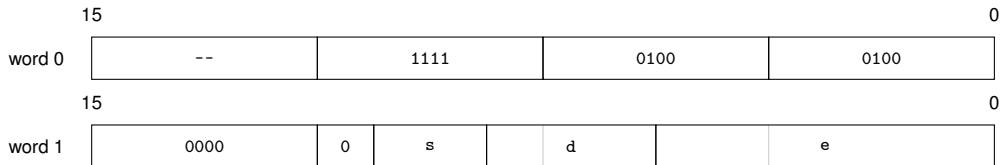
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

`CLMUL.X(s:B/W/L/Q) Dn(d), <ea(e)>`

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

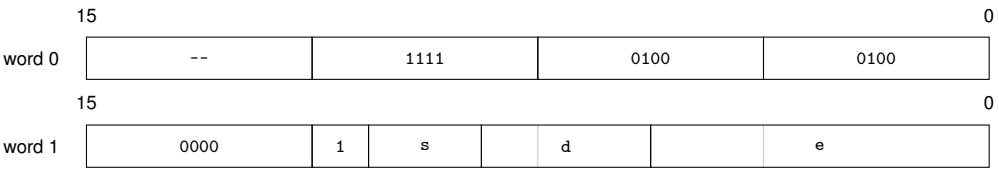


Instruction format for CLMUL.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the source operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

CLMUL.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for CLMUL.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

CLMULH**Carryless Multiply High****CLMULH**

Summary: Performs the carryless multiply high operation.

Operation: `dst_v = arch_carryless_multiply_high(dst_v, src_v, size_v);`

Assembler Syntax: `CLMULH.<B/W/L/Q> Dn, <ea>`
`CLMULH.<B/W/L/Q> <ea>, Dn`

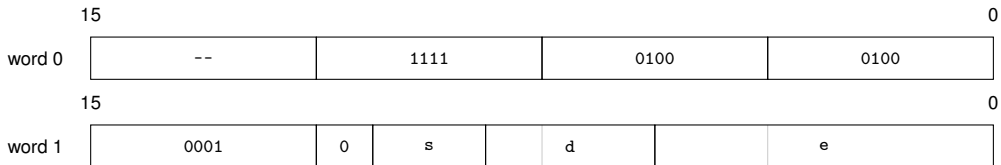
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

`CLMULH.X(s:B/W/L/Q) Dn(d), <ea(e)>`

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

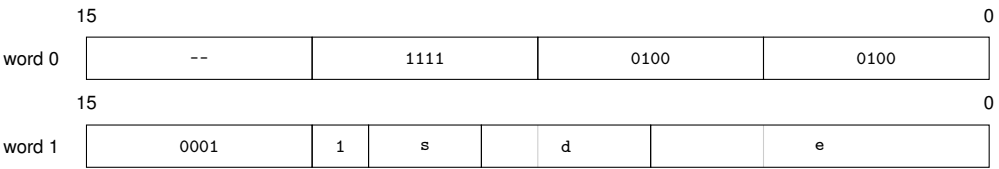


Instruction format for CLMULH.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the source operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

CLMULH.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for CLMULH.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

CLR**Clear****CLR****Summary:** Performs the clear operation.**Operation:** `dst_v = 0;`

Assembler Syntax: `CLR An`
`CLR.D Dn`
`CLR <ea>`

Attributes: Words = (1-8, 2-8)
Privilege = unprivileged
Flags = unchanged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:`CLR An(a)`

Form `An(a)`
Encoding primary opcode range 0xf18 to 0xf1f
Words 1-8
Privilege unprivileged
Operands destination operand is address register number

**Instruction format for CLR An(a)**

a address register number for the destination operand (primary word bits 2:0).

CLR.D Dn(d)

Form	Dn(d)
Encoding	primary opcode range 0xf20 to 0xf27
Words	1-8
Privilege	unprivileged
Operands	destination operand is data register number

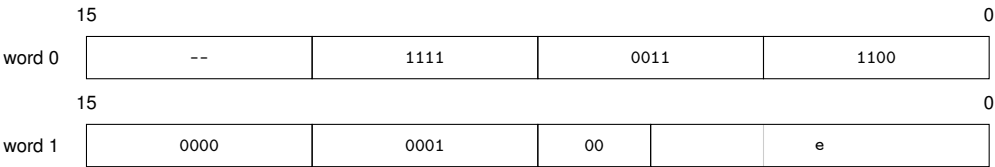


Instruction format for CLR.D Dn(d)

d data register number for the destination operand (primary word bits 2:0).

CLR <ea(e)>

Form	<ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	destination operand is compact effective-address field



Instruction format for CLR <ea(e)>

e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

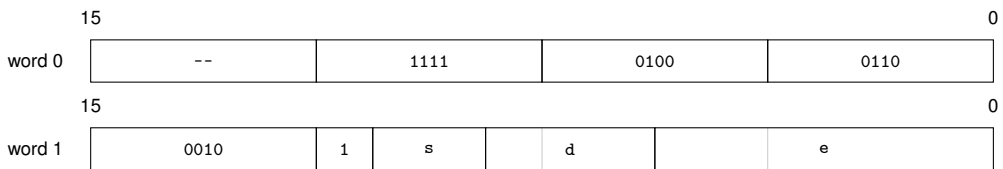
CLS**Count Leading Set Bits****CLS**

Summary:	Performs the count leading set bits operation.
Operation:	<code>dst_v = arch_count_leading_ones(src_v, size_v);</code>
Assembler Syntax:	<code>CLS.<B/W/L/Q> <ea>, Dn</code>
Attributes:	Size = (Byte, Word, Long, Quad) Words = (2-8) Privilege = unprivileged Flags = unchanged
Description:	Counts the leading one bits within the selected operand size and writes the count to the destination register. A zero operand returns 0; an operand whose selected bits are all one returns the operand width in bits.
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

`CLS.X(s:B/W/L/Q) <ea(e)>, Dn(d)`

Form	<code><ea(e)>, Dn(d)</code>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for CLS.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

CLZ

Count Leading Zeros

CLZ

Summary:

Operation:

Assembler Syntax:

Attributes:

Description:

Condition Codes:

Performs the count leading zeros operation.

`dst_v = arch_count_leading_zeros(src_v, size_v);`

`CLZ.<B/W/L/Q> <ea>, Dn`

Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Counts the leading zero bits within the selected operand size and writes the count to the destination register. A zero operand returns the operand width in bits.

Z N C V
- - - -
unchanged

Instruction Forms:

CLZ.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form

Encoding

Words

Privilege

Operands

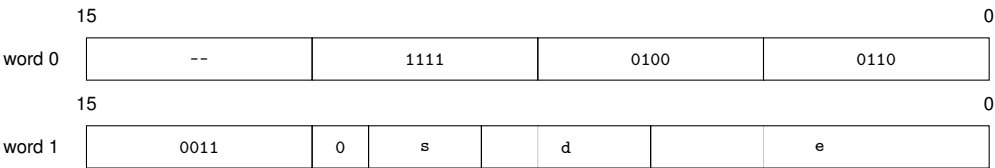
<ea(e)>, Dn(d)

extended opcode form

2-8

unprivileged

source operand is compact effective-address field; destination operand is data register number



Instruction format for CLZ.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e
- d
- s
- compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- data register number for the destination operand (extended opcode word bits 8:6).
- size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

CMP**Compare****CMP**

Summary:	Computes a comparison result and updates condition codes without storing the temporary value.
Operation:	<pre> result_v = dst_v - src_v; ZF = result_v == 0; NF = result_v < 0; CF = dst_v < src_v; VF = sborrow(dst_v, src_v); </pre>
Assembler Syntax:	<pre> CMP.<L/Q> Dn, Dn CMP.<W/L> <imm>, Dn CMP <ea>, An CMP.<B/W/L/Q> Dn, <ea> CMP.<B/W/L/Q> <ea>, Dn CMP.<B/W/L/Q> <ea>, <ea> CMP.<B/W/L/Q> <imm6>, <ea> </pre>
Attributes:	Size = (Byte, Word, Long, Quad, Long, Quad, Word, Long) Words = (1-8, 2-8) Privilege = unprivileged Flags = Z: result == 0; N: most significant bit(result); C: unsigned borrow; V: signed overflow
Condition Codes:	Z N C V * _ _ * Z: result == 0; N: most significant bit(result); C: unsigned borrow; V: signed overflow
Instruction Forms:	

CMP.X(z:L/Q) Dn(d), Dn(D)

Form Dn(d), Dn(D)
Encoding primary opcode range 0x380 to 0x3ff
Words 1-8
Privilege unprivileged
Operands source operand is data register number; destination operand is data register number

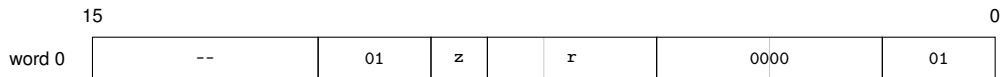


Instruction format for CMP.X(z:L/Q) Dn(d), Dn(D)

d data register number for the source operand (primary word bits 2:0).
D data register number for the destination operand (primary word bits 5:3).
z size selector for the operand size (primary word bits 6).

CMP.X(z:W/L) imm, Dn(r)

Form imm, Dn(r)
Encoding primary opcode range 0x401 to 0x7c1
Words 2-8
Privilege unprivileged
Operands imm; right operand is data register number

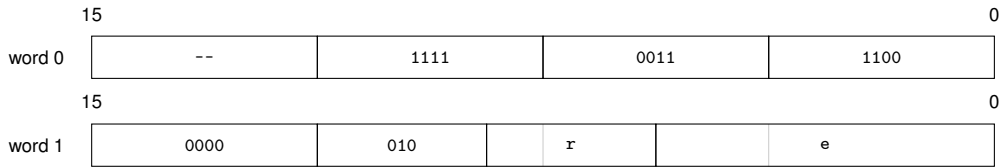


Instruction format for CMP.X(z:W/L) imm, Dn(r)

r data register number for the right operand (primary word bits 8:6).
z size selector for the operand size (primary word bits 9).

CMP <ea(e)>, An(r)

Form <ea(e)>, An(r)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands left operand is compact effective-address field; right operand is address register number

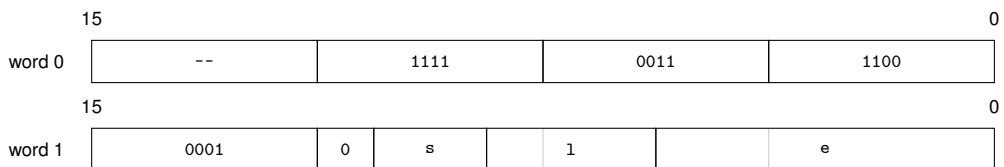


Instruction format for CMP <ea(e)>, An(r)

- e compact effective-address field for the left operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 r address register number for the right operand (extended opcode word bits 8:6).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

CMP.X(s:B/W/L/Q) Dn(l), <ea(e)>

Form Dn(l), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands left operand is data register number; right operand is compact effective-address field

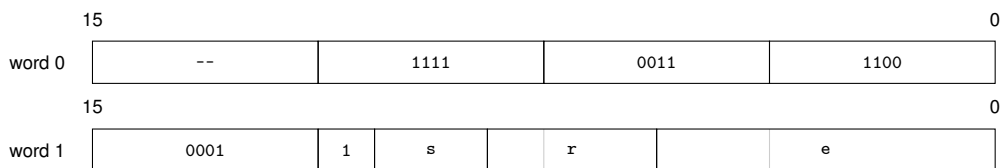


Instruction format for CMP.X(s:B/W/L/Q) Dn(l), <ea(e)>

- e compact effective-address field for the right operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 l data register number for the left operand (extended opcode word bits 8:6).
 s size selector for the operand size (extended opcode word bits 10:9).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

CMP.X(s:B/W/L/Q) <ea(e)>, Dn(r)

Form <ea(e)>, Dn(r)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands left operand is compact effective-address field; right operand is data register number

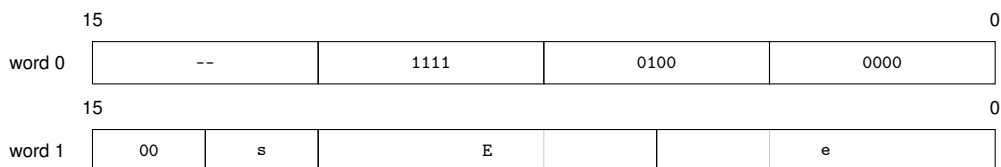


Instruction format for CMP.X(s:B/W/L/Q) <ea(e)>, Dn(r)

- e** compact effective-address field for the left operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- r** data register number for the right operand (extended opcode word bits 8:6).
- s** size selector for the operand size (extended opcode word bits 10:9).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

CMP.X(s:B/W/L/Q) <ea(e)>, <ea(E)>

Form <ea(e)>, <ea(E)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands left operand is compact effective-address field; right operand is compact effective-address field

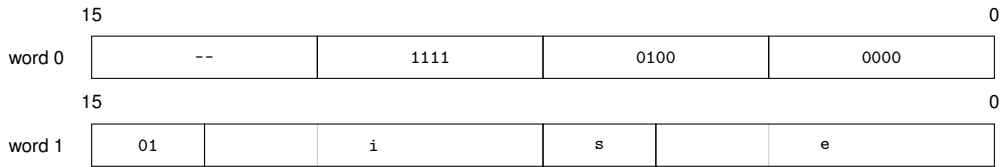


Instruction format for CMP.X(s:B/W/L/Q) <ea(e)>, <ea(E)>

- e** compact effective-address field for the left operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- E** compact effective-address field for the right operand (extended opcode word bits 11:6). EA field selects register, memory, immediate, and extended EA forms.
- s** size selector for the operand size (extended opcode word bits 13:12).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

`CMP.X(s:B/W/L/Q) <imm(i)>, <ea(e)>`

Form	<imm(i)>, <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	imm is unsigned 6-bit immediate literal; right operand is compact effective-address field



Instruction format for `CMP.X(s:B/W/L/Q) <imm(i)>, <ea(e)>`

- e compact effective-address field for the right operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - s size selector for the operand size (extended opcode word bits 7:6).
 - i unsigned 6-bit immediate literal for the imm (extended opcode word bits 13:8). The encoded value is zero-extended before use for integer forms with an explicit B/W/L/Q operation size.
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

CMPXCHG**Compare And Exchange****CMPXCHG**

Summary: Performs the compare and exchange operation.

Operation:

```
tmp_v = memory_v;  
if (tmp_v != expected_v) goto <cmpxchg_fail>;  
memory_v = desired_v;  
expected_v = tmp_v;  
ZF = 1;  
NF = 0;  
CF = 0;  
VF = 0;  
goto <cmpxchg_done>;  
<cmpxchg_fail>  
expected_v = tmp_v;  
ZF = 0;  
NF = 0;  
CF = 0;  
VF = 0;  
<cmpxchg_done>
```

Assembler Syntax: `CMPXCHG.<B/W/L/Q>/<ORDER> Dn, Dn, <ea>`

Attributes:

- Size = (Byte, Word, Long, Quad)
- Words = (3-8)
- Privilege = unprivileged
- Flags = Z: store succeeded; N: cleared; C: cleared; V: cleared

Description: Compares memory with the expected D register and conditionally stores the desired D register. After the operation, expected contains the observed old memory value. ZF=1 means the store happened; ZF=0 means memory was left unchanged.

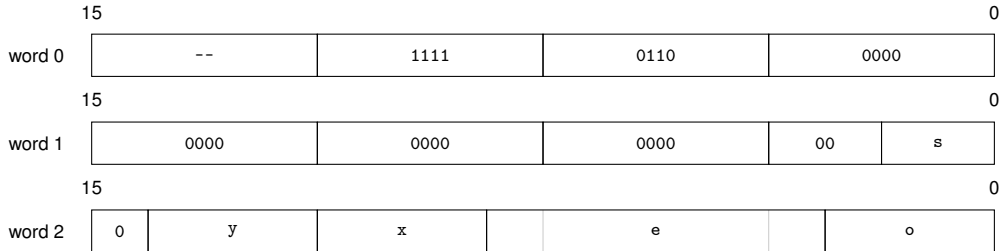
Condition Codes: **Z N C V**
* - - *

Z: store succeeded; N: cleared; C: cleared; V: cleared

Instruction Forms:

CMPXCHG.X(s:B/W/L/Q)/ORDER(o) Dn(x), Dn(y), <ea(e)>

Form	Dn(x), Dn(y), <ea(e)>
Encoding	extended opcode form
Words	3-8
Privilege	unprivileged
Operands	expected value is data register number; desired value is data register number; memory operand is compact effective-address field



Instruction format for CMPXCHG.X(s:B/W/L/Q)/ORDER(o) Dn(x), Dn(y), <ea(e)>

- s size selector for the operand size (extended opcode word bits 1:0).
 - o atomic memory-order value for the ordering mode (payload word 1 bits 2:0).
 - e compact effective-address field for the memory operand (payload word 1 bits 8:3). EA field selects register, memory, immediate, and extended EA forms.
 - x data register number for the expected value (payload word 1 bits 11:9).
 - y data register number for the desired value (payload word 1 bits 14:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

CPUID

CPU Identification

CPUID

Summary:

Operation:

Assembler Syntax:

Attributes:

Description:

Condition Codes:

Queries architectural discovery information through a selected D register.

reg_v = arch_cpuid(reg_v);

CPUID.D Dn

Words = (2-8)
Privilege = user allowed
Flags = unchanged

Queries architectural discovery information. The selected D register contains the CPUID query selector before execution and receives the selected 64-bit result after execution.

Z N C V
- - - -
unchanged

Instruction Forms:

CPUID.D Dn(d)

Form

Encoding

Words

Privilege

Operands

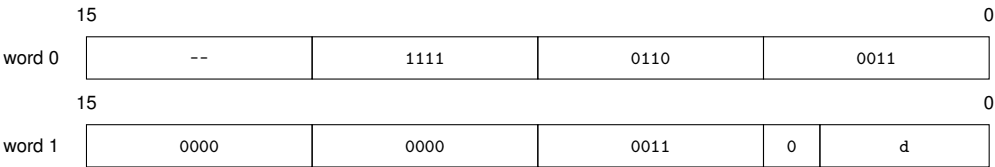
Dn(d)

extended opcode form

2-8

user allowed

register operand is data register number



Instruction format for CPUID.D Dn(d)

d data register number for the register operand (extended opcode word bits 2:0).

CTS**Count Trailing Set Bits****CTS**

Summary: Performs the count trailing set bits operation.

Operation: `dst_v = arch_count_trailing_ones(src_v, size_v);`

Assembler Syntax: `CTS.<B/W/L/Q> <ea>, Dn`

Attributes:
 Size = (Byte, Word, Long, Quad)
 Words = (2-8)
 Privilege = unprivileged
 Flags = unchanged

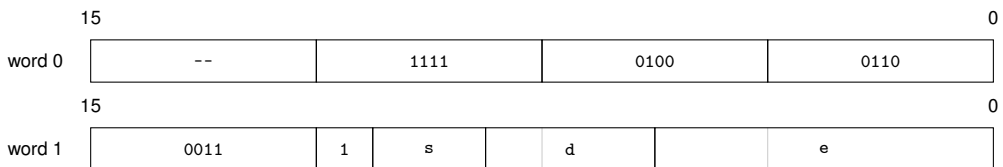
Description: Counts the trailing one bits within the selected operand size and writes the count to the destination register. A zero operand returns 0; an operand whose selected bits are all one returns the operand width in bits.

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

`CTS.X(s:B/W/L/Q) <ea(e)>, Dn(d)`

Form `<ea(e)>, Dn(d)`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is data register number



Instruction format for CTS.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

CTZ

Count Trailing Zeros

CTZ

Summary:Performs the count trailing zeros operation.

Operation:`dst_v = arch_count_trailing_zeros(src_v, size_v);`

Assembler Syntax:`CTZ.<B/W/L/Q> <ea>, Dn`

Attributes:Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Description:Counts the trailing zero bits within the selected operand size and writes the count to the destination register. A zero operand returns the operand width in bits.

Condition Codes:**Z N C V**
- - - -
unchanged

Instruction Forms:

`CTZ.X(s:B/W/L/Q) <ea(e)>, Dn(d)`

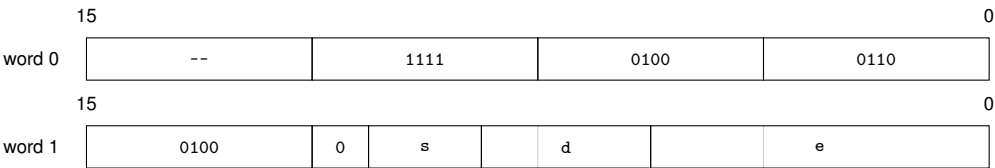
Form<ea(e)>, Dn(d)

Encodingextended opcode form

Words2-8

Privilegeunprivileged

Operandssource operand is compact effective-address field; destination operand is data register number



Instruction format for CTZ.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- ecompact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- ddata register number for the destination operand (extended opcode word bits 8:6).
- ssize selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

DEC**Decrement****DEC**

Summary: Performs the decrement operation.

Operation: $\text{result_v} = \text{dst_v} - 1;$
 $\text{dst_v} = \text{result_v};$
 $\text{ZF} = \text{result_v} == 0;$
 $\text{NF} = \text{result_v} < 0;$
 $\text{CF} = \text{dst_old_v} < 1:8;$
 $\text{VF} = \text{sborrow}(\text{dst_old_v}, 1:8);$

Assembler Syntax: `DEC.<B/W/L/Q> Dn`
`DEC.<B/W/L/Q> <ea>`

Attributes: Size = (Byte, Word, Long, Quad)
Words = (1-8, 2-8)
Privilege = unprivileged
Flags = Z: result == 0; N: most significant bit(result); C: unsigned borrow;
V: signed overflow

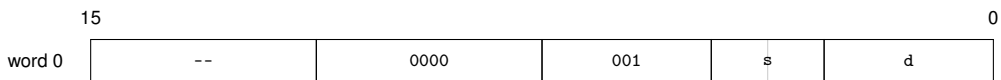
Condition Codes: **Z N C V**
* _ _ *

Z: result == 0; N: most significant bit(result); C: unsigned borrow; V:
signed overflow

Instruction Forms:

`DEC.X(s:B/W/L/Q) Dn(d)`

Form Dn(d)
Encoding primary opcode range 0x020 to 0x03f
Words 1-8
Privilege unprivileged
Operands destination operand is data register number

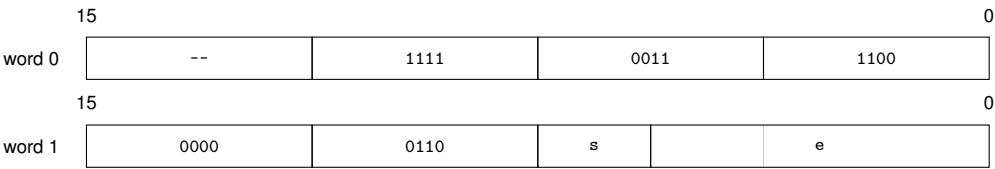


Instruction format for DEC.X(s:B/W/L/Q) Dn(d)

d data register number for the destination operand (primary word bits 2:0).
s size selector for the operand size (primary word bits 4:3).

DEC.X(s:B/W/L/Q) <ea(e)>

Form	<ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	destination operand is compact effective-address field



Instruction format for DEC.X(s:B/W/L/Q) <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 7:6). EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

DECN**Decrement Without Flags****DECN**

Summary: Performs the decrement without flags operation.

Operation: $\text{dst_v} = \text{dst_v} - 1;$

Assembler Syntax: `DECN.<B/W/L/Q> Dn`
`DECN.<B/W/L/Q> <ea>`

Attributes: Size = (Byte, Word, Long, Quad)
 Words = (1-8, 2-8)
 Privilege = unprivileged
 Flags = unchanged

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

`DECN.X(s:B/W/L/Q) Dn(d)`

Form Dn(d)
Encoding primary opcode range 0x2c0 to 0x2df
Words 1-8
Privilege unprivileged
Operands destination operand is data register number

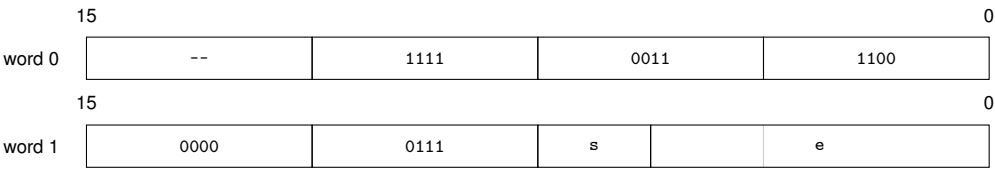


Instruction format for DECN.X(s:B/W/L/Q) Dn(d)

- d data register number for the destination operand (primary word bits 2:0).
 s size selector for the operand size (primary word bits 4:3).

DECN.X(s:B/W/L/Q) <ea(e)>

Form	<ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	destination operand is compact effective-address field



Instruction format for DECN.X(s:B/W/L/Q) <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 7:6). EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

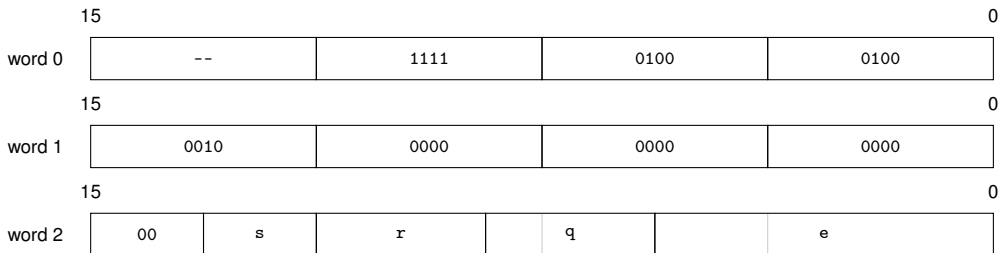
DIVMODS**Signed Divide With Remainder****DIVMODS**

Summary:	Performs the signed divide with remainder operation.
Operation:	<pre>tmp_v = arch_signed_modulo(quotient_v, src_v, size_v); quotient_v = arch_signed_divide(quotient_v, src_v, size_v); remainder_v = tmp_v;</pre>
Assembler Syntax:	<code>DIVMODS.<B/W/L/Q> <ea>, Dn, Dn</code>
Attributes:	Size = (Byte, Word, Long, Quad) Words = (3-8) Privilege = unprivileged
Description:	Performs the signed divide with remainder operation. The quotient register Dn(q) supplies the dividend before the operation, then receives the quotient. The remainder register Dn(r) receives the remainder.
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

`DIVMODS.X(s:B/W/L/Q) <ea(e)>, Dn(q), Dn(r)`

Form	<code><ea(e)>, Dn(q), Dn(r)</code>
Encoding	extended opcode form
Words	3-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; quotient/dividend register is data register number; remainder register is data register number



Instruction format for `DIVMODS.X(s:B/W/L/Q) <ea(e)>, Dn(q), Dn(r)`

- e compact effective-address field for the source operand (payload word 1 bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - q data register number for the quotient/dividend register (payload word 1 bits 8:6).
 - r data register number for the remainder register (payload word 1 bits 11:9).
 - s size selector for the operand size (payload word 1 bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

DIVMODU

Unsigned Divide With Remainder

DIVMODU

Summary:

Operation:

Assembler Syntax:

Attributes:

Description:

Condition Codes:

Performs the unsigned divide with remainder operation.

```
tmp_v = arch_unsigned_modulo(quotient_v, src_v, size_v);
quotient_v = arch_unsigned_divide(quotient_v, src_v, size_v);
remainder_v = tmp_v;
```

DIVMODU.<B/W/L/Q> <ea>, Dn, Dn

Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged

Performs the unsigned divide with remainder operation. The quotient register Dn(q) supplies the dividend before the operation, then receives the quotient. The remainder register Dn(r) receives the remainder.

Z N C V
- - - -
unchanged

Instruction Forms:

DIVMODU.X(s:B/W/L/Q) <ea(e)>, Dn(q), Dn(r)

Form

Encoding

Words

Privilege

Operands

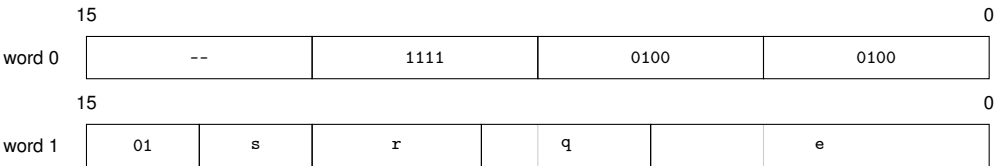
<ea(e)>, Dn(q), Dn(r)

extended opcode form

2-8

unprivileged

source operand is compact effective-address field; quotient/dividend register is data register number;
remainder register is data register number



Instruction format for DIVMODU.X(s:B/W/L/Q) <ea(e)>, Dn(q), Dn(r)

- e
- compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- q
- data register number for the quotient/dividend register (extended opcode word bits 8:6).
- r
- data register number for the remainder register (extended opcode word bits 11:9).
- s
- size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

DIVS**Signed Divide****DIVS**

Summary: Performs the signed divide operation.

Operation: `dst_v = arch_signed_divide(dst_v, src_v, size_v);`

Assembler Syntax: `DIVS.<B/W/L/Q> Dn, <ea>`
`DIVS.<B/W/L/Q> <ea>, Dn`

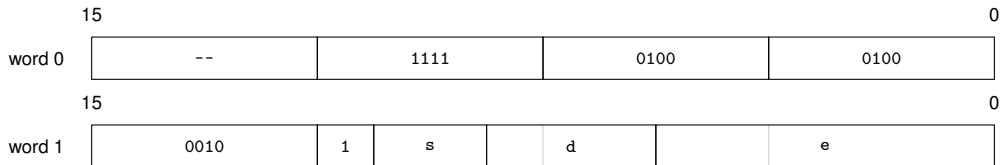
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

`DIVS.X(s:B/W/L/Q) Dn(d), <ea(e)>`

Form `Dn(d), <ea(e)>`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

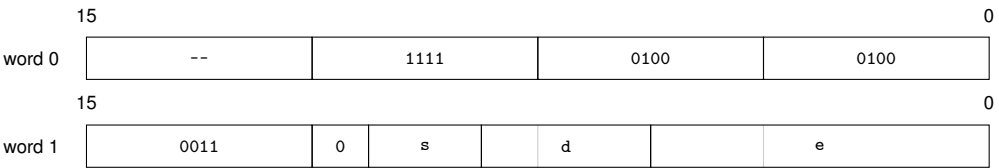


Instruction format for `DIVS.X(s:B/W/L/Q) Dn(d), <ea(e)>`

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the source operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

DIVS.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for DIVS.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

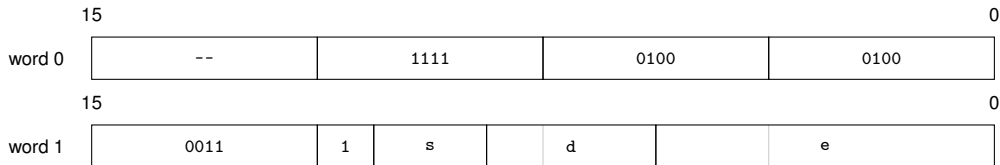
DIVU**Unsigned Divide****DIVU**

Summary:	Performs the unsigned divide operation.
Operation:	<code>dst_v = arch_unsigned_divide(dst_v, src_v, size_v);</code>
Assembler Syntax:	DIVU.<B/W/L/Q> Dn, <ea> DIVU.<B/W/L/Q> <ea>, Dn
Attributes:	Size = (Byte, Word, Long, Quad) Words = (2-8) Privilege = unprivileged
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

DIVU.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form	Dn(d), <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is data register number; destination operand is compact effective-address field

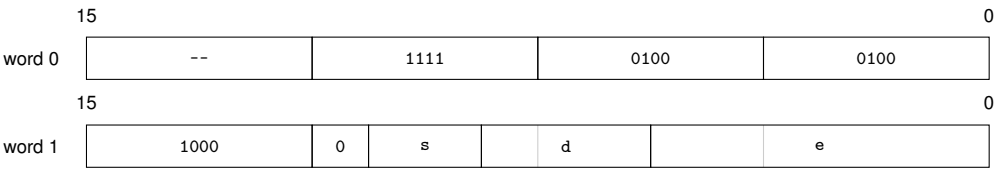


Instruction format for DIVU.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the source operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

DIVU.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for DIVU.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

DJcc**Decrement And Jump Conditional****DJcc**

Summary: Performs the decrement and jump conditional operation.

Operation: counter_v = counter_v - 1;
 if (counter_v == 0) goto <djcc_done>;
 if (cc_v == 0) goto <djcc_done>;
 goto [target_v];
 <djcc_done>

Assembler Syntax: DJcc.<B/W/L/Q> Dn, <ea>

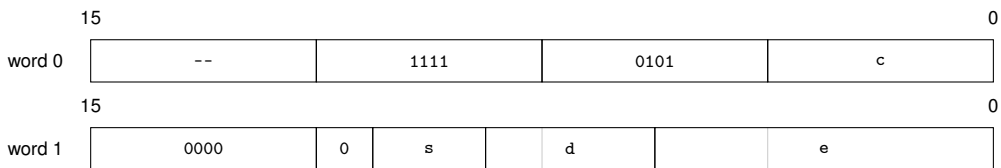
Attributes: Size = (Byte, Word, Long, Quad)
 Words = (2-8)
 Privilege = unprivileged
 Flags = unchanged

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

DJcc.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands condition code is condition-code selector; counter register is data register number; branch target is compact effective-address field



Instruction format for DJcc.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the branch target (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the counter register (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

EXTSL

Sign Extend To Long

EXTSL

Summary:

Operation:

Assembler Syntax:

Attributes:

Description:

Condition Codes:

Performs the sign extend to long operation.

`dst_v = arch_sign_extend(src_v, size_v, 4);`

`EXTSL.<B/W> Dn, Dn`
`EXTSL.<B/W> Dn, <ea>`
`EXTSL.<B/W> <ea>, Dn`
`EXTSL.<B/W> <ea>, <ea>`

Size = (Byte, Word)
Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Sign-extends the source operand from the instruction size to long size. The low source-size bytes of the source are copied into the destination, and the remaining high destination bits are filled with the source sign bit.

Z N C V
- - - -
unchanged

Instruction Forms:

EXTSL.X(s:B/W) Dn(d), Dn(D)

Form

Encoding

Words

Privilege

Operands

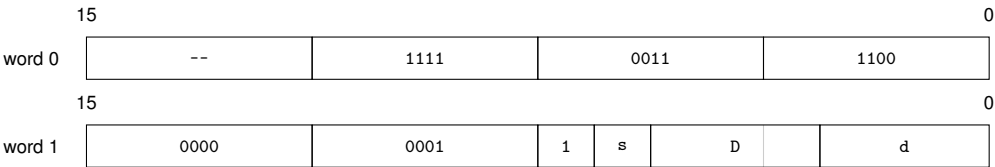
Dn(d), Dn(D)

extended opcode form

2-8

unprivileged

source operand is data register number; destination operand is data register number

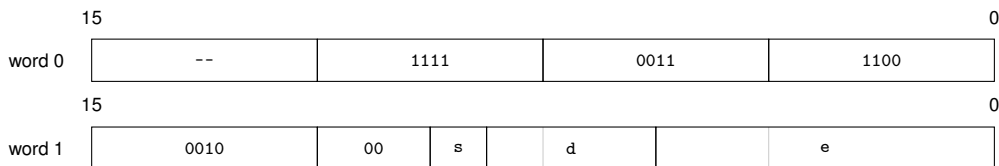


Instruction format for EXTSL.X(s:B/W) Dn(d), Dn(D)

- d
- D
- s
- data register number for the source operand (extended opcode word bits 2:0).
- data register number for the destination operand (extended opcode word bits 5:3).
- size selector for the operand size (extended opcode word bits 6).

EXTSL.X(s:B/W) Dn(d), <ea(e)>

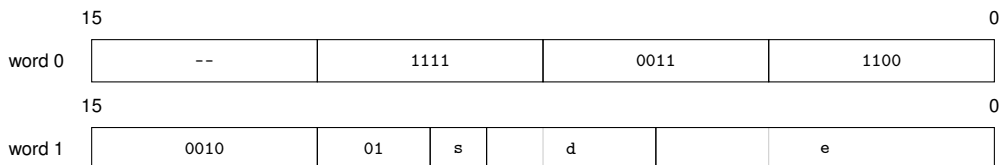
Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

**Instruction format for EXTSL.X(s:B/W) Dn(d), <ea(e)>**

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
d data register number for the source operand (extended opcode word bits 8:6).
s size selector for the operand size (extended opcode word bits 9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

EXTSL.X(s:B/W) <ea(e)>, Dn(d)

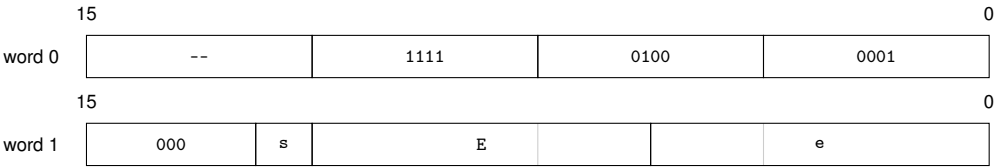
Form <ea(e)>, Dn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is data register number

**Instruction format for EXTSL.X(s:B/W) <ea(e)>, Dn(d)**

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
d data register number for the destination operand (extended opcode word bits 8:6).
s size selector for the operand size (extended opcode word bits 9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

EXTSL.X(s:B/W) <ea(e)>, <ea(E)>

Form	<ea(e)>, <ea(E)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is compact effective-address field



Instruction format for EXTSL.X(s:B/W) <ea(e)>, <ea(E)>

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- E compact effective-address field for the destination operand (extended opcode word bits 11:6). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 12).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

EXTSQ**Sign Extend To Quad****EXTSQ**

Summary: Performs the sign extend to quad operation.

Operation: `dst_v = arch_sign_extend(src_v, size_v, 8);`

Assembler Syntax: `EXTSQ.<B/W/L> Dn, Dn`
`EXTSQ.<B/W/L> Dn, <ea>`
`EXTSQ.<B/W/L> <ea>, Dn`
`EXTSQ.<B/W/L> <ea>, <ea>`

Attributes: Size = (Byte, Word, Long)
 Words = (1-8, 2-8)
 Privilege = unprivileged
 Flags = unchanged

Description: Sign-extends the source operand from the instruction size to quad size. The low source-size bytes of the source are copied into the destination, and the remaining high destination bits are filled with the source sign bit.

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

`EXTSQ.X(s:B/W/L/invalid) Dn(d), Dn(D)`

Form Dn(d), Dn(D)
Encoding primary opcode range 0x100 to 0x1ff
Words 1-8
Privilege unprivileged
Operands source operand is data register number; destination operand is data register number

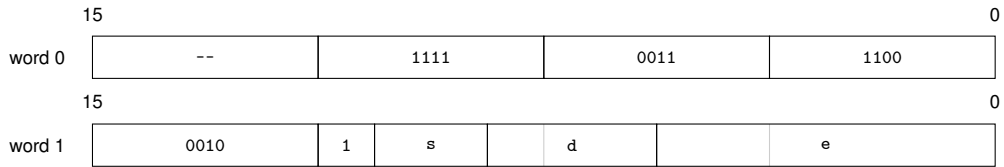


Instruction format for ETSQ.X(s:B/W/L/invalid) Dn(d), Dn(D)

d data register number for the source operand (primary word bits 2:0).
 D data register number for the destination operand (primary word bits 5:3).
 s size selector for the operand size (primary word bits 7:6).

EXTSQ.X(s:B/W/L/invalid) Dn(d), <ea(e)>

Form	Dn(d), <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is data register number; destination operand is compact effective-address field

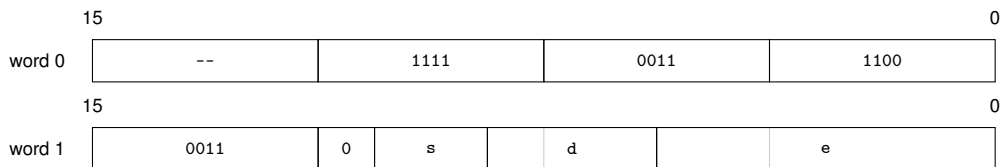


Instruction format for EXTSQ.X(s:B/W/L/invalid) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the source operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

EXTSQ.X(s:B/W/L/invalid) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number

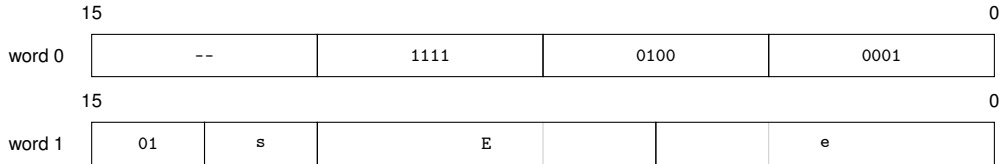


Instruction format for EXTSQ.X(s:B/W/L/invalid) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the destination operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

EXTSQ.X(s:B/W/L/invalid) <ea(e)>, <ea(E)>

Form	<ea(e)>, <ea(E)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is compact effective-address field



Instruction format for EXTSQ.X(s:B/W/L/invalid) <ea(e)>, <ea(E)>

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- E compact effective-address field for the destination operand (extended opcode word bits 11:6). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 13:12).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

EXTSW

Sign Extend To Word

EXTSW

Summary:

Operation:

Assembler Syntax:

Attributes:

Description:

Condition Codes:

Performs the sign extend to word operation.

`dst_v = arch_sign_extend(src_v, size_v, 2);`

`EXTSW Dn, Dn`
`EXTSW Dn, <ea>`
`EXTSW <ea>, Dn`
`EXTSW <ea>, <ea>`

Size = (Byte)
Words = (1-8, 2-8)
Privilege = unprivileged
Flags = unchanged

Sign-extends the source operand from byte size to word size. The low byte of the source is copied into the destination, and the remaining high destination bits are filled with the source sign bit.

Z N C V
- - - -
unchanged

Instruction Forms:

EXTSW Dn(d), Dn(D)

Form	Dn(d), Dn(D)
Encoding	primary opcode range 0x040 to 0x07f
Words	1-8
Privilege	unprivileged
Operands	source operand is data register number; destination operand is data register number

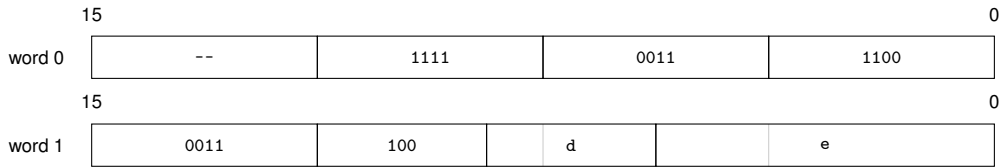


Instruction format for EXTSW Dn(d), Dn(D)

- d
- data register number for the source operand (primary word bits 2:0).
- D
- data register number for the destination operand (primary word bits 5:3).

EXTSW Dn(d), <ea(e)>

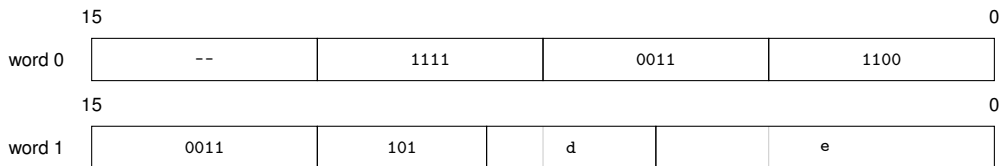
Form	Dn(d), <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is data register number; destination operand is compact effective-address field

**Instruction format for EXTSW Dn(d), <ea(e)>**

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the source operand (extended opcode word bits 8:6).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

EXTSW <ea(e)>, Dn(d)

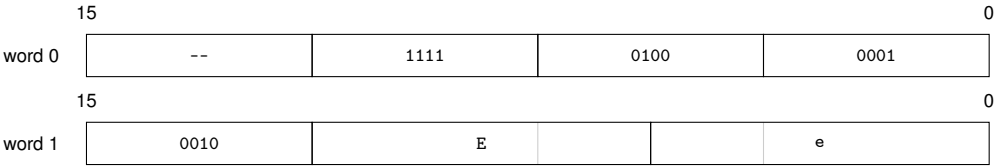
Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number

**Instruction format for EXTSW <ea(e)>, Dn(d)**

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

EXTSW <ea(e)>, <ea(E)>

Form	<ea(e)>, <ea(E)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is compact effective-address field



Instruction format for EXTSW <ea(e)>, <ea(E)>

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - E compact effective-address field for the destination operand (extended opcode word bits 11:6). EA field selects register, memory, immediate, and extended EA forms.
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

EXTZL**Zero Extend To Long****EXTZL**

Summary: Performs the zero extend to long operation.

Operation: `dst_v = arch_zero_extend(src_v, size_v, 4);`

Assembler Syntax: `EXTZL.<B/W> Dn, Dn`
`EXTZL.<B/W> Dn, <ea>`
`EXTZL.<B/W> <ea>, Dn`
`EXTZL.<B/W> <ea>, <ea>`

Attributes: Size = (Byte, Word)
 Words = (1-8, 2-8)
 Privilege = unprivileged
 Flags = unchanged

Description: Zero-extends the source operand from the instruction size to long size. The low source-size bytes of the source are copied into the destination, and the remaining high destination bits are filled with zero.

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

`EXTZL.X(s:B/W) Dn(d), Dn(D)`

Form Dn(d), Dn(D)
Encoding primary opcode range 0xe00 to 0xe7f
Words 1-8
Privilege unprivileged
Operands source operand is data register number; destination operand is data register number

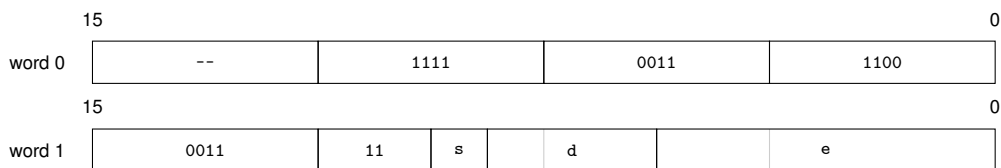


Instruction format for EXTZL.X(s:B/W) Dn(d), Dn(D)

d data register number for the source operand (primary word bits 2:0).
 D data register number for the destination operand (primary word bits 5:3).
 s size selector for the operand size (primary word bits 6).

EXTZL.X(s:B/W) Dn(d), <ea(e)>

Form	Dn(d), <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is data register number; destination operand is compact effective-address field

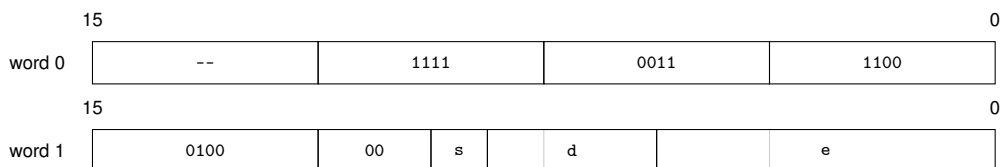


Instruction format for EXTZL.X(s:B/W) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the source operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

EXTZL.X(s:B/W) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number

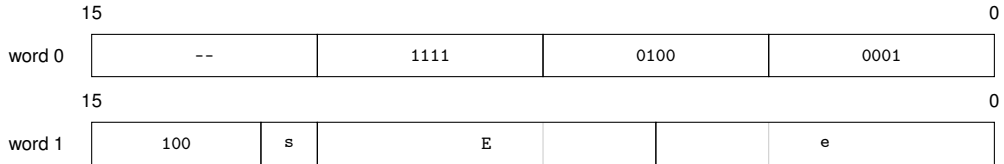


Instruction format for EXTZL.X(s:B/W) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the destination operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

EXTZL.X(s:B/W) <ea(e)>, <ea(E)>

Form	<ea(e)>, <ea(E)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is compact effective-address field



Instruction format for EXTZL.X(s:B/W) <ea(e)>, <ea(E)>

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- E compact effective-address field for the destination operand (extended opcode word bits 11:6). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 12).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

EXTZQ

Zero Extend To Quad

EXTZQ

Summary:

Operation:

Assembler Syntax:

Attributes:

Description:

Condition Codes:

Performs the zero extend to quad operation.

`dst_v = arch_zero_extend(src_v, size_v, 8);`

`EXTZQ.<B/W/L> Dn, Dn`
`EXTZQ.<B/W/L> Dn, <ea>`
`EXTZQ.<B/W/L> <ea>, Dn`
`EXTZQ.<B/W/L> <ea>, <ea>`

Size = (Byte, Word, Long)
Words = (1-8, 2-8)
Privilege = unprivileged
Flags = unchanged

Zero-extends the source operand from the instruction size to quad size. The low source-size bytes of the source are copied into the destination, and the remaining high destination bits are filled with zero.

Z N C V
- - - -
unchanged

Instruction Forms:

EXTZQ.X(s:B/W/L/invalid) Dn(d), Dn(D)

Form

Encoding

Words

Privilege

Operands

Dn(d), Dn(D)

primary opcode range 0x200 to 0x2ff

1-8

unprivileged

source operand is data register number; destination operand is data register number

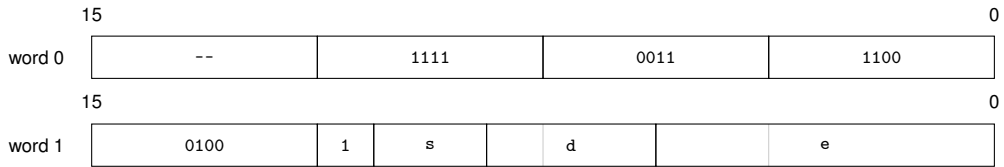


Instruction format for EXTZQ.X(s:B/W/L/invalid) Dn(d), Dn(D)

- d
- D
- s
- data register number for the source operand (primary word bits 2:0).
- data register number for the destination operand (primary word bits 5:3).
- size selector for the operand size (primary word bits 7:6).

EXTZQ.X(s:B/W/L/invalid) Dn(d), <ea(e)>

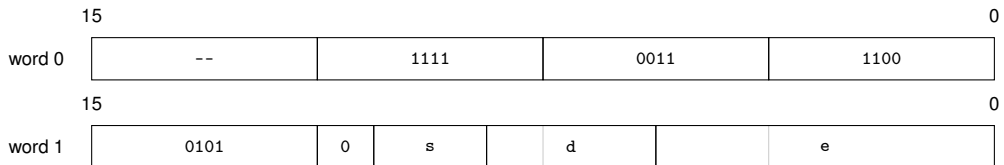
Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

**Instruction format for EXTZQ.X(s:B/W/L/invalid) Dn(d), <ea(e)>**

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
d data register number for the source operand (extended opcode word bits 8:6).
s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

EXTZQ.X(s:B/W/L/invalid) <ea(e)>, Dn(d)

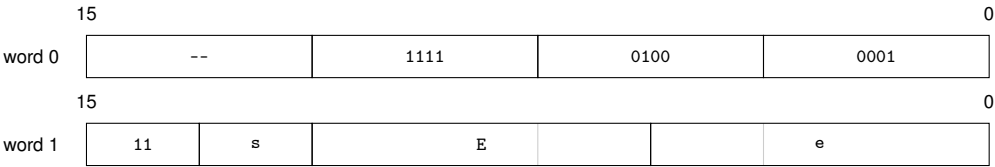
Form <ea(e)>, Dn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is data register number

**Instruction format for EXTZQ.X(s:B/W/L/invalid) <ea(e)>, Dn(d)**

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
d data register number for the destination operand (extended opcode word bits 8:6).
s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

EXTZQ.X(s:B/W/L/invalid) <ea(e)>, <ea(E)>

Form	<ea(e)>, <ea(E)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is compact effective-address field



Instruction format for EXTZQ.X(s:B/W/L/invalid) <ea(e)>, <ea(E)>

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- E compact effective-address field for the destination operand (extended opcode word bits 11:6). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 13:12). EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

EXTZW**Zero Extend To Word****EXTZW**

Summary: Performs the zero extend to word operation.

Operation: `dst_v = arch_zero_extend(src_v, size_v, 2);`

Assembler Syntax: `EXTZW <ea>, <ea>`

Attributes:
 Size = (Byte)
 Words = (2-8)
 Privilege = unprivileged
 Flags = unchanged

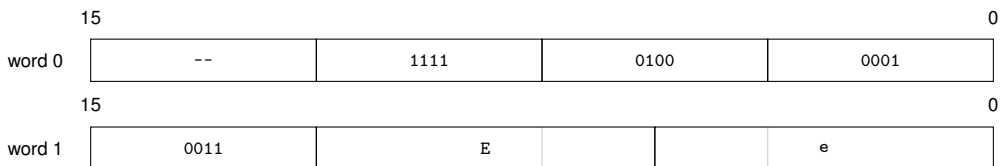
Description: Zero-extends the source operand from byte size to word size. The low byte of the source is copied into the destination, and the remaining high destination bits are filled with zero.

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

`EXTZW <ea(e)>, <ea(E)>`

Form `<ea(e)>, <ea(E)>`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is compact effective-address field



Instruction format for EXTZW <ea(e)>, <ea(E)>

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- E compact effective-address field for the destination operand (extended opcode word bits 11:6). EA field selects register, memory, immediate, and extended EA forms.
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FETCHADD

Atomic Fetch Add

FETCHADD

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the atomic fetch add operation.

tmp_v = memory_v;
memory_v = memory_v + src_v;
src_v = tmp_v;

FETCHADD.<B/W/L/Q>/<ORDER> Dn, <ea>

Size = (Byte, Word, Long, Quad)
Words = (3-8)
Privilege = unprivileged

Z N C V
- - - -
unchanged

Instruction Forms:

FETCHADD.X(s:B/W/L/Q)/ORDER(o) Dn(d), <ea(e)>

Form

Encoding

Words

Privilege

Operands

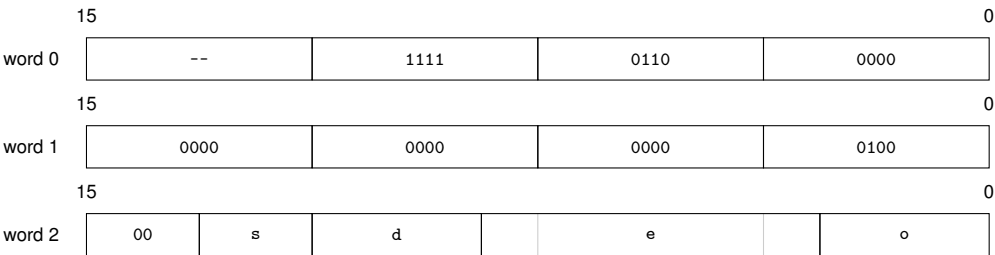
Dn(d), <ea(e)>

extended opcode form

3-8

unprivileged

source operand is data register number; memory operand is compact effective-address field



Instruction format for **FETCHADD.X(s:B/W/L/Q)/ORDER(o) Dn(d), <ea(e)>**

- o
- e
- d
- s
- atomic memory-order value for the ordering mode (payload word 1 bits 2:0).
- compact effective-address field for the memory operand (payload word 1 bits 8:3). EA field selects register, memory, immediate, and extended EA forms.
- data register number for the source operand (payload word 1 bits 11:9).
- size selector for the operand size (payload word 1 bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FETCHAND

Atomic Fetch And

FETCHAND

Summary: Performs the atomic fetch and operation.

Operation: tmp_v = memory_v;
memory_v = memory_v & src_v;
src_v = tmp_v;

Assembler Syntax: FETCHAND.<B/W/L/Q>/<ORDER> Dn, <ea>

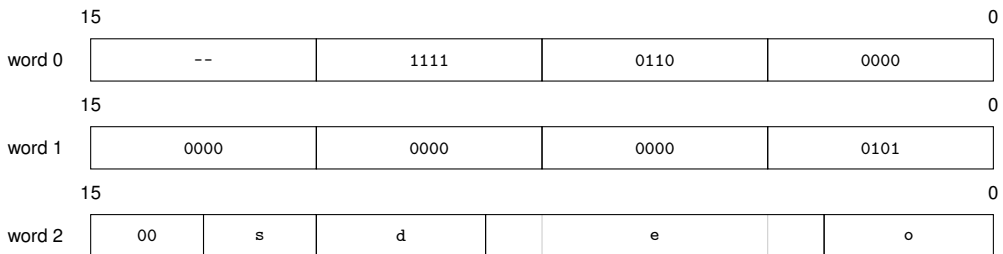
Attributes: Size = (Byte, Word, Long, Quad)
Words = (3-8)
Privilege = unprivileged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

FETCHAND.X(s:B/W/L/Q)/ORDER(o) Dn(d), <ea(e)>

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands source operand is data register number; memory operand is compact effective-address field



Instruction format for FETCHAND.X(s:B/W/L/Q)/ORDER(o) Dn(d), <ea(e)>

- o atomic memory-order value for the ordering mode (payload word 1 bits 2:0).
 - e compact effective-address field for the memory operand (payload word 1 bits 8:3). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the source operand (payload word 1 bits 11:9).
 - s size selector for the operand size (payload word 1 bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FETCHOR

Atomic Fetch Or

FETCHOR

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the atomic fetch or operation.

tmp_v = memory_v;
memory_v = memory_v | src_v;
src_v = tmp_v;

FETCHOR.<B/W/L/Q>/<ORDER> Dn, <ea>

Size = (Byte, Word, Long, Quad)
Words = (3-8)
Privilege = unprivileged

Z N C V
- - - -
unchanged

Instruction Forms:

FETCHOR.X(s:B/W/L/Q)/ORDER(o) Dn(d), <ea(e)>

Form

Encoding

Words

Privilege

Operands

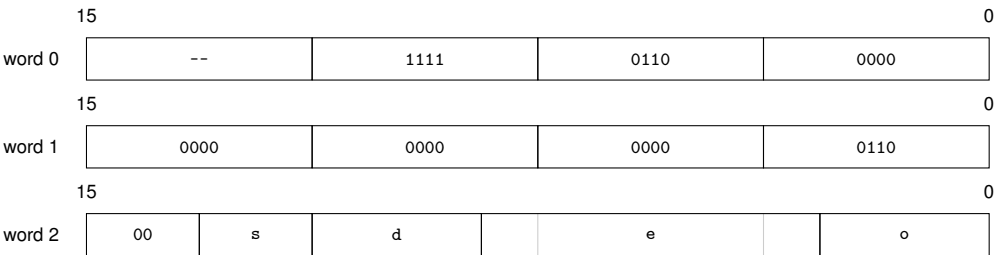
Dn(d), <ea(e)>

extended opcode form

3-8

unprivileged

source operand is data register number; memory operand is compact effective-address field



Instruction format for FETCHOR.X(s:B/W/L/Q)/ORDER(o) Dn(d), <ea(e)>

- o
- e
- d
- s
- atomic memory-order value for the ordering mode (payload word 1 bits 2:0).
- compact effective-address field for the memory operand (payload word 1 bits 8:3). EA field selects register, memory, immediate, and extended EA forms.
- data register number for the source operand (payload word 1 bits 11:9).
- size selector for the operand size (payload word 1 bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FETCHSUB**Atomic Fetch Subtract****FETCHSUB**

Summary: Performs the atomic fetch subtract operation.

Operation: $\text{tmp_v} = \text{memory_v};$
 $\text{memory_v} = \text{memory_v} - \text{src_v};$
 $\text{src_v} = \text{tmp_v};$

Assembler Syntax: `FETCHSUB.<B/W/L/Q>/<ORDER> Dn, <ea>`

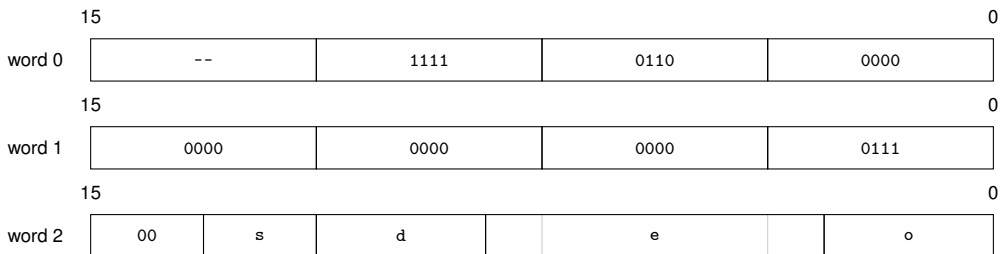
Attributes: Size = (Byte, Word, Long, Quad)
 Words = (3-8)
 Privilege = unprivileged

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

`FETCHSUB.X(s:B/W/L/Q)/ORDER(o) Dn(d), <ea(e)>`

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands source operand is data register number; memory operand is compact effective-address field



Instruction format for `FETCHSUB.X(s:B/W/L/Q)/ORDER(o) Dn(d), <ea(e)>`

- o** atomic memory-order value for the ordering mode (payload word 1 bits 2:0).
 - e** compact effective-address field for the memory operand (payload word 1 bits 8:3). EA field selects register, memory, immediate, and extended EA forms.
 - d** data register number for the source operand (payload word 1 bits 11:9).
 - s** size selector for the operand size (payload word 1 bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FETCHXOR

Atomic Fetch Xor

FETCHXOR

Summary: Performs the atomic fetch xor operation.

Operation: tmp_v = memory_v;
memory_v = memory_v ^ src_v;
src_v = tmp_v;

Assembler Syntax: FETCHXOR.<B/W/L/Q>/<ORDER> Dn, <ea>

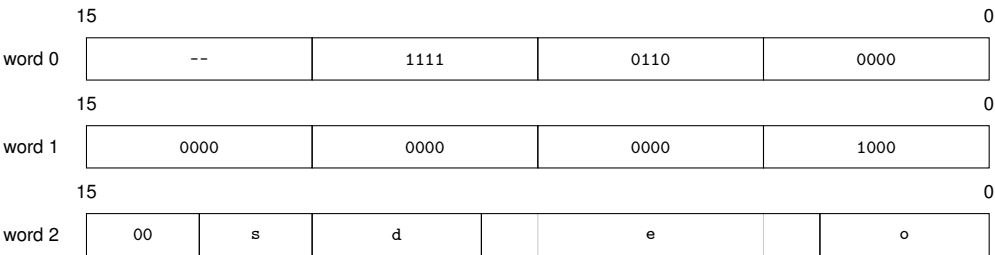
Attributes: Size = (Byte, Word, Long, Quad)
Words = (3-8)
Privilege = unprivileged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

FETCHXOR.X(s:B/W/L/Q)/ORDER(o) Dn(d), <ea(e)>

Form	Dn(d), <ea(e)>
Encoding	extended opcode form
Words	3-8
Privilege	unprivileged
Operands	source operand is data register number; memory operand is compact effective-address field



Instruction format for **FETCHXOR.X(s:B/W/L/Q)/ORDER(o) Dn(d), <ea(e)>**

- o atomic memory-order value for the ordering mode (payload word 1 bits 2:0).
 - e compact effective-address field for the memory operand (payload word 1 bits 8:3). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the source operand (payload word 1 bits 11:9).
 - s size selector for the operand size (payload word 1 bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FLSHDCACHE**Flush Data Cache****FLSHDCACHE**

Summary: Performs the flush data cache operation.

Operation: arch_cache_flush_data(target_v);

Assembler Syntax: FLSHDCACHE <ea>

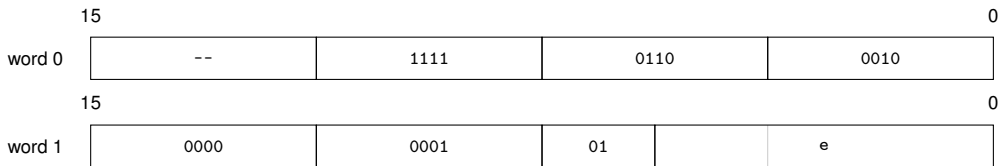
Attributes: Words = (2-8)
Privilege = supervisor or policy controlled
Flags = unchanged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

FLSHDCACHE <ea(e)>

Form <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege supervisor or policy controlled
Operands branch target is ea or range



Instruction format for FLSHDCACHE <ea(e)>

- e compact effective-address field for the branch target (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

GETDB

Get Data Register Bank

GETDB

Summary:

Reads the current data-register bank selector into a D register.

Operation:

`dst_v = DBANK & 0xf;`

Assembler Syntax:

`GETDB.D Dn`

Attributes:

Words = (2-8)
Privilege = user allowed
Flags = unchanged

Description:

Reads the current DBANK selector into the destination D register. The result is zero-extended.

Condition Codes:

Z N C V
- - - -
unchanged

Instruction Forms:

GETDB.D Dn(d)

Form

Dn(d)

Encoding

extended opcode form

Words

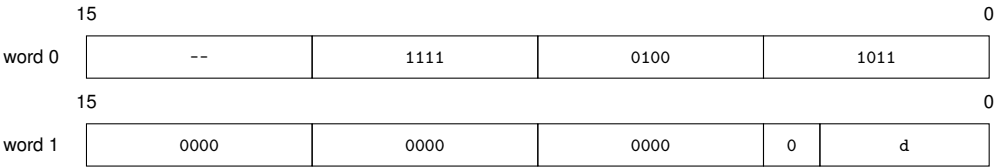
2-8

Privilege

user allowed

Operands

destination operand is data register number



Instruction format for GETDB.D Dn(d)

d data register number for the destination operand (extended opcode word bits 2:0).

HALT

Halt

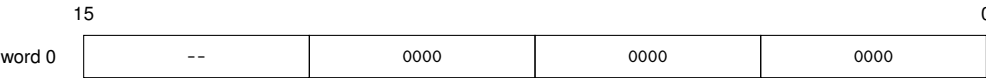
HALT

Summary:	Enter halted supervisor state until an enabled event resumes execution.
Operation:	arch_enter_halt();
Assembler Syntax:	HALT
Attributes:	Words = (1-8) Privilege = supervisor Flags = unchanged
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

HALT

Form	HALT
Encoding	primary opcode 0x000
Words	1-8
Privilege	supervisor
Operands	no explicit operands



Instruction format for HALT

No explicit operand fields are encoded in this form.

IJcc

Increment And Jump Conditional

IJcc

Summary: Performs the increment and jump conditional operation.

Operation: `index_v = index_v + 1;`
`if (index_v == bound_v) goto <ijcc_done>;`
`if (cc_v == 0) goto <ijcc_done>;`
`goto [target_v];`
`<ijcc_done>`

Assembler Syntax: `IJcc.<L/Q> Dn, Dn, <ea>`

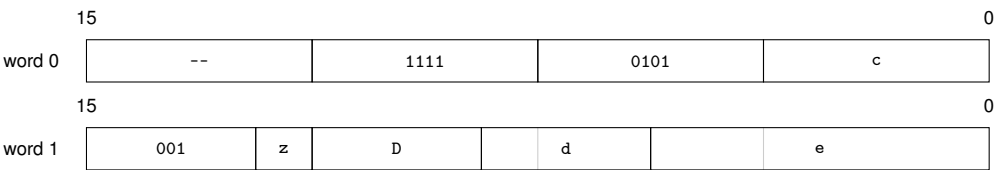
Attributes: Size = (Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

`IJcc.X(z:L/Q) Dn(d), Dn(D), <ea(e)>`

Form	Dn(d), Dn(D), <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	condition code is condition-code selector; index is data register number; bound is data register number; branch target is compact effective-address field



Instruction format for IJcc.X(z:L/Q) Dn(d), Dn(D), <ea(e)>

- e compact effective-address field for the branch target (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the index (extended opcode word bits 8:6).
 - D data register number for the bound (extended opcode word bits 11:9).
 - z size selector for the operand size (extended opcode word bits 12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

ILLEGAL**Illegal Instruction****ILLEGAL**

Summary: Raise illegal-instruction exception.

Operation: arch_raise_illegal_instruction();

Assembler Syntax: ILLEGAL

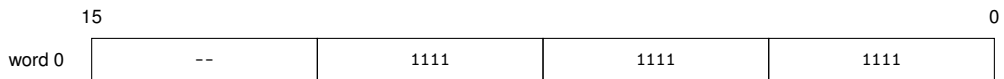
Attributes: Words = (1-8)
Privilege = unprivileged
Flags = unchanged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

ILLEGAL

Form ILLEGAL
Encoding primary opcode 0xff
Words 1-8
Privilege unprivileged
Operands no explicit operands



Instruction format for ILLEGAL

No explicit operand fields are encoded in this form.

INC

Increment

INC

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the increment operation.

result_v = dst_v + 1;
dst_v = result_v;
ZF = result_v == 0;
NF = result_v < 0;
CF = carry(dst_old_v, 1:8);
VF = scarry(dst_old_v, 1:8);

INC.<B/W/L/Q> Dn
INC.<B/W/L/Q> <ea>

Size = (Byte, Word, Long, Quad)
Words = (1-8, 2-8)
Privilege = unprivileged
Flags = Z: result == 0; N: most significant bit(result); C: unsigned carry out; V: signed overflow

Z N C V
* - - *

Z: result == 0; N: most significant bit(result); C: unsigned carry out; V: signed overflow

Instruction Forms:

INC.X(s:B/W/L/Q) Dn(d)

Form

Encoding

Words

Privilege

Operands

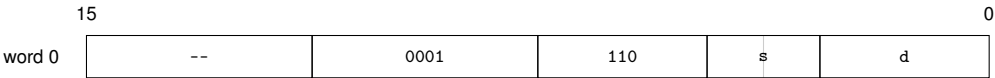
Dn(d)

primary opcode range 0x1c0 to 0x1df

1-8

unprivileged

destination operand is data register number



Instruction format for INC.X(s:B/W/L/Q) Dn(d)

d

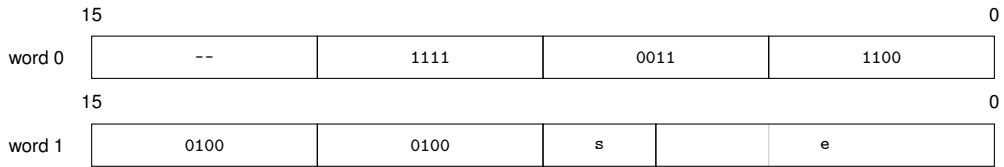
s

data register number for the destination operand (primary word bits 2:0).

size selector for the operand size (primary word bits 4:3).

INC.X(s:B/W/L/Q) <ea(e)>

Form <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands destination operand is compact effective-address field



Instruction format for INC.X(s:B/W/L/Q) <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 7:6).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

INCN

Increment Without Flags

INCN

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the increment without flags operation.

$\text{dst_v} = \text{dst_v} + 1;$

$\text{INCN}.\text{<B/W/L/Q> Dn}$
 $\text{INCN}.\text{<B/W/L/Q> <ea>}$

Size = (Byte, Word, Long, Quad)
Words = (1-8, 2-8)
Privilege = unprivileged
Flags = unchanged

Z N C V
- - - -
unchanged

Instruction Forms:

$\text{INCN}.\text{X}(\text{s}:\text{B/W/L/Q}) \text{ Dn}(\text{d})$

Form

Encoding

Words

Privilege

Operands

Dn(d)

primary opcode range 0x2e0 to 0x2ff

1-8

unprivileged

destination operand is data register number



Instruction format for INCN.X(s:B/W/L/Q) Dn(d)

d

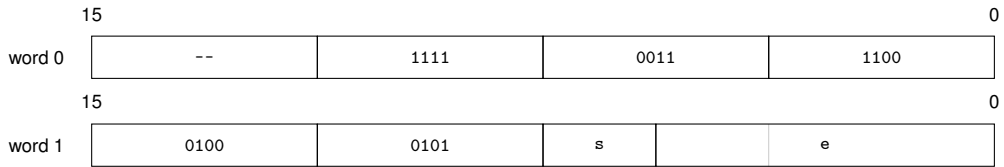
s

data register number for the destination operand (primary word bits 2:0).

size selector for the operand size (primary word bits 4:3).

INCN.X(s:B/W/L/Q) <ea(e)>

Form	<ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	destination operand is compact effective-address field



Instruction format for INCN.X(s:B/W/L/Q) <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 7:6).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

INVASID

Invalidate TLB By ASID

INVASID

Summary:

Invalidate TLB entries for ASID.

Operation:

arch_tlb_invalidate_asid(asid_v & 0xffff);

Assembler Syntax:

INVASID <imm16>

Attributes:

Words = (3-8)
Privilege = supervisor

Condition Codes:

Z N C V
- - - -
unchanged

Instruction Forms:

INVASID imm16

Form

imm16

Encoding

extended opcode form

Words

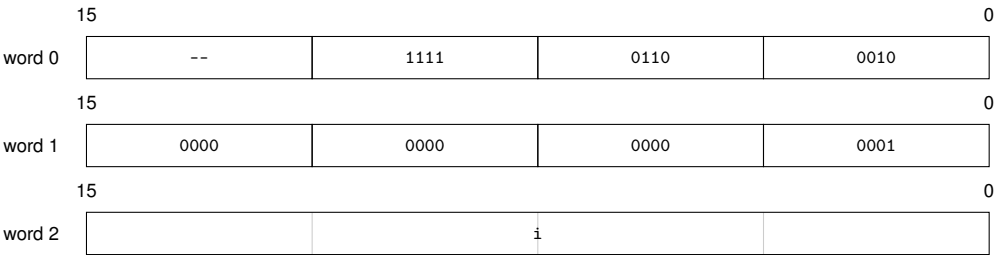
3-8

Privilege

supervisor

Operands

address-space identifier is 16-bit immediate selector



Instruction format for INVASID imm16

i imm16 for the address-space identifier (payload word 1 bits 15:0).

INVDCACHE**Invalidate Data Cache****INVDCACHE**

Summary: Invalidate data cache block or range.

Operation: arch_cache_invalidate_data(target_v);

Assembler Syntax: INVDCACHE <ea>

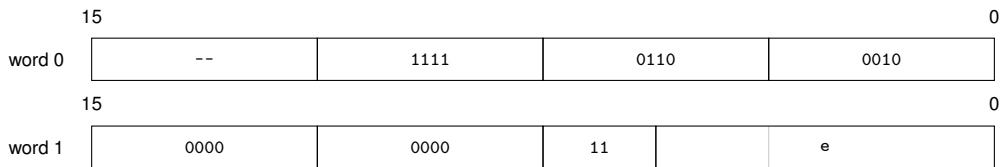
Attributes: Words = (2-8)
Privilege = supervisor or policy controlled
Flags = unchanged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

INVDCACHE <ea(e)>

Form <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege supervisor or policy controlled
Operands branch target is ea or range

**Instruction format for INVDCACHE <ea(e)>**

- e compact effective-address field for the branch target (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

INVICACHE

Invalidate Instruction Cache

INVICACHE

Summary:

Invalidate instruction cache block or range.

Operation:

arch_cache_invalidate_instruction(target_v);

Assembler Syntax:

INVICACHE <ea>

Attributes:

Words = (2-8)
Privilege = supervisor or policy controlled
Flags = unchanged

Condition Codes:

Z N C V
- - - -
unchanged

Instruction Forms:

INVICACHE <ea(e)>

Form

<ea(e)>

Encoding

extended opcode form

Words

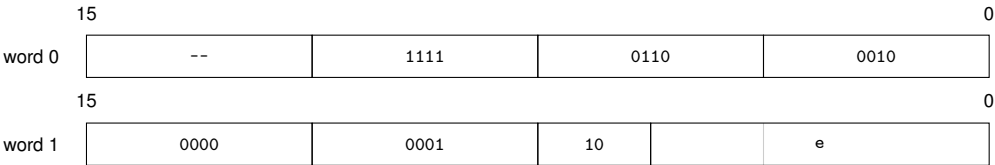
2-8

Privilege

supervisor or policy controlled

Operands

branch target is ea or range



Instruction format for INVICACHE <ea(e)>

e

compact effective-address field for the branch target (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

INVPAGE**Invalidate TLB Page****INVPAGE**

Summary: Invalidate TLB entry for linear page.

Operation: `arch_tlb_invalidate_page(page_v);`

Assembler Syntax: `INVPAGE <ea>`

Attributes: Words = (2-8)
Privilege = supervisor

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

`INVPAGE <ea(e)>`

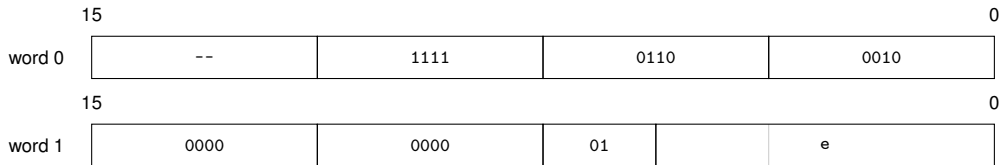
Form `<ea(e)>`

Encoding extended opcode form

Words 2-8

Privilege supervisor

Operands page is compact effective-address field



Instruction format for INVPAGE <ea(e)>

- e compact effective-address field for the page (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

INVTLB

Invalidate TLB

INVTLB

Summary:

Invalidate all TLB entries.

Operation:

arch_tlb_invalidate_all();

Assembler Syntax:

INVTLB

Attributes:

Words = (2-8)
Privilege = supervisor

Condition Codes:

Z N C V
- - - -
unchanged

Instruction Forms:

INVTLB

Form

INVTLB

Encoding

extended opcode form

Words

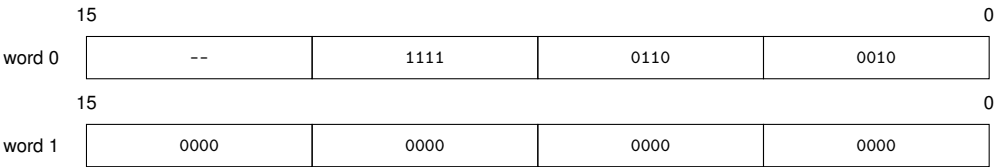
2-8

Privilege

supervisor

Operands

no explicit operands



Instruction format for INVTLB

No explicit operand fields are encoded in this form.

IRET**Interrupt Return****IRET**

Summary: Restore all saved control state from an interrupt return frame.

Operation:

```

local frame_base_v:8 = SP;
local frame_control_v:8 = *:8 frame_base_v;
local return_pc_v:8 = *:8 (frame_base_v + 0x18);
local return_sp_v:8 = *:8 (frame_base_v + 0x20);
local return_cs_v:8 = *:8 (frame_base_v + 0x28);
local return_ds_v:8 = *:8 (frame_base_v + 0x30);
local return_ss_v:8 = *:8 (frame_base_v + 0x38);
DBANK = (frame_control_v >> 27) & 0xf;
tmp_v = (frame_control_v >> 32) & 0xffff;
ZF = (tmp_v >> 3) & 1;
NF = (tmp_v >> 2) & 1;
CF = (tmp_v >> 1) & 1;
VF = tmp_v & 1;
STATUS = (frame_control_v >> 48) & 0xffff;
CS = return_cs_v;
DS = return_ds_v;
SS = return_ss_v;
SP = return_sp_v;
return [return_pc_v];

```

Assembler Syntax: IRET

Attributes:

- Words = (1-8)
- Privilege = supervisor
- Flags = unchanged

Description: Restore all saved control state from an interrupt return frame. IRET is the interrupt, trap, and exception return instruction. It restores all saved interrupt-frame state, including the saved FRAME_CONTROL state, and does not validate FRAME_CONTROL.frame_type; malformed return frames are software errors and do not raise a separate architectural exception.

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

IRET

Form	IRET
Encoding	primary opcode 0xf3a
Words	1-8
Privilege	supervisor
Operands	no explicit operands



Instruction format for IRET

No explicit operand fields are encoded in this form.

JMP**Jump****JMP**

Summary: Transfers control to the target address.

Operation: goto [target_v];

Assembler Syntax: JMP.<W/L> <imm>
JMP.<W/L> <ea>

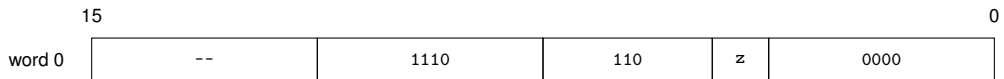
Attributes: Size = (Word, Long)
Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

JMP.X(z:W/L) imm

Form imm
Encoding canonical alias in primary opcode space
Words 2-8
Privilege unprivileged
Operands branch target is relative imm
Notes alias_of=Jcc.IMM

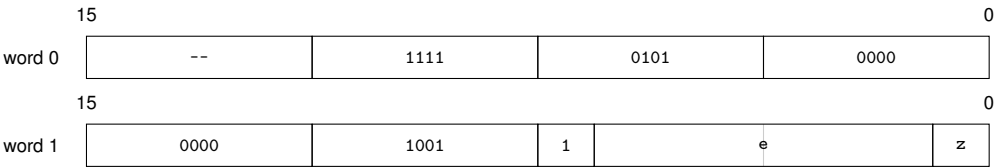


Instruction format for JMP.X(z:W/L) imm

z size selector for the operand size (primary word bits 4).

JMP.X(z:W/L) <ea(e)>

Form	<ea(e)>
Encoding	canonical alias in extended opcode space
Words	2-8
Privilege	unprivileged
Operands	branch target is compact effective-address field
Notes	alias_of=Jcc.EA



Instruction format for JMP.X(z:W/L) <ea(e)>

- z size selector for the operand size (extended opcode word bits 0).
 - e compact effective-address field for the branch target (extended opcode word bits 6:1). EA field selects register, memory, immediate, and extended EA forms.
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

Jcc**Jump Conditional****Jcc**

Summary: Transfers control when the encoded condition is true.

Operation: if (cc_v == 0) goto <jcc_done>;
goto [target_v];
<jcc_done>

Assembler Syntax: Jcc.<W/L> <imm>
Jcc.<W/L> <ea>

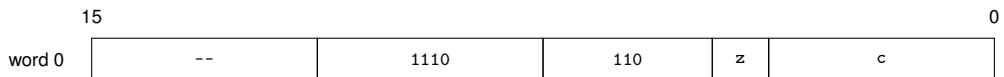
Attributes: Size = (Word, Long)
Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

Jcc.X(z:W/L) imm

Form imm
Encoding primary opcode range 0xec0 to 0xedf
Words 2-8
Privilege unprivileged
Operands condition code is condition-code selector; branch target is relative imm

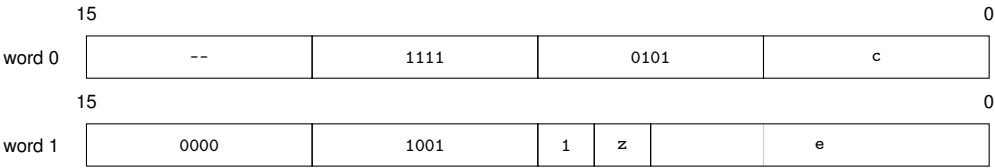


Instruction format for Jcc.X(z:W/L) imm

c condition-code selector for the condition code (primary word bits 3:0).
z size selector for the operand size (primary word bits 4).

Jcc.X(z:W/L) <ea(e)>

Form	<ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	condition code is condition-code selector; branch target is compact effective-address field



Instruction format for Jcc.X(z:W/L) <ea(e)>

- e compact effective-address field for the branch target (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - z size selector for the operand size (extended opcode word bits 6).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

LCALL**Long Call****LCALL**

Summary: Pushes a far return frame and atomically transfers control to a new CS:PC pair.

Operation: local return_pc_v:8 = inst_next;
 SP = SP - 16;
 *:8 SP = return_pc_v;
 *:8 (SP + 8) = CS;
 CS = new_cs_v;
 call [target_v];

Assembler Syntax: LCALL Dn, <ea>

Attributes: Words = (2-8)
 Privilege = unprivileged
 Flags = unchanged

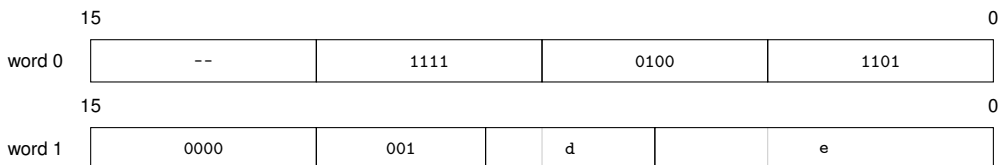
Description: Long call is the CS-changing call instruction. It first constructs the far return frame on SS:SP, then commits the new CS and PC together. The two-operand form takes the new CS image from Dn and the target offset/address from the target EA. A segment-immediate plus offset-immediate form is not encoded because two 64-bit immediates would exceed the 8-word instruction limit.

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

LCALL Dn(d), <ea(e)>

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands new code-segment value is data register number; branch target is compact effective-address field



Instruction format for LCALL Dn(d), <ea(e)>

- e compact effective-address field for the branch target (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the new code-segment value (extended opcode word bits 8:6).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

LEA**Load Effective Address****LEA**

Summary: Performs the load effective address operation.

Operation: `dst_v = src_v;`

Assembler Syntax: `LEA <ea>, An`

Attributes:
 Words = (2-8)
 Privilege = unprivileged
 Flags = unchanged

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

`LEA <ea(e)>, An(a)`

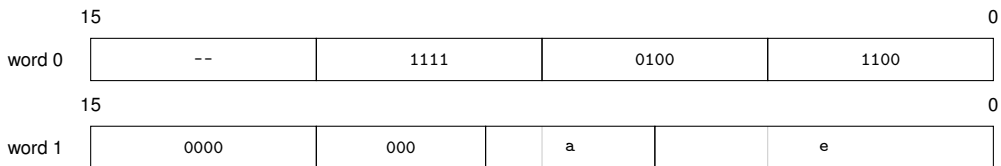
Form `<ea(e)>, An(a)`

Encoding extended opcode form

Words 2-8

Privilege unprivileged

Operands source operand is compact effective-address field; destination operand is address register number



Instruction format for LEA <ea(e)>, An(a)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- a address register number for the destination operand (extended opcode word bits 8:6).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

LJMP

Long Jump

LJMP

Summary:

Atomically transfers control to a new CS:PC pair.

Operation:

CS = new_cs_v;
goto [target_v];

Assembler Syntax:

LJMP Dn, <ea>

Attributes:

Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Description:

Long jump is the CS-changing jump instruction. It commits the new CS and PC as a single architectural control-transfer step, so software does not need to write CS directly while execution is still using the old PC. The two-operand form takes the new CS image from Dn and the target offset/address from the target EA. A segment-immediate plus offset-immediate form is not encoded because two 64-bit immediates would exceed the 8-word instruction limit.

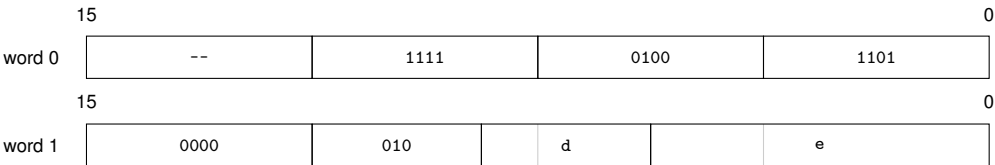
Condition Codes:

Z N C V
- - - -
unchanged

Instruction Forms:

LJMP Dn(d), <ea(e)>

Form	Dn(d), <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	new code-segment value is data register number; branch target is compact effective-address field



Instruction format for LJMP Dn(d), <ea(e)>

- e compact effective-address field for the branch target (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the new code-segment value (extended opcode word bits 8:6).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

LRET**Long Return****LRET**

Summary: Pops a far return frame and atomically restores CS:PC.

Operation: local return_pc_v:8 = *:8 SP;
 local return_cs_v:8 = *:8 (SP + 8);
 SP = SP + 16;
 CS = return_cs_v;
 return [return_pc_v];

Assembler Syntax: LRET

Attributes: Words = (2-8)
 Privilege = unprivileged
 Flags = unchanged

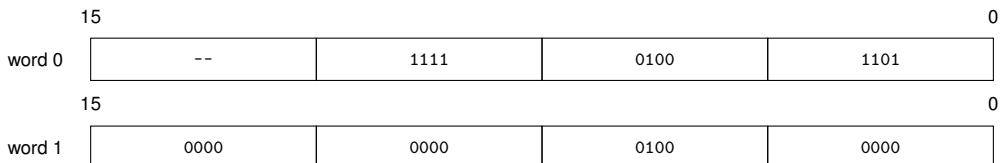
Description: Long return reads the far return frame from SS:SP and commits CS and PC together. The frame contains the return PC Q value at the current SP and the return CS Q value at SP+8; after both values are fetched, SP is advanced by 16 bytes and the control state is committed.

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

LRET

Form LRET
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands no explicit operands



Instruction format for LRET

No explicit operand fields are encoded in this form.

MADD

Multiply Add

MADD

Summary:

Adds the low-half product of a source operand and D register into a D accumulator.

Operation:

$acc_v = acc_v + (src_v * multiplier_v);$

Assembler Syntax:

MADD.<B/W/L/Q> <ea>, Dn, Dn

Attributes:

Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Description:

Adds the low-half product of the source operand and multiplier D register into the accumulator D register. The source operand may use any effective-address form, including direct D-register EA forms. The accumulator is always a D register and condition codes are unchanged.

Condition Codes:

Z N C V
- - - -
unchanged

Instruction Forms:

MADD.X(s:B/W/L/Q) <ea(e)>, Dn(d), Dn(D)

Form

<ea(e)>, Dn(d), Dn(D)

Encoding

extended opcode form

Words

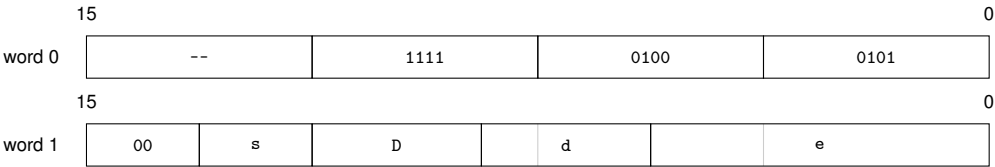
2-8

Privilege

unprivileged

Operands

source operand is compact effective-address field; multiplier is data register number; acc is data register number



Instruction format for MADD.X(s:B/W/L/Q) <ea(e)>, Dn(d), Dn(D)

- e
- compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d
- data register number for the multiplier (extended opcode word bits 8:6).
- D
- data register number for the acc (extended opcode word bits 11:9).
- s
- size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MAXS**Signed Maximum****MAXS**

Summary: Performs the signed maximum operation.

Operation: if (dst_v s< src_v) goto <minmax_done>;
dst_v = src_v;
<minmax_done>

Assembler Syntax: MAXS.<B/W/L/Q> Dn, <ea>
MAXS.<B/W/L/Q> <ea>, Dn

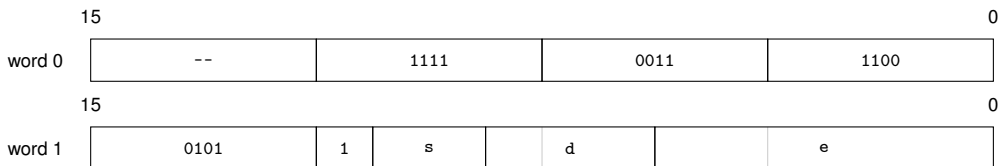
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = may update Z/N

Condition Codes: **Z N C V**
* * - -
may update Z/N

Instruction Forms:

MAXS.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

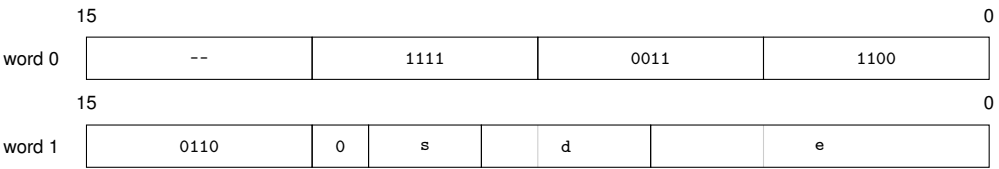


Instruction format for MAXS.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the source operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MAXS.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for MAXS.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MAXU**Unsigned Maximum****MAXU**

Summary: Performs the unsigned maximum operation.

Operation: if (dst_v < src_v) goto <minmax_done>;
dst_v = src_v;
<minmax_done>

Assembler Syntax: MAXU.<B/W/L/Q> Dn, <ea>
MAXU.<B/W/L/Q> <ea>, Dn

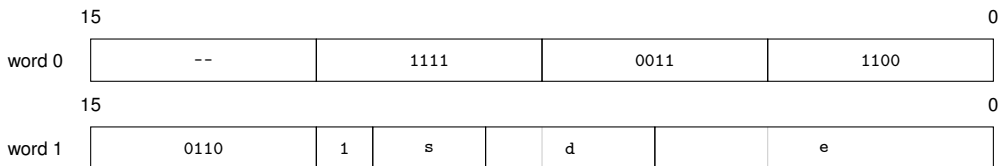
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = may update Z/N

Condition Codes: **Z N C V**
* * - -
may update Z/N

Instruction Forms:

MAXU.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

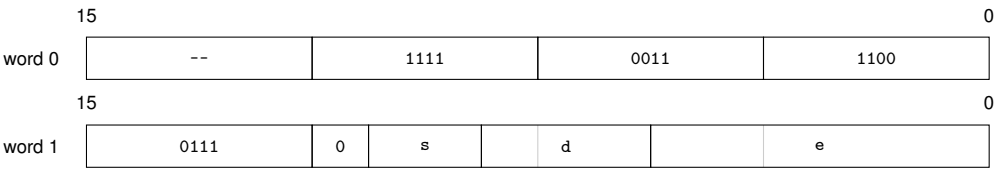


Instruction format for MAXU.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the source operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MAXU.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for MAXU.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MINS**Signed Minimum****MINS**

Summary: Performs the signed minimum operation.

Operation: if (dst_v s> src_v) goto <minmax_done>;
dst_v = src_v;
<minmax_done>

Assembler Syntax: MINS.<B/W/L/Q> Dn, <ea>
MINS.<B/W/L/Q> <ea>, Dn

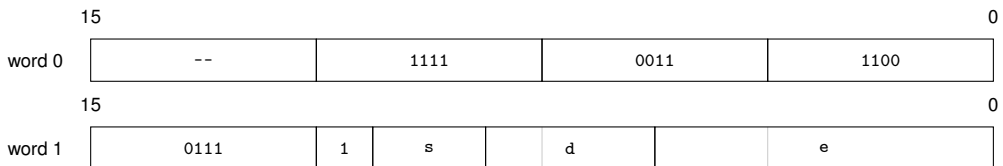
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = may update Z/N

Condition Codes: **Z N C V**
* * - -
may update Z/N

Instruction Forms:

MINS.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

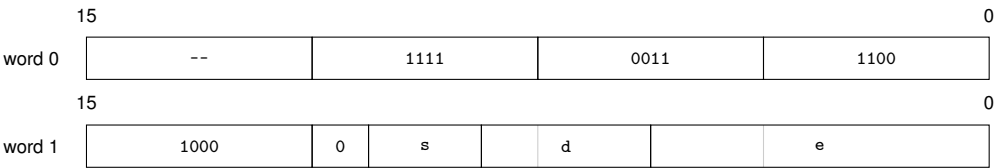


Instruction format for MINS.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the source operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MINS.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for MINS.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MINU**Unsigned Minimum****MINU**

Summary: Performs the unsigned minimum operation.

Operation: if (dst_v > src_v) goto <minmax_done>;
dst_v = src_v;
<minmax_done>

Assembler Syntax: MINU.<B/W/L/Q> Dn, <ea>
MINU.<B/W/L/Q> <ea>, Dn

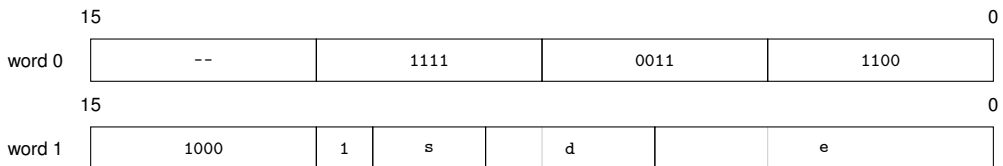
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = may update Z/N

Condition Codes: **Z N C V**
* * - -
may update Z/N

Instruction Forms:

MINU.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

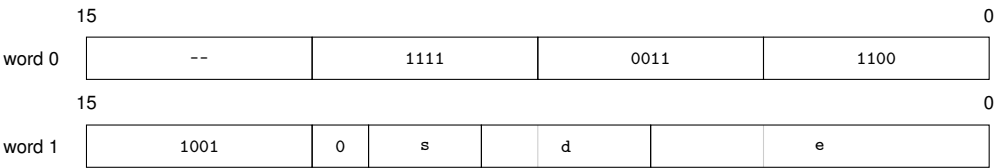


Instruction format for MINU.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the source operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MINU.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for MINU.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MODS**Signed Modulo****MODS**

Summary: Performs the signed modulo operation.

Operation: `dst_v = arch_signed_modulo(dst_v, src_v, size_v);`

Assembler Syntax: `MODS.<B/W/L/Q> Dn, <ea>`
`MODS.<B/W/L/Q> <ea>, Dn`

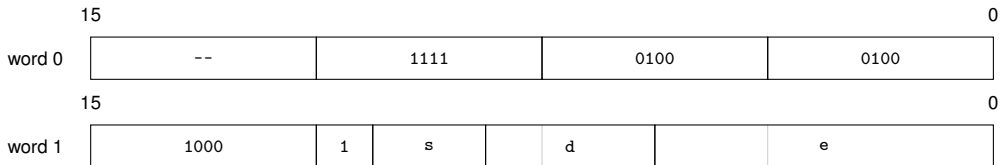
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

`MODS.X(s:B/W/L/Q) Dn(d), <ea(e)>`

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

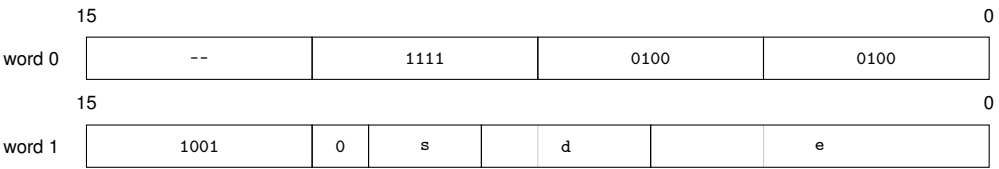


Instruction format for MODS.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the source operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MODS.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for MODS.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MODU**Unsigned Modulo****MODU**

Summary: Performs the unsigned modulo operation.

Operation: `dst_v = arch_unsigned_modulo(dst_v, src_v, size_v);`

Assembler Syntax: `MODU.<B/W/L/Q> Dn, <ea>`
`MODU.<B/W/L/Q> <ea>, Dn`

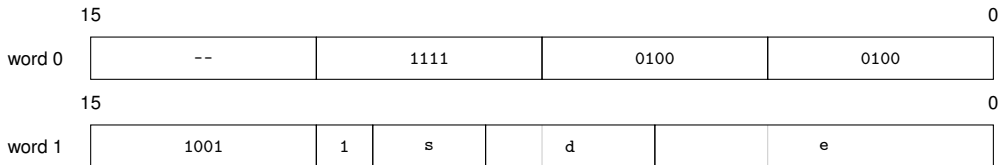
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

`MODU.X(s:B/W/L/Q) Dn(d), <ea(e)>`

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

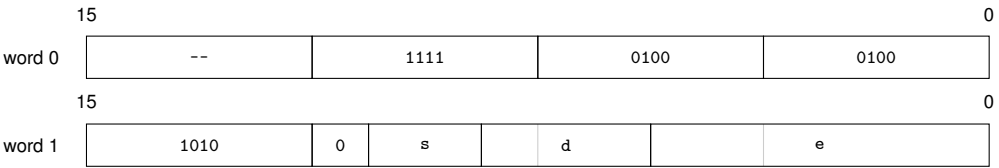


Instruction format for MODU.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the source operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MODU.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for MODU.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MOV**Move****MOV**

Summary: Copies the source operand to the destination operand.

Operation: `dst_v = src_v;`

Assembler Syntax: `MOV.<L/Q> Dn, <ea>`
`MOV.<L/Q> <ea>, Dn`
`MOV <imm64>, An`
`MOV.<B/W/L/Q> Dn, <ea>`
`MOV.<B/W/L/Q> <ea>, Dn`
`MOV.<B/W/L/Q> <ea>, <ea>`

Attributes: Size = (Byte, Word, Long, Quad, Long, Quad)
Words = (1-8, 2-8, 5-8)
Privilege = unprivileged
Flags = unchanged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

`MOV.X(z:L/Q) Dn(d), <ea(e)>`

Form Dn(d), <ea(e)>
Encoding primary opcode range 0x400 to 0x7ff
Words 1-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

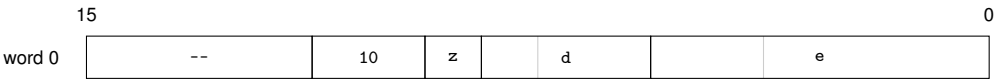


Instruction format for MOV.X(z:L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (primary word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the source operand (primary word bits 8:6).
- z size selector for the operand size (primary word bits 9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MOV.X(z:L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	primary opcode range 0x800 to 0xbff
Words	1-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for MOV.X(z:L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (primary word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (primary word bits 8:6).
- z size selector for the operand size (primary word bits 9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MOV imm64, An(a)

Form	imm64, An(a)
Encoding	primary opcode range 0xf28 to 0xf2f
Words	5-8
Privilege	unprivileged
Operands	source operand is imm64; destination operand is address register number

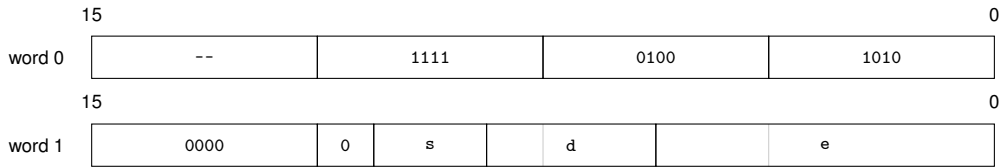


Instruction format for MOV imm64, An(a)

- a address register number for the destination operand (primary word bits 2:0).

MOV.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

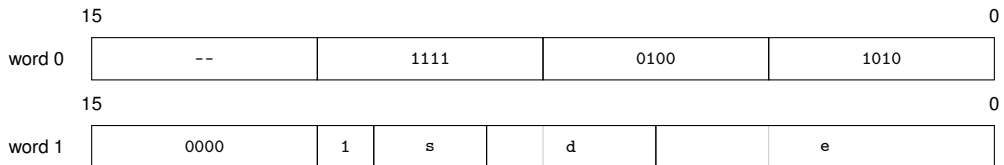


Instruction format for MOV.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the source operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MOV.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form <ea(e)>, Dn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is data register number

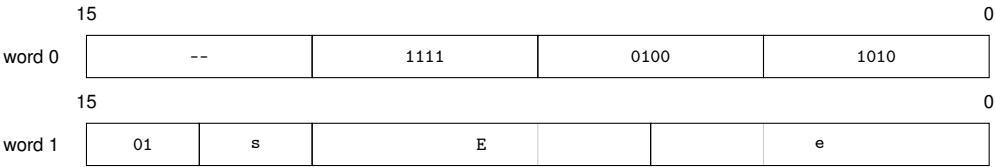


Instruction format for MOV.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the destination operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MOV.X(s:B/W/L/Q) <ea(e)>, <ea(E)>

Form	<ea(e)>, <ea(E)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is compact effective-address field



Instruction format for MOV.X(s:B/W/L/Q) <ea(e)>, <ea(E)>

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- E compact effective-address field for the destination operand (extended opcode word bits 11:6). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 13:12). EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MOVSETAD**Move D Registers To A Registers****MOVSETAD**

Summary: Copies selected D registers from a data-register bank to matching A registers.

Operation: MOVSETAD copies each selected D register to the matching A register. The one-operand form reads D[i] from the current data-register bank; the banked form reads D[i] from the explicitly named source DBANK. Bits 0 through 7 select register numbers 0 through 7.

Assembler Syntax: MOVSETAD <bitmap16>
MOVSETAD DBn, <bitmap16>

Attributes: Words = (2-8, 3-8)
Privilege = unprivileged
Flags = unchanged

Description: Copies selected D registers to matching A registers. The one-operand form uses the current DBANK selector. The banked form names an explicit source DBANK and does not change the current selector.

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

MOVSETAD <bitmap>

Form <bitmap>
Encoding primary opcode 0xf34
Words 2-8
Privilege unprivileged
Operands register bitmap is 16-bit register bitmap

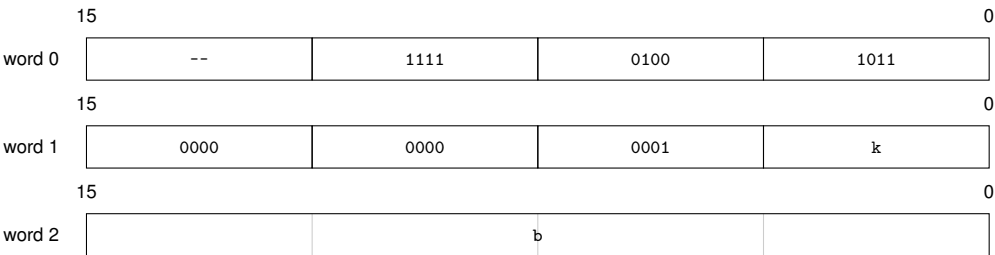


Instruction format for MOVSETAD <bitmap>

No explicit operand fields are encoded in this form.

MOVSETAD DBn(k), <bitmap>

Form	DBn(k), <bitmap>
Encoding	extended opcode form
Words	3-8
Privilege	unprivileged
Operands	src bank is dbank; register bitmap is 16-bit register bitmap



Instruction format for MOVSETAD DBn(k), <bitmap>

- k dbank for the src bank (extended opcode word bits 3:0).
- b 16-bit register bitmap for the register bitmap (payload word 1 bits 15:0).

MOVSETDA**Move A Registers To D Registers****MOVSETDA**

Summary:	Copies selected A registers to matching D registers in a data-register bank.
Operation:	MOVSETDA copies each selected A register to the matching D register. The one-operand form writes D[i] in the current data-register bank; the banked form writes D[i] in the explicitly named destination DBANK. Bits 0 through 7 select register numbers 0 through 7.
Assembler Syntax:	MOVSETDA <bitmap16> MOVSETDA DBn, <bitmap16>
Attributes:	Words = (2-8, 3-8) Privilege = unprivileged Flags = unchanged
Description:	Copies selected A registers to matching D registers. The one-operand form uses the current DBANK selector. The banked form names an explicit destination DBANK and does not change the current selector.
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

MOVSETDA <bitmap>

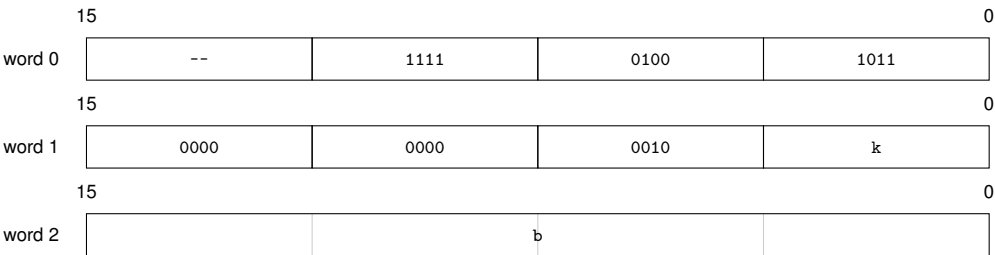
Form	<bitmap>
Encoding	primary opcode 0xf35
Words	2-8
Privilege	unprivileged
Operands	register bitmap is 16-bit register bitmap

**Instruction format for MOVSETDA <bitmap>**

No explicit operand fields are encoded in this form.

MOVSETDA DBn(k), <bitmap>

Form	DBn(k), <bitmap>
Encoding	extended opcode form
Words	3-8
Privilege	unprivileged
Operands	dst bank is dbank; register bitmap is 16-bit register bitmap



Instruction format for MOVSETDA DBn(k), <bitmap>

- k dbank for the dst bank (extended opcode word bits 3:0).
- b 16-bit register bitmap for the register bitmap (payload word 1 bits 15:0).

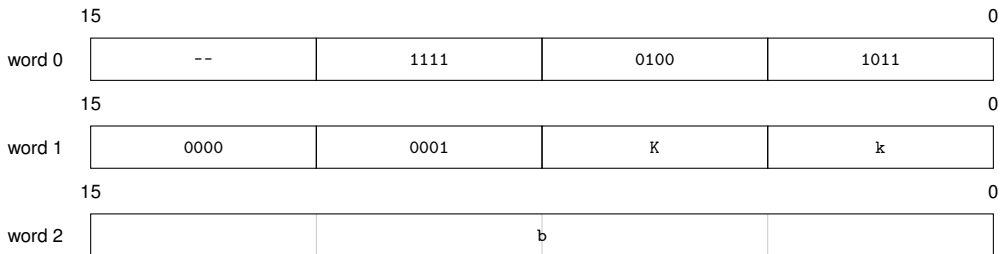
MOVSETDD**Move D Register Set Between Banks****MOVSETDD**

Summary:	Copies selected D registers from one data-register bank to another.
Operation:	MOVSETDD copies each selected D register from the source data-register bank to the destination data-register bank. Bits 0 through 7 select D0 through D7 respectively. The current DBANK selector is not changed.
Assembler Syntax:	MOVSETDD DBn, DBn, <bitmap16>
Attributes:	Words = (3-8) Privilege = unprivileged Flags = unchanged
Description:	Copies selected D registers from the source DBANK to the destination DBANK according to the bitmap operand. The current DBANK selector is not changed.
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

MOVSETDD DBn(k), DBn(K), <bitmap>

Form	DBn(k), DBn(K), <bitmap>
Encoding	extended opcode form
Words	3-8
Privilege	unprivileged
Operands	dst bank is dbank; src bank is dbank; register bitmap is 16-bit register bitmap

**Instruction format for MOVSETDD DBn(k), DBn(K), <bitmap>**

- k dbank for the dst bank (extended opcode word bits 3:0).
- K dbank for the src bank (extended opcode word bits 7:4).
- b 16-bit register bitmap for the register bitmap (payload word 1 bits 15:0).

MOVcc

Move Conditional

MOVcc

Summary: Performs the move conditional operation.

Operation: if (cc_v == 0) goto <movcc_done>;
dst_v = src_v;
<movcc_done>

Assembler Syntax: MOVcc.<B/W/L/Q> An, <ea>
MOVcc.<B/W/L/Q> Dn, <ea>
MOVcc.<B/W/L/Q> <ea>, An
MOVcc.<B/W/L/Q> <ea>, Dn

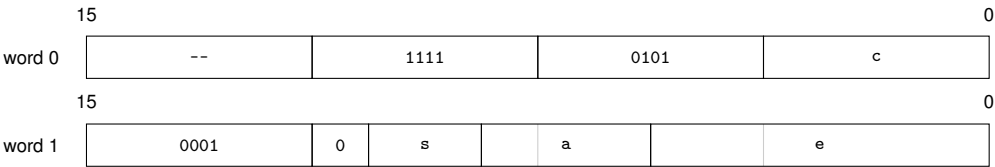
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

MOVcc.X(s:B/W/L/Q) An(a), <ea(e)>

Form	An(a), <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	condition code is condition-code selector; source operand is address register number; destination operand is compact effective-address field

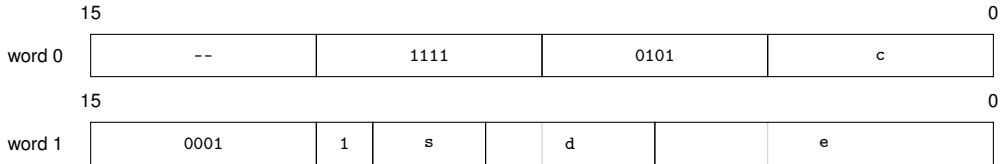


Instruction format for MOVcc.X(s:B/W/L/Q) An(a), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- a address register number for the source operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MOVcc.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands condition code is condition-code selector; source operand is data register number; destination operand is compact effective-address field

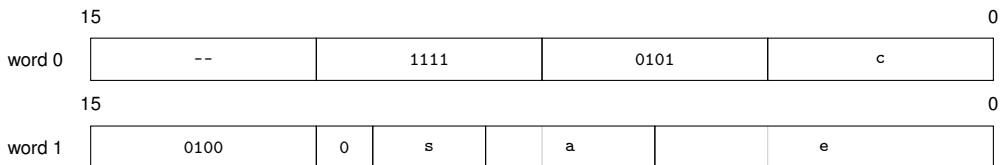


Instruction format for MOVcc.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the source operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MOVcc.X(s:B/W/L/Q) <ea(e)>, An(a)

Form <ea(e)>, An(a)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands condition code is condition-code selector; source operand is compact effective-address field; destination operand is address register number

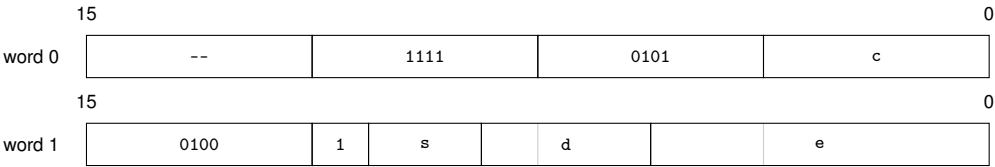


Instruction format for MOVcc.X(s:B/W/L/Q) <ea(e)>, An(a)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- a address register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MOVcc.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	condition code is condition-code selector; source operand is compact effective-address field; destination operand is data register number



Instruction format for MOVcc.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the destination operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MSUB**Multiply Subtract****MSUB**

Summary: Subtracts the low-half product of a source operand and D register from a D accumulator.

Operation: $\text{acc_v} = \text{acc_v} - (\text{src_v} * \text{multiplier_v});$

Assembler Syntax: `MSUB.<B/W/L/Q> <ea>, Dn, Dn`

Attributes:
 Size = (Byte, Word, Long, Quad)
 Words = (2-8)
 Privilege = unprivileged
 Flags = unchanged

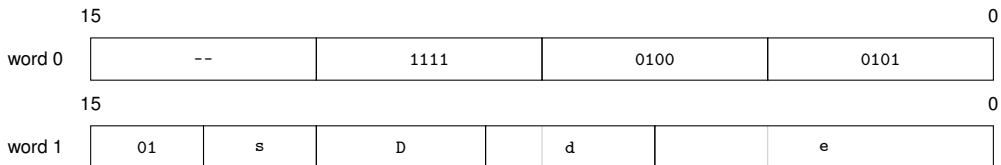
Description: Subtracts the low-half product of the source operand and multiplier D register from the accumulator D register. The source operand may use any effective-address form, including direct D-register EA forms. The accumulator is always a D register and condition codes are unchanged.

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

`MSUB.X(s:B/W/L/Q) <ea(e)>, Dn(d), Dn(D)`

Form `<ea(e)>, Dn(d), Dn(D)`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; multiplier is data register number; acc is data register number



Instruction format for MSUB.X(s:B/W/L/Q) <ea(e)>, Dn(d), Dn(D)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the multiplier (extended opcode word bits 8:6).
 - D data register number for the acc (extended opcode word bits 11:9).
 - s size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MULHS

Signed Multiply High

MULHS

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the signed multiply high operation.

`dst_v = arch_signed_multiply_high(dst_v, src_v, size_v);`

`MULHS.<B/W/L/Q> Dn, <ea>`
`MULHS.<B/W/L/Q> <ea>, Dn`

Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged

Z N C V
- - - -
unchanged

Instruction Forms:

`MULHS.X(s:B/W/L/Q) Dn(d), <ea(e)>`

Form

Encoding

Words

Privilege

Operands

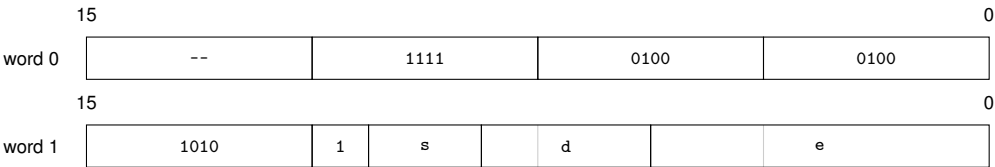
Dn(d), <ea(e)>

extended opcode form

2-8

unprivileged

source operand is data register number; destination operand is compact effective-address field

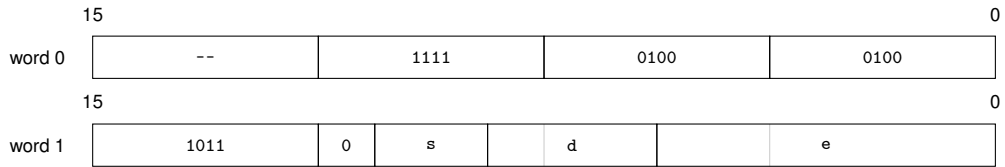


Instruction format for MULHS.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the source operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MULHS.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for MULHS.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the destination operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MULHSU

Signed By Unsigned Multiply High

MULHSU

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the signed by unsigned multiply high operation.

`dst_v = arch_signed_unsigned_multiply_high(dst_v, src_v, size_v);`

`MULHSU.<B/W/L/Q> Dn, <ea>`
`MULHSU.<B/W/L/Q> <ea>, Dn`

Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged

Z N C V
- - - -
unchanged

Instruction Forms:

`MULHSU.X(s:B/W/L/Q) Dn(d), <ea(e)>`

Form

Encoding

Words

Privilege

Operands

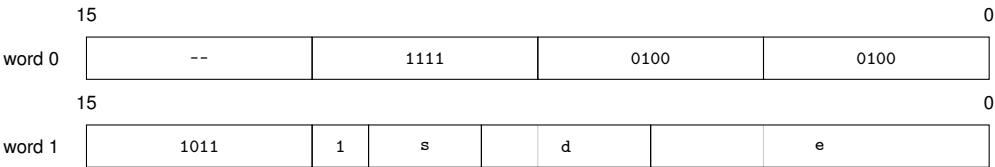
Dn(d), <ea(e)>

extended opcode form

2-8

unprivileged

source operand is data register number; destination operand is compact effective-address field

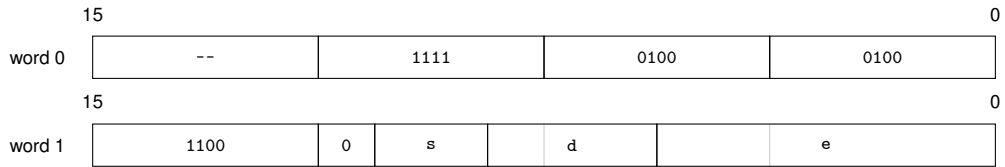


Instruction format for MULHSU.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e
- d
- s
- compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- data register number for the source operand (extended opcode word bits 8:6).
- size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MULHSU.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for MULHSU.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the destination operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MULHU

Unsigned Multiply High

MULHU

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the unsigned multiply high operation.

`dst_v = arch_unsigned_multiply_high(dst_v, src_v, size_v);`

`MULHU.<B/W/L/Q> Dn, <ea>`
`MULHU.<B/W/L/Q> <ea>, Dn`

Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged

Z N C V
- - - -
unchanged

Instruction Forms:

MULHU.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form

Encoding

Words

Privilege

Operands

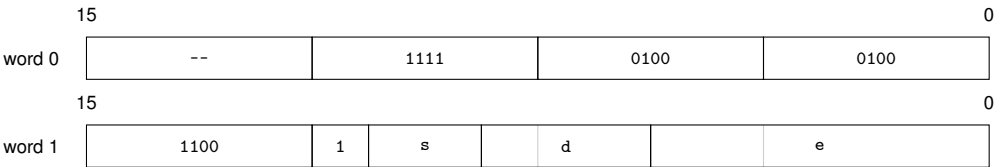
Dn(d), <ea(e)>

extended opcode form

2-8

unprivileged

source operand is data register number; destination operand is compact effective-address field

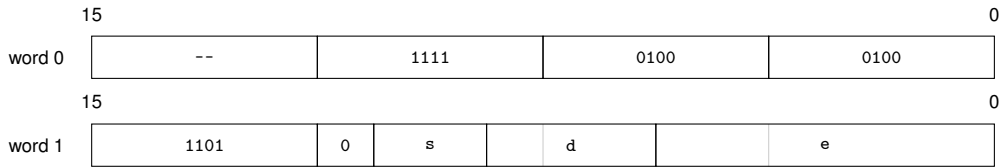


Instruction format for MULHU.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the source operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MULHU.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for MULHU.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the destination operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MULS

Signed Multiply

MULS

Summary: Performs the signed multiply operation.

Operation: `dst_v = dst_v * src_v;`

Assembler Syntax: `MULS.<B/W/L/Q> Dn, <ea>`
`MULS.<B/W/L/Q> <ea>, Dn`

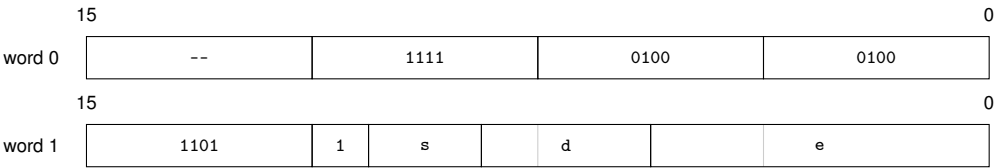
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = may update FLAGS

Condition Codes: **Z N C V**
- - - -
may update FLAGS

Instruction Forms:

`MULS.X(s:B/W/L/Q) Dn(d), <ea(e)>`

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

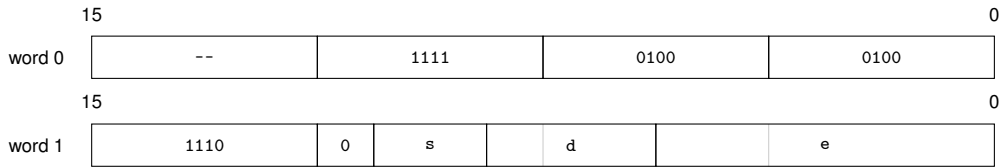


Instruction format for `MULS.X(s:B/W/L/Q) Dn(d), <ea(e)>`

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the source operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MULS.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for MULS.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the destination operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MULU

Unsigned Multiply

MULU

Summary: Performs the unsigned multiply operation.

Operation: `dst_v = dst_v * src_v;`

Assembler Syntax: `MULU.<B/W/L/Q> Dn, <ea>`
`MULU.<B/W/L/Q> <ea>, Dn`

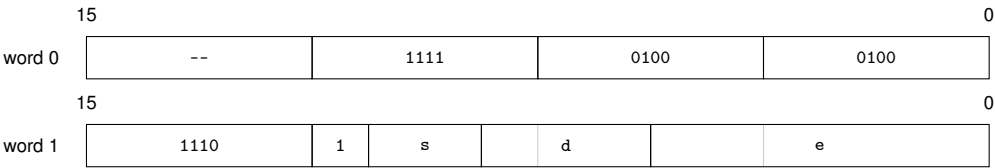
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = may update FLAGS

Condition Codes: **Z N C V**
- - - -
may update FLAGS

Instruction Forms:

`MULU.X(s:B/W/L/Q) Dn(d), <ea(e)>`

Form	Dn(d), <ea(e)>		
Encoding	extended opcode form		
Words	2-8		
Privilege	unprivileged		
Operands	source operand is data register number; destination operand is compact effective-address field		

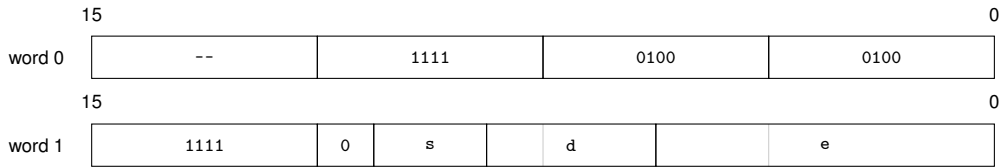


Instruction format for MULU.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the source operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

MULU.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for MULU.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the destination operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

NEG

Negate

NEG

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the negate operation.

result_v = 0 - dst_v;
dst_v = result_v;
ZF = result_v == 0;
NF = result_v s< 0;
CF = dst_old_v != 0;
VF = sborrow(0:8, dst_old_v);

NEG.<B/W/L/Q> Dn
NEG.<B/W/L/Q> <ea>

Size = (Byte, Word, Long, Quad)
Words = (1-8, 2-8)
Privilege = unprivileged
Flags = Z: result == 0; N: most significant bit(result); C: unsigned borrow;
V: signed overflow

Z N C V
* - - *

Z: result == 0; N: most significant bit(result); C: unsigned borrow; V:
signed overflow

Instruction Forms:

NEG.X(s:B/W/L/Q) Dn(d)

Form

Encoding

Words

Privilege

Operands

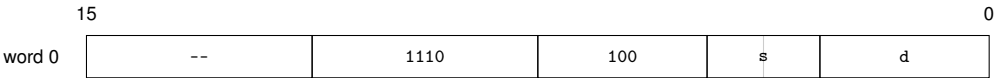
Dn(d)

primary opcode range 0xe80 to 0xe9f

1-8

unprivileged

destination operand is data register number



Instruction format for NEG.X(s:B/W/L/Q) Dn(d)

d

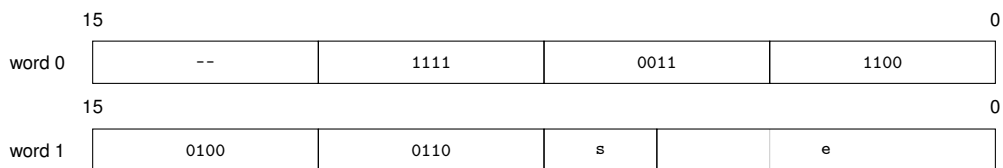
s

data register number for the destination operand (primary word bits 2:0).

size selector for the operand size (primary word bits 4:3).

NEG.X(s:B/W/L/Q) <ea(e)>

Form	<ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	destination operand is compact effective-address field



Instruction format for NEG.X(s:B/W/L/Q) <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 7:6).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

NOP

No Operation

NOP

Summary:

No architectural state change.

Operation:

goto <nop_done>;
<nop_done>

Assembler Syntax:

NOP

Attributes:

Words = (1-8)
Privilege = unprivileged
Flags = unchanged

Condition Codes:

Z N C V

- - - -

unchanged

Instruction Forms:

NOP

Form

NOP

Encoding

primary opcode 0xf4f

Words

1-8

Privilege

unprivileged

Operands

no explicit operands



Instruction format for NOP

No explicit operand fields are encoded in this form.

NOT**Bitwise Not****NOT**

Summary:	Performs the bitwise not operation.
Operation:	<pre>dst_v = ~dst_v; ZF = dst_v == 0; NF = dst_v < 0; CF = 0; VF = 0;</pre>
Assembler Syntax:	<pre>NOT.<B/W/L/Q> Dn NOT.<B/W/L/Q> <ea></pre>
Attributes:	Size = (Byte, Word, Long, Quad) Words = (1-8, 2-8) Privilege = unprivileged Flags = Z: result == 0; N: most significant bit(result); C: 0; V: 0
Condition Codes:	Z N C V * _ _ * Z: result == 0; N: most significant bit(result); C: 0; V: 0

Instruction Forms:

NOT.X(s:B/W/L/Q) Dn(d)

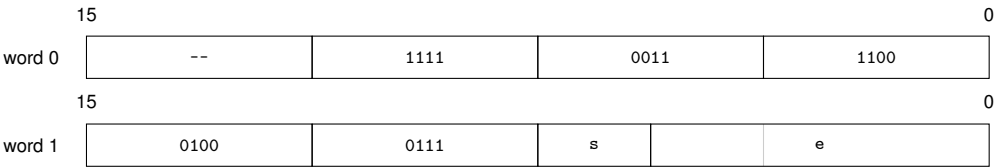
Form	Dn(d)
Encoding	primary opcode range 0xea0 to 0xebf
Words	1-8
Privilege	unprivileged
Operands	destination operand is data register number

**Instruction format for NOT.X(s:B/W/L/Q) Dn(d)**

- d data register number for the destination operand (primary word bits 2:0).
s size selector for the operand size (primary word bits 4:3).

NOT.X(s:B/W/L/Q) <ea(e)>

Form	<ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	destination operand is compact effective-address field



Instruction format for NOT.X(s:B/W/L/Q) <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 7:6). EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

OR**Bitwise Or****OR**

Summary: Computes the bitwise OR of the source and destination operands.

Operation: $\text{result_v} = \text{dst_v} \mid \text{src_v};$
 $\text{dst_v} = \text{result_v};$
 $\text{ZF} = \text{result_v} == 0;$
 $\text{NF} = \text{result_v} \text{ s} < 0;$
 $\text{CF} = 0;$
 $\text{VF} = 0;$

Assembler Syntax: $\text{OR}.\langle \text{L/Q} \rangle \text{ Dn}, \text{ Dn}$
 $\text{OR}.\langle \text{W/L} \rangle \langle \text{imm} \rangle, \text{ Dn}$
 $\text{OR}.\langle \text{B/W/L/Q} \rangle \text{ Dn}, \langle \text{ea} \rangle$
 $\text{OR}.\langle \text{B/W/L/Q} \rangle \langle \text{ea} \rangle, \text{ Dn}$
 $\text{OR}.\langle \text{B/W/L/Q} \rangle \langle \text{imm6} \rangle, \langle \text{ea} \rangle$

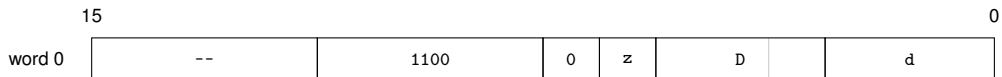
Attributes: Size = (Byte, Word, Long, Quad, Long, Quad, Word, Long)
Words = (1-8, 2-8)
Privilege = unprivileged
Flags = Z: result == 0; N: most significant bit(result); C: 0; V: 0

Condition Codes: **Z N C V**
* - - *
Z: result == 0; N: most significant bit(result); C: 0; V: 0

Instruction Forms:

$\text{OR}.\text{X}(\text{z:L/Q}) \text{ Dn}(\text{d}), \text{ Dn}(\text{D})$

Form $\text{Dn}(\text{d}), \text{ Dn}(\text{D})$
Encoding primary opcode range 0xc00 to 0xc7f
Words 1-8
Privilege unprivileged
Operands source operand is data register number; destination operand is data register number



Instruction format for OR.X(z:L/Q) Dn(d), Dn(D)

d data register number for the source operand (primary word bits 2:0).
D data register number for the destination operand (primary word bits 5:3).
z size selector for the operand size (primary word bits 6).

OR.X(z:W/L) imm, Dn(d)

Form	imm, Dn(d)
Encoding	primary opcode range 0xee0 to 0xeef
Words	2-8
Privilege	unprivileged
Operands	imm; destination operand is data register number

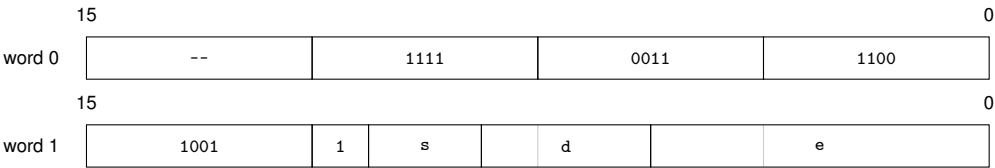


Instruction format for OR.X(z:W/L) imm, Dn(d)

- d data register number for the destination operand (primary word bits 2:0).
- z size selector for the operand size (primary word bits 3).

OR.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form	Dn(d), <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is data register number; destination operand is compact effective-address field

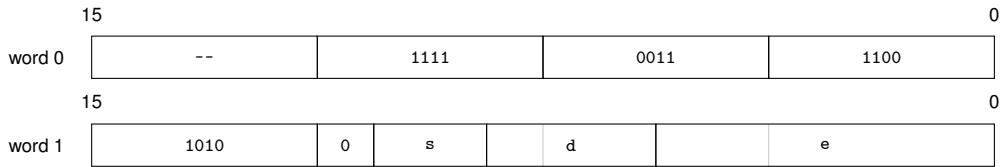


Instruction format for OR.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the source operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

OR.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form <ea(e)>, Dn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is data register number

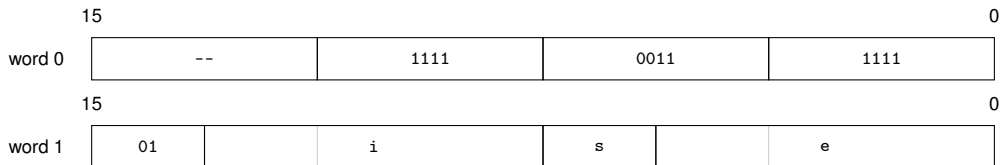


Instruction format for OR.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
d data register number for the destination operand (extended opcode word bits 8:6).
s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

OR.X(s:B/W/L/Q) <imm(i)>, <ea(e)>

Form <imm(i)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands imm is unsigned 6-bit immediate literal; destination operand is compact effective-address field



Instruction format for OR.X(s:B/W/L/Q) <imm(i)>, <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
s size selector for the operand size (extended opcode word bits 7:6).
i unsigned 6-bit immediate literal for the imm (extended opcode word bits 13:8). The encoded value is zero-extended before use for integer forms with an explicit B/W/L/Q operation size.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

PARITY**Parity Test****PARITY**

Summary: Computes operand parity and writes the predicate to FLAGS.Z.

Operation:

```

tmp_v = src_v;
if (size_v == 8) goto <parity_mask_done>;
if (size_v != 4) goto <parity_not_l>;
tmp_v = tmp_v & 0xffffffff;
goto <parity_mask_done>;
<parity_not_l>
if (size_v != 2) goto <parity_b>;
tmp_v = tmp_v & 0xffff;
goto <parity_mask_done>;
<parity_b>
tmp_v = tmp_v & 0xff;
<parity_mask_done>
result_v = popcount(tmp_v);
ZF = result_v & 1;

```

Assembler Syntax: `PARITY.<B/W/L/Q> <ea>`

Attributes:

- Size = (Byte, Word, Long, Quad)
- Words = (2-8)
- Privilege = unprivileged
- Flags = Z: population count(src) & 1; N: unchanged; C: unchanged; V: unchanged

Description: Computes the parity of the selected operand size and writes the parity bit directly to FLAGS.Z. Z is set to `popcount(src) & 1`, so Z=1 indicates odd parity and Z=0 indicates even parity. No integer destination register is written. N, C, and V are unchanged.

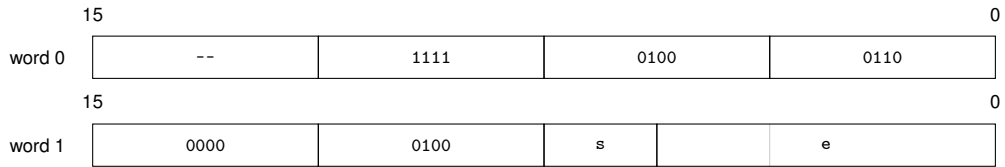
Condition Codes: **Z N C V**
- - - -

Z: population count(src) & 1; N: unchanged; C: unchanged; V: unchanged

Instruction Forms:

PARITY.X(s:B/W/L/Q) <ea(e)>

Form <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field



Instruction format for PARITY.X(s:B/W/L/Q) <ea(e)>

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 7:6).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

POP

Pop

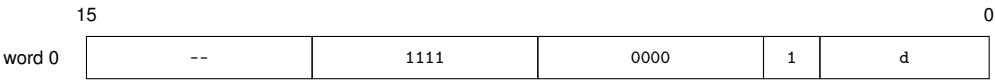
POP

Summary:	Performs the pop operation.
Operation:	reg_v = *:8 SP; SP = SP + 8;
Assembler Syntax:	POP.D Dn POP An
Attributes:	Words = (1-8) Privilege = unprivileged Flags = unchanged
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

POP.D Dn(d)

Form	Dn(d)
Encoding	primary opcode range 0xf08 to 0xf0f
Words	1-8
Privilege	unprivileged
Operands	register operand is data register number



Instruction format for POP.D Dn(d)

d data register number for the register operand (primary word bits 2:0).

POP An(a)

Form	An(a)
Encoding	primary opcode range 0xf10 to 0xf17
Words	1-8
Privilege	unprivileged
Operands	register operand is address register number

**Instruction format for POP An(a)**

a address register number for the register operand (primary word bits 2:0).

POPCNT**Population Count****POPCNT**

Summary: Performs the population count operation.

Operation:

```

tmp_v = src_v;
if (size_v == 8) goto <popcnt_mask_done>;
if (size_v != 4) goto <popcnt_not_l>;
tmp_v = tmp_v & 0xffffffff;
goto <popcnt_mask_done>;
<popcnt_not_l>
if (size_v != 2) goto <popcnt_b>;
tmp_v = tmp_v & 0xffff;
goto <popcnt_mask_done>;
<popcnt_b>
tmp_v = tmp_v & 0xff;
<popcnt_mask_done>
dst_v = popcount(tmp_v);

```

Assembler Syntax: POPCNT.<B/W/L/Q> <ea>, Dn

Attributes:

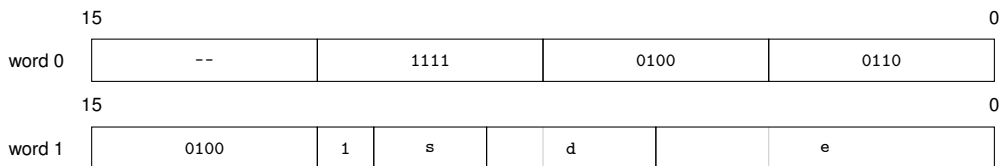
- Size = (Byte, Word, Long, Quad)
- Words = (2-8)
- Privilege = unprivileged
- Flags = unchanged

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

POPCNT.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for POPCNT.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the destination operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

POPM

Pop Multiple

POPM

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Pop selected A7-A0 then D7-D0 registers in inverse bitmap order.

For each set bit in the bitmap, POPM pops one 64-bit stack slot into the corresponding register. Registers are processed in inverse bitmap order, A7 through A0 followed by D7 through D0. Each selected register is loaded from SP before SP is incremented by 8.

POPM <bitmap16>

Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Z N C V
- - - -
unchanged

Instruction Forms:

POPM <bitmap>

Form

Encoding

Words

Privilege

Operands

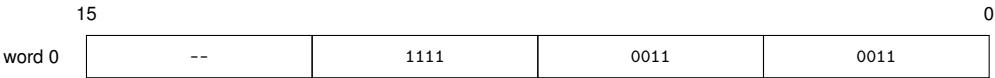
<bitmap>

primary opcode 0xf33

2-8

unprivileged

register bitmap is 16-bit register bitmap



Instruction format for POPM <bitmap>

No explicit operand fields are encoded in this form.

PREFETCH

Prefetch

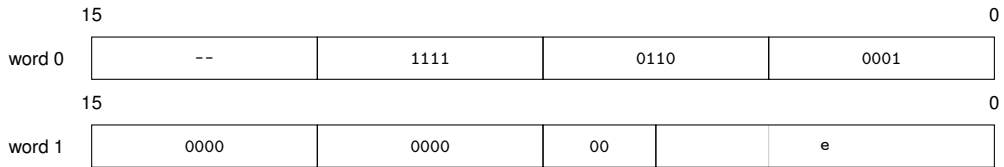
PREFETCH

Summary:	Hint that EA will be read soon; no architectural fault unless policy says otherwise.
Operation:	arch_prefetch(src_v);
Assembler Syntax:	PREFETCH <ea>
Attributes:	Words = (2-8) Privilege = user allowed Flags = unchanged
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

PREFETCH <ea(e)>

Form	<ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	user allowed
Operands	source operand is compact effective-address field

**Instruction format for PREFETCH <ea(e)>**

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

PTATTR

Page Table Attributes

PTATTR

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Perform page walk and return page attributes/status.

arch_page_attributes(src_v);

PTATTR <ea>

Words = (2-8)
Privilege = supervisor
Flags = operation-defined status result

Z N C V
- - - -
operation-defined status result

Instruction Forms:

PTATTR <ea(e)>

Form

Encoding

Words

Privilege

Operands

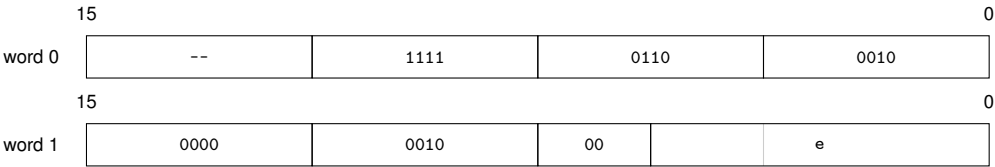
<ea(e)>

extended opcode form

2-8

supervisor

source operand is linear or ea; outputs



Instruction format for PTATTR <ea(e)>

e

compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

PTQUERY**Page Table Query****PTQUERY**

Summary: Perform page walk and return query result/status.

Operation: arch_page_query(src_v);

Assembler Syntax: PTQUERY <ea>

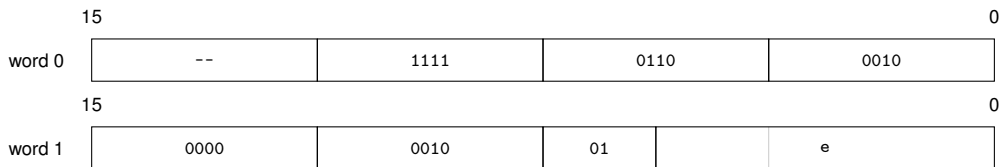
Attributes: Words = (2-8)
Privilege = supervisor
Flags = operation-defined status result

Condition Codes: **Z N C V**
- - - -
operation-defined status result

Instruction Forms:

PTQUERY <ea(e)>

Form <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege supervisor
Operands source operand is linear or ea; outputs



Instruction format for PTQUERY <ea(e)>

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

PUSH

Push

PUSH

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the push operation.

SP = SP - 8;
*:8 SP = reg_v;

PUSH.D Dn
PUSH An

Words = (1-8)
Privilege = unprivileged
Flags = unchanged

Z N C V
- - - -
unchanged

Instruction Forms:

PUSH.D Dn(d)

Form

Encoding

Words

Privilege

Operands

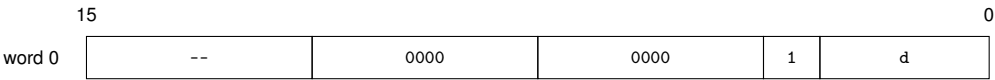
Dn(d)

primary opcode range 0x008 to 0x00f

1-8

unprivileged

register operand is data register number



Instruction format for PUSH.D Dn(d)

d data register number for the register operand (primary word bits 2:0).

PUSH An(a)

Form An(a)
Encoding primary opcode range 0xf00 to 0xf07
Words 1-8
Privilege unprivileged
Operands register operand is address register number



Instruction format for PUSH An(a)

a address register number for the register operand (primary word bits 2:0).

PUSHM

Push Multiple

PUSHM

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Push selected D0-D7 then A0-A7 registers according to bitmap order.

For each set bit in the bitmap, PUSHM pushes the corresponding 64-bit register value onto the stack. Registers are processed in increasing bitmap order, D0 through D7 followed by A0 through A7. Each selected register decrements SP by 8 before storing the register value at the new SP.

PUSHM <bitmap16>

Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Z N C V
- - - -
unchanged

Instruction Forms:

PUSHM <bitmap>

Form

Encoding

Words

Privilege

Operands

<bitmap>

primary opcode 0xf32

2-8

unprivileged

register bitmap is 16-bit register bitmap



Instruction format for PUSHM <bitmap>

No explicit operand fields are encoded in this form.

RCL**Rotate Through Carry Left****RCL**

Summary: Performs the rotate through carry left operation.

Operation: `tmp_v = count_v & 63;`
`dst_v = (dst_v << tmp_v) | (dst_v >> (64 - tmp_v));`
`ZF = dst_v == 0;`
`NF = dst_v > 0;`
`CF = 0;`
`VF = 0;`

Assembler Syntax: `RCL.<B/W/L/Q> Dn, Dn`
`RCL.<B/W/L/Q> Dn, <ea>`
`RCL.<B/W/L/Q> <count:imm6>, <ea>`

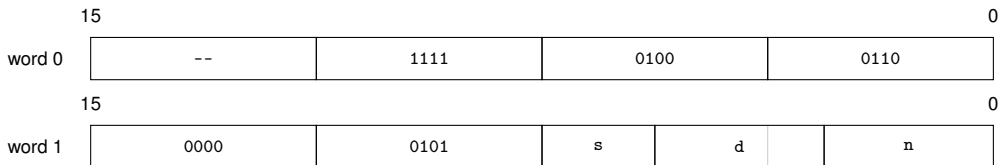
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = mode: per shift

Condition Codes: **Z N C V**
- - - -
mode: per shift

Instruction Forms:

`RCL.X(s:B/W/L/Q) Dn(n), Dn(n)`

Form Dn(n), Dn(n)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands count is data register number; destination operand is data register number

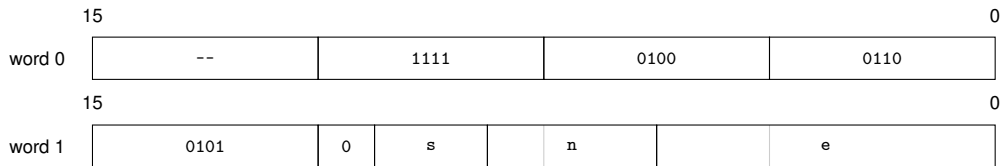


Instruction format for RCL.X(s:B/W/L/Q) Dn(n), Dn(n)

n data register number for the count (extended opcode word bits 2:0).
d data register number for the count (extended opcode word bits 5:3).
s size selector for the operand size (extended opcode word bits 7:6).

RCL.X(s:B/W/L/Q) Dn(n), <ea(e)>

Form Dn(n), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands count is data register number; destination operand is compact effective-address field

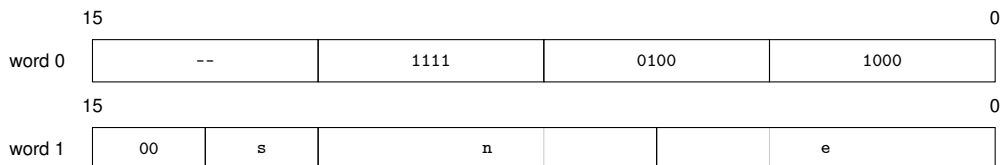


Instruction format for RCL.X(s:B/W/L/Q) Dn(n), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- n data register number for the count (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

RCL.X(s:B/W/L/Q) <count(n)>, <ea(e)>

Form <count(n)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands count is selector imm6; destination operand is compact effective-address field



Instruction format for RCL.X(s:B/W/L/Q) <count(n)>, <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- n selector6 for the count (extended opcode word bits 11:6).
- s size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

RCR**Rotate Through Carry Right****RCR**

Summary: Performs the rotate through carry right operation.

Operation: `tmp_v = count_v & 63;`
`dst_v = (dst_v >> tmp_v) | (dst_v << (64 - tmp_v));`
`ZF = dst_v == 0;`
`NF = dst_v < 0;`
`CF = 0;`
`VF = 0;`

Assembler Syntax: `RCR.<B/W/L/Q> Dn, Dn`
`RCR.<B/W/L/Q> Dn, <ea>`
`RCR.<B/W/L/Q> <count:imm6>, <ea>`

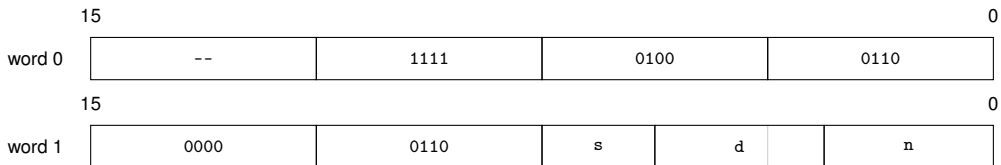
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = mode: per shift

Condition Codes: **Z N C V**
- - - -
mode: per shift

Instruction Forms:

`RCR.X(s:B/W/L/Q) Dn(n), Dn(n)`

Form Dn(n), Dn(n)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands count is data register number; destination operand is data register number

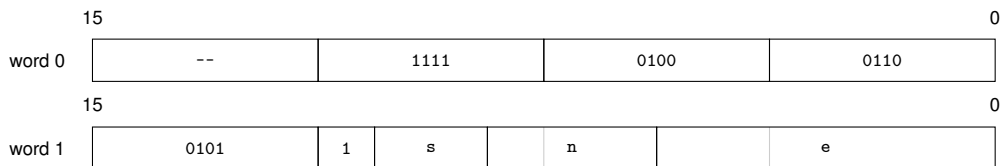


Instruction format for RCR.X(s:B/W/L/Q) Dn(n), Dn(n)

n data register number for the count (extended opcode word bits 2:0).
d data register number for the count (extended opcode word bits 5:3).
s size selector for the operand size (extended opcode word bits 7:6).

RCR.X(s:B/W/L/Q) Dn(n), <ea(e)>

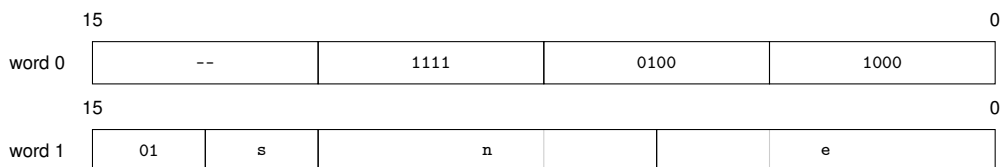
Form Dn(n), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands count is data register number; destination operand is compact effective-address field

**Instruction format for RCR.X(s:B/W/L/Q) Dn(n), <ea(e)>**

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- n data register number for the count (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

RCR.X(s:B/W/L/Q) <count(n)>, <ea(e)>

Form <count(n)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands count is selector imm6; destination operand is compact effective-address field

**Instruction format for RCR.X(s:B/W/L/Q) <count(n)>, <ea(e)>**

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- n selector6 for the count (extended opcode word bits 11:6).
- s size selector for the operand size (extended opcode word bits 13:12).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

RDCR**Read Control Register****RDCR**

Summary: Read a privileged control register.

Operation:

```

if (cr_v != 0x0000) goto <rdcr_ascr>;
dst_v = PTCR;
goto <rdcr_done>;
<rdcr_ascr>
if (cr_v != 0x0001) goto <rdcr_icr>;
dst_v = ASCR;
goto <rdcr_done>;
<rdcr_icr>
if (cr_v != 0x0002) goto <rdcr_spc>;
dst_v = ICR;
goto <rdcr_done>;
<rdcr_spc>
if (cr_v != 0x0100) goto <rdcr_scs>;
dst_v = SPC;
goto <rdcr_done>;
<rdcr_scs>
if (cr_v != 0x0101) goto <rdcr_sds>;
dst_v = SCS;
goto <rdcr_done>;
<rdcr_sds>
if (cr_v != 0x0102) goto <rdcr_sss0>;
dst_v = SDS;
goto <rdcr_done>;
<rdcr_sss0>
if (cr_v != 0x0200) goto <rdcr_ssp0>;
dst_v = SSS0;
goto <rdcr_done>;
<rdcr_ssp0>
if (cr_v != 0x0201) goto <rdcr_sss1>;
dst_v = SSP0;
goto <rdcr_done>;
<rdcr_sss1>
if (cr_v != 0x0210) goto <rdcr_ssp1>;
dst_v = SSS1;
goto <rdcr_done>;
<rdcr_ssp1>
if (cr_v != 0x0211) goto <rdcr_sss2>;
dst_v = SSP1;
goto <rdcr_done>;
<rdcr_sss2>
if (cr_v != 0x0220) goto <rdcr_ssp2>;
dst_v = SSS2;

```

```

goto <rdcr_done>;
<rdcr_ssp2>
if (cr_v != 0x0221) goto <rdcr_sss3>;
dst_v = SSP2;
goto <rdcr_done>;
<rdcr_sss3>
if (cr_v != 0x0230) goto <rdcr_ssp3>;
dst_v = SSS3;
goto <rdcr_done>;
<rdcr_ssp3>
if (cr_v != 0x0231) goto <rdcr_bootpc>;
dst_v = SSP3;
goto <rdcr_done>;
<rdcr_bootpc>
if (cr_v != 0x1000) goto <rdcr_bootcfg>;
dst_v = BOOTPC;
goto <rdcr_done>;
<rdcr_bootcfg>
if (cr_v != 0x1001) goto <rdcr_ptc>;
dst_v = BOOTCFG;
goto <rdcr_done>;
<rdcr_ptc>
if (cr_v != 0x1100) goto <rdcr_pmc>;
dst_v = PTC;
goto <rdcr_done>;
<rdcr_pmc>
if (cr_v != 0x1101) goto <rdcr_invalid>;
dst_v = PMC;
goto <rdcr_done>;
<rdcr_invalid>
arch_raise_invalid_control_state(cr_v);
<rdcr_done>

```

Assembler Syntax: RDCR.D <cr>, Dn

Attributes: Words = (3-8)
Privilege = supervisor
Flags = unchanged

Description: Read a privileged control register. RDCR is always a supervisor-only instruction; user-visible control state must use dedicated instructions gated by control-register policy bits.

Condition Codes: **Z N C V**

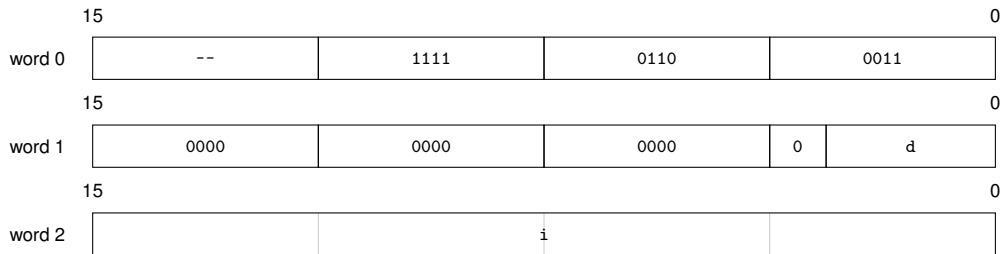
- - - -

unchanged

Instruction Forms:

RDCR.D <cr>, Dn(d)

Form <cr>, Dn(d)
Encoding extended opcode form
Words 3-8
Privilege supervisor
Operands cr; destination operand is data register number

**Instruction format for RDCR.D <cr>, Dn(d)**

- d data register number for the destination operand (extended opcode word bits 2:0).
i control-register selector for the control-register selector (payload word 1 bits 15:0).

RDFLAGS

Read Flags

RDFLAGS

Summary:

Read the integer condition-code flags into a D register.

Operation:

dst_v = zext(ZF) << 3;
tmp_v = zext(NF) << 2;
dst_v = dst_v | tmp_v;
tmp_v = zext(CF) << 1;
dst_v = dst_v | tmp_v;
dst_v = dst_v | zext(VF);

Assembler Syntax:

RDFLAGS.D Dn

Attributes:

Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Description:

RDFLAGS reads the architectural 16-bit FLAGS register, zero-extends it to 64 bits, and writes the result to the destination D register. FLAGS is not part of the S segment-register operand class.

Condition Codes:

Z N C V
- - - -
unchanged

Instruction Forms:

RDFLAGS.D Dn(d)

Form

Dn(d)

Encoding

extended opcode form

Words

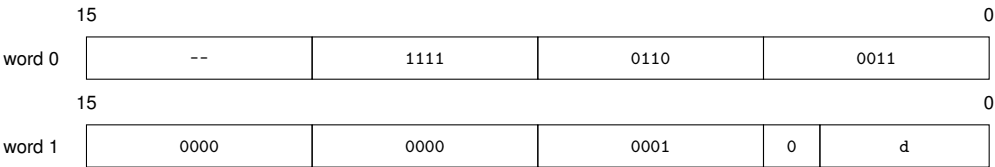
2-8

Privilege

unprivileged

Operands

destination operand is data register number



Instruction format for RDFLAGS.D Dn(d)

d data register number for the destination operand (extended opcode word bits 2:0).

RDFSTATUS**Read Floating-Point Status****RDFSTATUS**

Summary: Read the FPU status/control register into a D register.

Operation: `dst_v = FSTATUS;`

Assembler Syntax: `RDFSTATUS.D Dn`

Attributes:
 Words = (2-8)
 Privilege = unprivileged
 Flags = unchanged

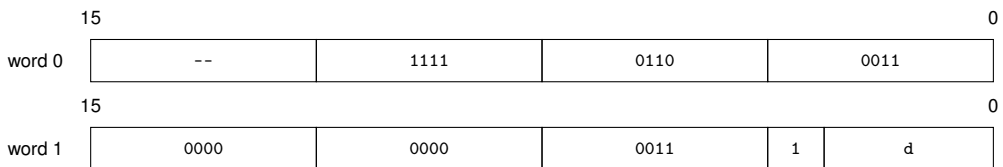
Description: RDFSTATUS reads the architectural 16-bit FSTATUS register, zero-extends it to 64 bits, and writes the result to the destination D register. FSTATUS contains floating-point exception trap enables, rounding mode, and non-default mode bits. Accrued floating-point exception flags are kept in the separate FFLAGS register. RDFSTATUS requires the floating-point extension.

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

`RDFSTATUS.D Dn(d)`

Form Dn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands destination operand is data register number



Instruction format for RDFSTATUS.D Dn(d)

d data register number for the destination operand (extended opcode word bits 2:0).

RDPMC

Read Performance Counter

RDPMC

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the read performance counter operation.

dst_v = PMC;

RDPMC <imm16>, Dn

Words = (3-8)
Privilege = configurable
Flags = unchanged

Z N C V
- - - -
unchanged

Instruction Forms:

RDPMC imm16, Dn(d)

Form

Encoding

Words

Privilege

Operands

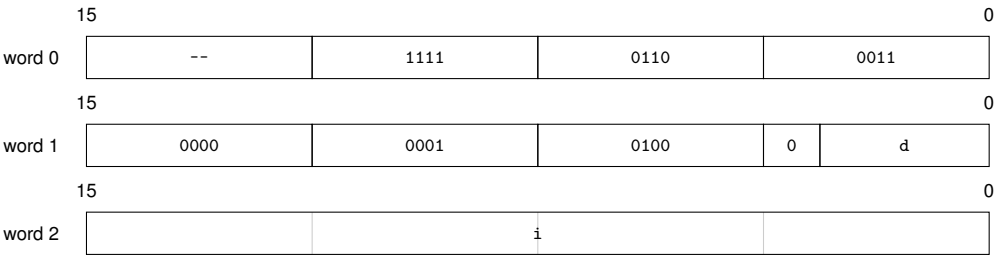
imm16, Dn(d)

extended opcode form

3-8

configurable

counter id is 16-bit immediate selector; destination operand is data register number



Instruction format for RDPMC imm16, Dn(d)

d

i

data register number for the destination operand (extended opcode word bits 2:0).

imm16 for the counter id (payload word 1 bits 15:0).

RDPTC**Read Page Table Counter****RDPTC**

Summary: Performs the read page table counter operation.

Operation: `dst_v = PTC;`

Assembler Syntax: `RDPTC.D Dn`

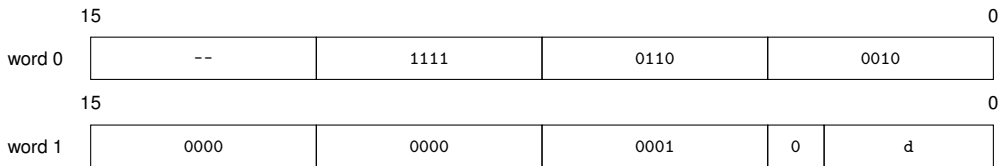
Attributes:
 Words = (2-8)
 Privilege = configurable
 Flags = unchanged

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

`RDPTC.D Dn(d)`

Form Dn(d)
Encoding extended opcode form
Words 2-8
Privilege configurable
Operands destination operand is data register number



Instruction format for RDPTC.D Dn(d)

d data register number for the destination operand (extended opcode word bits 2:0).

RDSEG

Read Segment Register

RDSEG

Summary:

Read a segment register image into a D register.

Operation:

dst_v = seg_v;

Assembler Syntax:

RDSEG Sreg, Dn

Attributes:

Words = (2-8)
Privilege = policy controlled
Flags = unchanged

Description:

RDSEG reads one of the S-class segment registers and writes the 64-bit segment register image to the destination D register. The S operand selector encoding is defined by the SREG selector table.

Condition Codes:

Z N C V

- - - -

unchanged

Instruction Forms:

RDSEG Sreg(g), Dn(d)

Form

Sreg(g), Dn(d)

Encoding

extended opcode form

Words

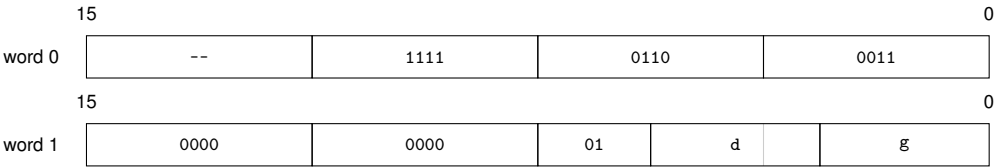
2-8

Privilege

policy controlled

Operands

seg is segment register selector; destination operand is data register number



Instruction format for RDSEG Sreg(g), Dn(d)

g

segment register selector for the seg (extended opcode word bits 2:0).

d

data register number for the destination operand (extended opcode word bits 5:3).

RDSTATUS**Read Status****RDSTATUS**

Summary: Read the processor STATUS register into a D register.

Operation: `dst_v = STATUS;`

Assembler Syntax: `RDSTATUS.D Dn`

Attributes:
 Words = (2-8)
 Privilege = unprivileged
 Flags = unchanged

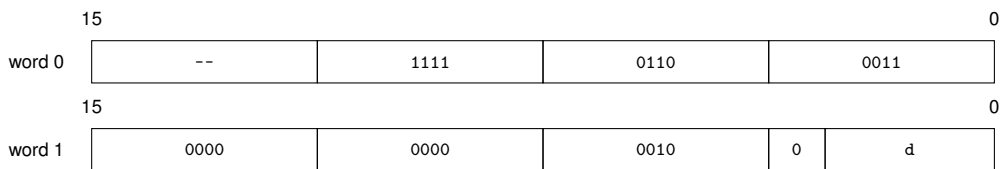
Description: RDSTATUS reads the architectural 16-bit STATUS register, zero-extends it to 64 bits, and writes the result to the destination D register. Reading STATUS is unprivileged; writing STATUS remains supervisor-only. STATUS is not part of the S segment-register operand class.

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

`RDSTATUS.D Dn(d)`

Form Dn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands destination operand is data register number



Instruction format for RDSTATUS.D Dn(d)

d data register number for the destination operand (extended opcode word bits 2:0).

RESET

Reset

RESET

Summary:	Perform warm reset; preserve BOOTPC and BOOTCFG.
Operation:	PC = BOOTPC; goto [PC];
Assembler Syntax:	RESET
Attributes:	Words = (1-8) Privilege = supervisor Flags = unchanged
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

RESET

Form	RESET
Encoding	primary opcode 0xf38
Words	1-8
Privilege	supervisor
Operands	no explicit operands



Instruction format for RESET

No explicit operand fields are encoded in this form.

RESTORE

Restore Processor State

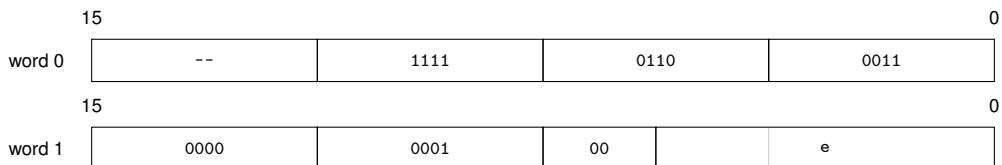
RESTORE

Summary:	Restore processor state from a CPUID-defined save area.
Operation:	RESTORE reads the fixed base block from the selected save-area address. It restores bank-0 D registers, A registers, FLAGS, and DBANK from the fixed base block; GS registers are restored only when their GS_VALID bits are set. User-mode RESTORE ignores the saved STATUS image, while supervisor RESTORE applies the normal STATUS write rules. Extension components whose state-block bitmap bits are set are restored from their CPUID-defined component offsets.
Assembler Syntax:	RESTORE <ea>
Attributes:	Words = (2-8) Privilege = unprivileged
Description:	Restore processor state from a CPUID-defined save area. The save area uses a fixed architectural layout discovered through CPUID; it does not contain a per-component size table. The save-area base must be 4 KiB aligned. RESTORE is the matching restore operation for SAVE and may restore base architectural state as well as enabled extension state from the same save image. User-mode RESTORE ignores the saved STATUS image; supervisor RESTORE applies the normal STATUS write rules.
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

RESTORE <ea(e)>

Form	<ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	memory operand is compact effective-address field



Instruction format for RESTORE <ea(e)>

- e compact effective-address field for the memory operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

RET

Return

RET

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the return operation.

local return_pc_v:8 = *:8 SP;
SP = SP + 8;
return [return_pc_v];

RET

Words = (1-8)
Privilege = unprivileged
Flags = unchanged

Z N C V
- - - -
unchanged

Instruction Forms:

RET	
Form	RET
Encoding	primary opcode 0xf30
Words	1-8
Privilege	unprivileged
Operands	no explicit operands



Instruction format for RET

No explicit operand fields are encoded in this form.

REVBIT**Reverse Bits****REVBIT**

Summary: Performs the reverse bits operation.

Operation:

```

dst_v = 0;
tmp_v = src_v;
result_v = size_v << 3;
<revbit_loop>
if (result_v == 0) goto <revbit_done>;
dst_v = dst_v << 1;
borrow_v = tmp_v & 1;
dst_v = dst_v | borrow_v;
tmp_v = tmp_v >> 1;
result_v = result_v - 1;
goto <revbit_loop>;
<revbit_done>

```

Assembler Syntax:

```

REVBIT.<B/W/L/Q> Dn, Dn
REVBIT.<B/W/L/Q> Dn, <ea>
REVBIT.<B/W/L/Q> <ea>, Dn

```

Attributes:

- Size = (Byte, Word, Long, Quad)
- Words = (2-8)
- Privilege = unprivileged
- Flags = unchanged

Condition Codes: **Z N C V**

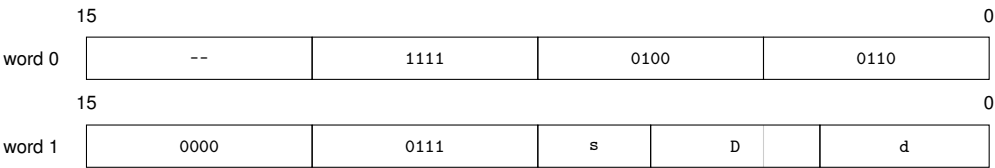
- - - -

unchanged

Instruction Forms:

REVBIT.X(s:B/W/L/Q) Dn(d) , Dn(D)

Form	Dn(d), Dn(D)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is data register number; destination operand is data register number

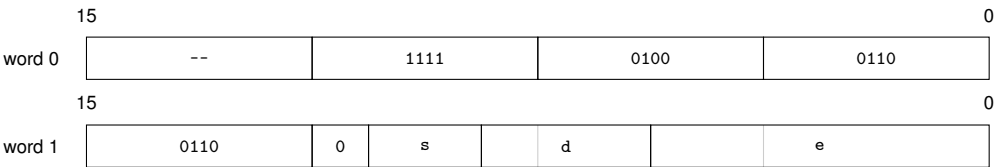


Instruction format for REVBIT.X(s:B/W/L/Q) Dn(d), Dn(D)

- d data register number for the source operand (extended opcode word bits 2:0).
- D data register number for the destination operand (extended opcode word bits 5:3).
- s size selector for the operand size (extended opcode word bits 7:6).

REVBIT.X(s:B/W/L/Q) Dn(d) , <ea(e)>

Form	Dn(d), <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is data register number; destination operand is compact effective-address field

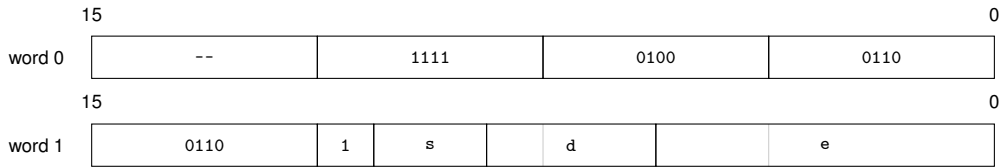


Instruction format for REVBIT.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the source operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

REVBIT.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for REVBIT.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - d data register number for the destination operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

REVBYTE**Reverse Bytes****REVBYTE**

Summary: Performs the reverse bytes operation.

Operation:

```
dst_v = 0;
tmp_v = src_v;
result_v = size_v;
<revbyte_loop>
if (result_v == 0) goto <revbyte_done>;
dst_v = dst_v << 8;
carry_v = 0xff;
borrow_v = tmp_v & carry_v;
dst_v = dst_v | borrow_v;
tmp_v = tmp_v >> 8;
result_v = result_v - 1;
goto <revbyte_loop>;
<revbyte_done>
```

Assembler Syntax:

```
REVBYTE.<B/W/L/Q> Dn, Dn
REVBYTE.<B/W/L/Q> Dn, <ea>
REVBYTE.<B/W/L/Q> <ea>, Dn
```

Attributes:

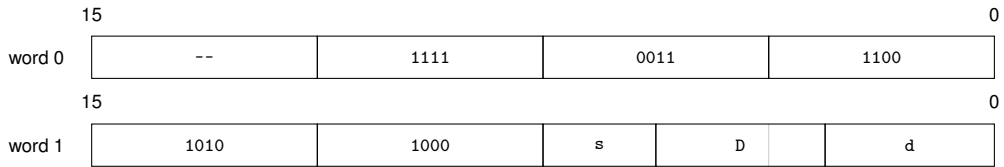
Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

REVBYTE.X(s:B/W/L/Q) Dn(d), Dn(D)

Form Dn(d), Dn(D)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is data register number

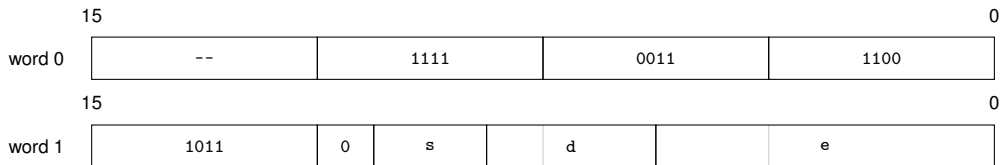


Instruction format for REVBYTE.X(s:B/W/L/Q) Dn(d), Dn(D)

d data register number for the source operand (extended opcode word bits 2:0).
D data register number for the destination operand (extended opcode word bits 5:3).
s size selector for the operand size (extended opcode word bits 7:6).

REVBYTE.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

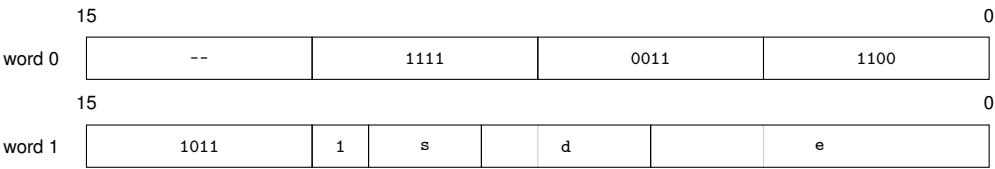


Instruction format for REVBYTE.X(s:B/W/L/Q) Dn(d), <ea(e)>

e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
d data register number for the source operand (extended opcode word bits 8:6).
s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

REVBYTE.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form	<ea(e)>, Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is data register number



Instruction format for REVBYTE.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

RFENCE

Read Fence

RFENCE

Summary:	Order prior reads before later reads.
Operation:	arch_fence_read();
Assembler Syntax:	RFENCE
Attributes:	Words = (1-8) Privilege = unprivileged Flags = unchanged
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

RFENCE

Form	RFENCE
Encoding	primary opcode 0x007
Words	1-8
Privilege	unprivileged
Operands	no explicit operands



Instruction format for RFENCE

No explicit operand fields are encoded in this form.

ROL

Rotate Left

ROL

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the rotate left operation.

tmp_v = count_v & 63;
dst_v = (dst_v << tmp_v) | (dst_v >> (64 - tmp_v));
ZF = dst_v == 0;
NF = dst_v s< 0;
CF = 0;
VF = 0;

ROL.<B/W/L/Q> Dn, Dn
ROL.<B/W/L/Q> Dn, <ea>
ROL.<B/W/L/Q> <count:imm6>, <ea>

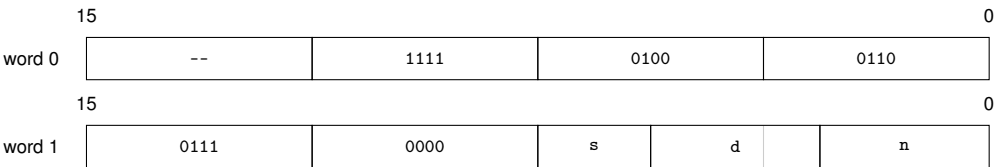
Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = mode: per shift

Z N C V
- - - -
mode: per shift

Instruction Forms:

ROL.X(s:B/W/L/Q) Dn(n), Dn(n)

Form	Dn(n), Dn(n)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	count is data register number; destination operand is data register number

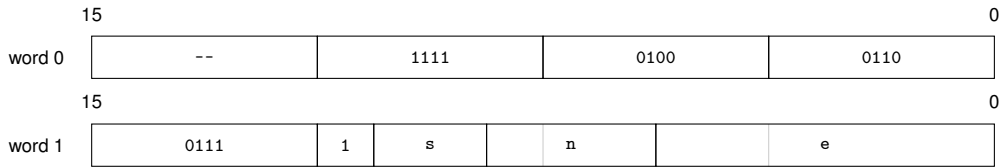


Instruction format for ROL.X(s:B/W/L/Q) Dn(n), Dn(n)

n data register number for the count (extended opcode word bits 2:0).
d data register number for the count (extended opcode word bits 5:3).
s size selector for the operand size (extended opcode word bits 7:6).

ROL.X(s:B/W/L/Q) Dn(n), <ea(e)>

Form Dn(n), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands count is data register number; destination operand is compact effective-address field

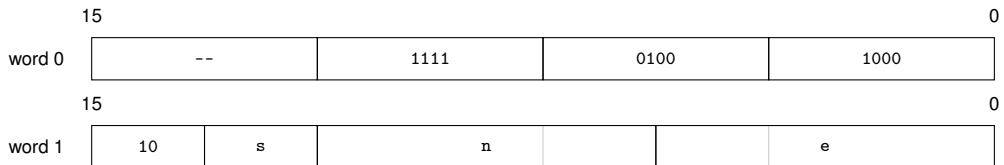


Instruction format for ROL.X(s:B/W/L/Q) Dn(n), <ea(e)>

- e** compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- n** data register number for the count (extended opcode word bits 8:6).
- s** size selector for the operand size (extended opcode word bits 10:9).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

ROL.X(s:B/W/L/Q) <count(n)>, <ea(e)>

Form <count(n)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands count is selector imm6; destination operand is compact effective-address field



Instruction format for ROL.X(s:B/W/L/Q) <count(n)>, <ea(e)>

- e** compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- n** selector6 for the count (extended opcode word bits 11:6).
- s** size selector for the operand size (extended opcode word bits 13:12).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

ROR

Rotate Right

ROR

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the rotate right operation.

tmp_v = count_v & 63;

dst_v = (dst_v >> tmp_v) | (dst_v << (64 - tmp_v));

ZF = dst_v == 0;

NF = dst_v s< 0;

CF = 0;

VF = 0;

ROR.<B/W/L/Q> Dn, Dn

ROR.<B/W/L/Q> Dn, <ea>

ROR.<B/W/L/Q> <count:imm6>, <ea>

Size = (Byte, Word, Long, Quad)

Words = (2-8)

Privilege = unprivileged

Flags = mode: per shift

Z N C V

- - - -

mode: per shift

Instruction Forms:

ROR.X(s:B/W/L/Q) Dn(n), Dn(n)

Form

Encoding

Words

Privilege

Operands

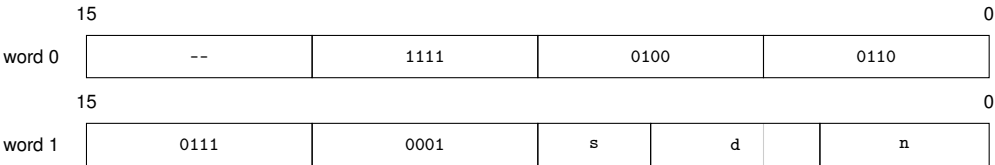
Dn(n), Dn(n)

extended opcode form

2-8

unprivileged

count is data register number; destination operand is data register number



Instruction format for ROR.X(s:B/W/L/Q) Dn(n), Dn(n)

n

d

s

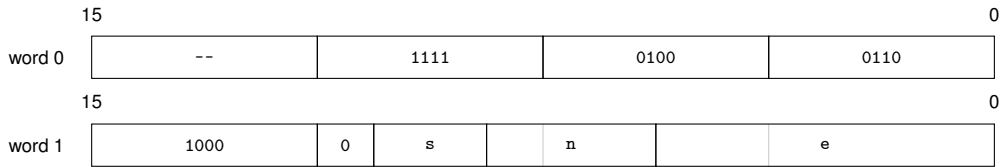
data register number for the count (extended opcode word bits 2:0).

data register number for the count (extended opcode word bits 5:3).

size selector for the operand size (extended opcode word bits 7:6).

ROR.X(s:B/W/L/Q) Dn(n), <ea(e)>

Form Dn(n), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands count is data register number; destination operand is compact effective-address field

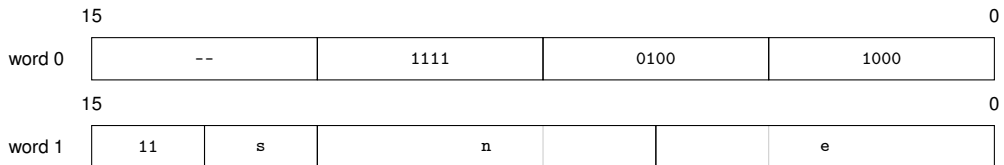


Instruction format for ROR.X(s:B/W/L/Q) Dn(n), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
n data register number for the count (extended opcode word bits 8:6).
s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

ROR.X(s:B/W/L/Q) <count(n)>, <ea(e)>

Form <count(n)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands count is selector imm6; destination operand is compact effective-address field



Instruction format for ROR.X(s:B/W/L/Q) <count(n)>, <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
n selector6 for the count (extended opcode word bits 11:6).
s size selector for the operand size (extended opcode word bits 13:12).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SAR

Shift Arithmetic Right

SAR

Summary: Performs the shift arithmetic right operation.

Operation: `dst_v = dst_v >> count_v;`
`ZF = dst_v == 0;`
`NF = dst_v s< 0;`
`CF = 0;`
`VF = 0;`

Assembler Syntax: `SAR.<B/W/L/Q> Dn, Dn`
`SAR.<B/W/L/Q> Dn, <ea>`
`SAR.<B/W/L/Q> <count:imm6>, <ea>`

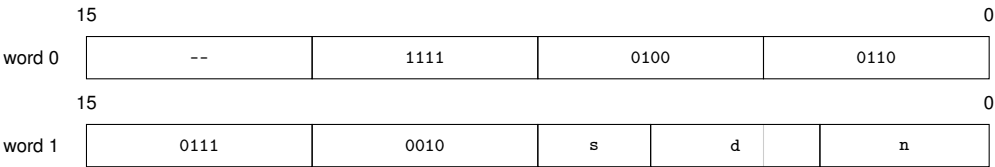
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = mode: per shift

Condition Codes: **Z N C V**
- - - -
mode: per shift

Instruction Forms:

`SAR.X(s:B/W/L/Q) Dn(n), Dn(n)`

Form	Dn(n), Dn(n)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	count is data register number; destination operand is data register number

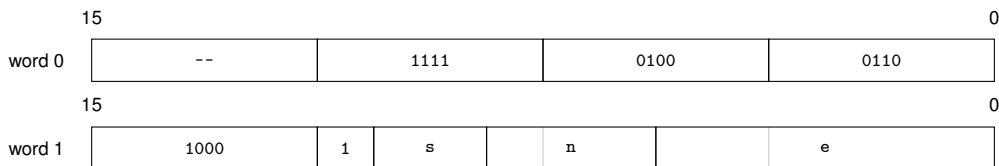


Instruction format for SAR.X(s:B/W/L/Q) Dn(n), Dn(n)

n data register number for the count (extended opcode word bits 2:0).
d data register number for the count (extended opcode word bits 5:3).
s size selector for the operand size (extended opcode word bits 7:6).

SAR.X(s:B/W/L/Q) Dn(n), <ea(e)>

Form Dn(n), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands count is data register number; destination operand is compact effective-address field

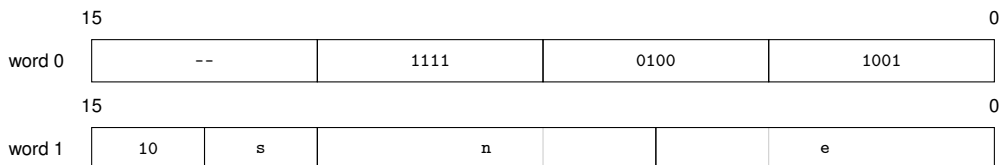


Instruction format for SAR.X(s:B/W/L/Q) Dn(n), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
n data register number for the count (extended opcode word bits 8:6).
s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SAR.X(s:B/W/L/Q) <count(n)>, <ea(e)>

Form <count(n)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands count is selector imm6; destination operand is compact effective-address field



Instruction format for SAR.X(s:B/W/L/Q) <count(n)>, <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
n selector6 for the count (extended opcode word bits 11:6).
s size selector for the operand size (extended opcode word bits 13:12).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SAVE

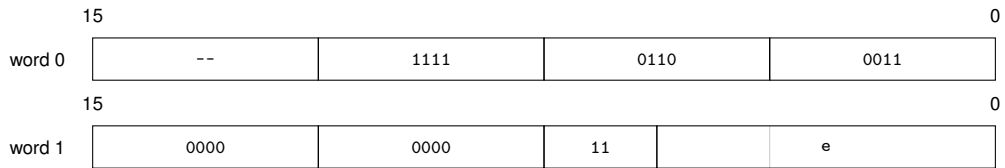
Save Processor State

SAVE

Summary:	Save modified processor state to a CPUID-defined save area.
Operation:	SAVE writes the fixed base block at the selected save-area address. It records the base header fields, the state-block bitmap, bank-0 D registers, A registers, GS registers whose validity bits are set, FLAGS, STATUS, and the current DBANK selector according to the SAVE/RESTORE Processor-State Save Area layout. Enabled extension state blocks are written to their CPUID-defined component offsets; architecturally clean extension blocks may be skipped and reported clear in the state-block bitmap.
Assembler Syntax:	SAVE <ea>
Attributes:	Words = (2-8) Privilege = unprivileged
Description:	Save modified processor state to a CPUID-defined save area. SAVE uses an XSAVEOPT-like policy for enabled extension state: state blocks that are architecturally clean may be skipped, while modified blocks are written to their CPUID-defined 64-byte-aligned slots. The fixed base block always records bank-0 D registers, A registers, FLAGS, STATUS, and the current DBANK selector. The memory image does not carry per-component size records; software allocates the maximum required save area size reported by CPUID. The save-area base must be 4 KiB aligned.
Condition Codes:	Z N C V - - - - unchanged
Instruction Forms:	

SAVE <ea(e)>

Form	<ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	memory operand is compact effective-address field



Instruction format for SAVE <ea(e)>

- e compact effective-address field for the memory operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SBB

Subtract With Borrow

SBB

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the subtract with borrow operation.

borrow_v = CF;

tmp_v = dst_v - src_v;

result_v = tmp_v - borrow_v;

dst_v = result_v;

ZF = result_v == 0;

NF = result_v s< 0;

CF = (dst_old_v < src_v) || (tmp_v < borrow_v);

VF = sborrow(dst_old_v, src_v) || sborrow(tmp_v, borrow_v);

SBB.<B/W/L/Q> Dn, <ea>

SBB.<B/W/L/Q> <ea>, Dn

SBB.<B/W/L/Q> <imm6>, <ea>

Size = (Byte, Word, Long, Quad)

Words = (2-8)

Privilege = unprivileged

Flags = update Z/N/C/V

Z N C V

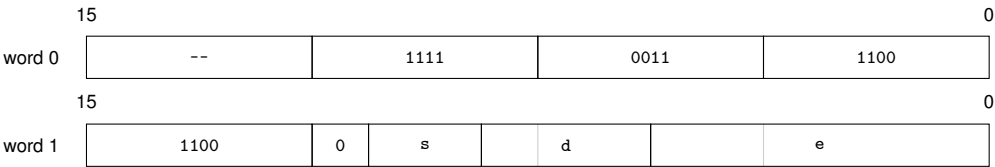
* * * *

update Z/N/C/V

Instruction Forms:

SBB.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form	Dn(d), <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is data register number; destination operand is compact effective-address field

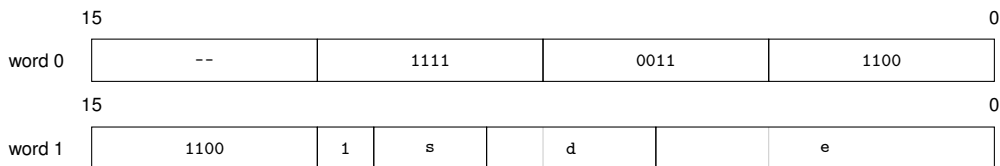


Instruction format for SBB.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the source operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SBB.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form <ea(e)>, Dn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is data register number

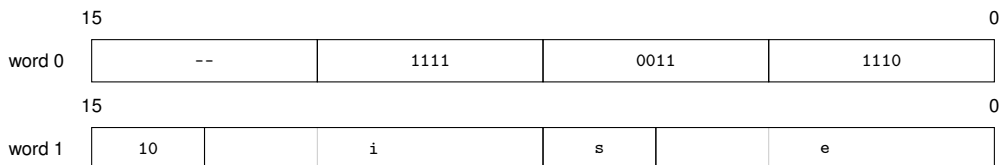


Instruction format for SBB.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the destination operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SBB.X(s:B/W/L/Q) <imm(i)>, <ea(e)>

Form <imm(i)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands imm is unsigned 6-bit immediate literal; destination operand is compact effective-address field



Instruction format for SBB.X(s:B/W/L/Q) <imm(i)>, <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 7:6).
- i unsigned 6-bit immediate literal for the imm (extended opcode word bits 13:8). The encoded value is zero-extended before use for integer forms with an explicit B/W/L/Q operation size.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SEGLEA

Segment Load Effective Address

SEGLEA

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the segment load effective address operation.

`dst_v = src_v;`

`SEGLEA <ea>, An`

Words = (2-8)
Privilege = unprivileged
Flags = V: set when segment bounds check fails

Z N C V
- - - *
V: set when segment bounds check fails

Instruction Forms:

SEGLEA <ea(e)>, An(a)

Form

Encoding

Words

Privilege

Operands

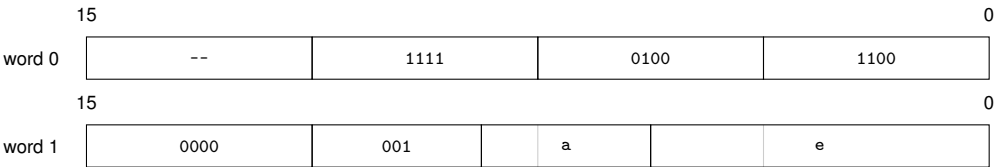
<ea(e)>, An(a)

extended opcode form

2-8

unprivileged

source operand is compact effective-address field; destination operand is address register number



Instruction format for SEGLEA <ea(e)>, An(a)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- a address register number for the destination operand (extended opcode word bits 8:6). EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SELDB**Select Data Register Bank****SELDB**

Summary: Selects the current data-register bank.

Operation:

Assembler Syntax: SELDB.D Dn
SELDB DBn

Attributes: Words = (2-8)
Privilege = user allowed
Flags = unchanged

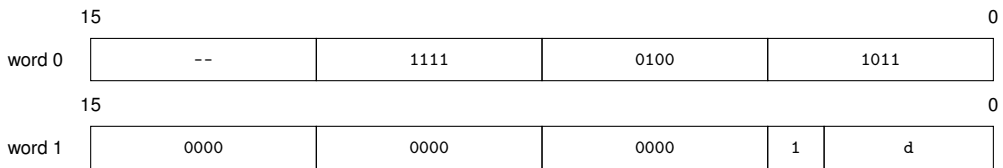
Description: Selects the current DBANK. The DBANK form names a literal bank selector. The D-register form takes the selector from the low bits of the source D register. If the selected bank is not implemented, the instruction reports INVALID_CONTROL_STATE.

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

SELDB.D Dn(d)

Form Dn(d)
Encoding extended opcode form
Words 2-8
Privilege user allowed
Operands selector is data register number

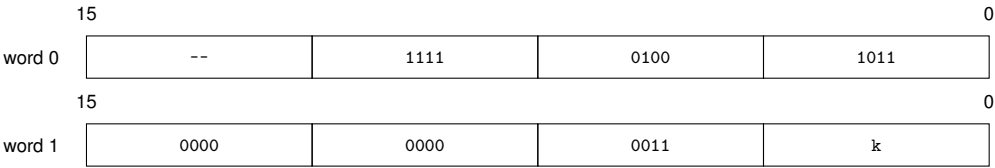


Instruction format for SELDB.D Dn(d)

d data register number for the selector (extended opcode word bits 2:0).

SELDB DBn(k)

Form	DBn(k)
Encoding	extended opcode form
Words	2-8
Privilege	user allowed
Operands	selector is dbank



Instruction format for SELDB DBn(k)

k dbank for the selector (extended opcode word bits 3:0).

SETcc**Set Conditional****SETcc**

Summary: Performs the set conditional operation.

Operation: `dst_v = cc_v;`

Assembler Syntax: `SETcc.<B/W/L/Q> <ea>`

Attributes:
 Size = (Byte, Word, Long, Quad)
 Words = (2-8)
 Privilege = unprivileged
 Flags = unchanged

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

`SETcc.X(s:B/W/L/Q) <ea(e)>`

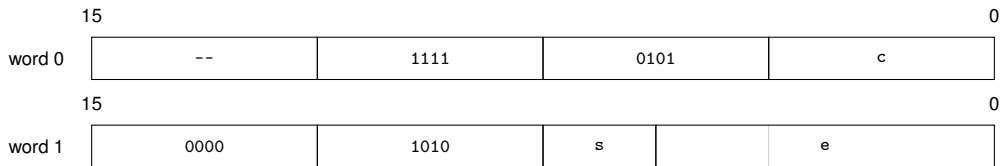
Form `<ea(e)>`

Encoding extended opcode form

Words 2-8

Privilege unprivileged

Operands condition code is condition-code selector; destination operand is compact effective-address field



Instruction format for SETcc.X(s:B/W/L/Q) <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 7:6).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SHL

Shift Left

SHL

Summary: Performs the shift left operation.

Operation: dst_v = dst_v << count_v;
ZF = dst_v == 0;
NF = dst_v s< 0;
CF = 0;
VF = 0;

Assembler Syntax: SHL.<B/W/L/Q> Dn, Dn
SHL.<B/W/L/Q> Dn, <ea>
SHL.<B/W/L/Q> <count:imm6>, <ea>

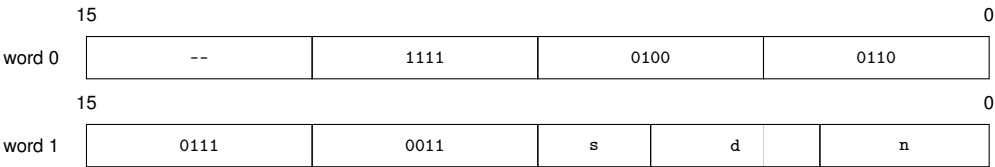
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = mode: per shift

Condition Codes: Z N C V
- - - -
mode: per shift

Instruction Forms:

SHL.X(s:B/W/L/Q) Dn(n), Dn(n)

Form	Dn(n), Dn(n)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	count is data register number; destination operand is data register number

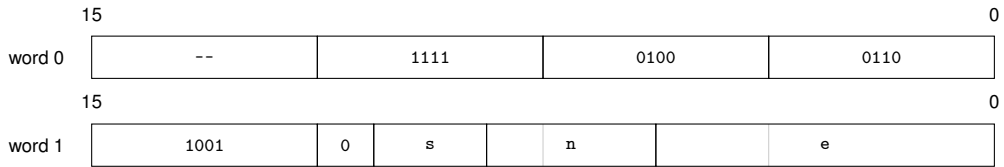


Instruction format for SHL.X(s:B/W/L/Q) Dn(n), Dn(n)

- n data register number for the count (extended opcode word bits 2:0).
- d data register number for the count (extended opcode word bits 5:3).
- s size selector for the operand size (extended opcode word bits 7:6).

SHL.X(s:B/W/L/Q) Dn(n), <ea(e)>

Form Dn(n), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands count is data register number; destination operand is compact effective-address field

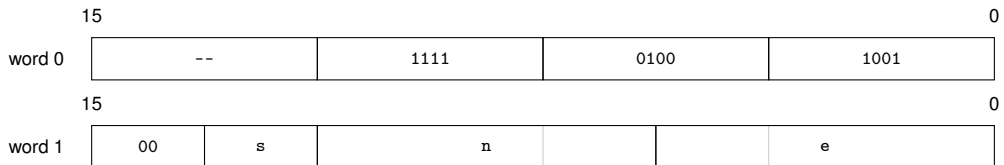


Instruction format for SHL.X(s:B/W/L/Q) Dn(n), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
n data register number for the count (extended opcode word bits 8:6).
s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SHL.X(s:B/W/L/Q) <count(n)>, <ea(e)>

Form <count(n)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands count is selector imm6; destination operand is compact effective-address field



Instruction format for SHL.X(s:B/W/L/Q) <count(n)>, <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
n selector6 for the count (extended opcode word bits 11:6).
s size selector for the operand size (extended opcode word bits 13:12).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SHR

Shift Right

SHR

Summary: Performs the shift right operation.

Operation: dst_v = dst_v >> count_v;
ZF = dst_v == 0;
NF = dst_v s< 0;
CF = 0;
VF = 0;

Assembler Syntax: SHR.<B/W/L/Q> Dn, Dn
SHR.<B/W/L/Q> Dn, <ea>
SHR.<B/W/L/Q> <count:imm6>, <ea>

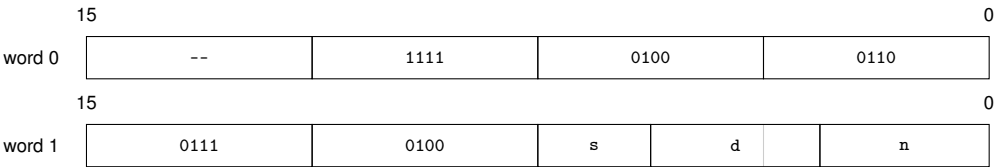
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = mode: per shift

Condition Codes: Z N C V
- - - -
mode: per shift

Instruction Forms:

SHR.X(s:B/W/L/Q) Dn(n), Dn(n)

Form	Dn(n), Dn(n)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	count is data register number; destination operand is data register number

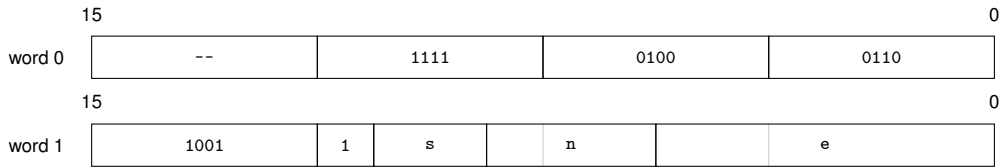


Instruction format for SHR.X(s:B/W/L/Q) Dn(n), Dn(n)

n data register number for the count (extended opcode word bits 2:0).
d data register number for the count (extended opcode word bits 5:3).
s size selector for the operand size (extended opcode word bits 7:6).

SHR.X(s:B/W/L/Q) Dn(n), <ea(e)>

Form Dn(n), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands count is data register number; destination operand is compact effective-address field

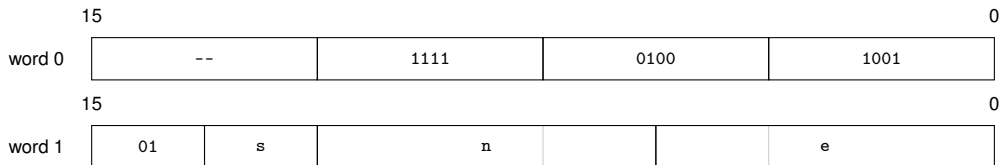


Instruction format for SHR.X(s:B/W/L/Q) Dn(n), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
n data register number for the count (extended opcode word bits 8:6).
s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SHR.X(s:B/W/L/Q) <count(n)>, <ea(e)>

Form <count(n)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands count is selector imm6; destination operand is compact effective-address field



Instruction format for SHR.X(s:B/W/L/Q) <count(n)>, <ea(e)>

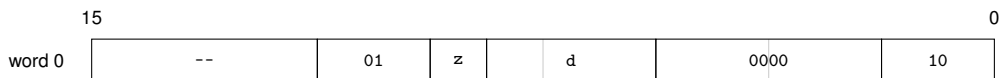
- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
n selector6 for the count (extended opcode word bits 11:6).
s size selector for the operand size (extended opcode word bits 13:12).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SUB**Subtract****SUB**

Summary:	Subtracts the source operand from the destination operand.
Operation:	<pre> result_v = dst_v - src_v; dst_v = result_v; ZF = result_v == 0; NF = result_v < 0; CF = dst_old_v < src_v; VF = sborrow(dst_old_v, src_v); </pre>
Assembler Syntax:	<pre> SUB.<W/L> <imm>, Dn SUB.<L/Q> Dn, Dn SUB <ea>, An SUB.<B/W/L/Q> Dn, <ea> SUB.<B/W/L/Q> <ea>, Dn SUB.<B/W/L/Q> <imm6>, <ea> </pre>
Attributes:	Size = (Byte, Word, Long, Quad, Long, Quad, Word, Long) Words = (1-8, 2-8) Privilege = unprivileged Flags = Z: result == 0; N: most significant bit(result); C: unsigned borrow; V: signed overflow
Condition Codes:	Z N C V * _ _ * Z: result == 0; N: most significant bit(result); C: unsigned borrow; V: signed overflow
Instruction Forms:	

SUB.X(z:W/L) imm, Dn(d)

Form imm, Dn(d)
Encoding primary opcode range 0x402 to 0x7c2
Words 2-8
Privilege unprivileged
Operands imm; destination operand is data register number

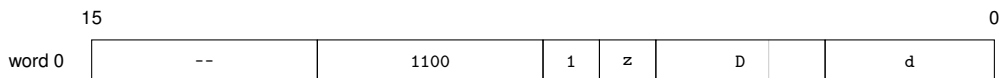


Instruction format for SUB.X(z:W/L) imm, Dn(d)

d data register number for the destination operand (primary word bits 8:6).
 z size selector for the operand size (primary word bits 9).

SUB.X(z:L/Q) Dn(d), Dn(D)

Form Dn(d), Dn(D)
Encoding primary opcode range 0xc80 to 0xcff
Words 1-8
Privilege unprivileged
Operands source operand is data register number; destination operand is data register number

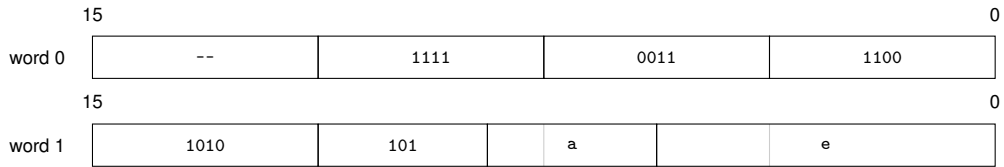


Instruction format for SUB.X(z:L/Q) Dn(d), Dn(D)

d data register number for the source operand (primary word bits 2:0).
 D data register number for the destination operand (primary word bits 5:3).
 z size selector for the operand size (primary word bits 6).

SUB <ea(e)>, An(a)

Form	<ea(e)>, An(a)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is address register number

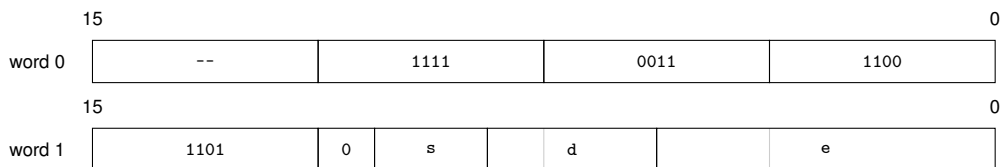


Instruction format for SUB <ea(e)>, An(a)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- a address register number for the destination operand (extended opcode word bits 8:6).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SUB.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form	Dn(d), <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is data register number; destination operand is compact effective-address field

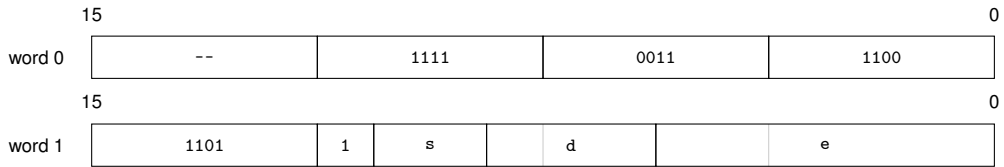


Instruction format for SUB.X(s:B/W/L/Q) Dn(d), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d data register number for the source operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SUB.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form <ea(e)>, Dn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is data register number

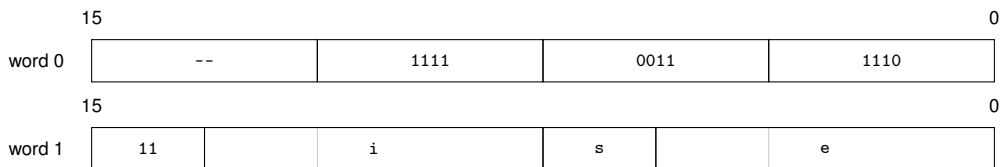


Instruction format for SUB.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
d data register number for the destination operand (extended opcode word bits 8:6).
s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SUB.X(s:B/W/L/Q) <imm(i)>, <ea(e)>

Form <imm(i)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands imm is unsigned 6-bit immediate literal; destination operand is compact effective-address field



Instruction format for SUB.X(s:B/W/L/Q) <imm(i)>, <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
s size selector for the operand size (extended opcode word bits 7:6).
i unsigned 6-bit immediate literal for the imm (extended opcode word bits 13:8). The encoded value is zero-extended before use for integer forms with an explicit B/W/L/Q operation size.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

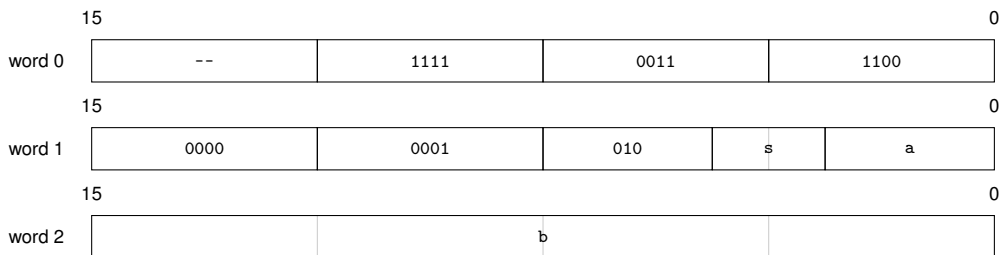
SUM**Register Set Sum****SUM**

Summary:	Sums selected D/A registers into a register.
Operation:	SUM reads every D/A register selected by the bitmap, adds the selected values using the instruction size, and writes the accumulated sum to the destination register. All selected source register values are read before the destination is written, so selecting the destination includes its original value.
Assembler Syntax:	SUM.<B/W/L/Q> <bitmap16>, An SUM.<B/W/L/Q> <bitmap16>, Dn
Attributes:	Size = (Byte, Word, Long, Quad) Words = (3-8) Privilege = unprivileged Flags = unchanged
Description:	Sums the selected D0-D7 and A0-A7 register values into a D- or A-register destination. The bitmap uses the same bit numbering as PUSHM and MOVSET operations. All selected source register values are read before the destination is written, so selecting the destination register includes its original value.
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

SUM.X(s:B/W/L/Q) <bitmap>, An(a)

Form	<bitmap>, An(a)
Encoding	extended opcode form
Words	3-8
Privilege	unprivileged
Operands	register bitmap is 16-bit register bitmap; destination operand is address register number

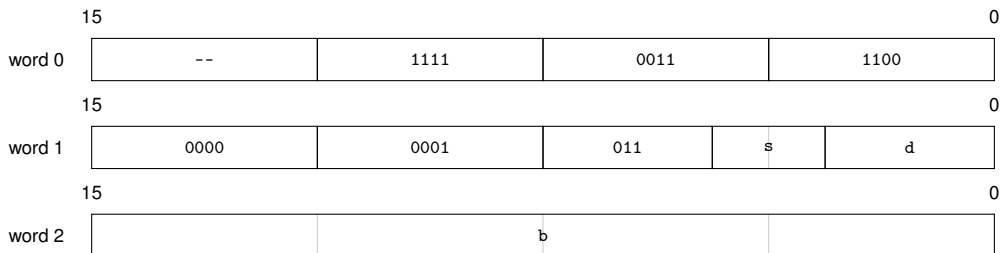


Instruction format for SUM.X(s:B/W/L/Q) <bitmap>, An(a)

- a address register number for the destination operand (extended opcode word bits 2:0).
- s size selector for the operand size (extended opcode word bits 4:3).
- b 16-bit register bitmap for the register bitmap (payload word 1 bits 15:0).

SUM.X(s:B/W/L/Q) <bitmap>, Dn(d)

Form <bitmap>, Dn(d)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands register bitmap is 16-bit register bitmap; destination operand is data register number



Instruction format for SUM.X(s:B/W/L/Q) <bitmap>, Dn(d)

- d data register number for the destination operand (extended opcode word bits 2:0).
- s size selector for the operand size (extended opcode word bits 4:3).
- b 16-bit register bitmap for the register bitmap (payload word 1 bits 15:0).

SWPT

Switch Page Table

SWPT

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Performs the switch page table operation.

PTCR = new_ptcr_v;
ASCR = 0;

SWPT.D Dn

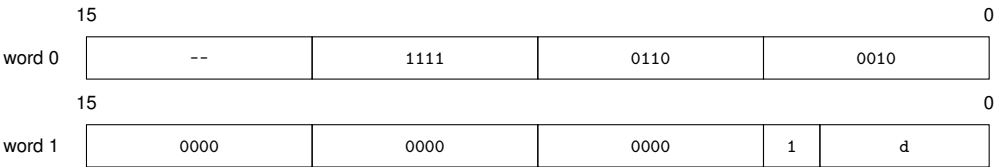
Words = (2-8)
Privilege = supervisor

Z N C V
- - - -
unchanged

Instruction Forms:

SWPT.D Dn(d)

Form	Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	supervisor
Operands	new page-table-control value is data register number



Instruction format for SWPT.D Dn(d)

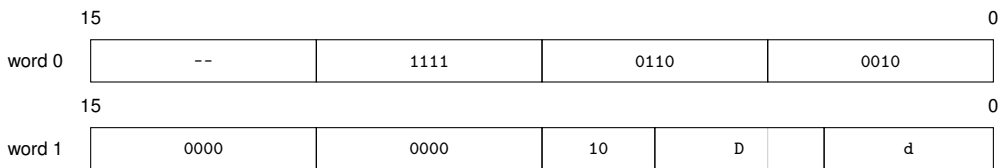
d data register number for the new page-table-control value (extended opcode word bits 2:0).

SWPTA**Switch Page Table With Context****SWPTA**

Summary:	Switches the page table and enables address-space context matching.
Operation:	PTCR = new_ptcr_v; ASCR = ((asid_v & 0xffff) << 16) 1;
Assembler Syntax:	SWPTA D _n , D _n
Attributes:	Words = (2-8) Privilege = supervisor
Description:	Updates PTCR from the first data register, writes the low 16 bits of the second data register into ASCR[31:16], and sets ASCR.AE.
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:SWPTA D_n(d), D_n(D)

Form	D _n (d), D _n (D)
Encoding	extended opcode form
Words	2-8
Privilege	supervisor
Operands	new page-table-control value is data register number; address-space identifier is data register number

**Instruction format for SWPTA D_n(d), D_n(D)**

- d data register number for the new page-table-control value (extended opcode word bits 2:0).
D data register number for the address-space identifier (extended opcode word bits 5:3).

SYNCCACHE

Synchronize Caches

SYNCCACHE

Summary:

Synchronize instruction/data cache visibility.

Operation:

arch_cache_synchronize(target_v);

Assembler Syntax:

SYNCCACHE <ea>

Attributes:

Words = (2-8)
Privilege = supervisor or policy controlled
Flags = unchanged

Condition Codes:

Z N C V
- - - -
unchanged

Instruction Forms:

SYNCCACHE <ea(e)>

Form

<ea(e)>

Encoding

extended opcode form

Words

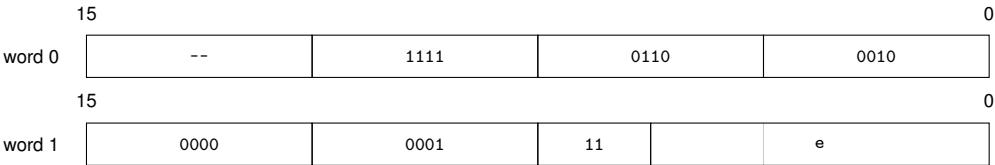
2-8

Privilege

supervisor or policy controlled

Operands

branch target is ea or range



Instruction format for SYNCCACHE <ea(e)>

e

compact effective-address field for the branch target (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

SYSCALL**System Call****SYSCALL**

Summary:	Enter supervisor mode through SPC/SCS/SDS.
Operation:	SYSCALL enters supervisor mode through the SPC/SCS/SDS supervisor-entry path. It writes a syscall stack frame containing the saved control state and return PC, resets DBANK to zero, sets STATUS.PM, loads CS from SCS and DS from SDS, and transfers control to SPC aligned to a 16-byte boundary.
Assembler Syntax:	SYSCALL
Attributes:	Words = (1-8) Privilege = unprivileged Flags = unchanged
Description:	Enter supervisor mode through the SPC/SCS/SDS supervisor-entry path. SYSCALL does not consume an interrupt vector. It saves FRAME_CONTROL, SS:SP, CS:PC, and DS, then sets DBANK to zero before entering the handler. It aligns the supervisor-entry address to 16 bytes, uses a 32-byte entry table, and sets STATUS.PM to one without automatically masking interrupts.
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

SYSCALL

Form	SYSCALL
Encoding	primary opcode 0xec1
Words	1-8
Privilege	unprivileged
Operands	no explicit operands

**Instruction format for SYSCALL**

No explicit operand fields are encoded in this form.

SYSRET

System Return

SYSRET

Summary:

Restore user control state from a syscall return frame.

Operation:

SYSRET restores user control state from the syscall stack frame at SP. It restores DBANK, FLAGS, STATUS, CS, DS, SS, SP, and the return PC from the frame written by SYSCALL, then resumes execution at the saved return PC.

Assembler Syntax:

SYSRET

Attributes:

Words = (1-8)
Privilege = supervisor
Flags = unchanged

Description:

Restore user control state from a syscall return frame. SYSRET is the matching return instruction for SYSCALL and reads the syscall frame as written, including the saved FRAME_CONTROL state.

Condition Codes:

Z N C V
- - - -
unchanged

Instruction Forms:

SYSRET	
Form	SYSRET
Encoding	primary opcode 0xf39
Words	1-8
Privilege	supervisor
Operands	no explicit operands



Instruction format for SYSRET

No explicit operand fields are encoded in this form.

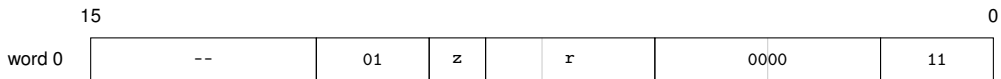
TEST**Test****TEST**

Summary:	Computes a comparison result and updates condition codes without storing the temporary value.
Operation:	<pre> result_v = dst_v & src_v; ZF = result_v == 0; NF = result_v < 0; CF = 0; VF = 0; </pre>
Assembler Syntax:	<pre> TEST.<W/L> <imm>, Dn TEST.<L/Q> Dn, Dn TEST.<B/W/L/Q> Dn, <ea> TEST.<B/W/L/Q> <ea>, Dn TEST.<B/W/L/Q> <imm6>, <ea> </pre>
Attributes:	Size = (Byte, Word, Long, Quad, Long, Quad, Word, Long) Words = (1-8, 2-8) Privilege = unprivileged Flags = Z: result == 0; N: most significant bit(result); C: 0; V: 0
Condition Codes:	Z N C V * - - * Z: result == 0; N: most significant bit(result); C: 0; V: 0

Instruction Forms:

TEST.X(z:W/L) imm, Dn(r)

Form	imm, Dn(r)
Encoding	primary opcode range 0x403 to 0x7c3
Words	2-8
Privilege	unprivileged
Operands	imm; right operand is data register number

**Instruction format for TEST.X(z:W/L) imm, Dn(r)**

- r data register number for the right operand (primary word bits 8:6).
 z size selector for the operand size (primary word bits 9).

TEST.X(z:L/Q) Dn(d), Dn(D)

Form	Dn(d), Dn(D)
Encoding	primary opcode range 0xd00 to 0xd7f
Words	1-8
Privilege	unprivileged
Operands	source operand is data register number; destination operand is data register number

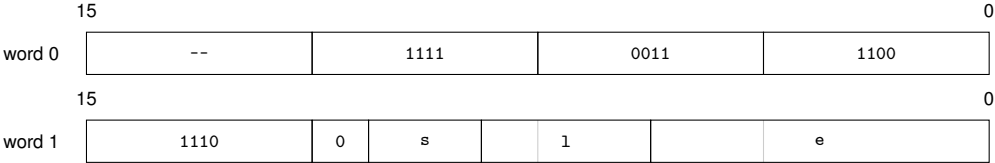


Instruction format for TEST.X(z:L/Q) Dn(d), Dn(D)

- d data register number for the source operand (primary word bits 2:0).
- D data register number for the destination operand (primary word bits 5:3).
- z size selector for the operand size (primary word bits 6).

TEST.X(s:B/W/L/Q) Dn(l), <ea(e)>

Form	Dn(l), <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	left operand is data register number; right operand is compact effective-address field

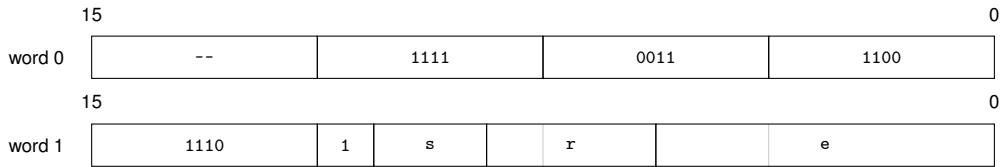


Instruction format for TEST.X(s:B/W/L/Q) Dn(l), <ea(e)>

- e compact effective-address field for the right operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - l data register number for the left operand (extended opcode word bits 8:6).
 - s size selector for the operand size (extended opcode word bits 10:9).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

TEST.X(s:B/W/L/Q) <ea(e)>, Dn(r)

Form <ea(e)>, Dn(r)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands left operand is compact effective-address field; right operand is data register number

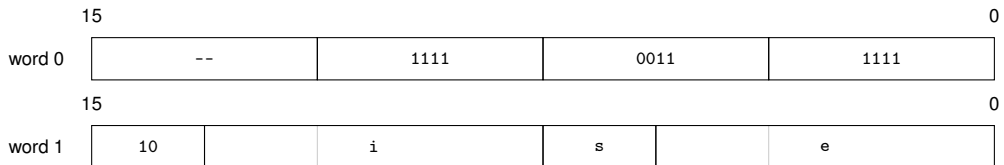


Instruction format for TEST.X(s:B/W/L/Q) <ea(e)>, Dn(r)

- e compact effective-address field for the left operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- r data register number for the right operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

TEST.X(s:B/W/L/Q) <imm(i)>, <ea(e)>

Form <imm(i)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands imm is unsigned 6-bit immediate literal; right operand is compact effective-address field



Instruction format for TEST.X(s:B/W/L/Q) <imm(i)>, <ea(e)>

- e compact effective-address field for the right operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- s size selector for the operand size (extended opcode word bits 7:6).
- i unsigned 6-bit immediate literal for the imm (extended opcode word bits 13:8). The encoded value is zero-extended before use for integer forms with an explicit B/W/L/Q operation size.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

TESTCANON

Test Canonical Address

TESTCANON

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Compute address and set V if noncanonical; do not read memory.

ZF = src_v == 0;
NF = src_v s< 0;
CF = 0;
VF = 0;

TESTCANON <ea>

Words = (2-8)
Privilege = unprivileged
Flags = V: set when the address is noncanonical

Z N C V
- - - *

V: set when the address is noncanonical

Instruction Forms:

TESTCANON <ea(e)>

Form

Encoding

Words

Privilege

Operands

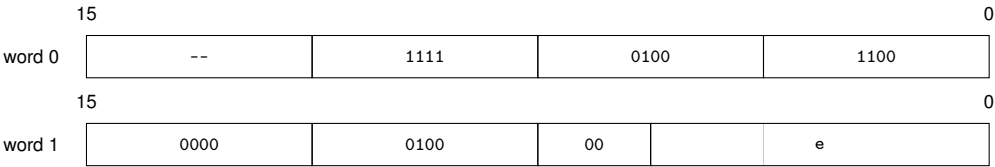
<ea(e)>

extended opcode form

2-8

unprivileged

source operand is compact effective-address field



Instruction format for TESTCANON <ea(e)>

e

compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

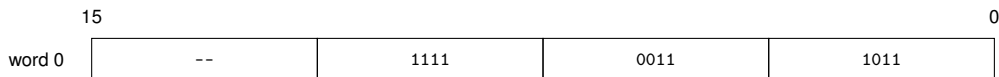
TRACE**Trace Marker****TRACE**

Summary:	Performs the trace marker operation.
Operation:	arch_trace_marker(marker_v & 0xffff);
Assembler Syntax:	TRACE <imm16>
Attributes:	Words = (2-8) Privilege = unprivileged Flags = unchanged
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

TRACE imm16

Form	imm16
Encoding	primary opcode 0xf3b
Words	2-8
Privilege	unprivileged
Operands	marker is 16-bit immediate selector

**Instruction format for TRACE imm16**

No explicit operand fields are encoded in this form.

TRAP

Trap

TRAP

Summary:

Raise architectural TRAP vector 0x05.

Operation:

arch_raise_trap(5:8);

Assembler Syntax:

TRAP

Attributes:

Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Condition Codes:

Z N C V
- - - -
unchanged

Instruction Forms:

TRAP

Form

TRAP

Encoding

canonical alias in extended opcode space

Words

2-8

Privilege

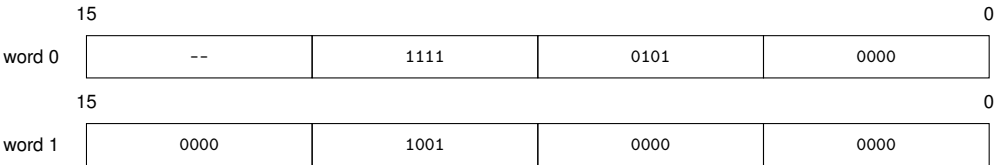
unprivileged

Operands

no explicit operands

Notes

alias_of=TRAPcc



Instruction format for TRAP

No explicit operand fields are encoded in this form.

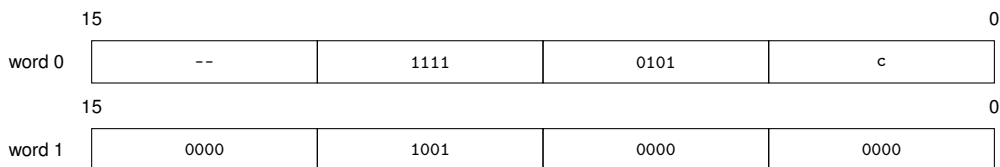
TRAPcc**Trap Conditional****TRAPcc**

Summary:	Raises architectural TRAP vector 0x05 when the encoded condition is true.
Operation:	if (cc_v == 0) goto <trapcc_done>; arch_raise_trap(5:8); <trapcc_done>
Assembler Syntax:	TRAPcc
Attributes:	Words = (2-8) Privilege = unprivileged Flags = unchanged
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

TRAPcc

Form	TRAPcc
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	condition code is condition-code selector

**Instruction format for TRAPcc**

No explicit operand fields are encoded in this form.

VTOP

Virtual To Physical Query

VTOP

Summary:

Operation:

Assembler Syntax:

Attributes:

Condition Codes:

Perform page walk and return translated physical address/status.

arch_page_translate(src_v);

VTOP <ea>

Words = (2-8)
Privilege = supervisor
Flags = operation-defined status result

Z N C V
- - - -
operation-defined status result

Instruction Forms:

VTOP <ea(e)>

Form

Encoding

Words

Privilege

Operands

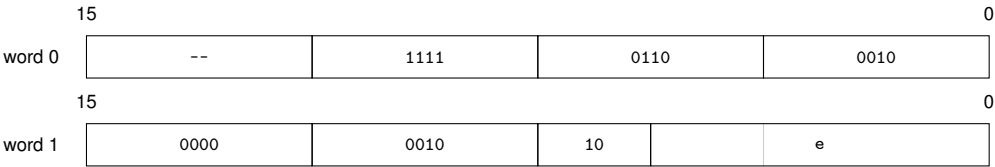
<ea(e)>

extended opcode form

2-8

supervisor

source operand is linear or ea; outputs



Instruction format for VTOP <ea(e)>

e

compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

WAIT

Wait

WAIT

Summary:	Wait until event or interrupt according to privilege policy.
Operation:	arch_wait_for_event();
Assembler Syntax:	WAIT
Attributes:	Words = (1-8) Privilege = policy controlled Flags = unchanged
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

WAIT

Form	WAIT
Encoding	primary opcode 0xed1
Words	1-8
Privilege	policy controlled
Operands	no explicit operands



Instruction format for WAIT

No explicit operand fields are encoded in this form.

WFENCE

Write Fence

WFENCE

Summary:

Order prior writes before later writes.

Operation:

arch_fence_write();

Assembler Syntax:

WFENCE

Attributes:

Words = (1-8)
Privilege = unprivileged
Flags = unchanged

Condition Codes:

Z N C V
- - - -
unchanged

Instruction Forms:

WFENCE

Form

WFENCE

Encoding

primary opcode 0x006

Words

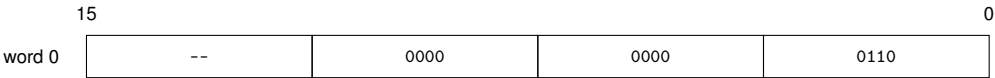
1-8

Privilege

unprivileged

Operands

no explicit operands



Instruction format for WFENCE

No explicit operand fields are encoded in this form.

WRBKDCACHE**Write Back Data Cache****WRBKDCACHE**

Summary: Write back data cache block or range.

Operation: `arch_cache_writeback_data(target_v);`

Assembler Syntax: `WRBKDCACHE <ea>`

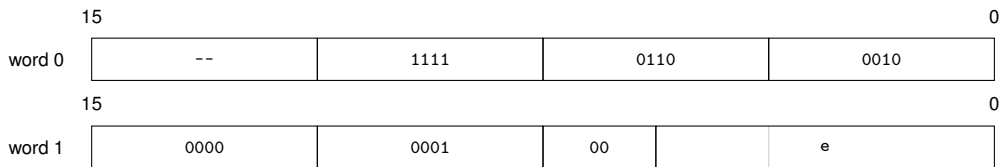
Attributes:
 Words = (2-8)
 Privilege = supervisor or policy controlled
 Flags = unchanged

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

`WRBKDCACHE <ea(e)>`

Form `<ea(e)>`
Encoding extended opcode form
Words 2-8
Privilege supervisor or policy controlled
Operands branch target is ea or range



Instruction format for WRBKDCACHE <ea(e)>

- e compact effective-address field for the branch target (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

WRCR**Write Control Register****WRCR**

Summary: Write a privileged control register from a D register.

Operation:

```

if (cr_v != 0x0000) goto <wrcr_ascr>;
PTCR = src_v;
goto <wrcr_done>;
<wrcr_ascr>
if (cr_v != 0x0001) goto <wrcr_icr>;
ASCR = src_v;
goto <wrcr_done>;
<wrcr_icr>
if (cr_v != 0x0002) goto <wrcr_spc>;
ICR = src_v;
goto <wrcr_done>;
<wrcr_spc>
if (cr_v != 0x0100) goto <wrcr_scs>;
SPC = src_v;
goto <wrcr_done>;
<wrcr_scs>
if (cr_v != 0x0101) goto <wrcr_sds>;
SCS = src_v;
goto <wrcr_done>;
<wrcr_sds>
if (cr_v != 0x0102) goto <wrcr_sss0>;
SDS = src_v;
goto <wrcr_done>;
<wrcr_sss0>
if (cr_v != 0x0200) goto <wrcr_ssp0>;
SSS0 = src_v;
goto <wrcr_done>;
<wrcr_ssp0>
if (cr_v != 0x0201) goto <wrcr_sss1>;
SSP0 = src_v;
goto <wrcr_done>;
<wrcr_sss1>
if (cr_v != 0x0210) goto <wrcr_ssp1>;
SSS1 = src_v;
goto <wrcr_done>;
<wrcr_ssp1>
if (cr_v != 0x0211) goto <wrcr_sss2>;
SSP1 = src_v;
goto <wrcr_done>;
<wrcr_sss2>
if (cr_v != 0x0220) goto <wrcr_ssp2>;
SSS2 = src_v;

```

```

goto <wrcr_done>;
<wrcr_ssp2>
if (cr_v != 0x0221) goto <wrcr_sss3>;
SSP2 = src_v;
goto <wrcr_done>;
<wrcr_sss3>
if (cr_v != 0x0230) goto <wrcr_ssp3>;
SSS3 = src_v;
goto <wrcr_done>;
<wrcr_ssp3>
if (cr_v != 0x0231) goto <wrcr_bootpc>;
SSP3 = src_v;
goto <wrcr_done>;
<wrcr_bootpc>
if (cr_v != 0x1000) goto <wrcr_bootcfg>;
BOOTPC = src_v;
goto <wrcr_done>;
<wrcr_bootcfg>
if (cr_v != 0x1001) goto <wrcr_ptc>;
BOOTCFG = src_v;
goto <wrcr_done>;
<wrcr_ptc>
if (cr_v != 0x1100) goto <wrcr_pmc>;
PTC = src_v;
goto <wrcr_done>;
<wrcr_pmc>
if (cr_v != 0x1101) goto <wrcr_invalid>;
PMC = src_v;
goto <wrcr_done>;
<wrcr_invalid>
arch_raise_invalid_control_state(cr_v);
<wrcr_done>

```

Assembler Syntax: WRCR.D Dn, <cr>

Attributes: Words = (3-8)
 Privilege = supervisor
 Flags = unchanged

Description: Write a privileged control register from a D register. WRCR is always a supervisor-only instruction; user-visible control state must use dedicated instructions gated by control-register policy bits.

Condition Codes: **Z N C V**

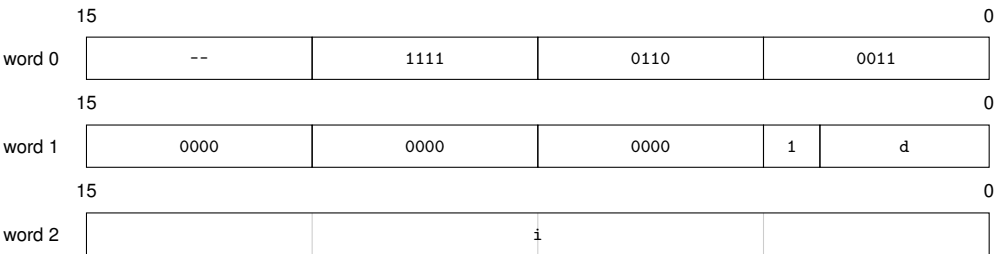
- - - -

unchanged

Instruction Forms:

WRCR.D Dn(d), <cr>

Form	Dn(d), <cr>
Encoding	extended opcode form
Words	3-8
Privilege	supervisor
Operands	source operand is data register number; cr



Instruction format for WRCR.D Dn(d), <cr>

- d data register number for the source operand (extended opcode word bits 2:0).
- i control-register selector for the control-register selector (payload word 1 bits 15:0).

WRFLAGS**Write Flags****WRFLAGS**

Summary: Write the integer condition-code flags from a D register.

Operation: $ZF = (src_v \gg 3) \& 1;$
 $NF = (src_v \gg 2) \& 1;$
 $CF = (src_v \gg 1) \& 1;$
 $VF = src_v \& 1;$

Assembler Syntax: WRFLAGS.D Dn

Attributes: Words = (2-8)
Privilege = unprivileged
Flags = writes FLAGS from source low bits

Description: WRFLAGS writes the low 16 bits from the source D register into the architectural FLAGS register. Reserved FLAGS bits must be zero. FLAGS is not part of the S segment-register operand class.

Condition Codes: **Z N C V**

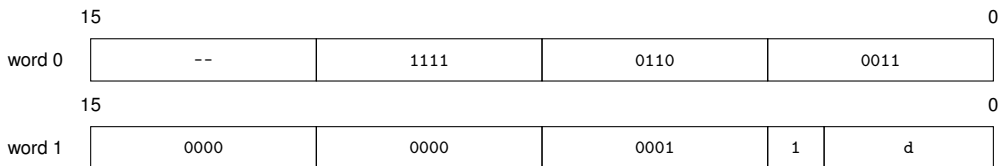
- - - -

writes FLAGS from source low bits

Instruction Forms:

WRFLAGS.D Dn(d)

Form Dn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number



Instruction format for WRFLAGS.D Dn(d)

d data register number for the source operand (extended opcode word bits 2:0).

WRFSTATUS

Write Floating-Point Status

WRFSTATUS

Summary:

Write the FPU status/control register from a D register.

Operation:

FSTATUS = src_v;

Assembler Syntax:

WRFSTATUS.D Dn

Attributes:

Words = (2-8)
Privilege = unprivileged
Flags = writes FSTATUS from source bits

Description:

WRFSTATUS writes the architectural 16-bit FSTATUS register from the low 16 bits of the source D register. Reserved FSTATUS bits must be zero. FFLAGS is a separate accrued floating-point exception flag register and is not written by WRFSTATUS. WRFSTATUS requires the floating-point extension.

Condition Codes:

Z N C V
- - - -
writes FSTATUS from source bits

Instruction Forms:

WRFSTATUS.D Dn(d)

Form

Dn(d)

Encoding

extended opcode form

Words

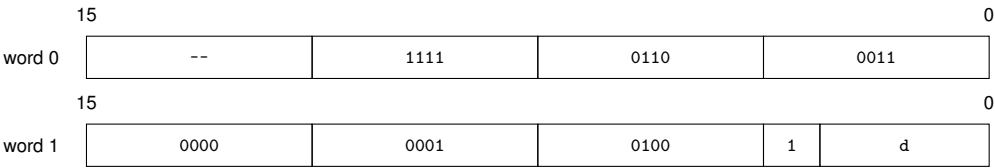
2-8

Privilege

unprivileged

Operands

source operand is data register number



Instruction format for WRFSTATUS.D Dn(d)

d data register number for the source operand (extended opcode word bits 2:0).

WRSEG**Write Segment Register****WRSEG**

Summary: Write a segment register image from a D register.

Operation: `seg_v = src_v;`

Assembler Syntax: `WRSEG Dn, Sreg`

Attributes:
 Words = (2-8)
 Privilege = policy controlled
 Flags = unchanged

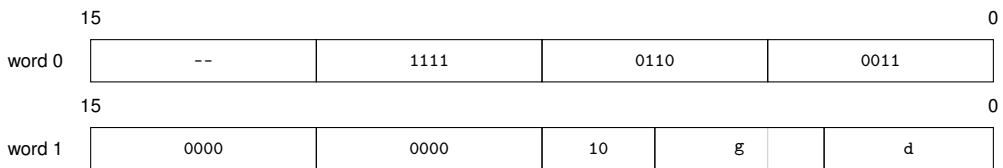
Description: WRSEG writes a 64-bit segment register image from the source D register to an S-class segment register. The S operand selector encoding is defined by the SREG selector table. General writes to CS are illegal; use LJMP, LCALL, LRET, SYSRET, or IRET for CS-changing control transfer.

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

`WRSEG Dn(d), Sreg(g)`

Form Dn(d), Sreg(g)
Encoding extended opcode form
Words 2-8
Privilege policy controlled
Operands source operand is data register number; seg is segment register selector



Instruction format for WRSEG Dn(d), Sreg(g)

d data register number for the source operand (extended opcode word bits 2:0).
 g segment register selector for the seg (extended opcode word bits 5:3).

WRSTATUS

Write Status

WRSTATUS

Summary:

Write the processor STATUS register from a D register.

Operation:

STATUS = src_v;

Assembler Syntax:

WRSTATUS.D Dn

Attributes:

Words = (2-8)
Privilege = supervisor
Flags = unchanged

Description:

WRSTATUS writes the low 16 bits from the source D register into the architectural STATUS register. Reserved STATUS bits must be zero. STATUS is not part of the S segment-register operand class.

Condition Codes:

Z N C V
- - - -
unchanged

Instruction Forms:

WRSTATUS.D Dn(d)

Form

Dn(d)

Encoding

extended opcode form

Words

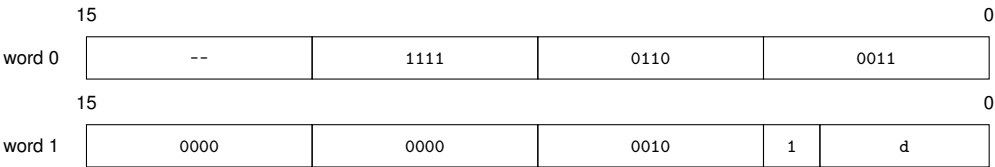
2-8

Privilege

supervisor

Operands

source operand is data register number



Instruction format for WRSTATUS.D Dn(d)

d data register number for the source operand (extended opcode word bits 2:0).

XCHG**Exchange****XCHG**

Summary: Performs the exchange operation.

Operation: tmp_v = lhs_v;
lhs_v = rhs_v;
rhs_v = tmp_v;

Assembler Syntax: XCHG.<B/W/L/Q> Dn, <ea>
XCHG.<B/W/L/Q> <ea>, Dn

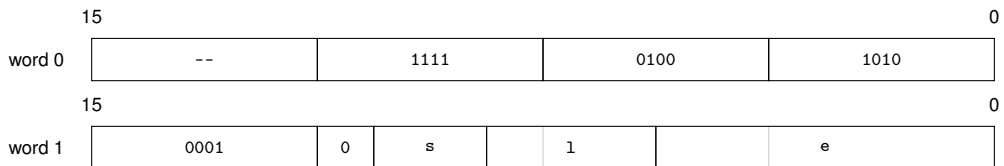
Attributes: Size = (Byte, Word, Long, Quad)
Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

XCHG.X(s:B/W/L/Q) Dn(l), <ea(e)>

Form Dn(l), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands left operand is data register number; right operand is compact effective-address field

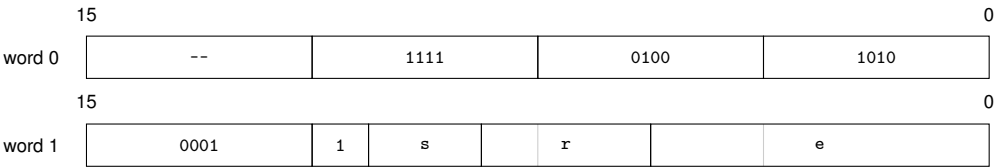


Instruction format for XCHG.X(s:B/W/L/Q) Dn(l), <ea(e)>

- e compact effective-address field for the right operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- l data register number for the left operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

XCHG.X(s:B/W/L/Q) <ea(e)>, Dn(r)

Form	<ea(e)>, Dn(r)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	left operand is compact effective-address field; right operand is data register number



Instruction format for XCHG.X(s:B/W/L/Q) <ea(e)>, Dn(r)

- e compact effective-address field for the left operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- r data register number for the right operand (extended opcode word bits 8:6).
- s size selector for the operand size (extended opcode word bits 10:9). EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

XCHGSETAD**Exchange D And A Register Set****XCHGSETAD**

Summary:	For each selected i: swap A[i] and D[i].
Operation:	XCHGSETAD swaps each selected A register with the matching D register. The one-operand form exchanges A[i] with D[i] in the current data-register bank; the banked form exchanges A[i] with D[i] in the explicitly named DBANK. Bits 0 through 7 select register numbers 0 through 7.
Assembler Syntax:	XCHGSETAD <bitmap16> XCHGSETAD DBn, <bitmap16>
Attributes:	Words = (2-8, 3-8) Privilege = unprivileged Flags = unchanged
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

XCHGSETAD <bitmap>

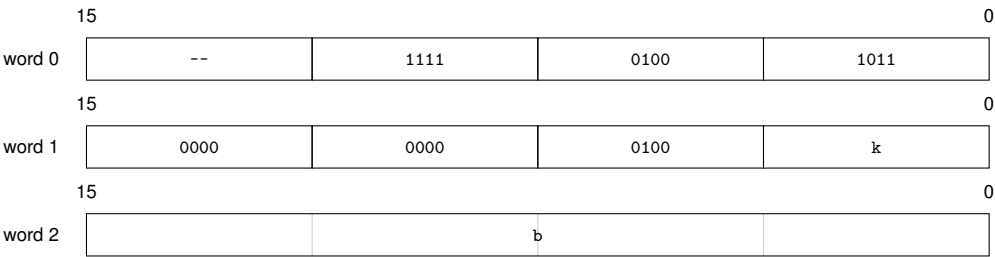
Form	<bitmap>
Encoding	primary opcode 0xf36
Words	2-8
Privilege	unprivileged
Operands	register bitmap is 16-bit register bitmap

**Instruction format for XCHGSETAD <bitmap>**

No explicit operand fields are encoded in this form.

XCHGSETAD DBn(k), <bitmap>

Form	DBn(k), <bitmap>
Encoding	extended opcode form
Words	3-8
Privilege	unprivileged
Operands	bank is dbank; register bitmap is 16-bit register bitmap



Instruction format for XCHGSETAD DBn(k), <bitmap>

- k dbank for the bank (extended opcode word bits 3:0).
- b 16-bit register bitmap for the register bitmap (payload word 1 bits 15:0).

XCHGSETDA**Exchange A And D Register Set****XCHGSETDA**

Summary:	For each selected i: swap D[i] and A[i].
Operation:	XCHGSETDA swaps each selected D register with the matching A register. The one-operand form exchanges D[i] with A[i] in the current data-register bank; the banked form exchanges D[i] in the explicitly named DBANK with A[i]. Bits 0 through 7 select register numbers 0 through 7.
Assembler Syntax:	XCHGSETDA <bitmap16> XCHGSETDA DBn, <bitmap16>
Attributes:	Words = (2-8, 3-8) Privilege = unprivileged Flags = unchanged
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

XCHGSETDA <bitmap>

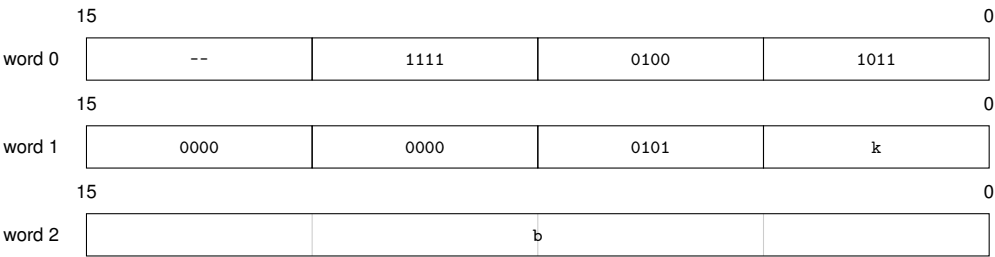
Form	<bitmap>
Encoding	primary opcode 0xf37
Words	2-8
Privilege	unprivileged
Operands	register bitmap is 16-bit register bitmap

**Instruction format for XCHGSETDA <bitmap>**

No explicit operand fields are encoded in this form.

XCHGSETDA DBn(k), <bitmap>

Form	DBn(k), <bitmap>
Encoding	extended opcode form
Words	3-8
Privilege	unprivileged
Operands	bank is dbank; register bitmap is 16-bit register bitmap



Instruction format for XCHGSETDA DBn(k), <bitmap>

- k dbank for the bank (extended opcode word bits 3:0).
- b 16-bit register bitmap for the register bitmap (payload word 1 bits 15:0).

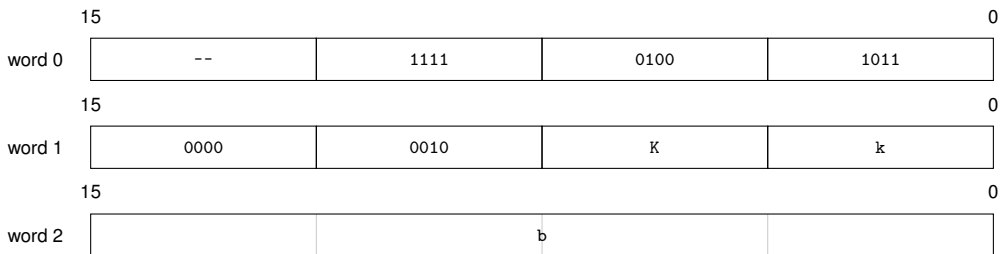
XCHGSETDD**Exchange D Register Sets Between Banks****XCHGSETDD**

Summary:	Swaps selected D registers between two data-register banks.
Operation:	XCHGSETDD swaps each selected D register between the two explicitly named data-register banks. Bits 0 through 7 select D0 through D7 respectively. The current DBANK selector is not changed.
Assembler Syntax:	XCHGSETDD DBn, DBn, <bitmap16>
Attributes:	Words = (3-8) Privilege = unprivileged Flags = unchanged
Description:	Swaps selected D registers between two explicitly named DBANK values according to the bitmap operand. The current DBANK selector is not changed.
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

XCHGSETDD DBn(k), DBn(K), <bitmap>

Form	DBn(k), DBn(K), <bitmap>
Encoding	extended opcode form
Words	3-8
Privilege	unprivileged
Operands	bank a is dbank; bank b is dbank; register bitmap is 16-bit register bitmap

**Instruction format for XCHGSETDD DBn(k), DBn(K), <bitmap>**

- k dbank for the bank a (extended opcode word bits 3:0).
- K dbank for the bank b (extended opcode word bits 7:4).
- b 16-bit register bitmap for the register bitmap (payload word 1 bits 15:0).

XOR

Bitwise Xor

XOR

Summary: Computes the bitwise XOR of the source and destination operands.

Operation: result_v = dst_v ^ src_v;
dst_v = result_v;
ZF = result_v == 0;
NF = result_v s< 0;
CF = 0;
VF = 0;

Assembler Syntax: XOR.<L/Q> Dn, Dn
XOR.<W/L> <imm>, Dn
XOR.<B/W/L/Q> Dn, <ea>
XOR.<B/W/L/Q> <ea>, Dn
XOR.<B/W/L/Q> <imm6>, <ea>

Attributes: Size = (Byte, Word, Long, Quad, Long, Quad, Word, Long)
Words = (1-8, 2-8)
Privilege = unprivileged
Flags = Z: result == 0; N: most significant bit(result); C: 0; V: 0

Condition Codes: **Z N C V**
* - - *
Z: result == 0; N: most significant bit(result); C: 0; V: 0

Instruction Forms:

XOR.X(z:L/Q) Dn(d), Dn(D)

Form Dn(d), Dn(D)
Encoding primary opcode range 0xd80 to 0xdff
Words 1-8
Privilege unprivileged
Operands source operand is data register number; destination operand is data register number



Instruction format for XOR.X(z:L/Q) Dn(d), Dn(D)

- d data register number for the source operand (primary word bits 2:0).
- D data register number for the destination operand (primary word bits 5:3).
- z size selector for the operand size (primary word bits 6).

XOR.X(z:W/L) imm, Dn(d)

Form imm, Dn(d)
Encoding primary opcode range 0xef0 to 0xef7
Words 2-8
Privilege unprivileged
Operands imm; destination operand is data register number

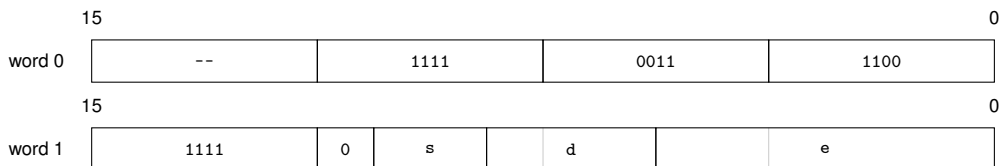


Instruction format for XOR.X(z:W/L) imm, Dn(d)

d data register number for the destination operand (primary word bits 2:0).
 z size selector for the operand size (primary word bits 3).

XOR.X(s:B/W/L/Q) Dn(d), <ea(e)>

Form Dn(d), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is compact effective-address field

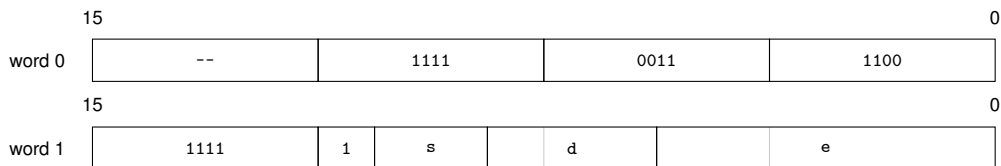


Instruction format for XOR.X(s:B/W/L/Q) Dn(d), <ea(e)>

e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 d data register number for the source operand (extended opcode word bits 8:6).
 s size selector for the operand size (extended opcode word bits 10:9).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

XOR.X(s:B/W/L/Q) <ea(e)>, Dn(d)

Form <ea(e)>, Dn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is data register number

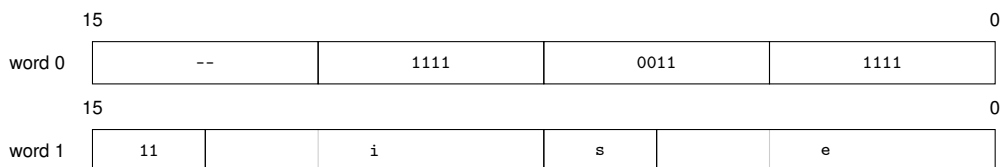


Instruction format for XOR.X(s:B/W/L/Q) <ea(e)>, Dn(d)

- e** compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d** data register number for the destination operand (extended opcode word bits 8:6).
- s** size selector for the operand size (extended opcode word bits 10:9).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

XOR.X(s:B/W/L/Q) <imm(i)>, <ea(e)>

Form <imm(i)>, <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands imm is unsigned 6-bit immediate literal; destination operand is compact effective-address field



Instruction format for XOR.X(s:B/W/L/Q) <imm(i)>, <ea(e)>

- e** compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- s** size selector for the operand size (extended opcode word bits 7:6).
- i** unsigned 6-bit immediate literal for the imm (extended opcode word bits 13:8). The encoded value is zero-extended before use for integer forms with an explicit B/W/L/Q operation size.
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

YIELD**Yield****YIELD**

Summary: Hint scheduler or implementation that the current thread may yield.

Operation: arch_yield();

Assembler Syntax: YIELD

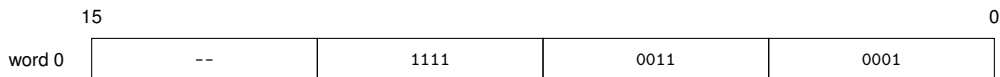
Attributes:
 Words = (1-8)
 Privilege = unprivileged
 Flags = unchanged

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

YIELD

Form YIELD
Encoding primary opcode 0xf31
Words 1-8
Privilege unprivileged
Operands no explicit operands

**Instruction format for YIELD**

No explicit operand fields are encoded in this form.

22 VIRTUALIZATION ACCELERATION INSTRUCTIONS SUMMARY

Mnemonic	Summary	Forms
ENCINST	Encode Instruction	1

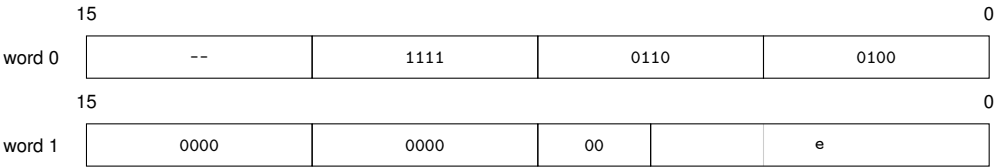
Table 22-1. Virtualization Acceleration Instructions Summary

ENCINST**Encode Instruction****ENCINST**

Summary:	Writes one canonical instruction encoding to a destination effective address.
Operation:	<code>D0 = arch_emit_instruction(dst_v, D0, D1, D2, D3);</code>
Assembler Syntax:	<code>ENCINST <ea></code>
Attributes:	Words = (2-8) Privilege = user allowed when ENCINST is present Flags = unchanged
Description:	ENCINST is an optional virtualization-acceleration instruction for software encoders, runtime code generators, hypervisors, and binary translation systems. Its only explicit operand is the destination effective address. The instruction descriptor is supplied through implicit D registers. ENCINST validates the descriptor, selects the canonical Bedrock encoding, and writes the resulting 16-bit-word instruction image to ordinary memory at the destination EA. Before execution, D0 is split into a high 32-bit form selector and a low 32-bit encode control word. D0 receives the number of bytes written on success or an implementation-defined negative encode-status value on descriptor failure. Memory permissions, segmentation, paging, and cache synchronization follow the ordinary store and SYNC-CACHE rules.
Condition Codes:	Z N C V - - - - unchanged
Instruction Forms:	

ENCINST <ea(e)>

Form	<ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	user allowed when ENCINST is present
Operands	destination operand is compact effective-address field



Instruction format for ENCINST <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms. EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

24 FLOATING-POINT INSTRUCTIONS SUMMARY

Mnemonic	Summary	Forms
FABS	Floating-Point Absolute Value	3
FADD	Floating-Point Add	2
FBNDII	Floating-Point Bounds Check Inclusive-Inclusive	2
FBNDIX	Floating-Point Bounds Check Inclusive-Exclusive	2
FBNDXI	Floating-Point Bounds Check Exclusive-Inclusive	2
FBNDXX	Floating-Point Bounds Check Exclusive-Exclusive	2
FCEIL	Floating-Point Ceiling	3
FCLASS	Floating-Point Classify	1
FCLR	Floating-Point Clear	2
FCMP	Floating-Point Compare	2
FCOPYSIGN	Floating-Point Copy Sign	1
FCVT	Floating-Point Convert Signed	3
FCVTU	Floating-Point Convert Unsigned	3
FDIV	Floating-Point Divide	2
FFLOOR	Floating-Point Floor	3
FGETEXP	Floating-Point Get Exponent	2
FGETMAN	Floating-Point Get Mantissa	2
FINT	Floating-Point Integral Value	3
FINTRZ	Floating-Point Integral Toward Zero	3
FMADD	Floating-Point Fused Multiply Add	3
FMAX	Floating-Point Maximum	2
FMIN	Floating-Point Minimum	2
FMOD	Floating-Point Modulo	2
FMOV	Floating-Point Move	3
FMOVCR	Floating-Point Move Constant	1
FMOV _{cc}	Floating-Point Conditional Move	3
FMSUB	Floating-Point Fused Multiply Subtract	3
FMUL	Floating-Point Multiply	2
FNEG	Floating-Point Negate	3
FNMADD	Floating-Point Fused Negative Multiply Add	3
FNMSUB	Floating-Point Fused Negative Multiply Subtract	3
FPOPM	Floating-Point Pop Multiple	1
FPUSHM	Floating-Point Push Multiple	1
FREM	Floating-Point Remainder	2
FROUND	Floating-Point Round	3
FSCALE	Floating-Point Scale	2
FSQRT	Floating-Point Square Root	3
FSUB	Floating-Point Subtract	2
FTEST	Floating-Point Test	2
FTRUNC	Floating-Point Truncate	3
FXCHG	Floating-Point Exchange	1

Table 24-1. Floating-Point Instructions Summary

FABS

Floating-Point Absolute Value

FABS

Summary: Performs the floating-point absolute value operation.

Operation: dst_v = abs(src_v);

Assembler Syntax: FABS.<S/D> <ea>, Fn
FABS.<S/D> Fn, <ea>
FABS.<S/D> Fn, Fn

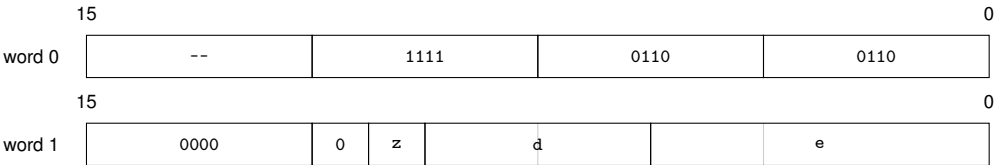
Attributes: Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

Floating-Point Status: NV DZ OF UF NX
- - - - -
FFLAGS unchanged

Instruction Forms:

FABS.X(z:S/D) <ea(e)>, Fn(d)

Form <ea(e)>, Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is floating-point register number

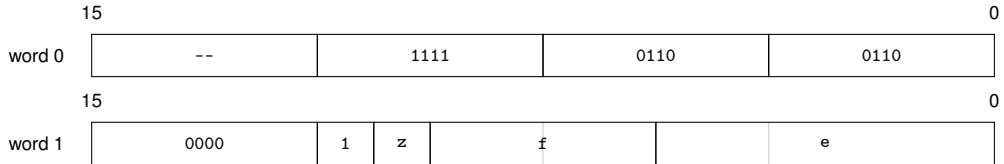


Instruction format for FABS.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FABS.X(z:S/D) Fn(f), <ea(e)>

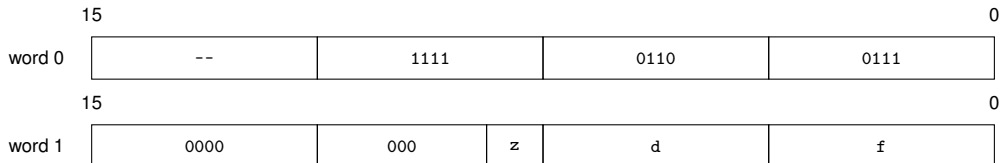
Form Fn(f), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is compact effective-address field

**Instruction format for FABS.X(z:S/D) Fn(f), <ea(e)>**

- e** compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- f** floating-point register number for the source operand (extended opcode word bits 9:6).
- z** size selector for the operand size (extended opcode word bits 10).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FABS.X(z:S/D) Fn(f), Fn(d)

Form Fn(f), Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number

**Instruction format for FABS.X(z:S/D) Fn(f), Fn(d)**

- f** floating-point register number for the source operand (extended opcode word bits 3:0).
- d** floating-point register number for the destination operand (extended opcode word bits 7:4).
- z** size selector for the operand size (extended opcode word bits 8).

FADD

Floating-Point Add

FADD

Summary: Adds the source operand to the destination operand.

Operation: `dst_v = dst_v f+ src_v;`

Assembler Syntax: `FADD.<S/D> <ea>, Fn`
`FADD.<S/D> Fn, Fn`

Attributes: Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

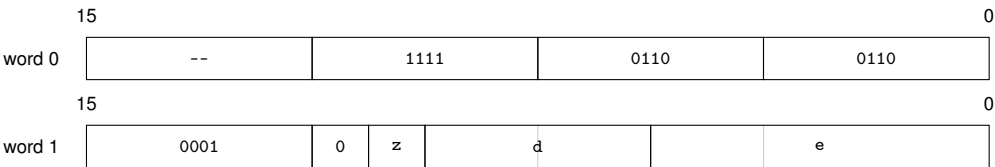
Floating-Point Status: **NV DZ OF UF NX**
* _ * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FADD.X(z:S/D) <ea(e)>, Fn(d)`

Form `<ea(e)>, Fn(d)`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is floating-point register number

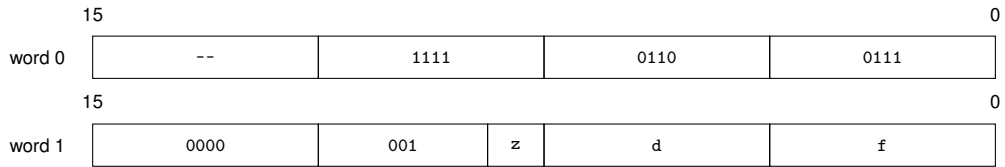


Instruction format for **FADD.X(z:S/D) <ea(e)>, Fn(d)**

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FADD.X(z:S/D) Fn(f), Fn(d)

Form	Fn(f), Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FADD.X(z:S/D) Fn(f), Fn(d)

- f floating-point register number for the source operand (extended opcode word bits 3:0).
- d floating-point register number for the destination operand (extended opcode word bits 7:4).
- z size selector for the operand size (extended opcode word bits 8).

FBNDII

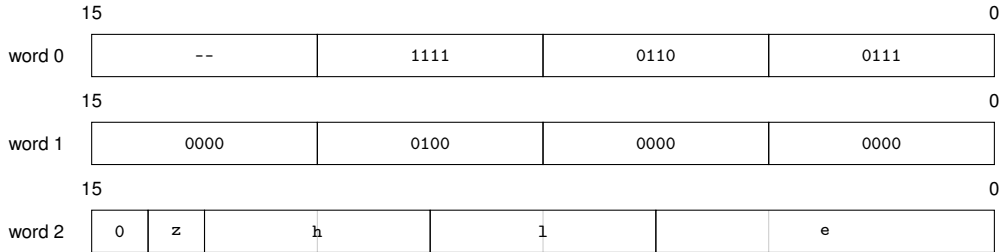
Floating-Point Bounds Check Inclusive-Inclusive

FBNDII

Summary:	Checks floating-point value membership in the inclusive-inclusive interval [lo, hi].
Operation:	ZF = 0; NF = 0; CF = 0; VF = 0; if (value_v < lo_v) goto <fbndii_bad>; if (value_v > hi_v) goto <fbndii_bad>; goto <fbndii_done>; <fbndii_bad> VF = 1; <fbndii_done>
Assembler Syntax:	FBNDII.<S/D> Fn, <ea>, Fn FBNDII.<S/D> Fn, Fn, Fn
Attributes:	Size = (Single, Double) Words = (3-8) Privilege = unprivileged Flags = V: set when the value is out of bounds or unordered
Description:	FBNDII treats both bounds as inclusive. It sets V when value is unordered or outside [lo, hi]; Z, N, and C are unchanged.
Condition Codes:	Z N C V - - - * V: set when the value is out of bounds or unordered
Instruction Forms:	

FBNDII.X(z:S/D) Fn(l), <ea(e)>, Fn(h)

Form Fn(l), <ea(e)>, Fn(h)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands lower bound operand is floating-point register number; value is compact effective-address field; upper bound operand is floating-point register number

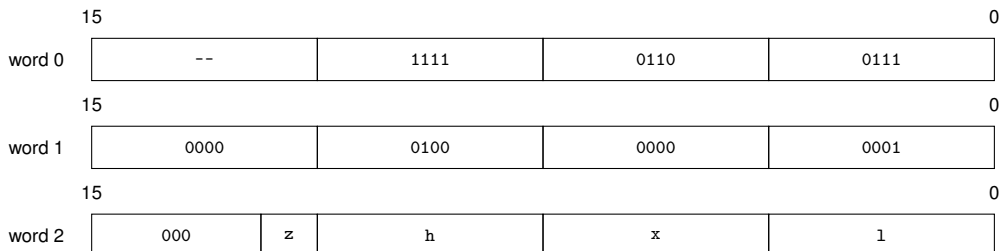


Instruction format for FBNDII.X(z:S/D) Fn(l), <ea(e)>, Fn(h)

- e compact effective-address field for the value (payload word 1 bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 l floating-point register number for the lower bound operand (payload word 1 bits 9:6).
 h floating-point register number for the upper bound operand (payload word 1 bits 13:10).
 z size selector for the operand size (payload word 1 bits 14).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FBNDII.X(z:S/D) Fn(l), Fn(x), Fn(h)

Form Fn(l), Fn(x), Fn(h)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands lower bound operand is floating-point register number; value is floating-point register number; upper bound operand is floating-point register number



Instruction format for FBNDII.X(z:S/D) Fn(l), Fn(x), Fn(h)

- l floating-point register number for the lower bound operand (payload word 1 bits 3:0).
 x floating-point register number for the value (payload word 1 bits 7:4).
 h floating-point register number for the upper bound operand (payload word 1 bits 11:8).
 z size selector for the operand size (payload word 1 bits 12).

FBNDIX

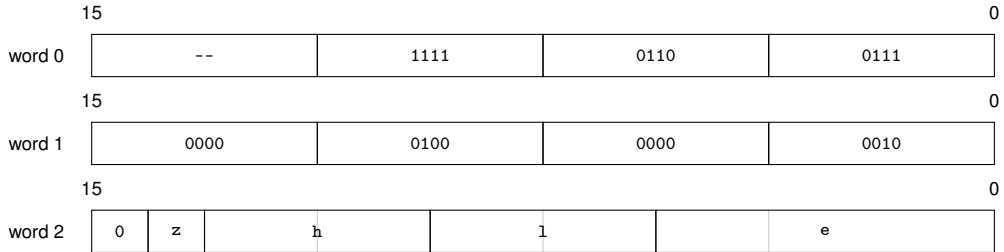
Floating-Point Bounds Check Inclusive-Exclusive

FBNDIX

Summary:	Checks floating-point value membership in the inclusive-exclusive interval [lo, hi).
Operation:	ZF = 0; NF = 0; CF = 0; VF = 0; if (value_v < lo_v) goto <fbndix_bad>; if (value_v >= hi_v) goto <fbndix_bad>; goto <fbndix_done>; <fbndix_bad> VF = 1; <fbndix_done>
Assembler Syntax:	FBNDIX.<S/D> Fn, <ea>, Fn FBNDIX.<S/D> Fn, Fn, Fn
Attributes:	Size = (Single, Double) Words = (3-8) Privilege = unprivileged Flags = V: set when the value is out of bounds or unordered
Description:	FBNDIX treats the low bound as inclusive and the high bound as exclusive. It sets V when value is unordered or outside [lo, hi); Z, N, and C are unchanged.
Condition Codes:	Z N C V - - - * V: set when the value is out of bounds or unordered
Instruction Forms:	

FBNDIX.X(z:S/D) Fn(l), <ea(e)>, Fn(h)

Form Fn(l), <ea(e)>, Fn(h)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands lower bound operand is floating-point register number; value is compact effective-address field; upper bound operand is floating-point register number

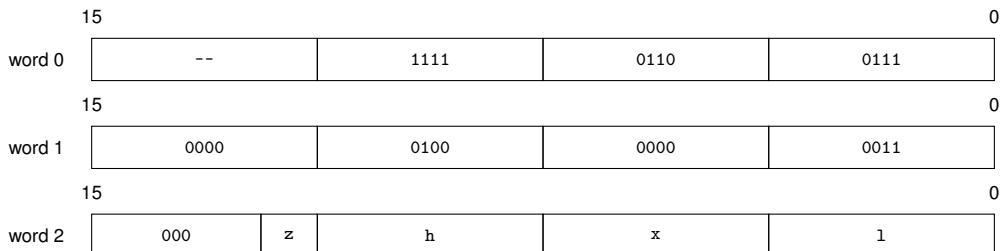


Instruction format for FBNDIX.X(z:S/D) Fn(l), <ea(e)>, Fn(h)

- e compact effective-address field for the value (payload word 1 bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 l floating-point register number for the lower bound operand (payload word 1 bits 9:6).
 h floating-point register number for the upper bound operand (payload word 1 bits 13:10).
 z size selector for the operand size (payload word 1 bits 14).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FBNDIX.X(z:S/D) Fn(l), Fn(x), Fn(h)

Form Fn(l), Fn(x), Fn(h)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands lower bound operand is floating-point register number; value is floating-point register number; upper bound operand is floating-point register number



Instruction format for FBNDIX.X(z:S/D) Fn(l), Fn(x), Fn(h)

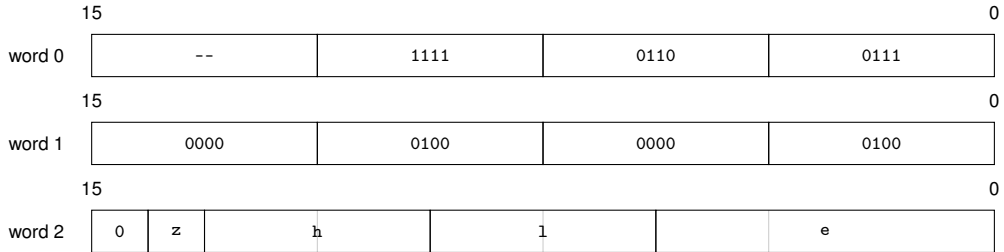
- l floating-point register number for the lower bound operand (payload word 1 bits 3:0).
 x floating-point register number for the value (payload word 1 bits 7:4).
 h floating-point register number for the upper bound operand (payload word 1 bits 11:8).
 z size selector for the operand size (payload word 1 bits 12).

FBNDXI**Floating-Point Bounds Check Exclusive-Inclusive****FBNDXI**

Summary:	Checks floating-point value membership in the exclusive-inclusive interval (lo, hi].
Operation:	ZF = 0; NF = 0; CF = 0; VF = 0; if (value_v <= lo_v) goto <fbndxi_bad>; if (value_v > hi_v) goto <fbndxi_bad>; goto <fbndxi_done>; <fbndxi_bad> VF = 1; <fbndxi_done>
Assembler Syntax:	FBNDXI.<S/D> Fn, <ea>, Fn FBNDXI.<S/D> Fn, Fn, Fn
Attributes:	Size = (Single, Double) Words = (3-8) Privilege = unprivileged Flags = V: set when the value is out of bounds or unordered
Description:	FBNDXI treats the low bound as exclusive and the high bound as inclusive. It sets V when value is unordered or outside (lo, hi]; Z, N, and C are unchanged.
Condition Codes:	Z N C V - - - * V: set when the value is out of bounds or unordered
Instruction Forms:	

FBNDXI.X(z:S/D) Fn(l), <ea(e)>, Fn(h)

Form Fn(l), <ea(e)>, Fn(h)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands lower bound operand is floating-point register number; value is compact effective-address field; upper bound operand is floating-point register number

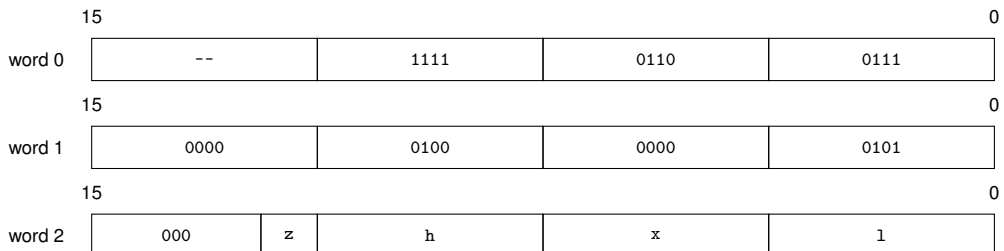


Instruction format for FBNDXI.X(z:S/D) Fn(l), <ea(e)>, Fn(h)

- e compact effective-address field for the value (payload word 1 bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 l floating-point register number for the lower bound operand (payload word 1 bits 9:6).
 h floating-point register number for the upper bound operand (payload word 1 bits 13:10).
 z size selector for the operand size (payload word 1 bits 14).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FBNDXI.X(z:S/D) Fn(l), Fn(x), Fn(h)

Form Fn(l), Fn(x), Fn(h)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands lower bound operand is floating-point register number; value is floating-point register number; upper bound operand is floating-point register number



Instruction format for FBNDXI.X(z:S/D) Fn(l), Fn(x), Fn(h)

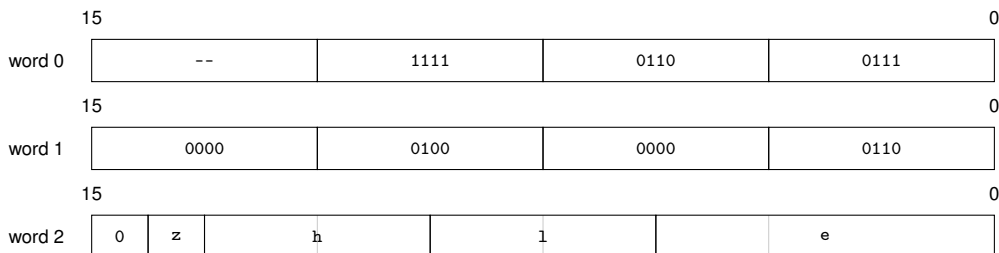
- l floating-point register number for the lower bound operand (payload word 1 bits 3:0).
 x floating-point register number for the value (payload word 1 bits 7:4).
 h floating-point register number for the upper bound operand (payload word 1 bits 11:8).
 z size selector for the operand size (payload word 1 bits 12).

FBNDXX**Floating-Point Bounds Check Exclusive-Exclusive****FBNDXX**

Summary:	Checks floating-point value membership in the exclusive-exclusive interval (lo, hi).
Operation:	ZF = 0; NF = 0; CF = 0; VF = 0; if (value_v <= lo_v) goto <fbndxx_bad>; if (value_v >= hi_v) goto <fbndxx_bad>; goto <fbndxx_done>; <fbndxx_bad> VF = 1; <fbndxx_done>
Assembler Syntax:	FBNDXX.<S/D> Fn, <ea>, Fn FBNDXX.<S/D> Fn, Fn, Fn
Attributes:	Size = (Single, Double) Words = (3-8) Privilege = unprivileged Flags = V: set when the value is out of bounds or unordered
Description:	FBNDXX treats both bounds as exclusive. It sets V when value is unordered or outside (lo, hi); Z, N, and C are unchanged.
Condition Codes:	Z N C V - - - * V: set when the value is out of bounds or unordered
Instruction Forms:	

FBNDXX.X(z:S/D) Fn(l), <ea(e)>, Fn(h)

Form Fn(l), <ea(e)>, Fn(h)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands lower bound operand is floating-point register number; value is compact effective-address field; upper bound operand is floating-point register number

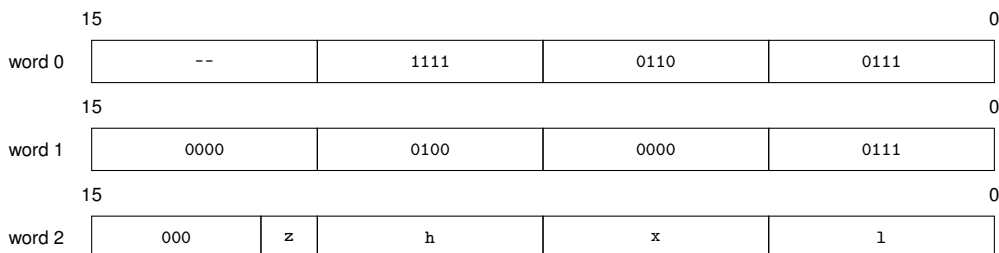


Instruction format for FBNDXX.X(z:S/D) Fn(l), <ea(e)>, Fn(h)

- e compact effective-address field for the value (payload word 1 bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 l floating-point register number for the lower bound operand (payload word 1 bits 9:6).
 h floating-point register number for the upper bound operand (payload word 1 bits 13:10).
 z size selector for the operand size (payload word 1 bits 14).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FBNDXX.X(z:S/D) Fn(l), Fn(x), Fn(h)

Form Fn(l), Fn(x), Fn(h)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands lower bound operand is floating-point register number; value is floating-point register number; upper bound operand is floating-point register number



Instruction format for FBNDXX.X(z:S/D) Fn(l), Fn(x), Fn(h)

- l floating-point register number for the lower bound operand (payload word 1 bits 3:0).
 x floating-point register number for the value (payload word 1 bits 7:4).
 h floating-point register number for the upper bound operand (payload word 1 bits 11:8).
 z size selector for the operand size (payload word 1 bits 12).

FCEIL

Floating-Point Ceiling

FCEIL

Summary:

Operation:

Assembler Syntax:

Attributes:

Floating-Point Status:

Performs the floating-point ceiling operation.

`dst_v = ceil(src_v);`

`FCEIL.<S/D> Fn, Fn`
`FCEIL.<S/D> <ea>, Fn`
`FCEIL.<S/D> Fn, <ea>`

Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

NV DZ OF UF NX
* - - - *

update when required: FFLAGS.NV (accrued invalid operation flag),
FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FCEIL.X(z:S/D) Fn(f), Fn(d)`

Form

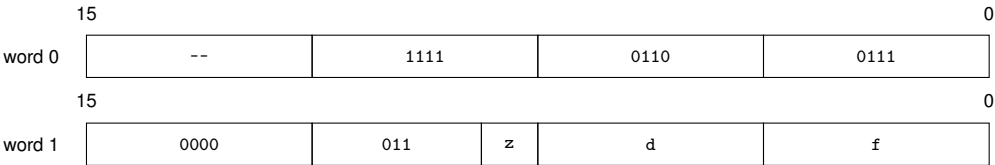
Encoding

Words

Privilege

Operands

`Fn(f), Fn(d)`
extended opcode form
2-8
unprivileged
source operand is floating-point register number; destination operand is floating-point register number

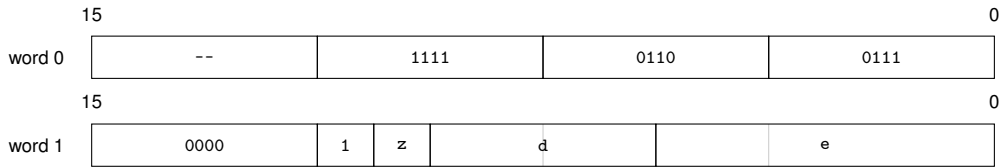


Instruction format for FCEIL.X(z:S/D) Fn(f), Fn(d)

- f
- d
- z
- floating-point register number for the source operand (extended opcode word bits 3:0).
- floating-point register number for the destination operand (extended opcode word bits 7:4).
- size selector for the operand size (extended opcode word bits 8).

FCEIL.X(z:S/D) <ea(e)>, Fn(d)

Form <ea(e)>, Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is floating-point register number

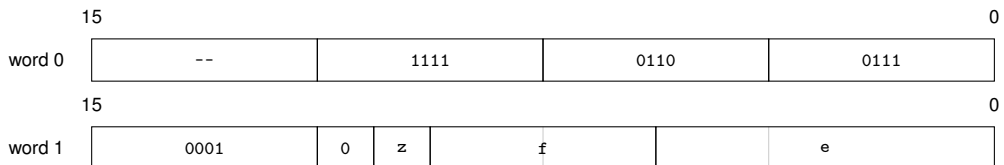


Instruction format for FCEIL.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
d floating-point register number for the destination operand (extended opcode word bits 9:6).
z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FCEIL.X(z:S/D) Fn(f), <ea(e)>

Form Fn(f), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is compact effective-address field



Instruction format for FCEIL.X(z:S/D) Fn(f), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
f floating-point register number for the source operand (extended opcode word bits 9:6).
z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FCLASS

Floating-Point Classify

FCLASS

Summary:

Operation:

Assembler Syntax:

Attributes:

Result Bitmap:

Performs the floating-point classify operation.

FCLASS classifies the source floating-point value according to the selected S or D format and writes a one-hot classification bitmap to the destination D register. The bitmap distinguishes negative infinity, negative normal, negative subnormal, negative zero, positive zero, positive subnormal, positive normal, positive infinity, signaling NaN, and quiet NaN.

FCLASS.<S/D> Fn, Dn

Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

FCLASS writes a 64-bit one-hot classification bitmap to the destination D register. Bit 0 is the least significant bit; bits 63..10 are written as zero.

Bit	Name	Meaning
0	NEG_INF	negative infinity
1	NEG_NORMAL	negative normalized finite value
2	NEG_SUBNORMAL	negative subnormal finite value
3	NEG_ZERO	negative zero
4	POS_ZERO	positive zero
5	POS_SUBNORMAL	positive subnormal finite value
6	POS_NORMAL	positive normalized finite value
7	POS_INF	positive infinity
8	SNAN	signaling NaN
9	QNAN	quiet NaN
63..10	0	reserved result bits, written as zero

Floating-Point Status:

NV DZ OF UF NX

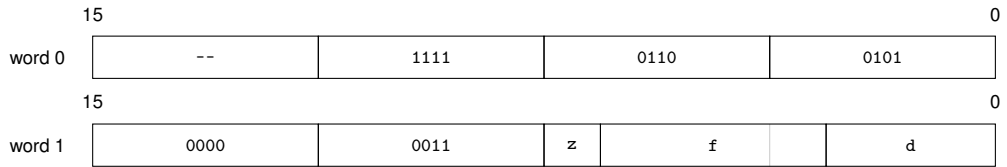
- - - - -

FFLAGS unchanged

Instruction Forms:

FCLASS.X(z:S/D) Fn(f), Dn(d)

Form	Fn(f), Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is data register number



Instruction format for FCLASS.X(z:S/D) Fn(f), Dn(d)

- d** data register number for the destination operand (extended opcode word bits 2:0).
- f** floating-point register number for the source operand (extended opcode word bits 6:3).
- z** size selector for the operand size (extended opcode word bits 7).

FCLR

Floating-Point Clear

FCLR

Summary:

Operation:

Assembler Syntax:

Attributes:

Floating-Point Status:

Performs the floating-point clear operation.

dst_v = 0;

FCLR Fn
FCLR <ea>

Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

NV DZ OF UF NX
- - - - -
FFLAGS unchanged

Instruction Forms:

FCLR Fn(d)

Form

Encoding

Words

Privilege

Operands

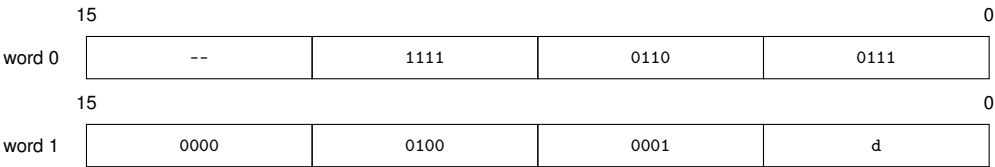
Fn(d)

extended opcode form

2-8

unprivileged

destination operand is floating-point register number



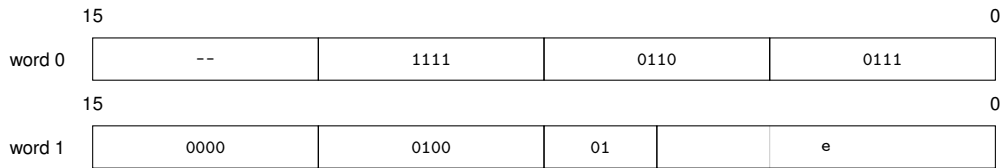
Instruction format for FCLR Fn(d)

d

floating-point register number for the destination operand (extended opcode word bits 3:0).

FCLR <ea(e)>

Form	<ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	destination operand is compact effective-address field

**Instruction format for FCLR <ea(e)>**

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FCMP

Floating-Point Compare

FCMP

Summary:

Operation:

Assembler Syntax:

Attributes:

Floating-Point Status:

NV DZ OF UF NX

* - - - -

update when required: FFLAGS.NV (accrued invalid operation flag)

Instruction Forms:

FCMP.X(z:S/D) Fn(f), Fn(d)

Form

Encoding

Words

Privilege

Operands

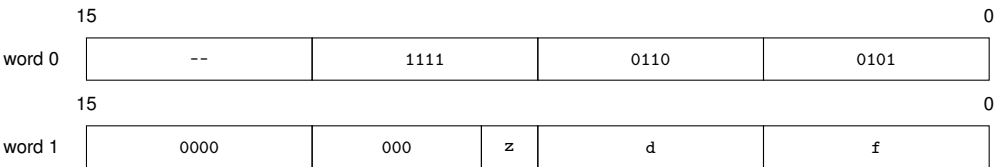
Fn(f), Fn(d)

extended opcode form

2-8

unprivileged

source operand is floating-point register number; destination operand is floating-point register number

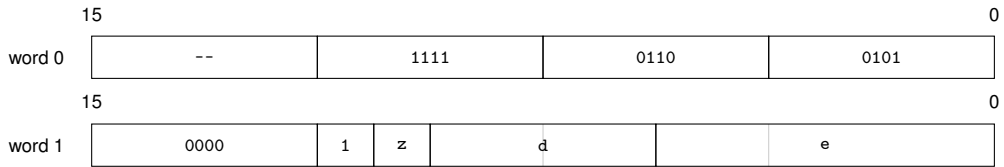


Instruction format for FCMP.X(z:S/D) Fn(f), Fn(d)

- f
- d
- z
- floating-point register number for the source operand (extended opcode word bits 3:0).
- floating-point register number for the destination operand (extended opcode word bits 7:4).
- size selector for the operand size (extended opcode word bits 8).

FCMP.X(z:S/D) <ea(e)>, Fn(d)

Form	<ea(e)>, Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is floating-point register number



Instruction format for FCMP.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FCOPYSIGN

Floating-Point Copy Sign

FCOPYSIGN

Summary:

Operation:

Assembler Syntax:

Attributes:

Floating-Point Status:

Performs the floating-point copy sign operation.

if (size_v != 4) goto <fcopysign_double>;
dst_v = (magnitude_src_v & 0x7fffffff) | (sign_src_v & 0x80000000);
goto <fcopysign_done>;
<fcopysign_double>
dst_v = (magnitude_src_v & 0x7fffffffffffffff) | (sign_src_v & 0x8000000000000000);
<fcopysign_done>

FCOPYSIGN.<S/D> Fn, Fn, Fn

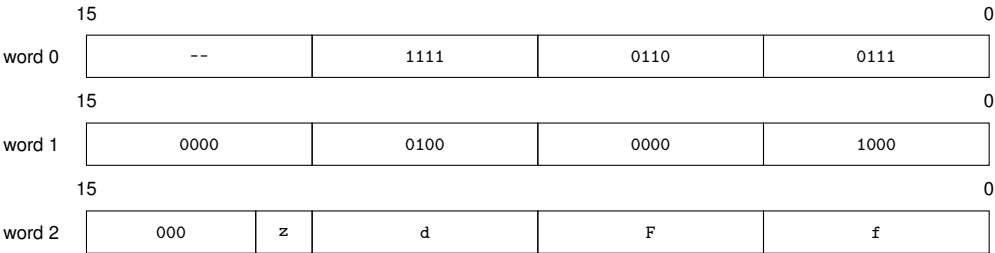
Size = (Single, Double)
Words = (3-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

NV DZ OF UF NX
- - - - -
FFLAGS unchanged

Instruction Forms:

FCOPYSIGN.X(z:S/D) Fn(f), Fn(F), Fn(d)

Form	Fn(f), Fn(F), Fn(d)
Encoding	extended opcode form
Words	3-8
Privilege	unprivileged
Operands	sign src is floating-point register number; magnitude src is floating-point register number; destination operand is floating-point register number



Instruction format for FCOPYSIGN.X(z:S/D) Fn(f), Fn(F), Fn(d)

- f floating-point register number for the sign src (payload word 1 bits 3:0).
- F floating-point register number for the magnitude src (payload word 1 bits 7:4).
- d floating-point register number for the destination operand (payload word 1 bits 11:8).
- z size selector for the operand size (payload word 1 bits 12).

FCVT**Floating-Point Convert Signed****FCVT**

Summary: Performs the floating-point convert signed operation.

Operation:

Assembler Syntax: FCVT Dn, Fn
 FCVT Fn, Dn
 FCVT Fn, Fn

Attributes: Words = (2-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

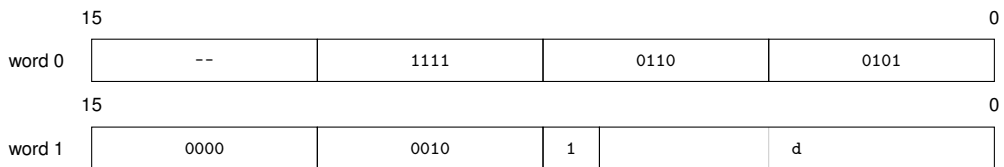
Floating-Point Status: **NV DZ OF UF NX**
 * _ * * *

update when required: FFLAGS.NV (accrued invalid operation flag),
 FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

FCVT Dn(d), Fn(d)

Form Dn(d), Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is floating-point register number

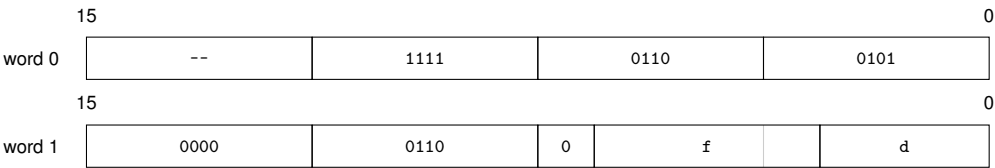


Instruction format for FCVT Dn(d), Fn(d)

- d data register number for the source operand (extended opcode word bits 2:0).
- d floating-point register number for the destination operand (extended opcode word bits 6:3).

FCVT Fn(f), Dn(d)

Form	Fn(f), Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is data register number

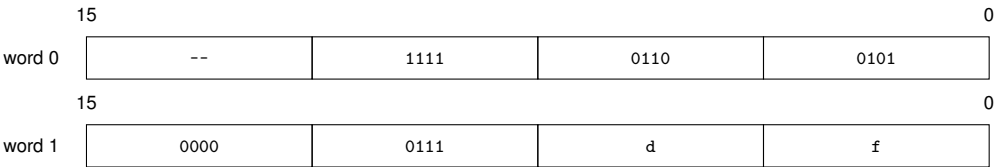


Instruction format for FCVT Fn(f), Dn(d)

- d data register number for the destination operand (extended opcode word bits 2:0).
- f floating-point register number for the source operand (extended opcode word bits 6:3).

FCVT Fn(f), Fn(d)

Form	Fn(f), Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FCVT Fn(f), Fn(d)

- f floating-point register number for the source operand (extended opcode word bits 3:0).
- d floating-point register number for the destination operand (extended opcode word bits 7:4).

FCVTU**Floating-Point Convert Unsigned****FCVTU**

Summary: Performs the floating-point convert unsigned operation.

Operation:

Assembler Syntax: FCVTU Dn, Fn
 FCVTU Fn, Dn
 FCVTU Fn, Fn

Attributes: Words = (2-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

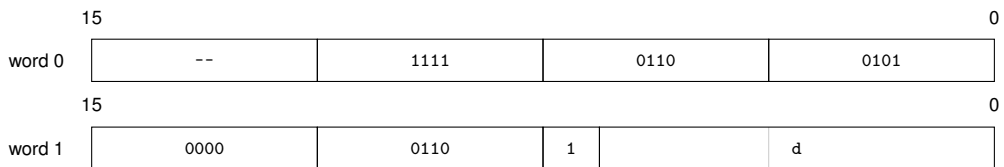
Floating-Point Status: **NV DZ OF UF NX**
 * _ * * *

update when required: FFLAGS.NV (accrued invalid operation flag),
 FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow
 flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

FCVTU Dn(d), Fn(d)

Form Dn(d), Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is data register number; destination operand is floating-point register number

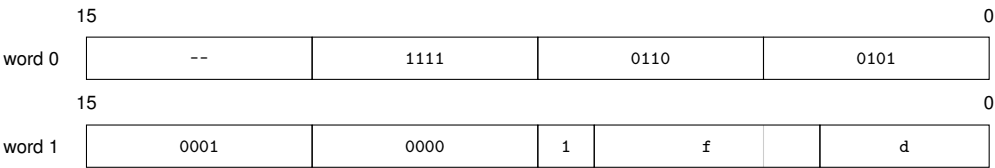


Instruction format for FCVTU Dn(d), Fn(d)

- d data register number for the source operand (extended opcode word bits 2:0).
- d floating-point register number for the destination operand (extended opcode word bits 6:3).

FCVTU Fn(f), Dn(d)

Form	Fn(f), Dn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is data register number

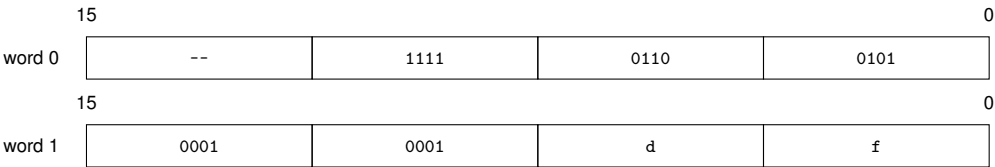


Instruction format for FCVTU Fn(f), Dn(d)

- d data register number for the destination operand (extended opcode word bits 2:0).
- f floating-point register number for the source operand (extended opcode word bits 6:3).

FCVTU Fn(f), Fn(d)

Form	Fn(f), Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FCVTU Fn(f), Fn(d)

- f floating-point register number for the source operand (extended opcode word bits 3:0).
- d floating-point register number for the destination operand (extended opcode word bits 7:4).

FDIV**Floating-Point Divide****FDIV**

Summary: Performs the floating-point divide operation.

Operation: $\text{dst_v} = \text{dst_v} \text{ f/ } \text{src_v};$

Assembler Syntax: `FDIV.<S/D> <ea>, Fn`
`FDIV.<S/D> Fn, Fn`

Attributes: Size = (Single, Double)
 Words = (2-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

Floating-Point Status: **NV DZ OF UF NX**
 * * * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.DZ (accrued divide-by-zero flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FDIV.X(z:S/D) <ea(e)>, Fn(d)`

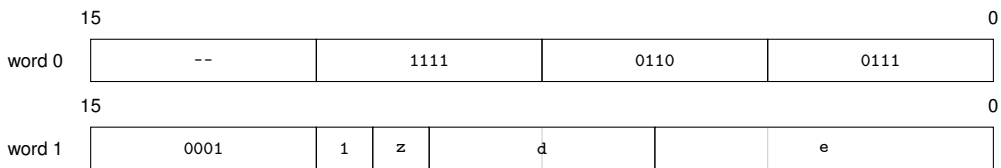
Form `<ea(e)>, Fn(d)`

Encoding extended opcode form

Words 2-8

Privilege unprivileged

Operands source operand is compact effective-address field; destination operand is floating-point register number

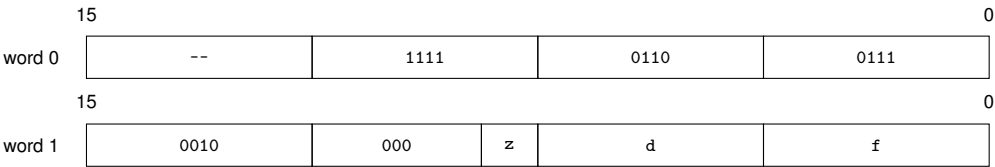


Instruction format for `FDIV.X(z:S/D) <ea(e)>, Fn(d)`

- e** compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d** floating-point register number for the destination operand (extended opcode word bits 9:6).
- z** size selector for the operand size (extended opcode word bits 10).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FDIV.X(z:S/D) Fn(f), Fn(d)

Form	Fn(f), Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FDIV.X(z:S/D) Fn(f), Fn(d)

- f floating-point register number for the source operand (extended opcode word bits 3:0).
- d floating-point register number for the destination operand (extended opcode word bits 7:4).
- z size selector for the operand size (extended opcode word bits 8).

FFLOOR**Floating-Point Floor****FFLOOR**

Summary: Performs the floating-point floor operation.

Operation: `dst_v = floor(src_v);`

Assembler Syntax: `FFLOOR.<S/D> Fn, Fn`
`FFLOOR.<S/D> <ea>, Fn`
`FFLOOR.<S/D> Fn, <ea>`

Attributes: Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

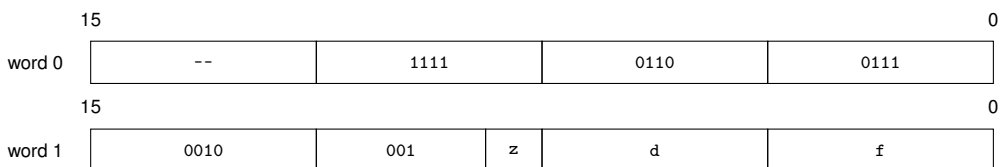
Floating-Point Status: **NV DZ OF UF NX**
* - - - *

update when required: FFLAGS.NV (accrued invalid operation flag),
FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FFLOOR.X(z:S/D) Fn(f), Fn(d)`

Form `Fn(f), Fn(d)`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number

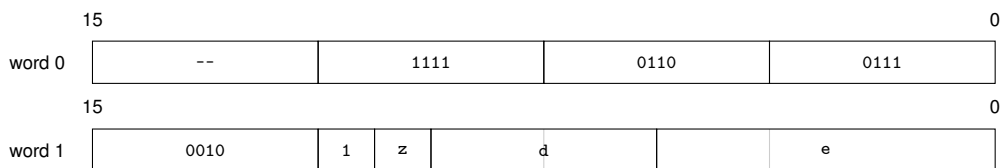


Instruction format for FFLOOR.X(z:S/D) Fn(f), Fn(d)

f floating-point register number for the source operand (extended opcode word bits 3:0).
d floating-point register number for the destination operand (extended opcode word bits 7:4).
z size selector for the operand size (extended opcode word bits 8).

FFLOOR.X(z:S/D) <ea(e)>, Fn(d)

Form	<ea(e)>, Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is floating-point register number

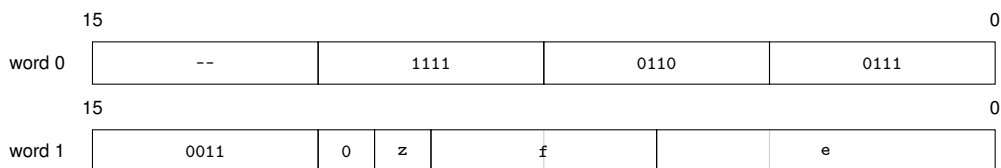


Instruction format for FFLOOR.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FFLOOR.X(z:S/D) Fn(f), <ea(e)>

Form	Fn(f), <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is compact effective-address field



Instruction format for FFLOOR.X(z:S/D) Fn(f), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- f floating-point register number for the source operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FGETEXP**Floating-Point Get Exponent****FGETEXP**

Summary: Performs the floating-point get exponent operation.

Operation:

```

tmp_v = src_v;
if (size_v != 4) goto <fgetexp_double>;
result_v = (tmp_v >> 23) & 0xff;
if (result_v != 0) goto <fgetexp_single_biased>;
result_v = 1;
<fgetexp_single_biased>
result_v = result_v - 127;
dst_v = int2float(result_v);
goto <fgetexp_done>;
<fgetexp_double>
result_v = (tmp_v >> 52) & 0x7ff;
if (result_v != 0) goto <fgetexp_double_biased>;
result_v = 1;
<fgetexp_double_biased>
result_v = result_v - 1023;
dst_v = int2float(result_v);
<fgetexp_done>

```

Assembler Syntax: FGETEXP.<S/D> Fn, Fn
 FGETEXP.<S/D> <ea>, Fn

Attributes: Size = (Single, Double)
 Words = (2-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

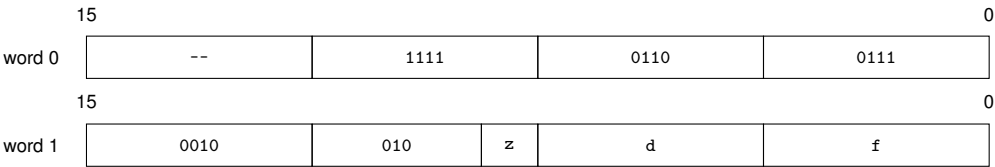
Floating-Point Status: **NV DZ OF UF NX**
 * - - - -

update when required: FFLAGS.NV (accrued invalid operation flag)

Instruction Forms:

FGETEXP.X(z:S/D) Fn(f), Fn(d)

Form	Fn(f), Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is floating-point register number

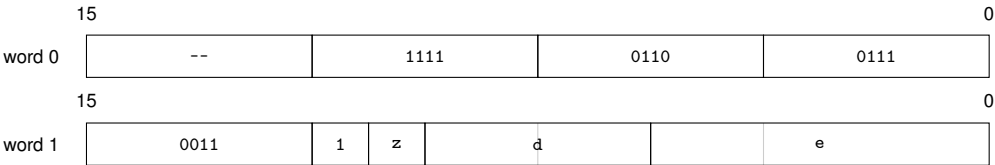


Instruction format for FGETEXP.X(z:S/D) Fn(f), Fn(d)

- f floating-point register number for the source operand (extended opcode word bits 3:0).
- d floating-point register number for the destination operand (extended opcode word bits 7:4).
- z size selector for the operand size (extended opcode word bits 8).

FGETEXP.X(z:S/D) <ea(e)>, Fn(d)

Form	<ea(e)>, Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is floating-point register number



Instruction format for FGETEXP.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FGETMAN**Floating-Point Get Mantissa****FGETMAN**

Summary: Performs the floating-point get mantissa operation.

Operation:

```

tmp_v = src_v;
if (size_v != 4) goto <fgetman_double>;
result_v = (tmp_v >> 23) & 0xff;
borrow_v = tmp_v & 0x7ffff;
if (result_v == 0) goto <fgetman_done>;
if (result_v == 0xff) goto <fgetman_done>;
carry_v = (tmp_v >> 31) << 31;
dst_v = carry_v | 0x3f800000;
dst_v = dst_v | borrow_v;
goto <fgetman_done>;
<fgetman_double>
result_v = (tmp_v >> 52) & 0x7ff;
borrow_v = tmp_v & 0xffffffffffff;
if (result_v == 0) goto <fgetman_done>;
if (result_v == 0x7ff) goto <fgetman_done>;
carry_v = (tmp_v >> 63) << 63;
dst_v = carry_v | 0x3ff0000000000000;
dst_v = dst_v | borrow_v;
<fgetman_done>

```

Assembler Syntax: FGETMAN.<S/D> Fn, Fn
 FGETMAN.<S/D> <ea>, Fn

Attributes: Size = (Single, Double)
 Words = (2-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

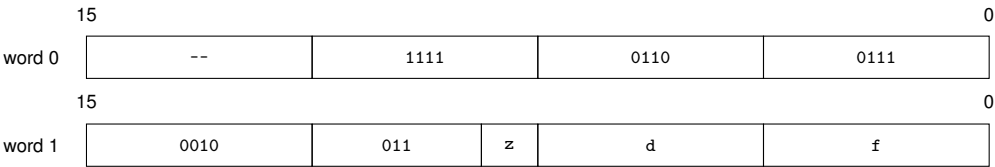
Floating-Point Status: **NV DZ OF UF NX**
 * - - - *

update when required: FFLAGS.NV (accrued invalid operation flag),
 FFLAGS.NX (accrued inexact flag)

Instruction Forms:

FGETMAN.X(z:S/D) Fn(f), Fn(d)

Form	Fn(f), Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is floating-point register number

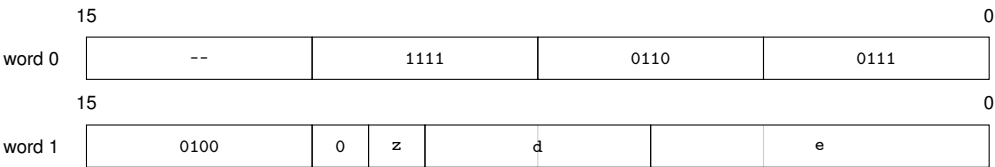


Instruction format for FGETMAN.X(z:S/D) Fn(f), Fn(d)

- f floating-point register number for the source operand (extended opcode word bits 3:0).
- d floating-point register number for the destination operand (extended opcode word bits 7:4).
- z size selector for the operand size (extended opcode word bits 8).

FGETMAN.X(z:S/D) <ea(e)>, Fn(d)

Form	<ea(e)>, Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is floating-point register number



Instruction format for FGETMAN.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FINT**Floating-Point Integral Value****FINT**

Summary: Performs the floating-point integral value operation.

Operation: `dst_v = round(src_v);`

Assembler Syntax: `FINT.<S/D> <ea>, Fn`
`FINT.<S/D> Fn, <ea>`
`FINT.<S/D> Fn, Fn`

Attributes: Size = (Single, Double)
 Words = (2-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

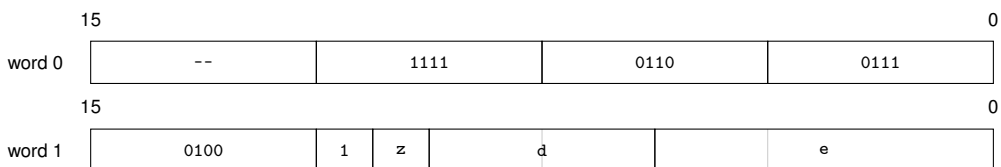
Floating-Point Status: **NV DZ OF UF NX**
 * - - - *

update when required: FFLAGS.NV (accrued invalid operation flag),
 FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FINT.X(z:S/D) <ea(e)>, Fn(d)`

Form `<ea(e)>, Fn(d)`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is floating-point register number

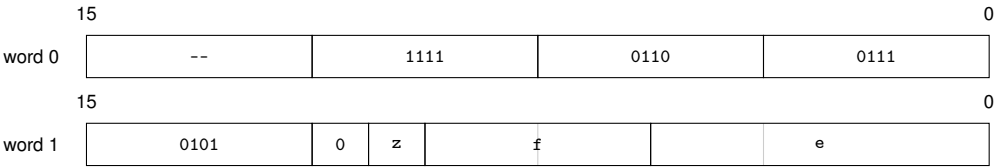


Instruction format for FINT.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FINT.X(z:S/D) Fn(f), <ea(e)>

Form	Fn(f), <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is compact effective-address field

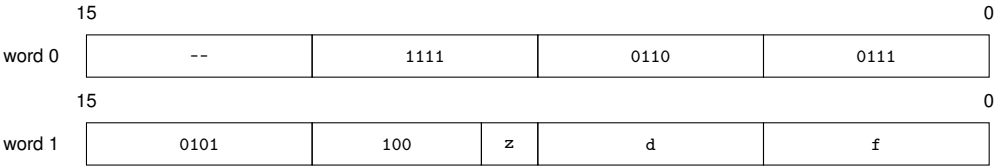


Instruction format for FINT.X(z:S/D) Fn(f), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- f floating-point register number for the source operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FINT.X(z:S/D) Fn(f), Fn(d)

Form	Fn(f), Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FINT.X(z:S/D) Fn(f), Fn(d)

- f floating-point register number for the source operand (extended opcode word bits 3:0).
- d floating-point register number for the destination operand (extended opcode word bits 7:4).
- z size selector for the operand size (extended opcode word bits 8).

FINTRZ**Floating-Point Integral Toward Zero****FINTRZ**

Summary: Performs the floating-point integral toward zero operation.

Operation: `local integral_v:8 = trunc(src_v);`
`dst_v = int2float(integral_v);`

Assembler Syntax: `FINTRZ.<S/D> Fn, Fn`
`FINTRZ.<S/D> <ea>, Fn`
`FINTRZ.<S/D> Fn, <ea>`

Attributes: Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

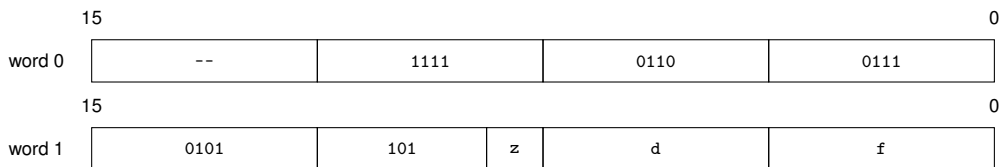
Floating-Point Status: **NV DZ OF UF NX**
* - - - *

update when required: FFLAGS.NV (accrued invalid operation flag),
FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FINTRZ.X(z:S/D) Fn(f), Fn(d)`

Form `Fn(f), Fn(d)`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number

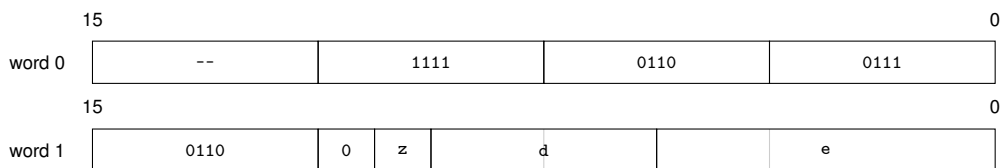


Instruction format for FINTRZ.X(z:S/D) Fn(f), Fn(d)

f floating-point register number for the source operand (extended opcode word bits 3:0).
d floating-point register number for the destination operand (extended opcode word bits 7:4).
z size selector for the operand size (extended opcode word bits 8).

FINTRZ.X(z:S/D) <ea(e)>, Fn(d)

Form <ea(e)>, Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is floating-point register number

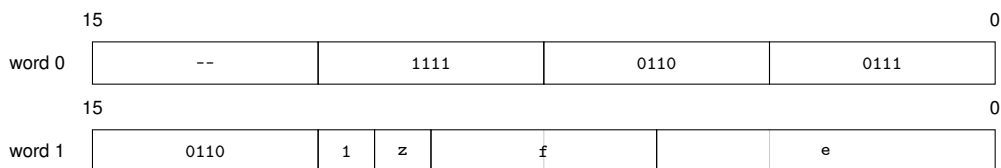


Instruction format for FINTRZ.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FINTRZ.X(z:S/D) Fn(f), <ea(e)>

Form Fn(f), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is compact effective-address field



Instruction format for FINTRZ.X(z:S/D) Fn(f), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- f floating-point register number for the source operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FMADD**Floating-Point Fused Multiply Add****FMADD**

Summary: Performs the floating-point fused multiply add operation.

Operation: $\text{dst_v} = (\text{lhs_v} \times \text{rhs_v}) + \text{dst_v};$

Assembler Syntax: `FMADD.<S/D> <ea>, Fn, Fn`
`FMADD.<S/D> Fn, <ea>, Fn`
`FMADD.<S/D> Fn, Fn, Fn`

Attributes: Size = (Single, Double)
 Words = (3-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

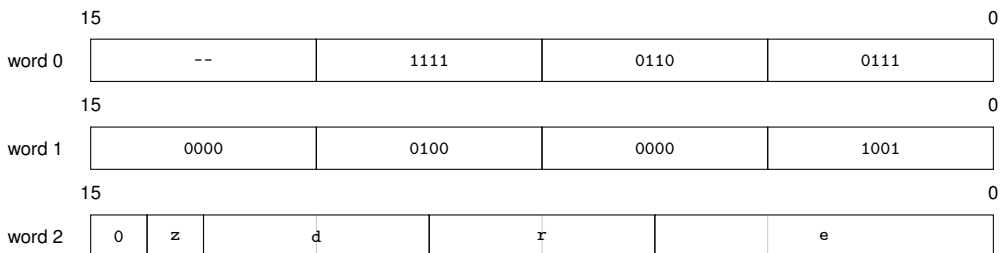
Floating-Point Status: **NV DZ OF UF NX**
 * _ * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FMADD.X(z:S/D) <ea(e)>, Fn(r), Fn(d)`

Form `<ea(e)>, Fn(r), Fn(d)`
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands left operand is compact effective-address field; right operand is floating-point register number; destination operand is floating-point register number

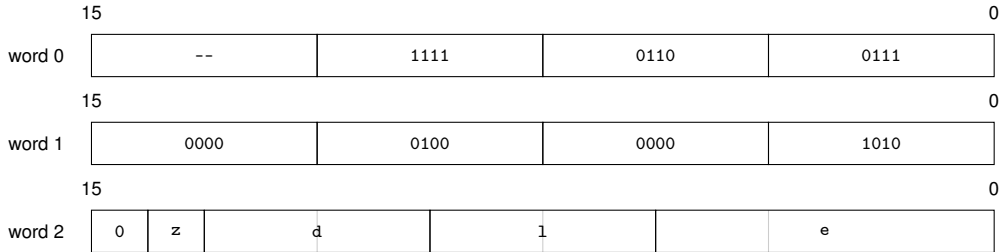


Instruction format for FMADD.X(z:S/D) <ea(e)>, Fn(r), Fn(d)

- e compact effective-address field for the left operand (payload word 1 bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - r floating-point register number for the right operand (payload word 1 bits 9:6).
 - d floating-point register number for the destination operand (payload word 1 bits 13:10).
 - z size selector for the operand size (payload word 1 bits 14).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FMADD.X(z:S/D) Fn(l), <ea(e)>, Fn(d)

Form Fn(l), <ea(e)>, Fn(d)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands left operand is floating-point register number; right operand is compact effective-address field; destination operand is floating-point register number

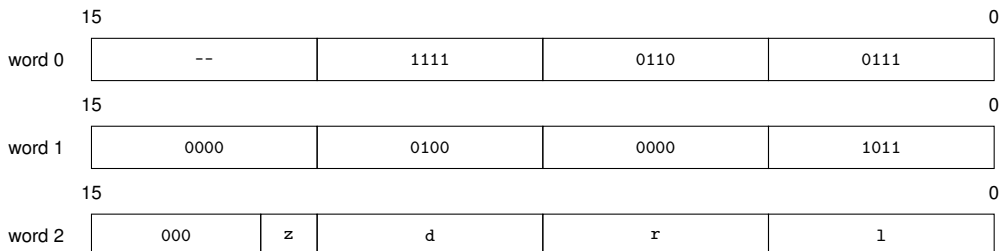


Instruction format for FMADD.X(z:S/D) Fn(l), <ea(e)>, Fn(d)

- e compact effective-address field for the right operand (payload word 1 bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 l floating-point register number for the left operand (payload word 1 bits 9:6).
 d floating-point register number for the destination operand (payload word 1 bits 13:10).
 z size selector for the operand size (payload word 1 bits 14).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FMADD.X(z:S/D) Fn(l), Fn(r), Fn(d)

Form Fn(l), Fn(r), Fn(d)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands left operand is floating-point register number; right operand is floating-point register number; destination operand is floating-point register number



Instruction format for FMADD.X(z:S/D) Fn(l), Fn(r), Fn(d)

- l floating-point register number for the left operand (payload word 1 bits 3:0).
 r floating-point register number for the right operand (payload word 1 bits 7:4).
 d floating-point register number for the destination operand (payload word 1 bits 11:8).
 z size selector for the operand size (payload word 1 bits 12).

FMAX**Floating-Point Maximum****FMAX**

Summary: Performs the floating-point maximum operation.

Operation: if (dst_v f> src_v) goto <fmax_done>;
dst_v = src_v;
<fmax_done>

Assembler Syntax: FMAX.<S/D> Fn, Fn
FMAX.<S/D> <ea>, Fn

Attributes: Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

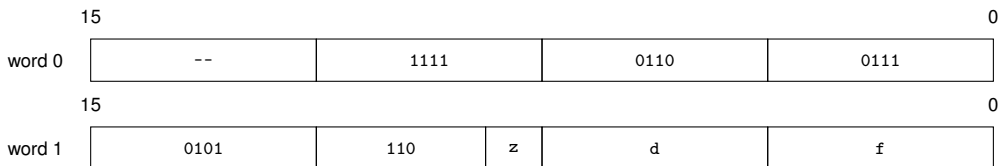
Floating-Point Status: **NV DZ OF UF NX**
* - - - -

update when required: FFLAGS.NV (accrued invalid operation flag)

Instruction Forms:

FMAX.X(z:S/D) Fn(f), Fn(d)

Form Fn(f), Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number

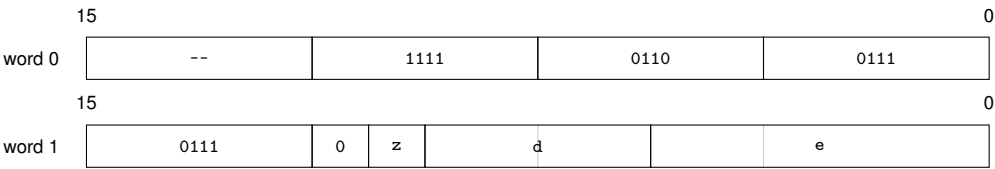


Instruction format for FMAX.X(z:S/D) Fn(f), Fn(d)

f floating-point register number for the source operand (extended opcode word bits 3:0).
d floating-point register number for the destination operand (extended opcode word bits 7:4).
z size selector for the operand size (extended opcode word bits 8).

FMAX.X(z:S/D) <ea(e)>, Fn(d)

Form	<ea(e)>, Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is floating-point register number



Instruction format for FMAX.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FMIN**Floating-Point Minimum****FMIN**

Summary: Performs the floating-point minimum operation.

Operation: if (dst_v f< src_v) goto <fmin_done>;
dst_v = src_v;
<fmin_done>

Assembler Syntax: FMIN.<S/D> Fn, Fn
FMIN.<S/D> <ea>, Fn

Attributes: Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

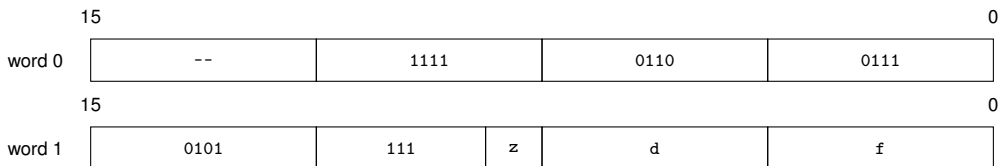
Floating-Point Status: **NV DZ OF UF NX**
* - - - -

update when required: FFLAGS.NV (accrued invalid operation flag)

Instruction Forms:

FMIN.X(z:S/D) Fn(f), Fn(d)

Form Fn(f), Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number

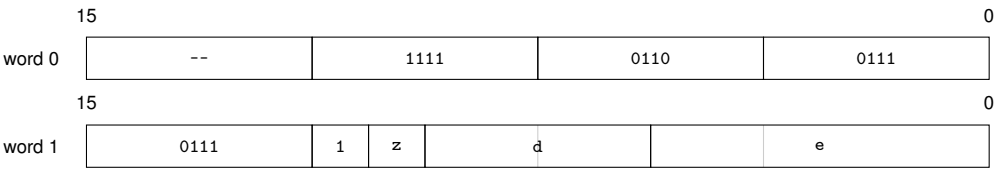


Instruction format for FMIN.X(z:S/D) Fn(f), Fn(d)

f floating-point register number for the source operand (extended opcode word bits 3:0).
d floating-point register number for the destination operand (extended opcode word bits 7:4).
z size selector for the operand size (extended opcode word bits 8).

FMIN.X(z:S/D) <ea(e)>, Fn(d)

Form	<ea(e)>, Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is floating-point register number



Instruction format for FMIN.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FMOD**Floating-Point Modulo****FMOD**

Summary: Performs the floating-point modulo operation.

Operation: `dst_v = fp_mod(dst_v, src_v);`

Assembler Syntax: `FMOD.<S/D> <ea>, Fn`
`FMOD.<S/D> Fn, Fn`

Attributes: Size = (Single, Double)
 Words = (2-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

Floating-Point Status: **NV DZ OF UF NX**
 * - - - -

update when required: FFLAGS.NV (accrued invalid operation flag)

Instruction Forms:

`FMOD.X(z:S/D) <ea(e)>, Fn(d)`

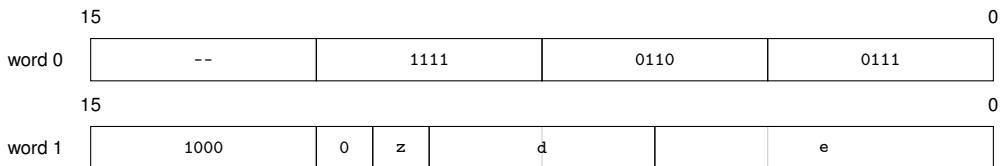
Form `<ea(e)>, Fn(d)`

Encoding extended opcode form

Words 2-8

Privilege unprivileged

Operands source operand is compact effective-address field; destination operand is floating-point register number

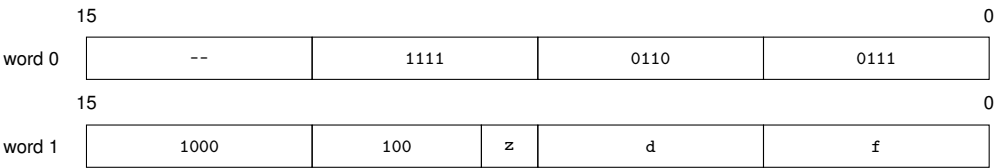


Instruction format for FMOD.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FMOD.X(z:S/D) Fn(f) , Fn(d)

Form	Fn(f), Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FMOD.X(z:S/D) Fn(f), Fn(d)

- f floating-point register number for the source operand (extended opcode word bits 3:0).
- d floating-point register number for the destination operand (extended opcode word bits 7:4).
- z size selector for the operand size (extended opcode word bits 8).

FMOV**Floating-Point Move****FMOV**

Summary: Copies the source operand to the destination operand.

Operation: `dst_v = src_v;`

Assembler Syntax: `FMOV.<S/D> Fn, Fn`
`FMOV.<S/D> <ea>, Fn`
`FMOV.<S/D> Fn, <ea>`

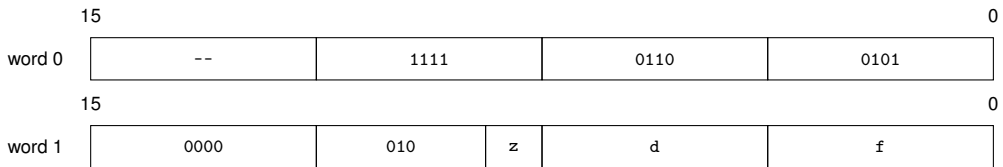
Attributes: Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

`FMOV.X(z:S/D) Fn(f), Fn(d)`

Form `Fn(f), Fn(d)`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number

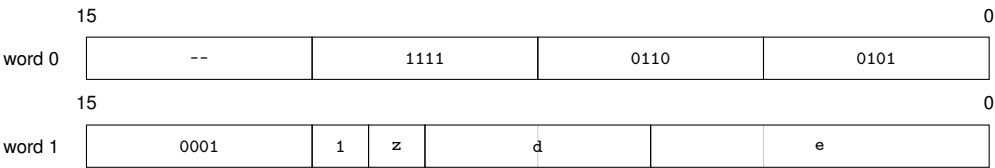


Instruction format for FMOV.X(z:S/D) Fn(f), Fn(d)

f floating-point register number for the source operand (extended opcode word bits 3:0).
d floating-point register number for the destination operand (extended opcode word bits 7:4).
z size selector for the operand size (extended opcode word bits 8).

FMOV.X(z:S/D) <ea(e)>, Fn(d)

Form	<ea(e)>, Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is floating-point register number

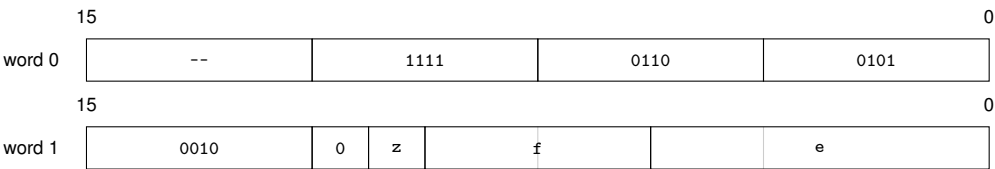


Instruction format for FMOV.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FMOV.X(z:S/D) Fn(f), <ea(e)>

Form	Fn(f), <ea(e)>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is compact effective-address field



Instruction format for FMOV.X(z:S/D) Fn(f), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- f floating-point register number for the source operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FMOVCR**Floating-Point Move Constant****FMOVCR**

Summary: Performs the floating-point move constant operation.

Operation: $\text{dst_v} = \text{fp_const}(\text{constant_v} \ \& \ 0\text{xffff})$;

Assembler Syntax: `FMOVCR <imm16>, Fn`

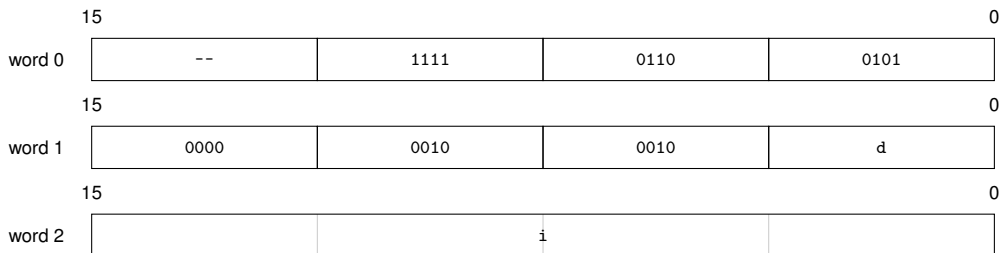
Attributes:
 Words = (3-8)
 Privilege = unprivileged
 Flags = unchanged

Condition Codes: **Z N C V**
 - - - -
 unchanged

Instruction Forms:

`FMOVCR imm16, Fn(d)`

Form imm16, Fn(d)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands constant id is 16-bit immediate selector; destination operand is floating-point register number



Instruction format for FMOVCR imm16, Fn(d)

- d floating-point register number for the destination operand (extended opcode word bits 3:0).
 i imm16 for the constant id (payload word 1 bits 15:0).

FMOVcc

Floating-Point Conditional Move

FMOVcc

Summary: Performs the floating-point conditional move operation.

Operation: if (cc_v == 0) goto <fmovcc_done>;
dst_v = src_v;
<fmovcc_done>

Assembler Syntax: FMOVcc Fn, Fn
FMOVcc.<S/D> <ea>, Fn
FMOVcc.<S/D> Fn, <ea>

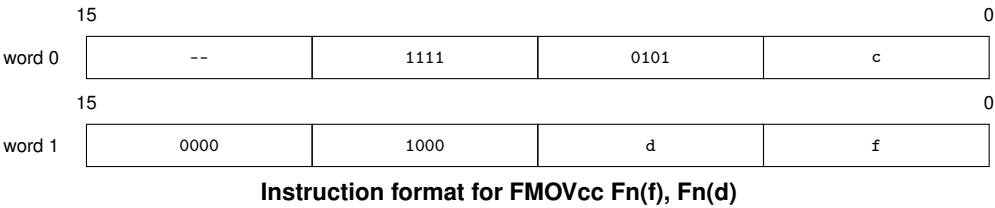
Attributes: Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
Flags = unchanged

Condition Codes: **Z N C V**
- - - -
unchanged

Instruction Forms:

FMOVcc Fn(f), Fn(d)

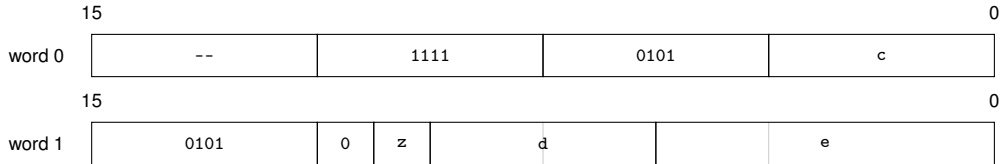
Form	Fn(f), Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	condition code is condition-code selector; source operand is floating-point register number; destination operand is floating-point register number



f floating-point register number for the source operand (extended opcode word bits 3:0).
d floating-point register number for the destination operand (extended opcode word bits 7:4).

FMOVcc.X(z:S/D) <ea(e)>, Fn(d)

Form <ea(e)>, Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands condition code is condition-code selector; source operand is compact effective-address field; destination operand is floating-point register number

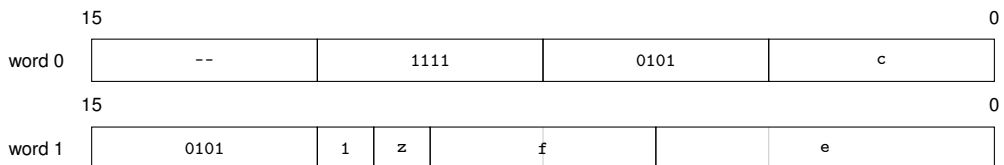


Instruction format for FMOVcc.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
d floating-point register number for the destination operand (extended opcode word bits 9:6).
z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FMOVcc.X(z:S/D) Fn(f), <ea(e)>

Form Fn(f), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands condition code is condition-code selector; source operand is floating-point register number; destination operand is compact effective-address field



Instruction format for FMOVcc.X(z:S/D) Fn(f), <ea(e)>

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
f floating-point register number for the source operand (extended opcode word bits 9:6).
z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FMSUB

Floating-Point Fused Multiply Subtract

FMSUB

Summary: Performs the floating-point fused multiply subtract operation.

Operation: `dst_v = dst_v f- (lhs_v f* rhs_v);`

Assembler Syntax: `FMSUB.<S/D> <ea>, Fn, Fn`
`FMSUB.<S/D> Fn, <ea>, Fn`
`FMSUB.<S/D> Fn, Fn, Fn`

Attributes: Size = (Single, Double)
Words = (3-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

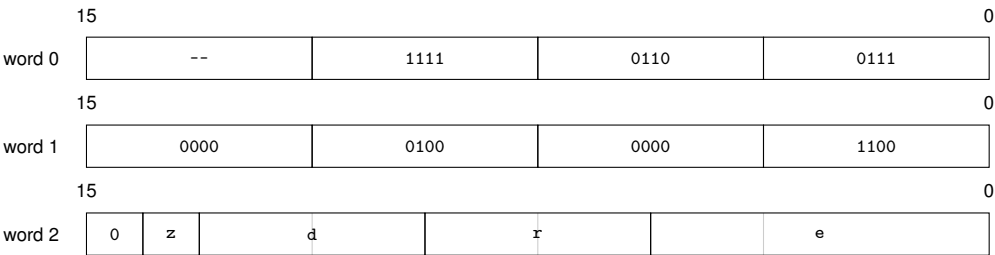
Floating-Point Status: **NV DZ OF UF NX**
* _ * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FMSUB.X(z:S/D) <ea(e)>, Fn(r), Fn(d)`

Form `<ea(e)>, Fn(r), Fn(d)`
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands left operand is compact effective-address field; right operand is floating-point register number; destination operand is floating-point register number

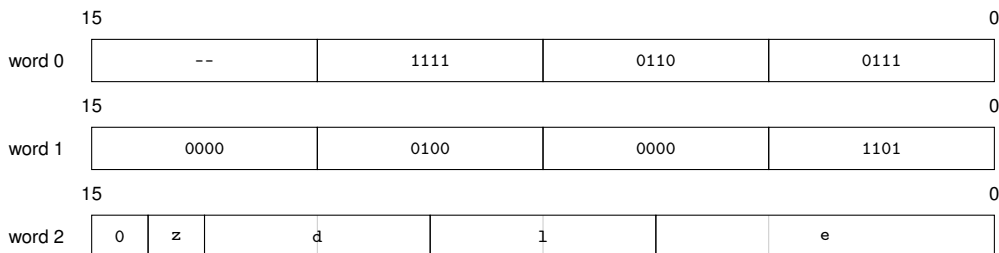


Instruction format for `FMSUB.X(z:S/D) <ea(e)>, Fn(r), Fn(d)`

- `e` compact effective-address field for the left operand (payload word 1 bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - `r` floating-point register number for the right operand (payload word 1 bits 9:6).
 - `d` floating-point register number for the destination operand (payload word 1 bits 13:10).
 - `z` size selector for the operand size (payload word 1 bits 14).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FMSUB.X(z:S/D) Fn(l), <ea(e)>, Fn(d)

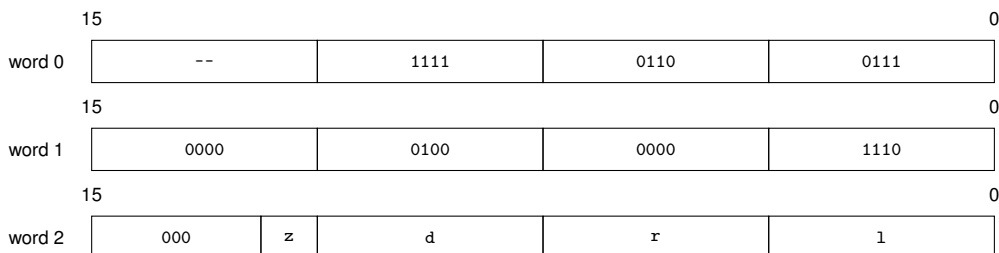
Form Fn(l), <ea(e)>, Fn(d)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands left operand is floating-point register number; right operand is compact effective-address field; destination operand is floating-point register number

**Instruction format for FMSUB.X(z:S/D) Fn(l), <ea(e)>, Fn(d)**

- e compact effective-address field for the right operand (payload word 1 bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 l floating-point register number for the left operand (payload word 1 bits 9:6).
 d floating-point register number for the destination operand (payload word 1 bits 13:10).
 z size selector for the operand size (payload word 1 bits 14).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FMSUB.X(z:S/D) Fn(l), Fn(r), Fn(d)

Form Fn(l), Fn(r), Fn(d)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands left operand is floating-point register number; right operand is floating-point register number; destination operand is floating-point register number

**Instruction format for FMSUB.X(z:S/D) Fn(l), Fn(r), Fn(d)**

- l floating-point register number for the left operand (payload word 1 bits 3:0).
 r floating-point register number for the right operand (payload word 1 bits 7:4).
 d floating-point register number for the destination operand (payload word 1 bits 11:8).
 z size selector for the operand size (payload word 1 bits 12).

FMUL

Floating-Point Multiply

FMUL

Summary: Performs the floating-point multiply operation.

Operation: `dst_v = dst_v f* src_v;`

Assembler Syntax: `FMUL.<S/D> Fn, Fn`
`FMUL.<S/D> <ea>, Fn`

Attributes: Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

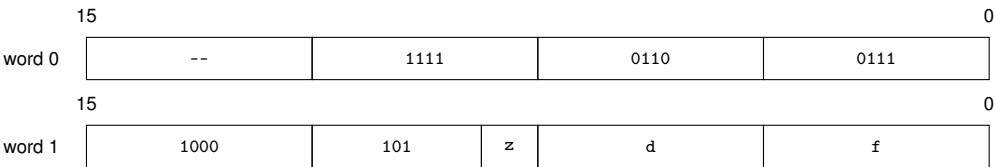
Floating-Point Status: **NV DZ OF UF NX**
* _ * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FMUL.X(z:S/D) Fn(f), Fn(d)`

Form	<code>Fn(f), Fn(d)</code>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is floating-point register number

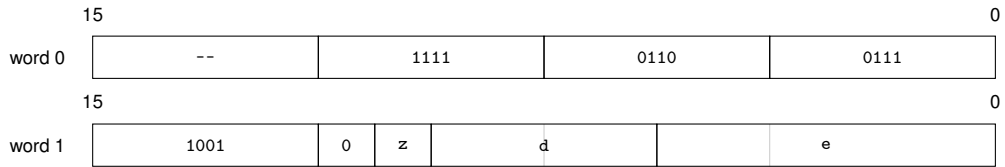


Instruction format for `FMUL.X(z:S/D) Fn(f), Fn(d)`

- `f` floating-point register number for the source operand (extended opcode word bits 3:0).
- `d` floating-point register number for the destination operand (extended opcode word bits 7:4).
- `z` size selector for the operand size (extended opcode word bits 8).

FMUL.X(z:S/D) <ea(e)>, Fn(d)

Form	<ea(e)>, Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is floating-point register number



Instruction format for FMUL.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FNEG

Floating-Point Negate

FNEG

Summary: Performs the floating-point negate operation.

Operation: dst_v = f- src_v;

Assembler Syntax: FNEG.<S/D> Fn, Fn
FNEG.<S/D> <ea>, Fn
FNEG.<S/D> Fn, <ea>

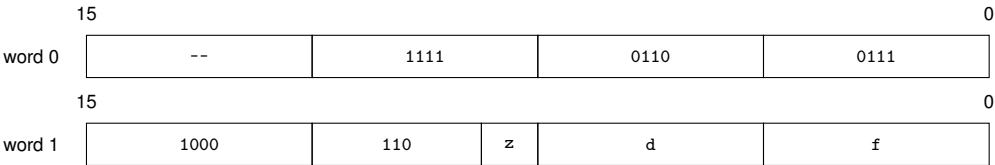
Attributes: Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

Floating-Point Status: NV DZ OF UF NX
- - - - -
FFLAGS unchanged

Instruction Forms:

FNEG.X(z:S/D) Fn(f), Fn(d)

Form Fn(f), Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number

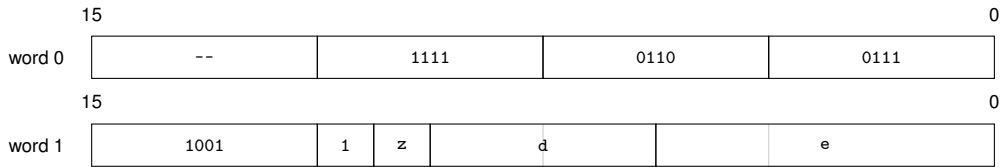


Instruction format for FNEG.X(z:S/D) Fn(f), Fn(d)

- f floating-point register number for the source operand (extended opcode word bits 3:0).
- d floating-point register number for the destination operand (extended opcode word bits 7:4).
- z size selector for the operand size (extended opcode word bits 8).

FNEG.X(z:S/D) <ea(e)>, Fn(d)

Form <ea(e)>, Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is floating-point register number

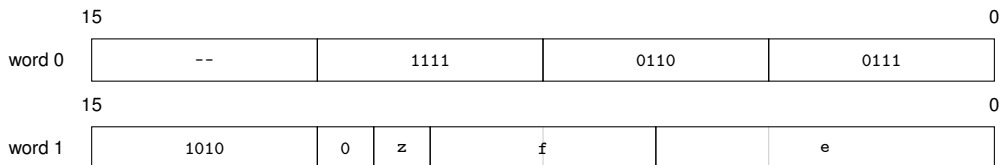


Instruction format for FNEG.X(z:S/D) <ea(e)>, Fn(d)

- e** compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
d floating-point register number for the destination operand (extended opcode word bits 9:6).
z size selector for the operand size (extended opcode word bits 10).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FNEG.X(z:S/D) Fn(f), <ea(e)>

Form Fn(f), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is compact effective-address field



Instruction format for FNEG.X(z:S/D) Fn(f), <ea(e)>

- e** compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
f floating-point register number for the source operand (extended opcode word bits 9:6).
z size selector for the operand size (extended opcode word bits 10).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FNMADD

Floating-Point Fused Negative Multiply Add

FNMADD

Summary: Performs the floating-point fused negative multiply add operation.

Operation: $\text{dst_v} = (\text{lhs_v} \text{ f* } \text{rhs_v}) \text{ f+ } \text{dst_v};$
 $\text{dst_v} = \text{f- } \text{dst_v};$

Assembler Syntax: `FNMADD.<S/D> <ea>, Fn, Fn`
`FNMADD.<S/D> Fn, <ea>, Fn`
`FNMADD.<S/D> Fn, Fn, Fn`

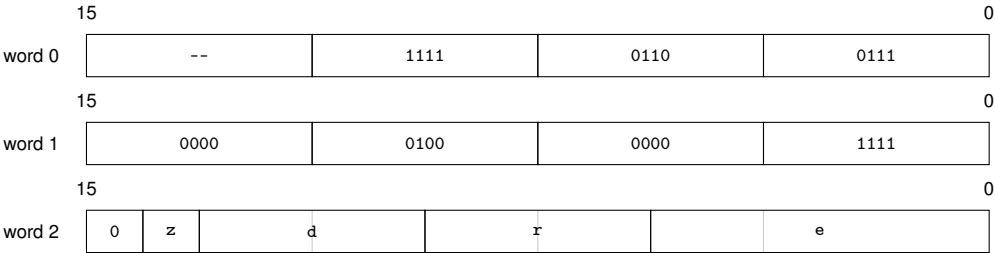
Attributes: Size = (Single, Double)
Words = (3-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

Floating-Point Status: **NV DZ OF UF NX**
* _ * * *
update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FNMADD.X(z:S/D) <ea(e)>, Fn(r), Fn(d)`

Form `<ea(e)>, Fn(r), Fn(d)`
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands left operand is compact effective-address field; right operand is floating-point register number; destination operand is floating-point register number

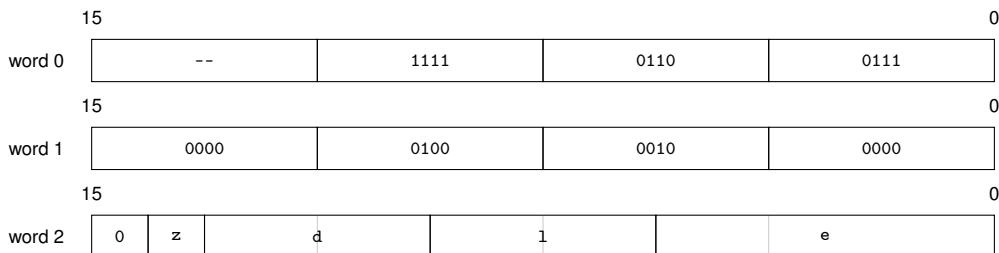


Instruction format for FNMADD.X(z:S/D) <ea(e)>, Fn(r), Fn(d)

- e compact effective-address field for the left operand (payload word 1 bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - r floating-point register number for the right operand (payload word 1 bits 9:6).
 - d floating-point register number for the destination operand (payload word 1 bits 13:10).
 - z size selector for the operand size (payload word 1 bits 14).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FNMAADD.X(z:S/D) Fn(l), <ea(e)>, Fn(d)

Form Fn(l), <ea(e)>, Fn(d)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands left operand is floating-point register number; right operand is compact effective-address field; destination operand is floating-point register number

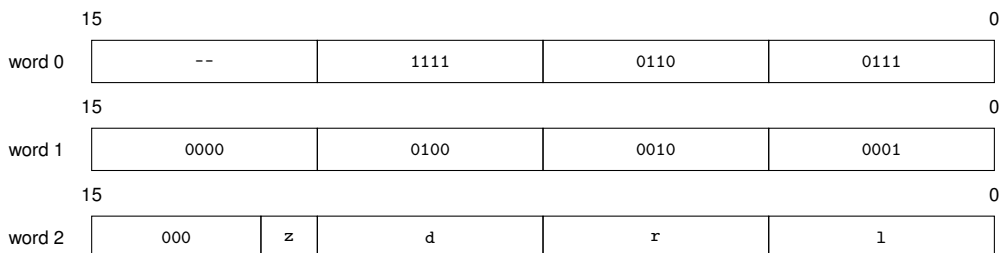


Instruction format for FNMAADD.X(z:S/D) Fn(l), <ea(e)>, Fn(d)

- e** compact effective-address field for the right operand (payload word 1 bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
l floating-point register number for the left operand (payload word 1 bits 9:6).
d floating-point register number for the destination operand (payload word 1 bits 13:10).
z size selector for the operand size (payload word 1 bits 14).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FNMAADD.X(z:S/D) Fn(l), Fn(r), Fn(d)

Form Fn(l), Fn(r), Fn(d)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands left operand is floating-point register number; right operand is floating-point register number; destination operand is floating-point register number



Instruction format for FNMAADD.X(z:S/D) Fn(l), Fn(r), Fn(d)

- l** floating-point register number for the left operand (payload word 1 bits 3:0).
r floating-point register number for the right operand (payload word 1 bits 7:4).
d floating-point register number for the destination operand (payload word 1 bits 11:8).
z size selector for the operand size (payload word 1 bits 12).

FNMSUB

Floating-Point Fused Negative Multiply Subtract

FNMSUB

Summary: Performs the floating-point fused negative multiply subtract operation.

Operation: $\text{dst_v} = \text{dst_v} \text{ f- } (\text{lhs_v} \text{ f* } \text{rhs_v});$
 $\text{dst_v} = \text{f- } \text{dst_v};$

Assembler Syntax: `FNMSUB.<S/D> <ea>, Fn, Fn`
`FNMSUB.<S/D> Fn, <ea>, Fn`
`FNMSUB.<S/D> Fn, Fn, Fn`

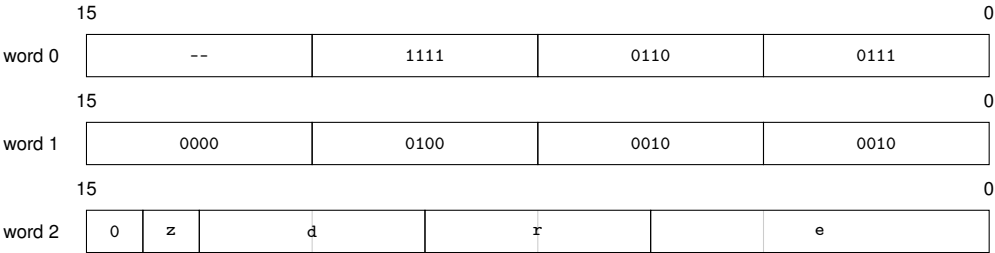
Attributes: Size = (Single, Double)
Words = (3-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

Floating-Point Status: **NV DZ OF UF NX**
* _ * * *
update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FNMSUB.X(z:S/D) <ea(e)>, Fn(r), Fn(d)`

Form `<ea(e)>, Fn(r), Fn(d)`
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands left operand is compact effective-address field; right operand is floating-point register number; destination operand is floating-point register number

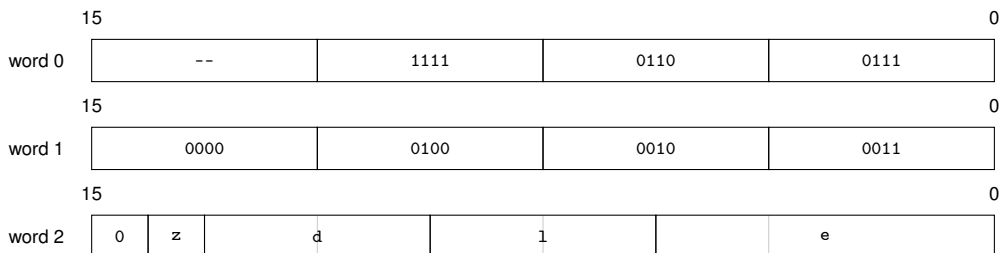


Instruction format for FNMSUB.X(z:S/D) <ea(e)>, Fn(r), Fn(d)

- e compact effective-address field for the left operand (payload word 1 bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 - r floating-point register number for the right operand (payload word 1 bits 9:6).
 - d floating-point register number for the destination operand (payload word 1 bits 13:10).
 - z size selector for the operand size (payload word 1 bits 14).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FNMSUB.X(z:S/D) Fn(l), <ea(e)>, Fn(d)

Form Fn(l), <ea(e)>, Fn(d)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands left operand is floating-point register number; right operand is compact effective-address field; destination operand is floating-point register number

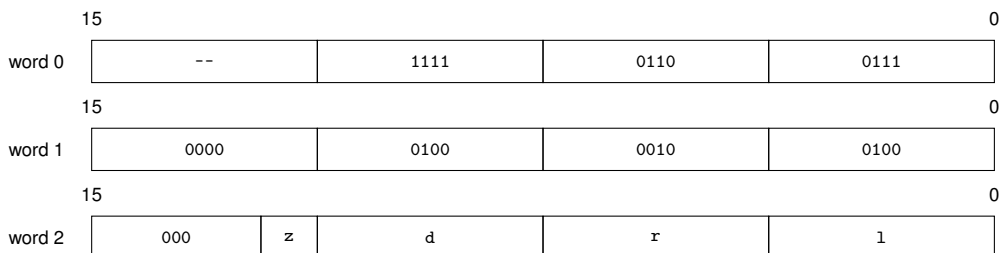


Instruction format for FNMSUB.X(z:S/D) Fn(l), <ea(e)>, Fn(d)

- e compact effective-address field for the right operand (payload word 1 bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
 l floating-point register number for the left operand (payload word 1 bits 9:6).
 d floating-point register number for the destination operand (payload word 1 bits 13:10).
 z size selector for the operand size (payload word 1 bits 14).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FNMSUB.X(z:S/D) Fn(l), Fn(r), Fn(d)

Form Fn(l), Fn(r), Fn(d)
Encoding extended opcode form
Words 3-8
Privilege unprivileged
Operands left operand is floating-point register number; right operand is floating-point register number; destination operand is floating-point register number



Instruction format for FNMSUB.X(z:S/D) Fn(l), Fn(r), Fn(d)

- l floating-point register number for the left operand (payload word 1 bits 3:0).
 r floating-point register number for the right operand (payload word 1 bits 7:4).
 d floating-point register number for the destination operand (payload word 1 bits 11:8).
 z size selector for the operand size (payload word 1 bits 12).

FPOPM

Floating-Point Pop Multiple

FPOPM

Summary:

Pop selected F15-F0 registers in inverse bitmap order.

Operation:

For each set bit in the bitmap, FPOPM pops one 64-bit stack slot into the corresponding F register. F registers are processed in inverse order from F15 through F0. Each selected register is loaded from SP before SP is incremented by 8.

Assembler Syntax:

FPOPM <bitmap16>

Attributes:

Words = (3-8)
Privilege = unprivileged

Description:

Pop selected F15-F0 registers in inverse bitmap order. Bit n names Fn.

Condition Codes:

Z N C V

- - - -

unchanged

Instruction Forms:

FPOPM <bitmap>

Form

<bitmap>

Encoding

extended opcode form

Words

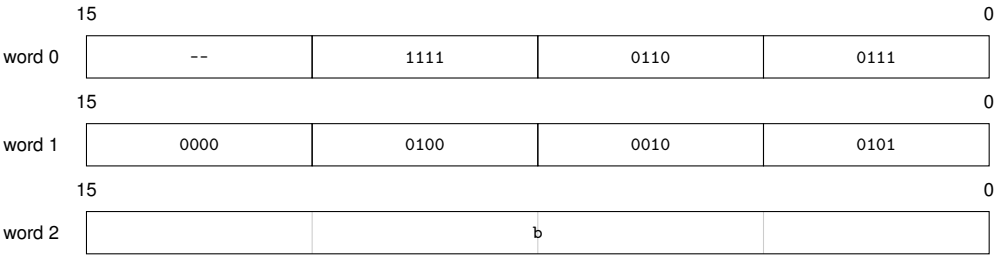
3-8

Privilege

unprivileged

Operands

register bitmap is 16-bit floating-point register bitmap



Instruction format for FPOPM <bitmap>

b 16-bit floating-point register bitmap for the register bitmap (payload word 1 bits 15:0).

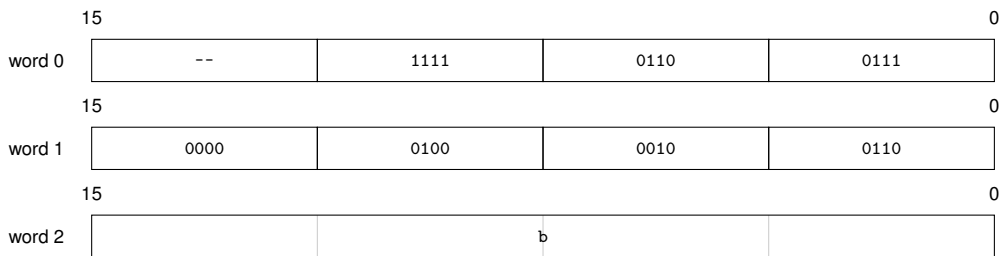
FPUSHM**Floating-Point Push Multiple****FPUSHM**

Summary:	Push selected F0-F15 registers according to bitmap order.
Operation:	For each set bit in the bitmap, FPUSHM pushes the corresponding 64-bit F register value onto the stack. F registers are processed in increasing order from F0 through F15. Each selected register decrements SP by 8 before storing the register value at the new SP.
Assembler Syntax:	FPUSHM <bitmap16>
Attributes:	Words = (3-8) Privilege = unprivileged
Description:	Push selected F0-F15 registers according to bitmap order. Bit n names Fn.
Condition Codes:	Z N C V - - - - unchanged

Instruction Forms:

FPUSHM <bitmap>

Form	<bitmap>
Encoding	extended opcode form
Words	3-8
Privilege	unprivileged
Operands	register bitmap is 16-bit floating-point register bitmap

**Instruction format for FPUSHM <bitmap>**

b 16-bit floating-point register bitmap for the register bitmap (payload word 1 bits 15:0).

FREM

Floating-Point Remainder

FREM

Summary:

Operation:

Assembler Syntax:

Attributes:

Floating-Point Status:

Performs the floating-point remainder operation.

`dst_v = fp_rem(dst_v, src_v);`

`FREM.<S/D> Fn, Fn`
`FREM.<S/D> <ea>, Fn`

Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

NV DZ OF UF NX
* - - - -

update when required: FFLAGS.NV (accrued invalid operation flag)

Instruction Forms:

`FREM.X(z:S/D) Fn(f), Fn(d)`

Form

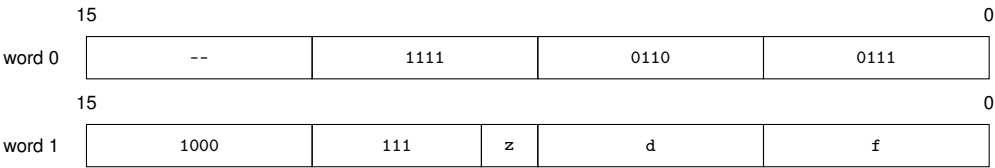
Encoding

Words

Privilege

Operands

`Fn(f), Fn(d)`
extended opcode form
2-8
unprivileged
source operand is floating-point register number; destination operand is floating-point register number

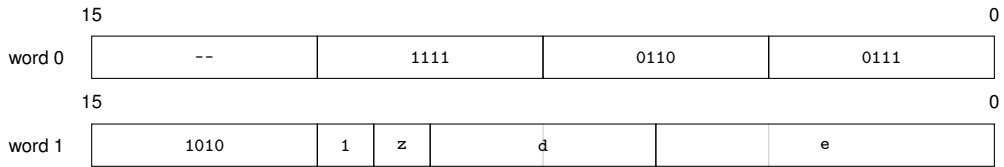


Instruction format for FREM.X(z:S/D) Fn(f), Fn(d)

- f
- d
- z
- floating-point register number for the source operand (extended opcode word bits 3:0).
- floating-point register number for the destination operand (extended opcode word bits 7:4).
- size selector for the operand size (extended opcode word bits 8).

FREM.X(z:S/D) <ea(e)>, Fn(d)

Form	<ea(e)>, Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is floating-point register number



Instruction format for FREM.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FROUND

Floating-Point Round

FROUND

Summary:

Operation:

Assembler Syntax:

Attributes:

Floating-Point Status:

Performs the floating-point round operation.

`dst_v = round(src_v);`

`FROUND.<S/D> <ea>, Fn`
`FROUND.<S/D> Fn, <ea>`
`FROUND.<S/D> Fn, Fn`

Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

NV DZ OF UF NX
* - - - *

update when required: FFLAGS.NV (accrued invalid operation flag),
FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FROUND.X(z:S/D) <ea(e)>, Fn(d)`

Form

Encoding

Words

Privilege

Operands

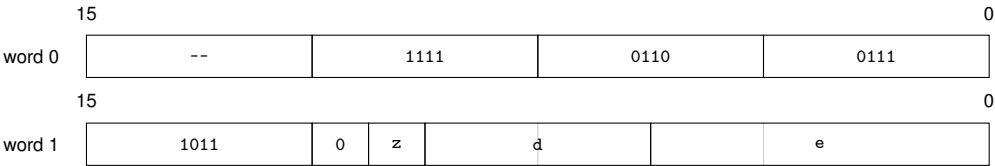
<ea(e)>, Fn(d)

extended opcode form

2-8

unprivileged

source operand is compact effective-address field; destination operand is floating-point register number

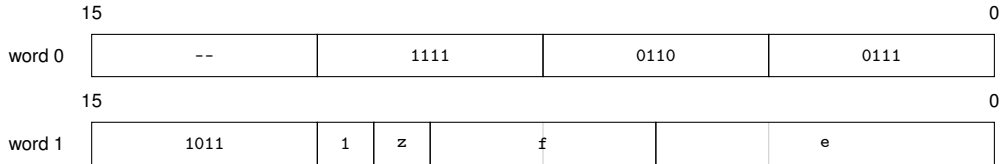


Instruction format for FROUND.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FROUND.X(z:S/D) Fn(f), <ea(e)>

Form Fn(f), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is compact effective-address field

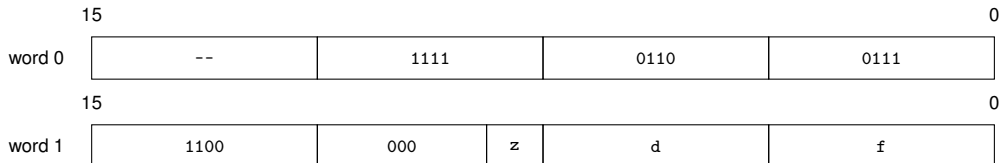


Instruction format for FROUND.X(z:S/D) Fn(f), <ea(e)>

- e** compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- f** floating-point register number for the source operand (extended opcode word bits 9:6).
- z** size selector for the operand size (extended opcode word bits 10).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FROUND.X(z:S/D) Fn(f), Fn(d)

Form Fn(f), Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FROUND.X(z:S/D) Fn(f), Fn(d)

- f** floating-point register number for the source operand (extended opcode word bits 3:0).
- d** floating-point register number for the destination operand (extended opcode word bits 7:4).
- z** size selector for the operand size (extended opcode word bits 8).

FSCALE

Floating-Point Scale

FSCALE

Summary: Performs the floating-point scale operation.

Operation: dst_v = fp_scale(dst_v, src_v);

Assembler Syntax: FSCALE.<S/D> Fn, Fn
FSCALE.<S/D> <ea>, Fn

Attributes: Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

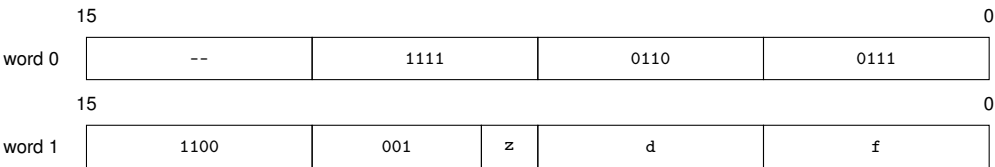
Floating-Point Status: **NV DZ OF UF NX**
* _ * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

FSCALE.X(z:S/D) Fn(f), Fn(d)

Form Fn(f), Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number

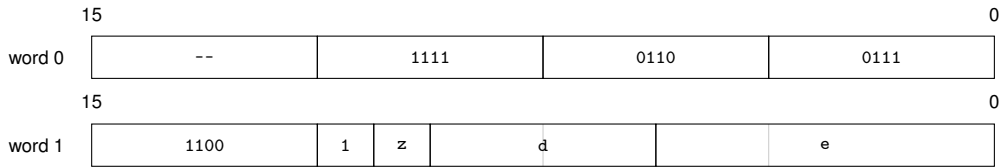


Instruction format for FSCALE.X(z:S/D) Fn(f), Fn(d)

- f floating-point register number for the source operand (extended opcode word bits 3:0).
- d floating-point register number for the destination operand (extended opcode word bits 7:4).
- z size selector for the operand size (extended opcode word bits 8).

FSCALE.X(z:S/D) <ea(e)>, Fn(d)

Form	<ea(e)>, Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is floating-point register number



Instruction format for FSCALE.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FSQRT

Floating-Point Square Root

FSQRT

Summary:

Operation:

Assembler Syntax:

Attributes:

Floating-Point Status:

Performs the floating-point square root operation.

`dst_v = sqrt(src_v);`

`FSQRT.<S/D> Fn, Fn`
`FSQRT.<S/D> <ea>, Fn`
`FSQRT.<S/D> Fn, <ea>`

Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

NV DZ OF UF NX
* - - - *

update when required: FFLAGS.NV (accrued invalid operation flag),
FFLAGS.NX (accrued inexact flag)

Instruction Forms:

FSQRT.X(z:S/D) Fn(f), Fn(d)

Form

Encoding

Words

Privilege

Operands

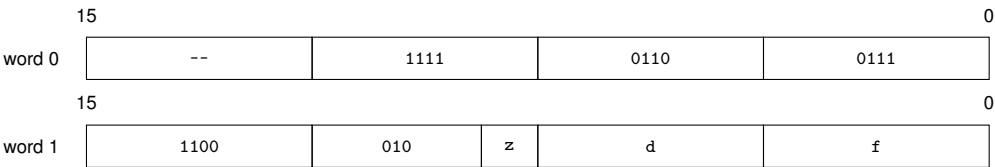
Fn(f), Fn(d)

extended opcode form

2-8

unprivileged

source operand is floating-point register number; destination operand is floating-point register number

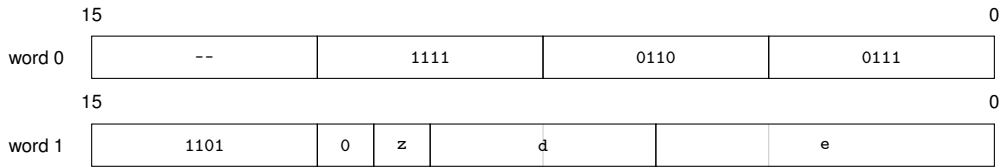


Instruction format for FSQRT.X(z:S/D) Fn(f), Fn(d)

- f
- d
- z
- floating-point register number for the source operand (extended opcode word bits 3:0).
- floating-point register number for the destination operand (extended opcode word bits 7:4).
- size selector for the operand size (extended opcode word bits 8).

FSQRT.X(z:S/D) <ea(e)>, Fn(d)

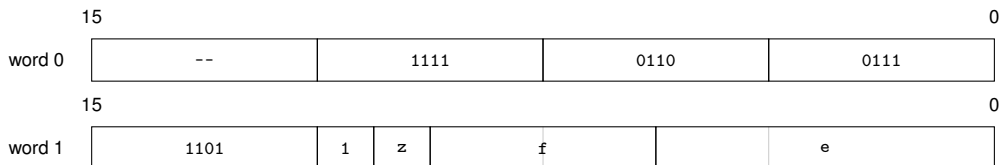
Form <ea(e)>, Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field; destination operand is floating-point register number

**Instruction format for FSQRT.X(z:S/D) <ea(e)>, Fn(d)**

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
d floating-point register number for the destination operand (extended opcode word bits 9:6).
z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FSQRT.X(z:S/D) Fn(f), <ea(e)>

Form Fn(f), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is compact effective-address field

**Instruction format for FSQRT.X(z:S/D) Fn(f), <ea(e)>**

- e compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
f floating-point register number for the source operand (extended opcode word bits 9:6).
z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FSUB

Floating-Point Subtract

FSUB

Summary: Subtracts the source operand from the destination operand.

Operation: `dst_v = dst_v f- src_v;`

Assembler Syntax: `FSUB.<S/D> Fn, Fn`
`FSUB.<S/D> <ea>, Fn`

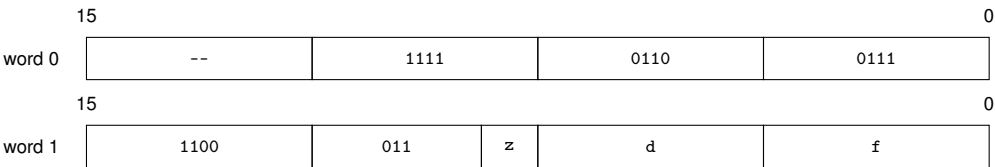
Attributes: Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

Floating-Point Status: **NV DZ OF UF NX**
* _ * * *
update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FSUB.X(z:S/D) Fn(f), Fn(d)`

Form	<code>Fn(f), Fn(d)</code>
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is floating-point register number

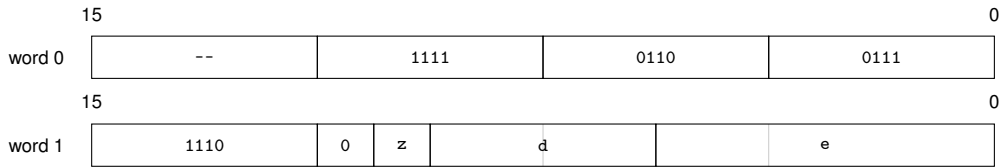


Instruction format for FSUB.X(z:S/D) Fn(f), Fn(d)

- `f` floating-point register number for the source operand (extended opcode word bits 3:0).
- `d` floating-point register number for the destination operand (extended opcode word bits 7:4).
- `z` size selector for the operand size (extended opcode word bits 8).

FSUB.X(z:S/D) <ea(e)>, Fn(d)

Form	<ea(e)>, Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is compact effective-address field; destination operand is floating-point register number



Instruction format for FSUB.X(z:S/D) <ea(e)>, Fn(d)

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- d floating-point register number for the destination operand (extended opcode word bits 9:6).
- z size selector for the operand size (extended opcode word bits 10).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FTEST

Floating-Point Test

FTEST

Summary:

Operation:

Assembler Syntax:

Attributes:

Floating-Point Status:

Performs the floating-point test operation.

ZF = src_v f== 0;
NF = src_v f< 0;
CF = 0;
VF = nan(src_v);

FTEST.<S/D> Fn
FTEST.<S/D> <ea>

Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

NV DZ OF UF NX
* - - - -

update when required: FFLAGS.NV (accrued invalid operation flag)

Instruction Forms:

FTEST.X(z:S/D) Fn(f)

Form

Encoding

Words

Privilege

Operands

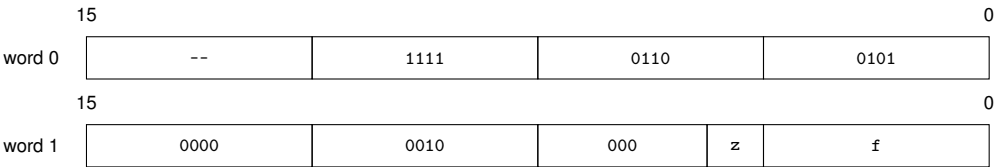
Fn(f)

extended opcode form

2-8

unprivileged

source operand is floating-point register number



Instruction format for FTEST.X(z:S/D) Fn(f)

f

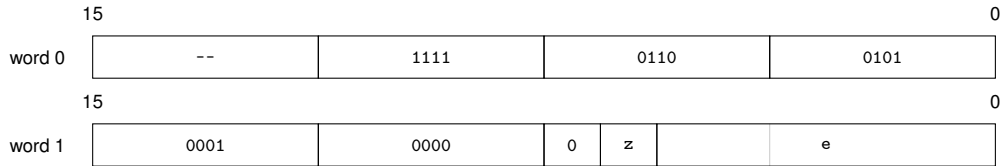
floating-point register number for the source operand (extended opcode word bits 3:0).

z

size selector for the operand size (extended opcode word bits 4).

FTEST.X(z:S/D) <ea(e)>

Form <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is compact effective-address field



Instruction format for FTEST.X(z:S/D) <ea(e)>

- e compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- z size selector for the operand size (extended opcode word bits 6).
EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FTRUNC

Floating-Point Truncate

FTRUNC

Summary:

Operation:

Assembler Syntax:

Attributes:

Floating-Point Status:

NV DZ OF UF NX

* - - - *

update when required: FFLAGS.NV (accrued invalid operation flag),
FFLAGS.NX (accrued inexact flag)

Instruction Forms:

FTRUNC.X(z:S/D) <ea(e)>, Fn(d)

Form

Encoding

Words

Privilege

Operands

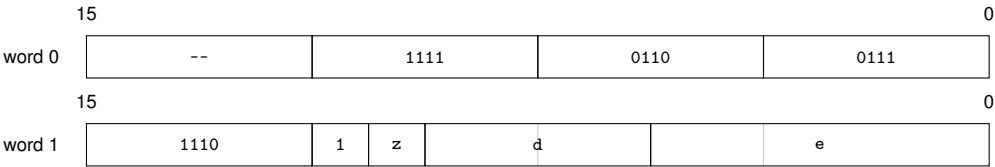
<ea(e)>, Fn(d)

extended opcode form

2-8

unprivileged

source operand is compact effective-address field; destination operand is floating-point register number

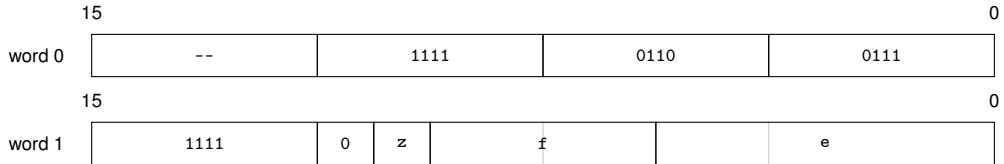


Instruction format for FTRUNC.X(z:S/D) <ea(e)>, Fn(d)

- e
- d
- z
- compact effective-address field for the source operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- floating-point register number for the destination operand (extended opcode word bits 9:6).
- size selector for the operand size (extended opcode word bits 10).
- EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FTRUNC.X(z:S/D) Fn(f), <ea(e)>

Form Fn(f), <ea(e)>
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is compact effective-address field

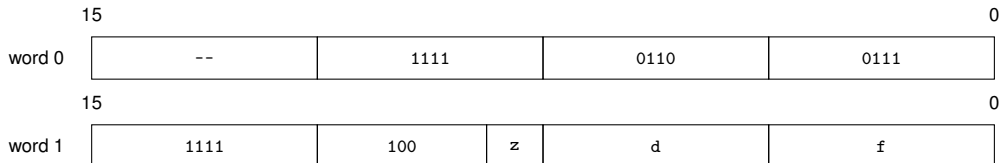


Instruction format for FTRUNC.X(z:S/D) Fn(f), <ea(e)>

- e** compact effective-address field for the destination operand (extended opcode word bits 5:0). EA field selects register, memory, immediate, and extended EA forms.
- f** floating-point register number for the source operand (extended opcode word bits 9:6).
- z** size selector for the operand size (extended opcode word bits 10).
 EA selections may append displacement, absolute-address, immediate, or extended-EA payload words.

FTRUNC.X(z:S/D) Fn(f), Fn(d)

Form Fn(f), Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FTRUNC.X(z:S/D) Fn(f), Fn(d)

- f** floating-point register number for the source operand (extended opcode word bits 3:0).
- d** floating-point register number for the destination operand (extended opcode word bits 7:4).
- z** size selector for the operand size (extended opcode word bits 8).

FXCHG

Floating-Point Exchange

FXCHG

Summary:

Operation:

Assembler Syntax:

Attributes:

Floating-Point Status:

Performs the floating-point exchange operation.

tmp_v = lhs_v;
lhs_v = rhs_v;
rhs_v = tmp_v;

FXCHG Fn, Fn

Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

NV DZ OF UF NX
- - - - -
FFLAGS unchanged

Instruction Forms:

FXCHG Fn(l), Fn(r)

Form

Encoding

Words

Privilege

Operands

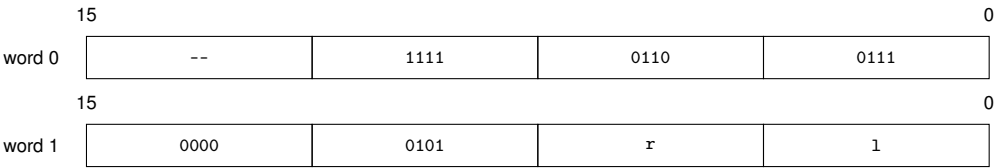
Fn(l), Fn(r)

extended opcode form

2-8

unprivileged

left operand is floating-point register number; right operand is floating-point register number



Instruction format for FXCHG Fn(l), Fn(r)

- l floating-point register number for the left operand (extended opcode word bits 3:0).
- r floating-point register number for the right operand (extended opcode word bits 7:4).

26 FLOATING-POINT TRANSCENDENTAL INSTRUCTIONS SUMMARY

Mnemonic	Summary	Forms
FACOS	Floating-Point Arc Cosine	1
FASIN	Floating-Point Arc Sine	1
FATAN	Floating-Point Arc Tangent	1
FATANH	Floating-Point Hyperbolic Arc Tangent	1
FCOS	Floating-Point Cosine	1
FCOSH	Floating-Point Hyperbolic Cosine	1
FETOX	Floating-Point e To x	1
FETOXM1	Floating-Point e To x Minus One	1
FLOG10	Floating-Point Log Base 10	1
FLOG2	Floating-Point Log Base 2	1
FLOGN	Floating-Point Natural Log	1
FLOGNP1	Floating-Point Natural Log Plus One	1
FSIN	Floating-Point Sine	1
FSINCOS	Floating-Point Sine And Cosine	1
FSINH	Floating-Point Hyperbolic Sine	1
FTAN	Floating-Point Tangent	1
FTANH	Floating-Point Hyperbolic Tangent	1
FTENTOX	Floating-Point 10 To x	1
FTWOTOX	Floating-Point 2 To x	1

Table 26-1. Floating-Point Transcendental Instructions Summary

FACOS

Floating-Point Arc Cosine

FACOS

Summary:

Operation:

Assembler Syntax:

Attributes:

Floating-Point Status:

Performs the floating-point arc cosine operation.

`dst_v = fp_acos(src_v);`

`FACOS.<S/D> Fn, Fn`

Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

NV DZ OF UF NX
*** - - * ***

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FACOS.X(z:S/D) Fn(f), Fn(d)`

Form

Encoding

Words

Privilege

Operands

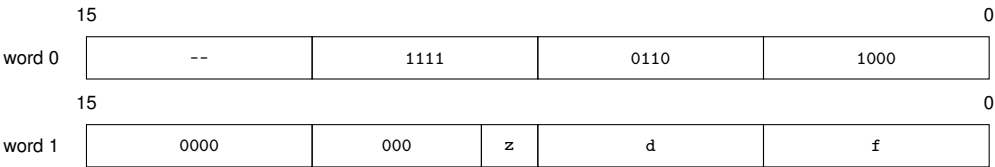
Fn(f), Fn(d)

extended opcode form

2-8

unprivileged

source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FACOS.X(z:S/D) Fn(f), Fn(d)

- f
- d
- z
- floating-point register number for the source operand (extended opcode word bits 3:0).
- floating-point register number for the destination operand (extended opcode word bits 7:4).
- size selector for the operand size (extended opcode word bits 8).

FASIN**Floating-Point Arc Sine****FASIN**

Summary: Performs the floating-point arc sine operation.

Operation: `dst_v = fp_asin(src_v);`

Assembler Syntax: `FASIN.<S/D> Fn, Fn`

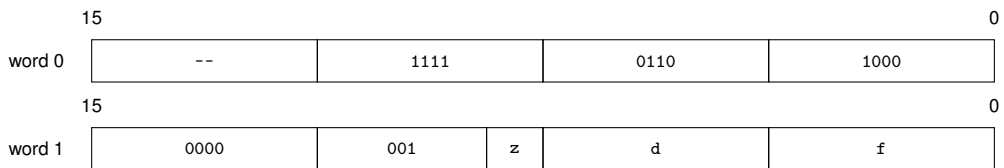
Attributes:
 Size = (Single, Double)
 Words = (2-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

Floating-Point Status: **NV DZ OF UF NX**
 * - - * *
 update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FASIN.X(z:S/D) Fn(f), Fn(d)`

Form `Fn(f), Fn(d)`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FASIN.X(z:S/D) Fn(f), Fn(d)

f floating-point register number for the source operand (extended opcode word bits 3:0).
d floating-point register number for the destination operand (extended opcode word bits 7:4).
z size selector for the operand size (extended opcode word bits 8).

FATAN

Floating-Point Arc Tangent

FATAN

Summary:

Operation:

Assembler Syntax:

Attributes:

Floating-Point Status:

Performs the floating-point arc tangent operation.

`dst_v = fp_atan(src_v);`

`FATAN.<S/D> Fn, Fn`

Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

NV DZ OF UF NX
* - - * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FATAN.X(z:S/D) Fn(f), Fn(d)`

Form

Encoding

Words

Privilege

Operands

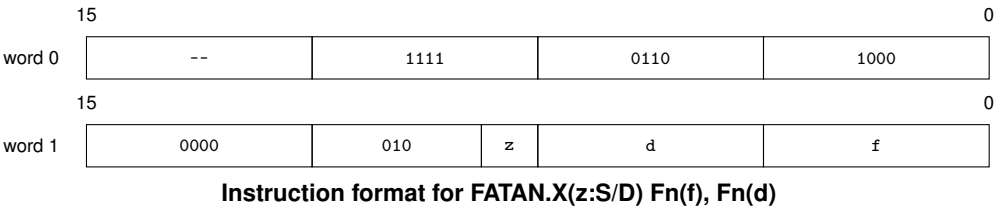
Fn(f), Fn(d)

extended opcode form

2-8

unprivileged

source operand is floating-point register number; destination operand is floating-point register number



- f
- d
- z
- floating-point register number for the source operand (extended opcode word bits 3:0).
- floating-point register number for the destination operand (extended opcode word bits 7:4).
- size selector for the operand size (extended opcode word bits 8).

FATANH**Floating-Point Hyperbolic Arc Tangent****FATANH**

Summary: Performs the floating-point hyperbolic arc tangent operation.

Operation: `dst_v = fp_atanh(src_v);`

Assembler Syntax: `FATANH.<S/D> Fn, Fn`

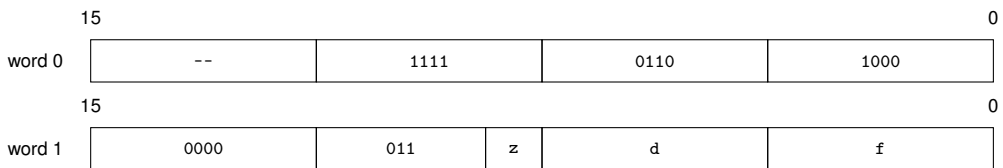
Attributes:
 Size = (Single, Double)
 Words = (2-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

Floating-Point Status: **NV DZ OF UF NX**
 * - - * *
 update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FATANH.X(z:S/D) Fn(f), Fn(d)`

Form `Fn(f), Fn(d)`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FATANH.X(z:S/D) Fn(f), Fn(d)

f floating-point register number for the source operand (extended opcode word bits 3:0).
d floating-point register number for the destination operand (extended opcode word bits 7:4).
z size selector for the operand size (extended opcode word bits 8).

FCOS

Floating-Point Cosine

FCOS

Summary: Performs the floating-point cosine operation.

Operation: dst_v = fp_cos(src_v);

Assembler Syntax: FCOS.<S/D> Fn, Fn

Attributes:
Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

Floating-Point Status:

NV DZ OF UF NX

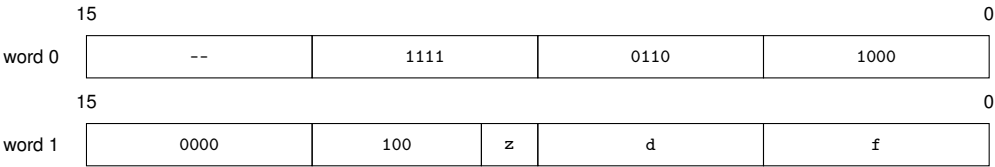
* _ * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

FCOS.X(z:S/D) Fn(f), Fn(d)

Form	Fn(f), Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FCOS.X(z:S/D) Fn(f), Fn(d)

- f floating-point register number for the source operand (extended opcode word bits 3:0).
- d floating-point register number for the destination operand (extended opcode word bits 7:4).
- z size selector for the operand size (extended opcode word bits 8).

FCOSH**Floating-Point Hyperbolic Cosine****FCOSH**

Summary: Performs the floating-point hyperbolic cosine operation.

Operation: `dst_v = fp_cosh(src_v);`

Assembler Syntax: `FCOSH.<S/D> Fn, Fn`

Attributes:
 Size = (Single, Double)
 Words = (2-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

Floating-Point Status:

NV DZ OF UF NX

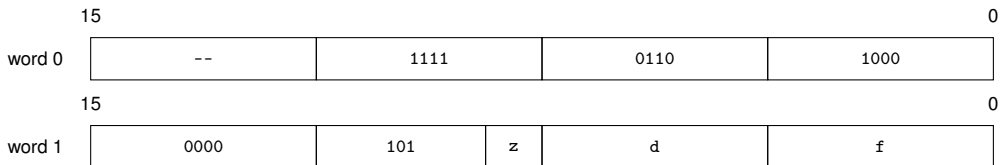
* _ * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FCOSH.X(z:S/D) Fn(f), Fn(d)`

Form `Fn(f), Fn(d)`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FCOSH.X(z:S/D) Fn(f), Fn(d)

f floating-point register number for the source operand (extended opcode word bits 3:0).
 d floating-point register number for the destination operand (extended opcode word bits 7:4).
 z size selector for the operand size (extended opcode word bits 8).

FETOX

Floating-Point e To x

FETOX

Summary: Performs the floating-point e to x operation.

Operation: dst_v = fp_exp(src_v);

Assembler Syntax: FETOX.<S/D> Fn, Fn

Attributes:
Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

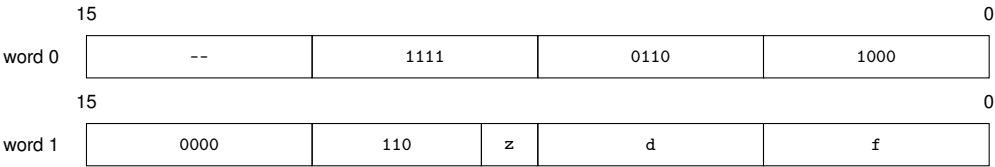
Floating-Point Status: **NV DZ OF UF NX**
* _ * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

FETOX.X(z:S/D) Fn(f), Fn(d)

Form Fn(f), Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FETOX.X(z:S/D) Fn(f), Fn(d)

- f floating-point register number for the source operand (extended opcode word bits 3:0).
- d floating-point register number for the destination operand (extended opcode word bits 7:4).
- z size selector for the operand size (extended opcode word bits 8).

FETOXM1**Floating-Point e To x Minus One****FETOXM1**

Summary: Performs the floating-point e to x minus one operation.

Operation: `dst_v = fp_expm1(src_v);`

Assembler Syntax: `FETOXM1.<S/D> Fn, Fn`

Attributes:
 Size = (Single, Double)
 Words = (2-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

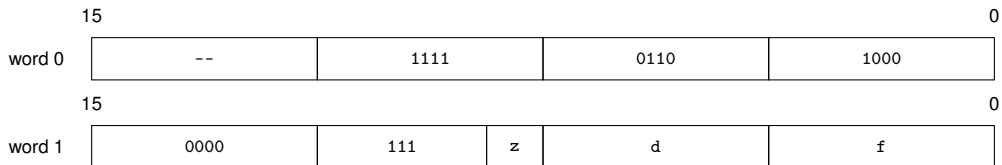
Floating-Point Status: **NV DZ OF UF NX**
 * _ * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FETOXM1.X(z:S/D) Fn(f), Fn(d)`

Form `Fn(f), Fn(d)`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FETOXM1.X(z:S/D) Fn(f), Fn(d)

f floating-point register number for the source operand (extended opcode word bits 3:0).
d floating-point register number for the destination operand (extended opcode word bits 7:4).
z size selector for the operand size (extended opcode word bits 8).

FLOG10

Floating-Point Log Base 10

FLOG10

Summary:

Performs the floating-point log base 10 operation.

Operation:

`dst_v = fp_log10(src_v);`

Assembler Syntax:

`FLOG10.<S/D> Fn, Fn`

Attributes:

Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

Floating-Point Status:

NV DZ OF UF NX
* * _ * *

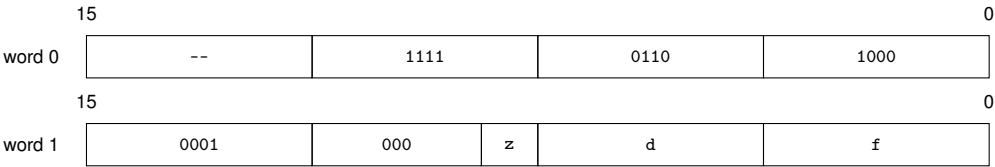
update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.DZ (accrued divide-by-zero flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

FLOG10.X(z:S/D) Fn(f), Fn(d)

Form
Encoding
Words
Privilege
Operands

Fn(f), Fn(d)
extended opcode form
2-8
unprivileged
source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FLOG10.X(z:S/D) Fn(f), Fn(d)

- f
- floating-point register number for the source operand (extended opcode word bits 3:0).
- d
- floating-point register number for the destination operand (extended opcode word bits 7:4).
- z
- size selector for the operand size (extended opcode word bits 8).

FLOG2**Floating-Point Log Base 2****FLOG2**

Summary: Performs the floating-point log base 2 operation.

Operation: `dst_v = fp_log2(src_v);`

Assembler Syntax: `FLOG2.<S/D> Fn, Fn`

Attributes:
 Size = (Single, Double)
 Words = (2-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

Floating-Point Status:

NV DZ OF UF NX

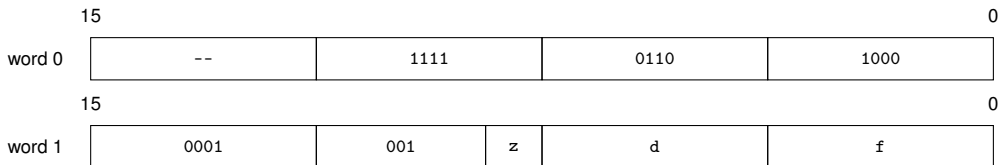
* * _ * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.DZ (accrued divide-by-zero flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FLOG2.X(z:S/D) Fn(f), Fn(d)`

Form `Fn(f), Fn(d)`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FLOG2.X(z:S/D) Fn(f), Fn(d)

f floating-point register number for the source operand (extended opcode word bits 3:0).
d floating-point register number for the destination operand (extended opcode word bits 7:4).
z size selector for the operand size (extended opcode word bits 8).

FLOGN

Floating-Point Natural Log

FLOGN

Summary:

Operation:

Assembler Syntax:

Attributes:

Floating-Point Status:

Performs the floating-point natural log operation.

`dst_v = fp_log(src_v);`

`FLOGN.<S/D> Fn, Fn`

Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

NV DZ OF UF NX
* * _ * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.DZ (accrued divide-by-zero flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FLOGN.X(z:S/D) Fn(f), Fn(d)`

Form

Encoding

Words

Privilege

Operands

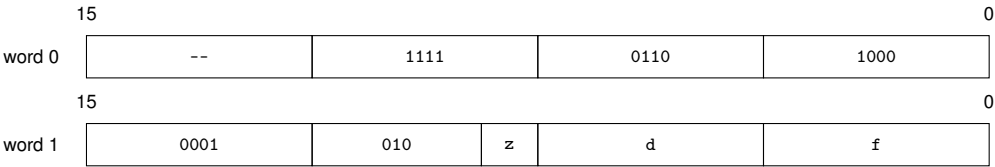
Fn(f), Fn(d)

extended opcode form

2-8

unprivileged

source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FLOGN.X(z:S/D) Fn(f), Fn(d)

- f
- d
- z
- floating-point register number for the source operand (extended opcode word bits 3:0).
- floating-point register number for the destination operand (extended opcode word bits 7:4).
- size selector for the operand size (extended opcode word bits 8).

FLOGNP1**Floating-Point Natural Log Plus One****FLOGNP1**

Summary: Performs the floating-point natural log plus one operation.

Operation: `dst_v = fp_log1p(src_v);`

Assembler Syntax: `FLOGNP1.<S/D> Fn, Fn`

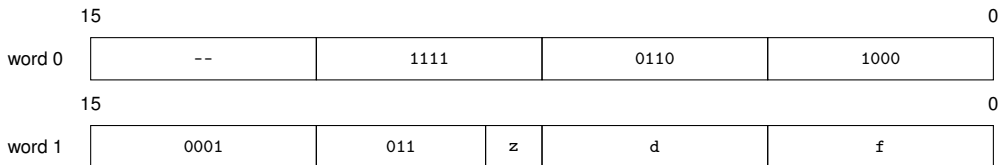
Attributes:
 Size = (Single, Double)
 Words = (2-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

Floating-Point Status: **NV DZ OF UF NX**
 * * _ * *
 update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.DZ (accrued divide-by-zero flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FLOGNP1.X(z:S/D) Fn(f), Fn(d)`

Form `Fn(f), Fn(d)`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FLOGNP1.X(z:S/D) Fn(f), Fn(d)

f floating-point register number for the source operand (extended opcode word bits 3:0).
d floating-point register number for the destination operand (extended opcode word bits 7:4).
z size selector for the operand size (extended opcode word bits 8).

FSIN

Floating-Point Sine

FSIN

Summary: Performs the floating-point sine operation.

Operation: dst_v = fp_sin(src_v);

Assembler Syntax: FSIN.<S/D> Fn, Fn

Attributes: Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

Floating-Point Status:

NV DZ OF UF NX

* _ * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

FSIN.X(z:S/D) Fn(f), Fn(d)

Form

Fn(f), Fn(d)

Encoding

extended opcode form

Words

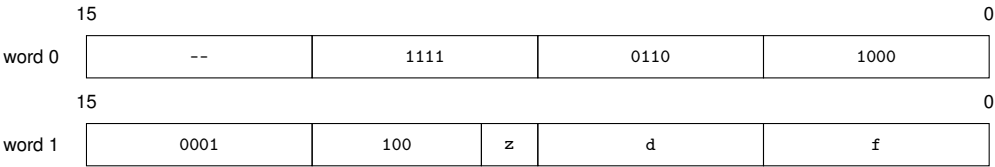
2-8

Privilege

unprivileged

Operands

source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FSIN.X(z:S/D) Fn(f), Fn(d)

- f floating-point register number for the source operand (extended opcode word bits 3:0).
- d floating-point register number for the destination operand (extended opcode word bits 7:4).
- z size selector for the operand size (extended opcode word bits 8).

FSINCOS**Floating-Point Sine And Cosine****FSINCOS**

Summary: Performs the floating-point sine and cosine operation.

Operation: `dst_v = fp_sincos(src_v);`

Assembler Syntax: `FSINCOS.<S/D> Fn, Fn`

Attributes:
 Size = (Single, Double)
 Words = (2-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

Floating-Point Status:

NV DZ OF UF NX

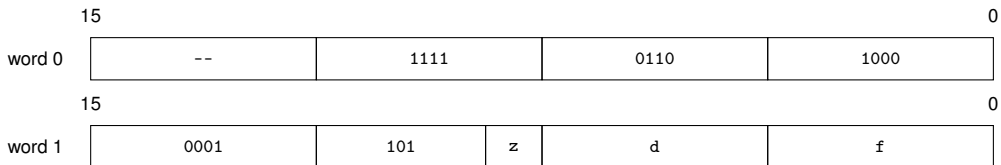
* _ * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FSINCOS.X(z:S/D) Fn(f), Fn(d)`

Form `Fn(f), Fn(d)`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FSINCOS.X(z:S/D) Fn(f), Fn(d)

f floating-point register number for the source operand (extended opcode word bits 3:0).
 d floating-point register number for the destination operand (extended opcode word bits 7:4).
 z size selector for the operand size (extended opcode word bits 8).

FSINH

Floating-Point Hyperbolic Sine

FSINH

Summary: Performs the floating-point hyperbolic sine operation.

Operation: dst_v = fp_sinh(src_v);

Assembler Syntax: FSINH.<S/D> Fn, Fn

Attributes: Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

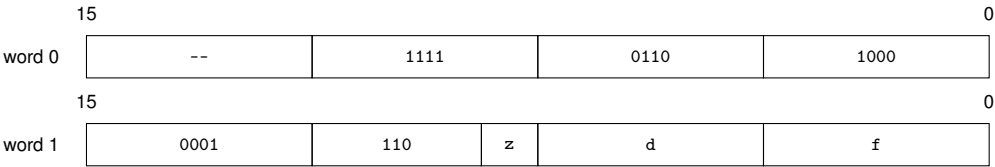
Floating-Point Status: **NV DZ OF UF NX**
* _ * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

FSINH.X(z:S/D) Fn(f), Fn(d)

Form Fn(f), Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FSINH.X(z:S/D) Fn(f), Fn(d)

- f floating-point register number for the source operand (extended opcode word bits 3:0).
- d floating-point register number for the destination operand (extended opcode word bits 7:4).
- z size selector for the operand size (extended opcode word bits 8).

FTAN**Floating-Point Tangent****FTAN**

Summary: Performs the floating-point tangent operation.

Operation: `dst_v = fp_tan(src_v);`

Assembler Syntax: `FTAN.<S/D> Fn, Fn`

Attributes:
 Size = (Single, Double)
 Words = (2-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

Floating-Point Status:

NV DZ OF UF NX

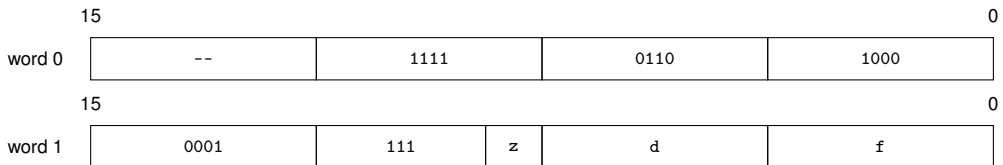
* _ * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FTAN.X(z:S/D) Fn(f), Fn(d)`

Form `Fn(f), Fn(d)`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FTAN.X(z:S/D) Fn(f), Fn(d)

f floating-point register number for the source operand (extended opcode word bits 3:0).
d floating-point register number for the destination operand (extended opcode word bits 7:4).
z size selector for the operand size (extended opcode word bits 8).

FTANH

Floating-Point Hyperbolic Tangent

FTANH

Summary: Performs the floating-point hyperbolic tangent operation.

Operation: dst_v = fp_tanh(src_v);

Assembler Syntax: FTANH.<S/D> Fn, Fn

Attributes:
Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

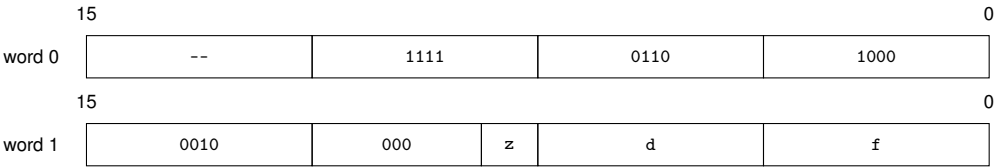
Floating-Point Status: **NV DZ OF UF NX**
* _ * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

FTANH.X(z:S/D) Fn(f), Fn(d)

Form	Fn(f), Fn(d)
Encoding	extended opcode form
Words	2-8
Privilege	unprivileged
Operands	source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FTANH.X(z:S/D) Fn(f), Fn(d)

- f floating-point register number for the source operand (extended opcode word bits 3:0).
- d floating-point register number for the destination operand (extended opcode word bits 7:4).
- z size selector for the operand size (extended opcode word bits 8).

FTENTOX**Floating-Point 10 To x****FTENTOX**

Summary: Performs the floating-point 10 to x operation.

Operation: `dst_v = fp_exp10(src_v);`

Assembler Syntax: `FTENTOX.<S/D> Fn, Fn`

Attributes:
 Size = (Single, Double)
 Words = (2-8)
 Privilege = unprivileged
 FFLAGS = see Floating-Point Status

Floating-Point Status:

NV DZ OF UF NX

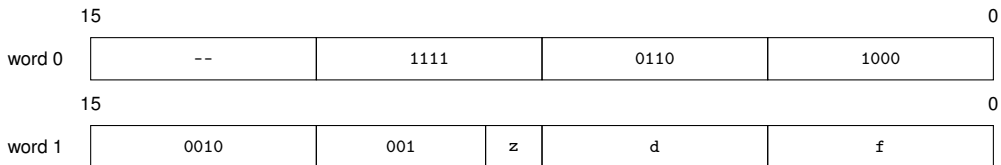
* _ * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

`FTENTOX.X(z:S/D) Fn(f), Fn(d)`

Form `Fn(f), Fn(d)`
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FTENTOX.X(z:S/D) Fn(f), Fn(d)

f floating-point register number for the source operand (extended opcode word bits 3:0).
d floating-point register number for the destination operand (extended opcode word bits 7:4).
z size selector for the operand size (extended opcode word bits 8).

FTWOTOX

Floating-Point 2 To x

FTWOTOX

Summary: Performs the floating-point 2 to x operation.

Operation: dst_v = fp_exp2(src_v);

Assembler Syntax: FTWOTOX.<S/D> Fn, Fn

Attributes:
Size = (Single, Double)
Words = (2-8)
Privilege = unprivileged
FFLAGS = see Floating-Point Status

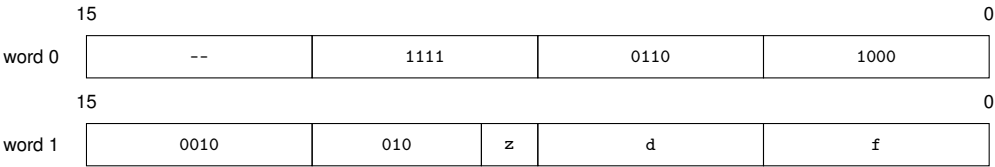
Floating-Point Status: **NV DZ OF UF NX**
* _ * * *

update: FFLAGS.NV (accrued invalid operation flag), FFLAGS.OF (accrued overflow flag), FFLAGS.UF (accrued underflow flag), FFLAGS.NX (accrued inexact flag)

Instruction Forms:

FTWOTOX.X(z:S/D) Fn(f), Fn(d)

Form Fn(f), Fn(d)
Encoding extended opcode form
Words 2-8
Privilege unprivileged
Operands source operand is floating-point register number; destination operand is floating-point register number



Instruction format for FTWOTOX.X(z:S/D) Fn(f), Fn(d)

- f floating-point register number for the source operand (extended opcode word bits 3:0).
- d floating-point register number for the destination operand (extended opcode word bits 7:4).
- z size selector for the operand size (extended opcode word bits 8).

28 C LIBRARY INSTRUCTION EXAMPLES

This section shows how the instruction set maps onto common C library kernels. The examples are schematic assembly sequences rather than ABI-mandated entry points. They use A0 as a source pointer, A1 as a destination pointer, D0 as the REP counter, and D1 as a temporary or fill-value register.

28.1 REP Counter Convention

REP and REPcc interpret the selected D register as an ordinary signed two's-complement counter. A zero counter performs no iteration. Positive counters are decremented toward zero; negative counters are incremented toward zero. The same architectural D-register value is visible to indexed effective-address calculation, so there is no REP-only counter representation for address expressions to reinterpret.

The examples below use a positive count and address-update operands such as [A0++]. The address update is part of the operand syntax: after each successful element access, the selected address register advances by the element size. Indexed forms remain useful for kernels whose source and destination strides differ.

For C library implementations, ordinary fast paths may normally load a `size_t` count directly into the REP counter when the implementation's maximum object size fits in the signed count range used by the chosen direction. A strictly conforming implementation that wants to define behavior for larger counts can branch to a slow path or split the transfer into multiple chunks.

28.2 memcpy

The forward byte-copy form uses source and destination base pointers and a positive signed count.

```
; memcpy(dst, src, n), byte copy forward
; A0 = src, A1 = dst
; D0 = n
REP D0, MOV.B [A0++], [A1++]
```

The same shape generalizes to wider element sizes by changing the instruction size and therefore the address-update step.

28.3 memmove

memmove chooses direction according to overlap. If the destination is below the source, or the ranges do not overlap, it can use the same forward sequence as memcpy. If the destination overlaps at a higher address, the library uses an analogous reverse address-update sequence.

```
; memmove(dst, src, n), non-overlapping or forward byte copy
; A0 = src, A1 = dst
; D0 = n
REP D0, MOV.B [A0++], [A1++]
```

28.4 memset

memset uses a register source and a memory destination. The counter/address convention is the same as memcpy forward traversal.

```
; memset(dst, c, n), byte fill
; A1 = dst, D1 = byte c
; D0 = n
REP D0, MOV.B D1, [A1++]
```

28.5 memcmp

memcmp can use REPEQ to continue while the observed comparison result is equal. The final FLAGS value is the result of the last completed comparison, including the terminating not-equal comparison when one occurs. The library tail can derive the required negative, zero, or positive return value from that final comparison state and the selected unsigned/signed byte policy.

```
; memcmp(a, b, n), forward byte compare
; A0 = a, A1 = b
; D0 = n
REPEQ D0, CMP.B [A0++], [A1++]
```

28.6 Bulk Character Extension

The orthogonal EXTS/EXTZ memory forms let a C library or codec routine widen elements while copying. The following shape zero-extends bytes into 32-bit elements, which is a natural kernel for ASCII or byte-string to UTF-32 expansion.

```
; widen n bytes to n 32-bit elements
; A0 = src, A1 = dst
; D0 = n
REP D0, EXTZL.B [A0++], [A1++]
```

29 RUNTIME INSTRUCTION EXAMPLES

This section shows how runtime support code can combine architectural discovery instructions with state-management instructions. The examples are schematic C helpers, not ABI-mandated entry points.

29.1 CPUID-Defined Program Context Save

SAVE and RESTORE use a CPUID-described memory image. A runtime that wants to save program-visible register and extension state first queries the required save-area size and component descriptors, then supplies a 4 KiB-aligned buffer to SAVE. The continuation state that is not part of the SAVE area, such as a scheduler or setjmp return point, is managed by the runtime layer that calls this helper.

The following C example is schematic. The two target hooks are ordinary runtime entry points implemented with the architectural CPUID `Dn` and SAVE `<ea>` instructions.

```
#include <stddef.h>
#include <stdint.h>

#define BR_CPUID_CLASS_IMPL      0x00000002u
#define BR_CPUID_SAVE_AREA_LAYOUT 0x0004u
#define BR_SAVE_ALIGNMENT      4096u

struct br_save_area_info {
    uint64_t size;
    uint16_t header_size;
    uint16_t component_count;
    uint16_t bitmap_words;
};

struct br_save_component {
    uint16_t id;
    uint16_t flags;
    uint32_t offset;
};

struct br_program_context {
    void *save_area;
    uint64_t save_area_size;
    uint16_t header_size;
    uint16_t component_count;
    uint16_t bitmap_words;
};

uint64_t bedrock_cpuid(uint64_t selector);
void bedrock_save(void *save_area);

static uint64_t br_cpuid_selector(uint32_t cls,
                                   uint16_t leaf,
                                   uint16_t index)
{
    return ((uint64_t)cls << 32) |
           ((uint64_t)leaf << 16) |
           (uint64_t)index;
}
```



```
static struct br_save_area_info br_query_save_area(void)
{
    uint64_t r0 = bedrock_cpuid(
        br_cpuid_selector(BR_CPUID_CLASS_IMPL,
                          BR_CPUID_SAVE_AREA_LAYOUT, 0));
    uint64_t r1 = bedrock_cpuid(
        br_cpuid_selector(BR_CPUID_CLASS_IMPL,
                          BR_CPUID_SAVE_AREA_LAYOUT, 1));
    struct br_save_area_info info;

    info.size = r0;
    info.header_size = (uint16_t)(r1 & 0xffffu);
    info.component_count = (uint16_t)((r1 >> 16) & 0xffffu);
    info.bitmap_words = (uint16_t)((r1 >> 32) & 0xffffu);
    return info;
}

static void br_query_save_components(struct br_save_component *out,
                                     uint16_t count)
{
    for (uint16_t i = 0; i != count; ++i) {
        uint64_t r = bedrock_cpuid(
            br_cpuid_selector(BR_CPUID_CLASS_IMPL,
                              BR_CPUID_SAVE_AREA_LAYOUT,
                              (uint16_t)(2u + i)));
        out[i].id = (uint16_t)(r & 0xffffu);
        out[i].flags = (uint16_t)((r >> 16) & 0xffffu);
        out[i].offset = (uint32_t)(r >> 32);
    }
}

static uintptr_t br_align_up(uintptr_t value, uintptr_t alignment)
{
    return (value + alignment - 1u) & ~(alignment - 1u);
}

int br_save_program_context(struct br_program_context *ctx,
                            struct br_save_component *components,
                            uint16_t component_capacity,
                            void *storage,
                            size_t storage_size)
{
    struct br_save_area_info info = br_query_save_area();
    uintptr_t raw = (uintptr_t)storage;
    uintptr_t base = br_align_up(raw, BR_SAVE_ALIGNMENT);
    size_t prefix = (size_t)(base - raw);

    if (prefix > storage_size)
        return -1;
    if (info.size > (uint64_t)(storage_size - prefix))
        return -1;
    if (info.component_count > component_capacity)
        return -1;

    br_query_save_components(components, info.component_count);

    ctx->save_area = (void *)base;
    ctx->save_area_size = info.size;
}
```

```
ctx->header_size = info.header_size;
ctx->component_count = info.component_count;
ctx->bitmap_words = info.bitmap_words;

bedrock_save(ctx->save_area);
return 0;
}
```

The component array is not part of the SAVE image. It is ordinary runtime metadata derived from CPUID so the context owner can inspect, copy, or migrate extension components without hard-coding implementation-specific offsets.